Technical Documentation Version 9.6

RiverWare Policy Language (RPL)

Center for Advanced Decision Support for Water and Environmental Systems
UNIVERSITY OF COLORADO **BOULDER**

Contents

Chapter 1
RPL Language Structure

RiverWare Policy Language (RPL) is a programming language included in RiverWare that allows modelers to express operating policy separate from the physical process model.

Topics

About RPL

One of the main goals of the RiverWare modeling system is to provide a way to express operating policy separate from the physical process model. Many older basin management models were *dedicated* software—they integrated complex operating policies into the code, intermingled with the code for the physical processes. This allowed ultimate flexibility and detail for the operations. Typically, general modeling tools allow only the specification of simple policies like guide curves.

RiverWare allows complex policies to be expressed in a general modeling framework. RiverWare achieves this through the RiverWare Policy Language (RPL), which the modeler can use to express policy. The policy statements are interpreted at runtime, and the policies interact with the simulation to drive the solution.

RPL Requirements

The requirements for RPL are as follows:

- Must be rich enough to express even the most complex operating policies,
- Must be an interpreted language, and
- Must be able to interact with RiverWare, primarily in being able to read and set slots in the model.

In the prototype development, Tcl (Tool Command Language) was used as the rule language. Tcl is a complete interpreted programming language that meets the requirements above (an interface was developed to allow Tcl to access model values, timesteps, etc.). Through observation of the use of that language, requirements for a more user-friendly language were developed. These include the following:

- Language must be easy to read, so that interested parties can look at the logic and understand the policy relatively easily,
- Language must be relatively easy to formulate, so the modeler does not have to learn a new complex programming language,
- The modeler does not have to debug spelling and similar syntax issues, i.e., that a syntax-directed editor is provided to ensure creation of valid expressions, and
- Performance of the interpretation and application of the language must be fast enough for acceptable run times.

Based on these needs, RPL was then developed. One of the primary design decisions for RPL was that it be, to a large extent, a *functional* programming language. This section describes the main characteristics of functional languages, how they differ from *imperative* languages like FORTRAN or C, and takes you through some exercises in formulating logic using this approach.

Imperative Versus Functional Programming

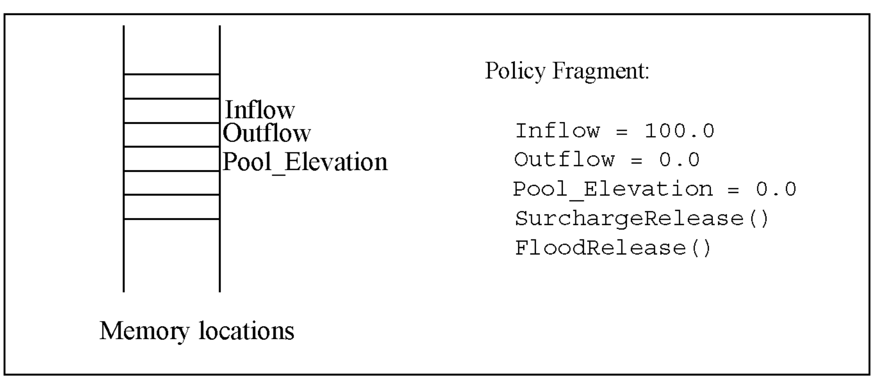
Imperative Programming Paradigm

Imperative (also called procedural) programming languages such as FORTRAN and C have been in use for a long time, and most engineers in the water management domain are familiar with the concepts. The structure of imperative, or statement-oriented languages is dominated by imperative statements, which, when evaluated in a given sequence, achieve the desired results of the program.

This type of language evolved as a natural extension of the Von Neumann computer architecture. Such languages are characterized by the existence of variables, the possibility of assigning values to variables, and mechanisms for repetition (iteration), mirroring the three computer architectural concepts of memory cells, storing or assigning values to memory cells, and the repetition of a set of instructions which are stored in memory.

Figure 1.1 illustrates a how an imperative programming language might be used to implement a portion of a water management policy:

Figure 1.1



In this example *Inflow*, *Outflow*, and *Pool_Elevation* are global variables; that is, any part of the code can read from or write values to these memory locations. The first several statements provide these memory locations with initial values. Then a subroutine is called to perform the “surcharge release” computation. We don’t know what that computation is, but we can guess that is sets *Outflow* and *Pool_Elevation* to reflect high priority policy considerations. It could do other things as well, like modify other values (e.g., *Storage*). Next, another routine is called which performs the “flood release” computation. Again, we don’t know what this routine does, but perhaps it might readjust the values set by the SurchargeRelease routine to manage flooding scenarios.

The use of memory locations is a key aspect of imperative programs. The two subroutines executed by the program fragment above presumably takes advantage of the memory locations corresponding to the global variables by reading them to get their values and assigning values to them.

Two advantages of the imperative programming approach are that programs tend to be efficient (because the program organization mirrors the machine architecture) and most water resources engineers are accustomed to this style of computation.

Referential Transparency

Imperative languages have a problem related to the issue of *referential transparency*. A system is said to be referentially transparent if the meaning of the whole can be determined solely from the meaning of its parts. Mathematical expressions are referentially transparent. Imperative languages are not referentially transparent because the value of a variable or the meaning of an expression (result of its evaluation) depends on the history of computation. Assignment statements, parameters passed by reference, and global variables are the main reasons that imperative languages are not referentially transparent. The lack of referential transparency means that it is possible to create programs which are difficult to read, modify and prove correct.

Functional Programming Paradigm

A functional language, by contrast, makes use of the mathematical properties of functions. Recall from mathematics that a *function is a mapping from a set of values in some domain to a single value*. Thus f(*x*,*y*,*z*) evaluates to a single value when specific values are given to x, y and z. A purely function programming language performs all of its computations by evaluating functions, i.e., by evaluating for various inputs the mathematical expressions which define the functions.

Notice that this description of purely functional programming languages contains no reference to memory locations. Since there is no setting of values in memory when evaluating a mathematical function, there are no hidden “side effects.” This lack of side effects allows functions to be combined hierarchically with predictable results. Knowledge of all the effects and predictability of the results makes the functional approach referentially transparent.

Let’s look at how the policy fragment from above might be written in a more functional manner:

```
Inflow = 100.0
Outflow = SurchargeRelease(Inflow)
Pool_Elevation = MassBalance(Inflow, Outflow)
Outflow = FloodRelease(Inflow)
Pool_Elevation = MassBalance(Inflow, Outflow)
```

If we allow functions to read (but not write!) global memory locations, then we don't need to pass the inputs to the function in explicitly, and this policy fragment might be written:

```
Inflow = 100.0
Outflow = SurchargeRelease()
Pool_Elevation = MassBalance()
Outflow = FloodRelease()
Pool_Elevation = MassBalance()
```

This code is quite similar to the original code fragment, but a couple of differences from the original code are worth highlighting. First, since a function produces only one return value, the functional computation requires separate functions for computing the value of *Outflow* and *Pool_Elevation*. Thus the SurchargeRelease function now only computes *Outflow*, and we have introduced the MassBalance function, which uses *Inflow* and *Outflow* to compute the *Pool_Elevation*.

> **Note:** This code is not strictly *functional*. We have retained from the original imperative program the idea of statements which assign values to memory locations, however we have moved the assignments to the outermost level. If we further stipulate that evaluating the three functions have no side effects (simply compute a value and don't change any memory locations), then this policy fragment becomes quite easy to read and understand. That is, it is immediately obvious which portions of the policy are affecting *Outflow*, which *Pool_Elevation*, and so on.

This is the philosophy of the RPL in a nutshell: restrict the setting of global values (slot or model values) to the outermost level of policy statements, all other policy computation is via function and expression evaluation. The key aspects of function and expression evaluation are as follows:

- Functions operate on zero or more input values.
- Functions evaluate to a single value (or to a list of values, considered a single item).
- Global memory values are not affected.

The advantages of this approach are that we can easily see where and how values are computed. We can look at the functions to see what they do; there are no hidden side effects.

Examples

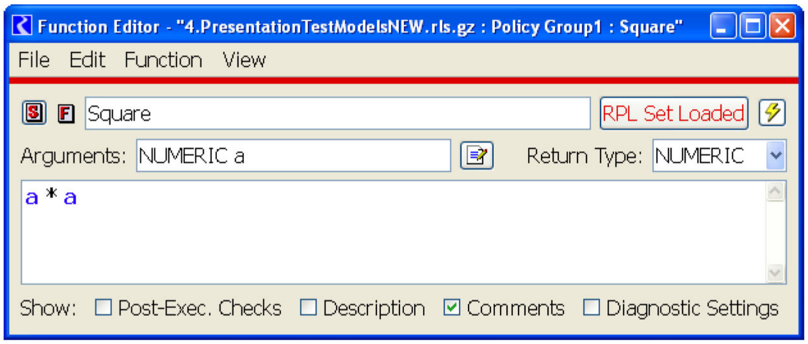
The purpose of the following examples is to familiarize the reader with the RPL approach to describing computation and to provide the opportunity to practice this style of problem-solving.

The following subroutine finds the square of a number.

```
SUBROUTINE Square( FLOAT a, FLOAT answer )
   answer = a * a
END
```

Figure 1.2 shows the same program written as a RPL function.

Figure 1.2

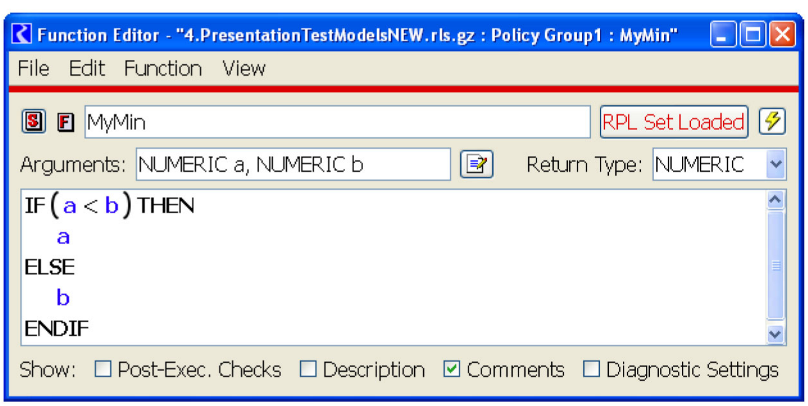


The following imperative-style function finds the minimum of two numbers.

```
FUNCTION Min( FLOAT a, FLOAT b )
   FLOAT answer
   IF ( a < b )
      answer = a
   ELSE
      answer = b
   END IF
   RETURN answer
END
```

Figure 1.3 shows the same function implemented within RPL.

Figure 1.3



Note: The RPL IF expression evaluates to a single value, as do all functions and expressions.

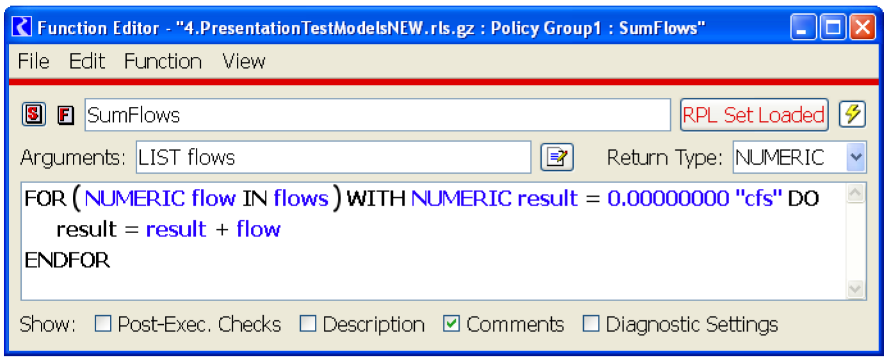
The following procedure takes an array of *numFlows* flow values and returns the sum of these flows:

```
SUBROUTINE SumFlows(FLOAT ARRAY flows,INTEGER numFlows,FLOAT total )
   INTEGER i = 0
   total = 0
   WHILE ( i < numFlows )
```

```
    total = total + flows[i]
    i = i + 1
  END WHILE
END
```

Figure 1.4 shows RPL code that accomplishes the same task.

Figure 1.4



Note: The FOR expression evaluates to a single value.

Following are the steps in evaluation of a FOR expression:

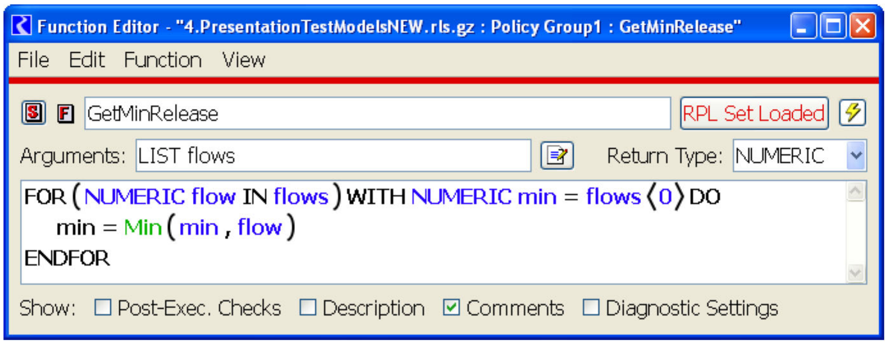
1. The list expression is evaluated.
2. The initialization expression is evaluated and the result is assigned to the loop variable.
3. The first/next item in the result of evaluating the list expression is assigned to the index variable.
4. The body is evaluated and the result is assigned to the loop variables.
5. If there are more values in the result of evaluating the list expression, return to the third step.
6. Return the value of the loop variable.

The following procedure takes an array of *numFlows* reservoir releases and returns the minimum of these values:

```
SUBROUTINE GetMinRelease(FLOAT ARRAY flows,INTEGER numFlows,FLOAT min )
  min = flows[0]
  INTEGER i = 1
  WHILE ( i < numFlows )
    IF ( min > flows[i] )
      min = flows[i]
    END IF
    i = i + 1
  END WHILE
END
```

The RPL version shows how iteration can be framed as an expression which evaluates to a single value (and uses the Min function defined above). Figure 1.5 illustrates.

Figure 1.5

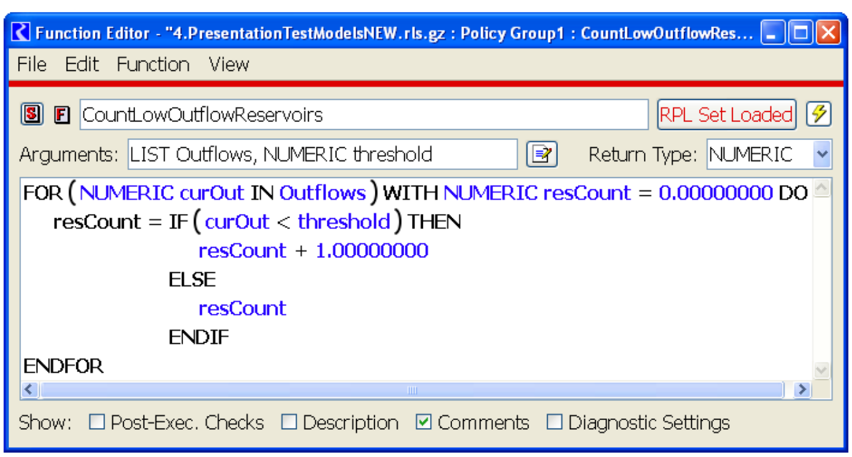


Given the pool elevation of *numRes reservoirs*, the following subroutine counts the number of reservoirs whose pool elevation is below a certain threshold.

```
SUBROUTINE CountLowReservoirs(FLOAT ARRAY PEs,INTEGER numRes,FLOAT threshold,INTEGER
resCount )
  resCount = 0
  INTEGER i = 0
  WHILE ( i < numRes )
    IF ( PEs[i] < threshold )
      resCount = resCount + 1
    END IF
    i = i + 1
  END WHILE
END
```

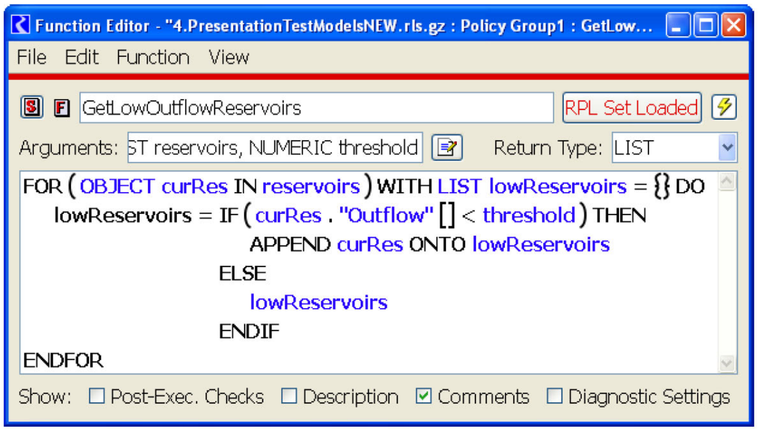
Figure 1.6 shows a possible solution.

Figure 1.6



To illustrate the use of some of the RPL built-in list operations, consider a variation on the previous example, in which we would like a list of the reservoirs whose pool elevation is below a certain threshold. Figure 1.7 illustrates.

Figure 1.7



RPL: A Hybrid Approach

The requirements of RPL point to an ideal language that has some elements of both imperative and functional programming languages. For example, the ultimate purpose of RPL as the rule language is to set slots in the model in order to drive the simulation. Therefore, it is necessary to have assignment statements, particularly to slots. The slots in the model maintain the state of the system for the rules, thus are analogous to values in memory. To maintain clarity of meaning for rule assignments, all assignments are done at the very top level of the rules. In fact, rules contain only slot assignment statements and print statements. Each slot is assigned the result of an expression or function evaluation, No slot assignments are “hidden” in lower level functions.

RiverWare rules have the following form:

```
Object.slot[timestep] = <expression>
```

where the expression could contain complex logic or be as simple as a single function call.

The rules need to look at the current state of the model, so the language must have the ability to read slot values from memory. This does not diminish the referential transparency of the rules because slot values in the model can never change while a rule is executing. Data that is associated with policy, for example reservoir guide curves and minimum flow values, are kept in custom slots in the model.

Beyond the assignments to slots, policy computation is performed exclusively by evaluating functions and expressions, providing the benefits of being able to follow the meaning of the rule and not having hidden side effects. To assist in policy that frequently references the same variable, the WITH expression allows the use of a local variable within an expression. This feature helps the user to write policies which are more efficient and simpler than they might otherwise be.

Chapter 2
RPL User Interface

This section describes basic rule, function, method, or constraint construction with the RPL Set Editor. The RPL Set Editor is the main window through which policy is managed. From this window, you can perform the following actions.

- Add, delete, open, name, prioritize, and enable or disable rules, functions, and methods.
- Open, close, save, load, export, import, and unload entire RPL sets.

Topics

About RPL Sets

Table 2.1 summarizes the available RPL sets.

Table 2.1

RPL Set	Set Color	Save Location	Link to More information
Expression Slot Set	Orange	In the model file	"Series Expression Slots" in *User Interface*
Global Function Set	Brown	Separate file ***OR*** in the model file	Global RPL Functions, page 81
Initialization Rules	Teal	In the model file	Initialization Rules Set, page 76
Iterative MRM Ruleset	Navy Blue	In the model file	"Iterative Runs" in *Solution Approaches*
Object Level Accounting Method Set	Green	In the model file	"Object-level Accounting Methods (OLAMs)" in *Accounting*
Optimization Goal Set	Purple	Separate file ***OR*** in the model file	"Create an Optimization Goal Set" in *Optimization*
Rulebased Simulation (RBS) Ruleset	Red	Separate file ***OR*** in the model file	"Rules and Rulesets" in *Solution Approaches*

When dealing with specific menus, this document will use the terminology *set* in place of any type of set. Each set has a unique menu name and should be used accordingly.

Set Name

Each set can have a user-specified name that is separate from the file path. The default for a newly created set is RPL Set N. Enter your desired name in the Name field shown in Figure 2.1.

Figure 2.1

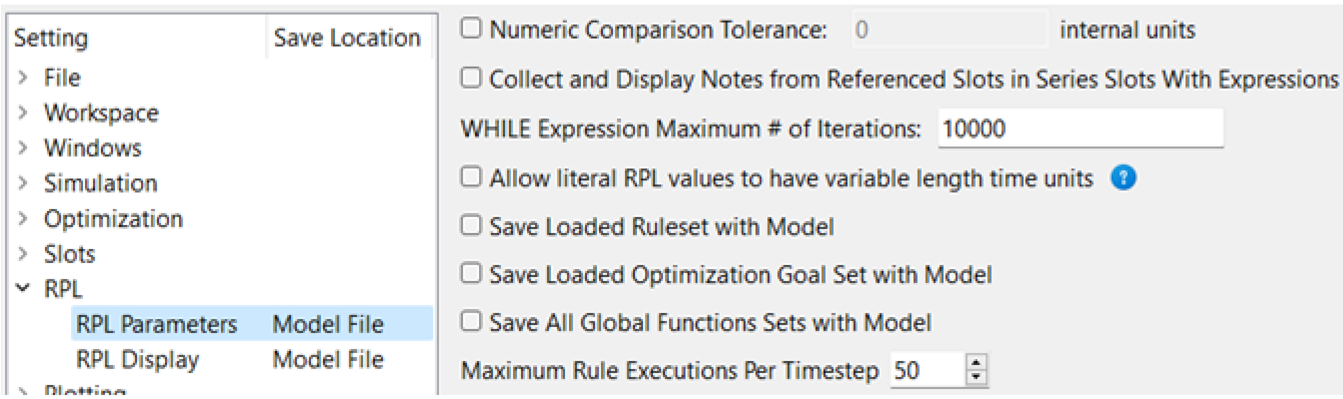
Note: Prior to RiverWare 7.0, the name and the file name were the same. When loading an old set into 7.0, the name defaults to the file name. You can change the name if desired.

For sets saved as a separate file, the File Path is listed as shown in Figure 2.1. If the set is saved in the model file, the text says **Save Location: Model File.**

RPL Parameters

The RPL Parameters are used to modify certain behaviors of RPL sets and RPL expressions. To access the RPL Parameters, use the Settings Manager or the workspace **Policy** and then **RPL Parameters** menu. The following figure shows the parameters.

Figure 2.2 RPL Parameters in the Settings Manager



The following sections describe the parameters or provide links to more information.

Numeric Comparison Tolerance

See “Setting Tolerance for Use in the Logical Comparison Operators” on page 112 for more information.

Collect and Display Notes

See “Collected Notes on Expression Slots” in *User Interface* for more information.

WHILE Expression Maximum Iterations

When a WHILE expression is executed, it counts the number of times the body is evaluated, and fails with a message if the iteration count ever exceeds the value of the RPL parameter. The default max iterations is 10,000, but may be modified as needed. See “Conditional and Iterative Operations Buttons” on page 119 for more information on the WHILE operator.

Allow Literal RPL values to have variable time units

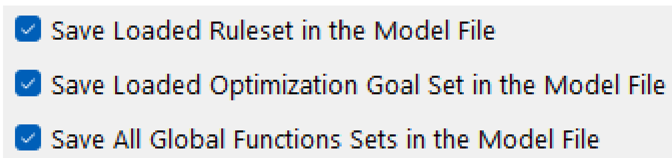
See “Use of Variable Length Time Units” on page 127 for more information.

Save Location

Based on the type of set, the set is saved in one of two locations, as shown in the Table 2.1.

- In the model file or
- In an external file

Four of the sets (expression slot set, initialization rules, iterative MRM rulesets, and accounting method set) are always saved in the model file. Global functions sets, goal sets, and rulesets may be saved in either location as specified in the RPL Parameter in the Settings Manager or from the workspace **Policy** and then **RPL Parameters** menu.



The **Save Loaded Ruleset in Model File** checkbox controls where rulebased simulation sets are saved. By default, this option is enabled, meaning the **Loaded** ruleset is saved in the model. Disabling means that the ruleset will need to be saved separately to an external file.

> **Note:** This toggle only applies to the *loaded* ruleset. Any other *opened* sets will not be saved with the model file.

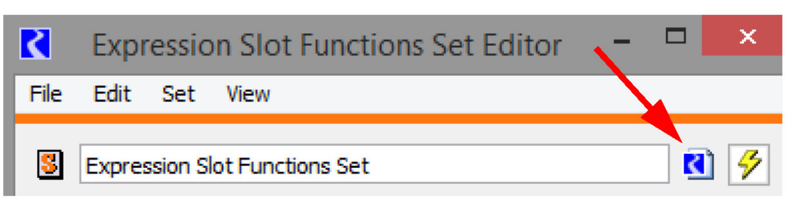
The RPL Parameters page contains an option to **Save All Global Functions Sets in Model File** toggle. By default, this option is enabled, meaning that all global functions sets are saved in the model. Disabling it means that all global functions sets will need to be saved separately to external files.

Once a set is saved with the model file, there is some risk that it may be lost if:

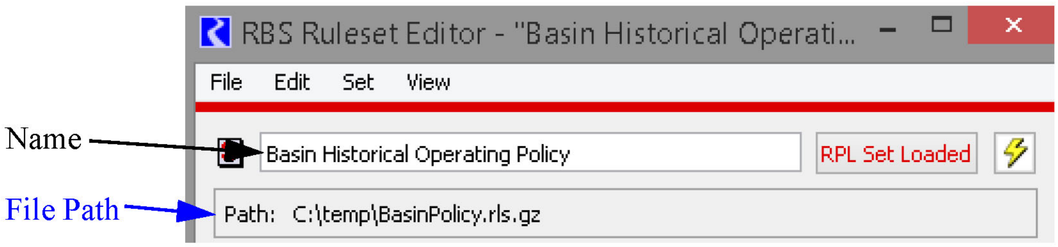
- The toggle above is unchecked and the model is saved.
- The set is unloaded and the model is saved.
- Another set is loaded and the model is saved.
- Closing the set and the model is saved.

In each case, a warning message is presented to allow you to confirm that the action is intended. A set saved with the model is automatically loaded when the model is opened. The set is then minimized. You can bring it to the front at anytime using the workspace **Policy** menu or the buttons on the bottom of the workspace.

Further, for any set that is saved with the model, a model file icon is displayed at the top right of the dialog.



Sets that are saved in an external file have the path shown below the set Name.



Maximum Rule Executions Per Timestep

This value indicates how many times a single rule can execute on each timestep. The default is 50, but many models need additional executions per timestep.

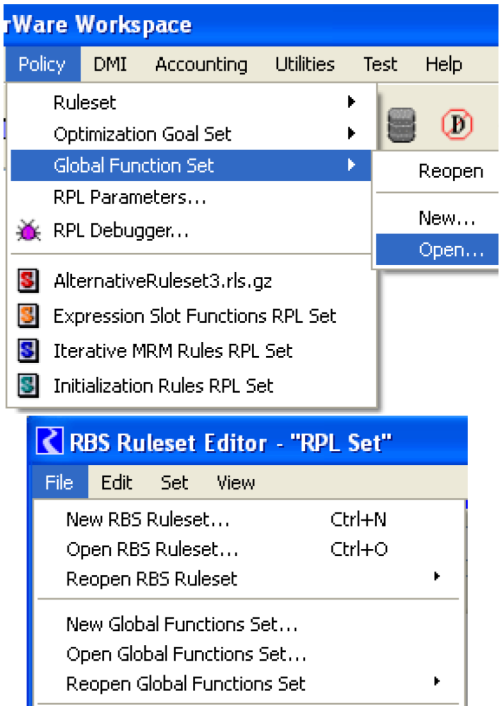
Managing RPL Sets

Following is a description of managing RPL sets. Much of this information is specific to the RPL set in question.

Actions Specific to Rulesets, Optimization Goal Sets, and Global Function Sets

The following actions are specific to managing a set (either Ruleset, Optimization Goal Set, or Global Function Set) and are performed on an entire set. Figure 2.3 illustrates.

Figure 2.3



- **New**. A new RPL set is created by selecting **Policy**, then **Set**, then **New** from the main RiverWare workspace or **File**, then **New Set** from an already opened set.
- **Open**. An existing RPL set (saved as a file) is opened by selecting **Policy**, then **Set**, then **Open** from the main RiverWare workspace or **File**, then **Open Set** from an already opened set. Either of these commands invokes a file chooser in which the desired set is selected. Each set is saved in a separate file. Several sets may be opened at once. Each opened set and any windows opened from these sets are identified by a unique color band across the top of their windows.
- **Reopen**. An existing RPL set (saved as a file) can be reopened by selecting **Policy**, then **Set**, then **Reopen** from the main RiverWare workspace or **File**, then **Reopen Set** from an already opened set. Either option then presents a cascading menu of recently opened sets and folders.
- **Reopen and Load**. An existing RPL set (saved as a file) can be reopened and loaded by selecting **Policy**, then **Set**, then **Reopen and Load** from the main RiverWare workspace. Then, a cascading menu of recently opened sets and folders is presented.
- **Show RPL set**. When a set is opened, the name of the set is added to the workspace **Policy** menu as shown in Figure 2.4. Only the short name is shown in the menu, but the full path is shown in the workspace status bar. Use this menu to raise/show the desired set. When the set is saved with the model file, the menu will show it as **RPL Set**.

  Note: The expression slot, initialization rules, MRM and accounting methods sets (saved with the model) are always shown too. Also, the workspace has buttons to show the *loaded* RBS ruleset  or optimization goal set  . The bar is colored when there is a loaded set, grey when there isn't. Select one of the buttons (when a set is loaded) to raise that set to the top.The workspace also shows color coded buttons for all the opened sets. Tooltips indicate their names, as shown in Figure 2.5.

Figure 2.4

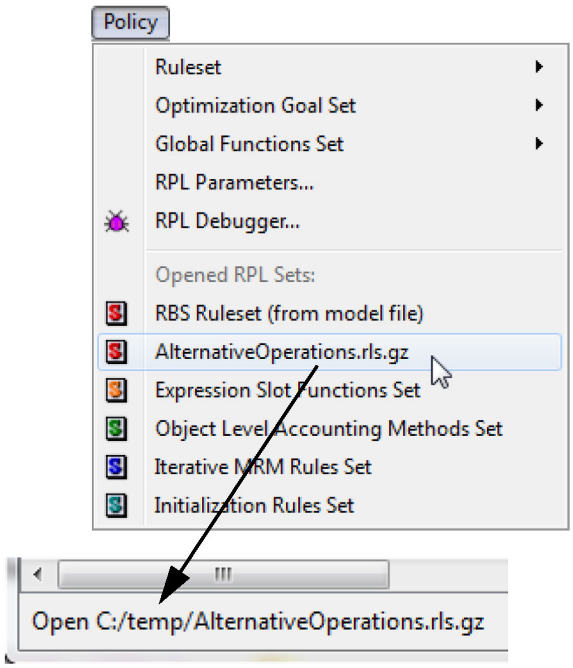


Figure 2.5

- **Save**. When saved to a separate file, save any changes by selecting **File**, then **Save Set**. The name and folder of the file to which the RPL set is saved is that from which it was last loaded or to which it was last saved.
- **Save As**. A new RPL set configured to be saved to a separate file must be given a name and saved for the first time by selecting **File**, then **Save Set As** in the **Set Editor** menu bar. This command is also used to save an open RPL set to a different filename and/or directory from which it was last loaded or saved. The **Save Set As** command invokes a file chooser in which the RPL set's new path and filename must be specified.

  Note: For sets saved in the model file, there is no **Save** or **Save As** menu; saving the model will save the sets. Use the **File**, then **Export Set** command to save the set to a separate file. This does not unload it from the model.

- **Load**. A ruleset or goal set is loaded into a model for use during a run by selecting the **RPL Set Not Loaded** button on the right side of the **Set Editor** menu bar. When the set is loaded, the button text changes to **RPL Set Loaded** and the colored bar along the top of the **Set Editor** becomes red. Only one ruleset at a time may be loaded for use by a model. Loading a set when another is already loaded will unload the first ruleset. Global Function Sets are not loaded, any open set applies to all other sets.

- **Unload**. A loaded set is unloaded from a model by selecting the **RPL Set Loaded** button on the right side of the **Set Editor** menu bar. When the set is unloaded, the button text changes to **RPL Set Not Loaded**, and the red bar along the top of the **Set Editor** reverts to its original color (blue, green, yellow, purple, etc.).
- Close **Window (Ctrl+W)**. An opened RPL set is closed but not removed by selecting **File**, then **Close Window** in the **Set Editor** menu bar. It can be re-shown using the **Policy** menu on the workspace.
- **Close (and Unload) Set**. An opened RPL set is closed by selecting **File**, then **Close (and Unload) Set** in the **Set Editor** menu bar. Closing a loaded set automatically unloads the set from the model. Closing does not automatically save changes to the set since the last save. This option is not available for those sets always saved with the model file.

Actions Specific to Accounting Method Set, Expression Slot Set, Initialization Rules, and Iterative MRM Sets

The following actions are specific to the Object Level Accounting Method set, the Expression Slot set, Initialization Rules, and the Iterative MRM set. Each of these RPL sets is saved with the model so they do not need to be opened or saved separately.

To show the Object Level Accounting Method set (when accounting is enabled):

- From the Workspace choose the **Policy**, then **Accounting Methods RPL Set** or choose **Accounting**, then **Open Accounting Methods RPL Set**

To show the Expression Slot RPL set:

- From the Workspace choose the **Policy**, then **Expression Slot RPL Set**

To show the Initialization Rules RPL set:

- From the Workspace choose the **Policy**, then **Initialization Rules Set**

To show the Iterative MRM RPL set:

- From the Workspace choose the **Policy**, then **MRM RPL Set.** The MRM RPL set can also be opened from the MRM run control dialog. See “Iterative Runs” in *Solution Approaches* for details.

These sets are always active in their respective context so it is not necessary to load or unload the sets. The dialog for the sets can be closed using the **File**, then **Close Set** menu. Again, each of these sets is saved with the model file so do not need to be separately saved.

The Initialization Rules Set Editor dialog does have a **File**, then **Save Initialization Rules Set As** menu item to save the initialization rules to a file. A **File**, then **Replace Initialization Rules Set from File** menu item allows the initialization rules set to be replaced by the contents of a specified file. These menu items allow an initialization rules set to be moved between models via a file. However there is only a single instance of initialization rules in a model and this set is still saved and loaded with the model file. See “Initialization Rules” in *Solution Approaches* for details on Initialization Rules.

Tour of a RPL Set

In RPL sets there is an upper-level construct specific to the RPL set. For example, in a ruleset, a rule is the construct; in an object level accounting method set, a method is the upper level construct. In this document, a *block* refers to the upper level construct for each of these sets and will be used in all descriptions. Table 2.2 lists the block for each type of set and the term used for the numbering.

Table 2.2 Table of blocks for each RPL Set

	RPL Set	Upper-level Block	Block Numbering
	Expression Slot Set	N/A	N/A
	Global Function Set	N/A	N/A
	Initialization Rules Set	Rule	Index
	Iterative MRM Ruleset	Rule	Index
	Object Level Accounting Method Set	Method	Index
	Optimization Goal Set	Goal	Priority
	RBS Ruleset	Rule	Priority

> **Note:** Expression Slot and Global Function Sets do not have the paradigm of a block. For Expression Slot Sets, the analogous upper level construct is the expression slot itself. Global Function Sets do not have the concept of a block as they contain only global utility groups and functions.

This remainder of this section describes the components of a RPL set when using the RPL set editor.

RPL Set Elements

A RPL set includes the following general elements:

- **Policy Groups:** Policy groups are containers for blocks and functions. The functions within a policy group are available only to blocks and functions within that policy group.
- **Utility Groups:** Utility groups are containers for functions. The functions within a utility group are available to blocks and functions within any policy group.
- **Blocks:** Blocks (Rules/Goals/Methods) are prioritized expressions of policy which assign slots and/or generate diagnostic messages.
- **Functions:** Functions are subroutines called from blocks or other functions, which evaluate to a value in one of the expression data types.
- **Report Groups:** Report groups provide an organizational grouping of related RPL and workspace objects.
- **Report Items:** Report items refer to blocks or functions in the RPL set or to workspace objects.

Table 2.3 shows all the RPL elements and their icons. The icons are shown in grey, but each set has its own color that is used for the letter.

Table 2.3

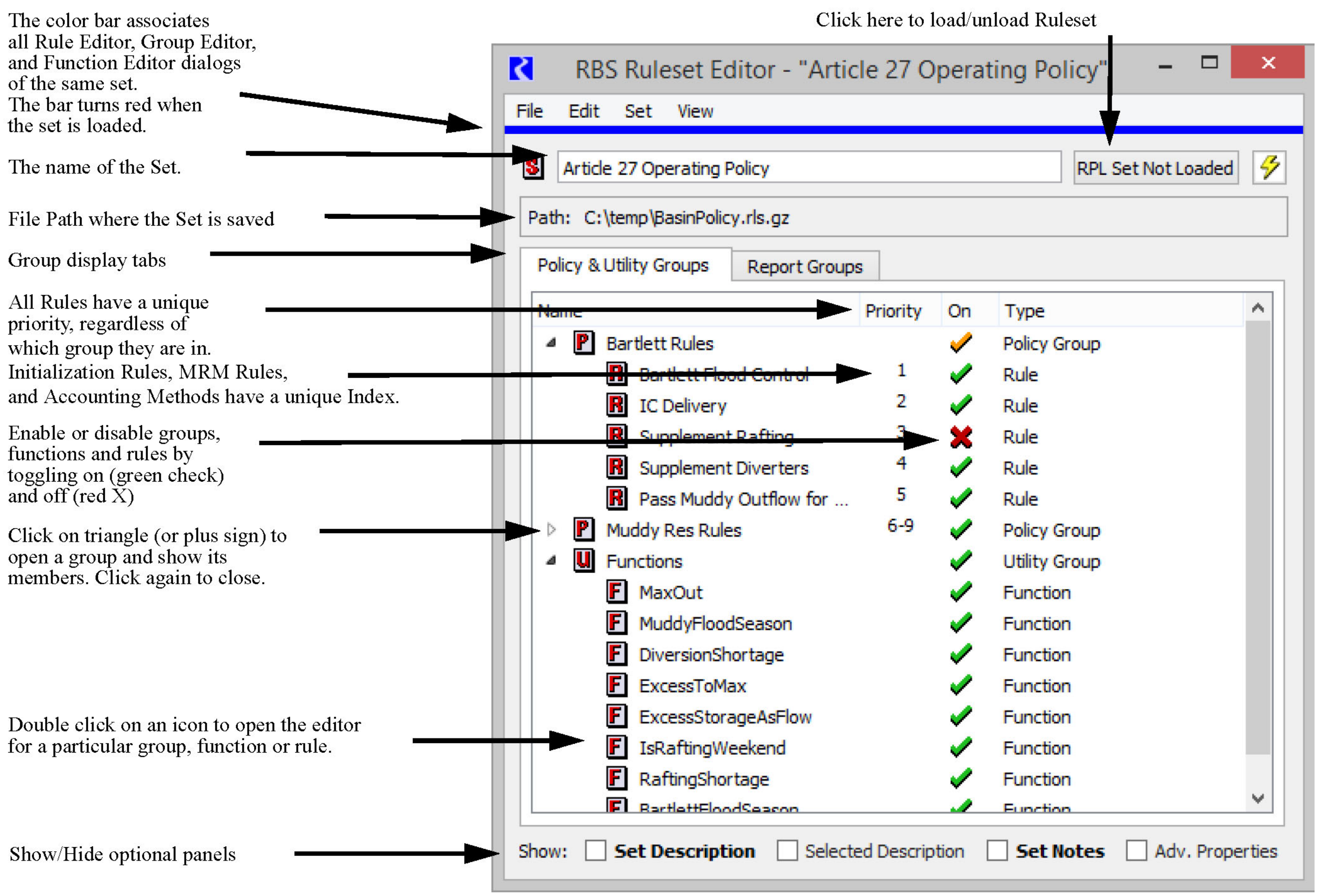
Letter	Icon	Item		Letter	Icon	Item
G		Goal		R		Rule
M		Method		S		Set
P		Policy Group		=		Statement
		Report Group		F		User-defined Function
		Report Item		U		Utility Group

Predefined functions use the F icon but always have the light blue color.

RPL Set Editor View

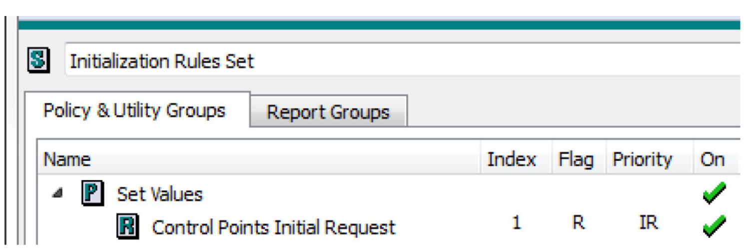
Information about a RPL set is displayed in the RPL Set Editor dialog. Each policy group, utility group, block, and function is represented in the RPL Set Editor on a different line. Figure 2.6 illustrates.

Figure 2.6



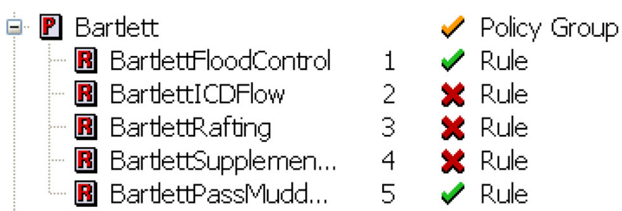
Each line includes the following information:

- **Name:** The name column contains an icon representing the type of the element and its name. All elements are assigned a default name when created.
- **Priority:** A priority is shown to the right of an element name, when appropriate. Policy groups, utility groups, and functions do not have priorities but policy groups display the range of priorities of its member rules. Priorities number indicates the relative importance of the block.
- **Index**: For Initialization Rulesets, Object Level Accounting Method Sets, and MRM Rulesets, the **Index** column is shown. The priority column may not be shown. Rules or methods have a unique **Index** which is how the rules and methods are referenced.



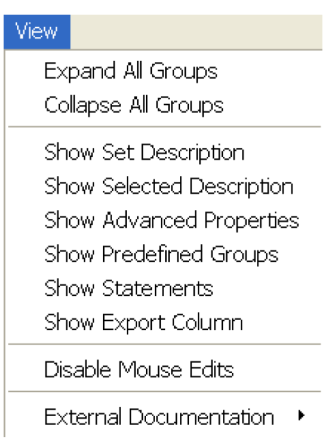
- **Flags**: Initialization Rules allow you to specify the flag that is set. For initialization Rulesets, a Flags column is shown.
- **On/Off Status:** All elements may be turned on or off independently by selecting the green check mark or red X in their line, unless mouse editing has been disabled. If mouse editing has been disabled, a block or function can be enabled or disabled by right-clicking. This will bring up a menu to enable or disable the element. A green check mark indicates that an element is enabled. A red X indicates that an element has been disabled. An element that is disabled cannot be used during the model run. Disabling a policy group or utility group disables all of the blocks and functions within that group. Selecting a group's red X now enables the entire group without making any changes. This prevents you from inadvertently changing the status of an item in a group that is disabled, only to find that it is now different when the entire group is re-enabled. If all items in a group are disabled (red X), then the entire group is considered disabled and the group will also have a red X. If you select the group's red X, a warning message will appear to indicate that at least one item in the group should be enabled. An orange check mark on a policy or utility groups specifies that one or more items in that group is disabled, as shown in Figure 2.7. As before, you can select the orange check to disable the entire group. Selecting it again, restores the previous state thus showing an orange check.

Figure 2.7



- **Type**: The type of each item in a set is shown in the **Type** column.

There are several additional pieces of information that can optionally be shown from a RPL set as accessed from the **View** menu.



Expand or Collapse All Groups

A RPL set's contents may be viewed with all groups expanded, some groups expanded, or all groups collapsed. Use one of the following methods to expand or collapse all groups simultaneously:

- Select **View**, then **Expand All Groups** or **Collapse All Groups** item in the **RPL Set Editor** menu bar.

- To expand or collapse a single group, select the triangle next to the group name.

  Note: The following four items can be shown using the **View** menu or selecting the toggle in the **Show**: row. If there is an entry in these panels (or a non-default value) the text is **bold**. For example, in the following screenshot, the **Set Description** and **Set Notes** have a non-default entry.

Show RPL Set Description

A description of the RPL set may be typed into the RPL Set Description: text frame and saved in the RPL set file. The description may contain any combination of letters, numbers, spaces and punctuation, except double-quotes (“ ”). The area is opened and closed by toggling the **View**, then **Show RPLSet Description** in the menu bar or using the **Set Description** toggle at the bottom of the window.

Show Selected Description

A special expanded area of the RPL set is used to enter a description of any specific block. The area is opened and closed by toggling the **View**, then **Show Selected Description** in the menu bar of the RPL Set Editor dialog or using the **Selected Description** toggle at the bottom of the window. Select a block name to view the description of that specific block.

Show Set Notes

Notes about the RPL set may be typed into the **Set Notes** text frame and saved in the RPL set file. The Notes may contain any combination of letters, numbers, spaces and punctuation, except double-quotes (“ ”). The area is opened and closed by toggling the **View**, then **Show Set Notes** in the menu bar or using the **Set Notes** toggle at the bottom of the window. Notes are often used for commenting on development practices; they provide a place to annotate when/why/who changed the set.

Show Advanced Properties

The **Advanced Properties** are accessed from the **View**, then **Show Advanced Properties** menu or using the **Adv. Properties** toggle at the bottom of the window The Advanced Properties area has the following components.

- **Agenda Order**. Rulesets can be configured to execute in either ascending or descending Priority/(it called Index in Initialization Rulesets) order. Descending order executes the highest priority block first, then each of the lower priority blocks (1,2,3,...). Ascending order executes the lowest priority block first, then each of the higher priority blocks (...,3,2,1). Ascending Priority (...3,2,1) is the default selection and the recommended priority order. The agenda order is only applicable and shown for RBS rulesets, initialization rules, iterative MRM rulesets and optimization goal sets. Also, optimization goal sets always must execute in the 1,2,3... order and cannot be changed.
- **Precision**. Specify the Set Precision, the number of digits to display after the decimal for literal numbers in RPL expressions. 2 is the default value. The displayed precision has no effect on the 17 significant figures of internal precision used for calculations.

  Tip: You can change the precision for any individual value by selecting the numeric expression and using the right-click Precision context menu. See “Precision of Numeric Values” on page 56.

Show Predefined Groups

In the **View**, then **Show Predefined Groups** menu, you can choose to show the groups for the predefined functions. When you also show Selected Description above, the documentation for that predefined function is shown in the description panel.

Show Statements

In the **View**, then **Show Statements** menu, you can choose to show statements (assign, print, etc) as another level in the RPL dialog tree-view. You can also change the name of statements once shown. To do this, right-selects that item to show the context-sensitive menu. Then, selecting **Rename** will open an inline name editor and you can enter a new name. In addition to the RPL Set view, this statement name will be shown in the RPL debugger and RPL Analysis Tool.

Show Export Columns

The **View**, then **Show Export Column** menu allows you to specify if the selected row should be exported. See “Exporting and Importing RPL Sets” on page 38 for details on Import and Export of RPL sets.

Disable Mouse Edits

It is easy to rename a group, block or function, change the priority of a block, and turn a block on or off. To prevent accidental changing of a block priority or turning the block on or off in the Set Editor as a result of mouse clicks, select **View**, then **Disable Mouse Edits.** This will prevent editing with mouse clicks in the RPL Set Editor. To edit a block or group after mouse editing is disabled, double-click or right-click the block, function, or group name to bring up an editor dialog. We highly recommend disabling mouse editing to prevent accidental changes to a RPL set.

External Documentation

View, edit, or configure the external documentation; see “RPL External Documentation” on page 131 for details.

Editing RPL Sets

Following is a description of how to manage and edit RPL sets once they are open. This includes creating, naming, and editing blocks/groups, adding statements, and validating blocks or the entire set. See “Editing a RPL Expression” on page 54 for details on editing individual expressions.

When a RPL set dialog opens, the **Policy & Utility Groups** tab is initially selected, presenting a comprehensive high-level view of the policy. Switching to the **Report Groups** tab provides a more selective, user-defined view.

> **Note:** Many actions described below are appropriate only for one or the other main tabs (e.g., adding a new Report Group), and these actions are disabled when they do not apply.

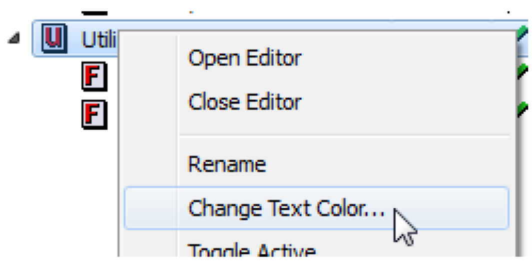
Blocks and Groups

Naming Groups

To name a block or group, right-click or double-click the **Name** field of the block or group. Right-clicking will bring up a menu from which the editor dialog can be opened. Double-clicking will bring up an editor dialog directly. Type the name in the name field.

Name Color

The color of each name in the set can be specified by right-clicking an item and choosing **Change Text Color**. The selected color is then used by the RPL set editor, the RPL set analysis tool, and the RPL object selector dialogs for the name of the selected object. For groups, the color is changed for the group name and all members of the group that have the default black color.



Adding Groups

Groups (of type Policy, Utility, Global Utility, or Report) are added to a RPL set by selecting the appropriate item from the RPL set **Set** menu (e.g., **Set**, then **Add Policy Group**, **Set**, then **Add Utility Group**, **Set**, then **Add Report Group,** or **Set**, then **Add Global Utility Group,** for global function sets). New policy and utility groups are added to a set directly below the currently highlighted group of that type.

Add Blocks and Functions

Blocks and functions are added to a RPL set by highlighting the group they are to be placed in and selecting **Set**, then **Add Block**, **Set**, then **Add User-Defined Function,** or **Set**, then **Add Global User-Defined Function** (for global function sets) from the RPL set menu bar. New blocks and functions are added to a RPL set directly below the currently highlighted element of that type or at the bottom of a highlighted policy or utility group.

Add Report Items

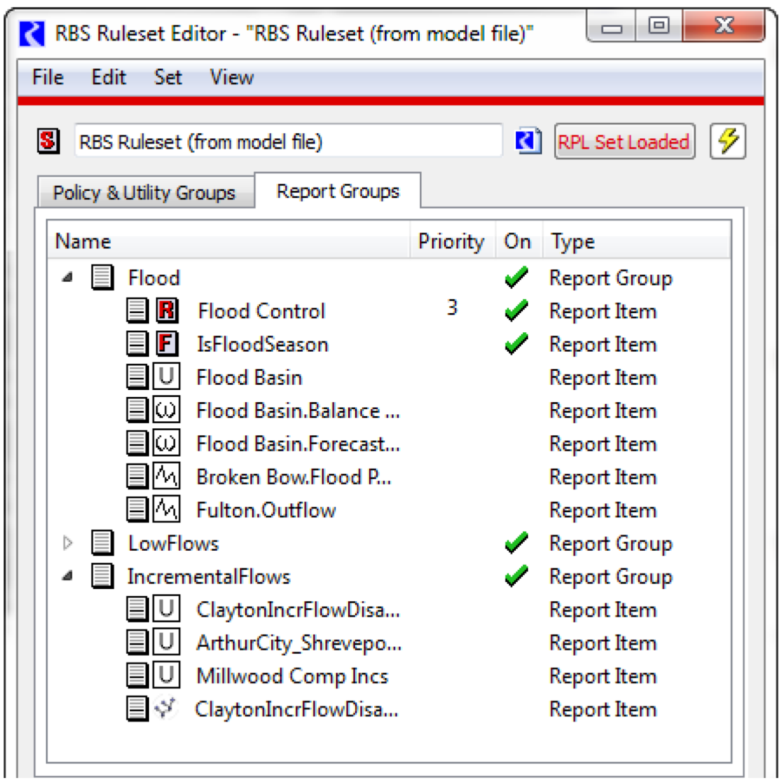
The **Report Groups** tab, shown in Figure 2.8, provides a more selective, user-defined view of the policy/slots and object. Items are added to a RPL report group (visible in the **Report Groups** tab) by highlighting the group they are to be placed in and selecting the desired action from the **Set**, then **Add Report Items** menu, which contains an option to open a selector for each of the following types of objects.

- **RPL Object Items** Blocks and functions in the current set. See “RPL Printing and Formatting” on page 86 for details on using the RPL Object selector.

- **Object Items** objects on the workspace.
- **Slot Items** slots on objects or accounts.
- **Subbasin Item** a single subbasin (automatic or user-defined).

Selecting the **Ok** button in the selection dialog will append the items for the selected objects to the end of the selected report group, but can be moved to any position by using the drag/drop technique.

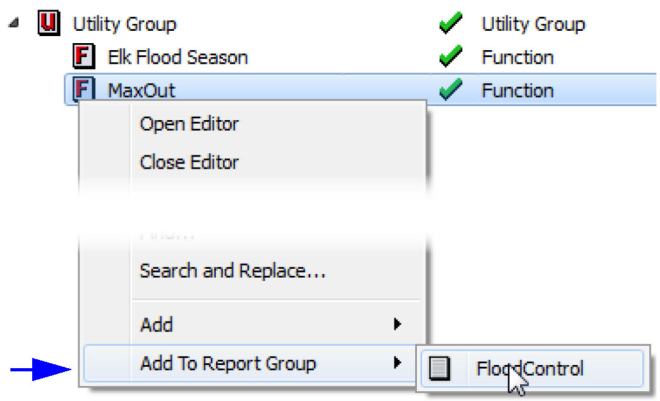
Figure 2.8



You can also right-click any RPL item (rule, function, etc), and choose Add to Report Group and select an existing Report Group. This will append the selected RPL item to the Report Group, as shown in Figure 2.9.

See “Report Layout Area” in *Output Utilities and Data Visualization* for details on including a Report Group in a model report.

Figure 2.9



Change Priorities

Blocks and functions may be rearranged in a set. By changing the relative priority of blocks, the order of firing, success of slot assignments, and order of dispatching is altered. Changing of priorities is often used to evaluate the effectiveness of different operational policies. Rearranging blocks and functions in a set is simple. To move a block in the list, select its icon and drag the block to the new desired location within the set. If you place it on another block, it will append after that block. Then, simply release the mouse button to drop the block in its new location. A dialog will ask you to confirm that you wish to complete the move. Select **OK** to confirm the move. All other blocks will be shifted to comply with the new priority structure. There is no need to reload a loaded set when rearranging blocks. Moving a block or a function is done in an identical manner. Predefined functions cannot be moved.

Delete Groups, Blocks and Functions

Groups, blocks and functions are deleted from a RPL set by highlighting them and selecting **Edit**, then **Delete**. Deleting a policy or utility group deletes the contents of the group as well. A confirmation dialog is shown.

Open and Close Policy, Utility, and Report Groups

Policy and utility groups which contain at least one block or function can be opened in a separate window by double-clicking their  , , or  icon or highlighting them and selecting **Set**, then **Open Editor** They may also be expanded within the **RPL Set Editor** window by selecting the white tree-view triangle to the left of their icon or selecting **View**, then **Expand**. An expanded policy or utility group can be collapsed by selecting the white tree-view triangle again or selecting **View**, then **Collapse**.

Open and Close Rules, Functions, Methods, Goals, and Report Items

Blocks and functions are opened in a RPL Viewer by double-clicking the icon or highlighting the item and selecting **Set**, then **Open Editor**. An opened block or function in the RPL Viewer may be closed by selecting red X on the tab or by selecting **File**, then **Close Window** from an individual RPL Editor.

Copy and Paste Groups, Blocks, and Functions

Copy and paste groups, blocks and functions within a set, between open sets, and between sets in different RiverWare sessions. Copy by highlighting and selecting **Edit**, then **Copy**. Paste below any highlighted item in a RPL set by selecting **Edit**, then **Append**. An item may also be duplicated by selecting **Edit**, then **Duplicate**.

> **Note:** A copied group/block/function can be pasted (or dragged and dropped) as a text representation into any text editor. This shows the internal representation of all information about that group, block, or function including the logic, description, active state, etc.

See “Exporting and Importing RPL Sets” on page 38 for details on copying large blocks or groups from one set to another set.

Exporting and Importing Groups and Blocks

In addition to copy and paste operations, it is also possible to export and import all or part of a RPL set. This is useful to share items between two RPL sets. See “Exporting and Importing RPL Sets” on page 38 for details on this feature.

Validity

Before loading a newly built set into a model (or running with one of the other types of sets), the validity of the set must be checked. This can be done manually or is automatically done at run start. Manually validating a RPL set, however, allows you to perform these checks without having to bring up the Run Control dialog, start the run, and stop the run or wait for it to finish. Validating a RPL set checks for unspecified expressions, illegal object and slot names, conflicting expression types, and syntax errors. It *does not* check the consistency of unit types in mathematical expressions. Unit consistency is done while the block is evaluated during the run.

- To check the entire set, in the RPL Set Editor, select **Set**, then **Check Validity.**
- To check a single block or user defined function, select **Block**, then **Check Validity.**

If the block or set is valid, a confirmation window appears. Select **OK** and continue. Otherwise, a validation error dialog appears and the diagnostic window posts the location of the invalid expression. You then must fix the block.

Comparing RPL Sets

When developing RPL sets, it is often desirable and/or necessary to compare two RPL sets and see what is different between the sets. Following are some examples of when you might want to compare two RPL sets:

- You have multiple copies of similar RPL sets that represent different policy alternatives. You need to determine what is different to make sure they represent the correct policy alternative.

- Your co-worker has made changes to the RPL set and you want to know what changes have been made. Perhaps then you will incorporate the changes into a master version.
- You need to identify all of the changes that have been made to the set since some previous version.

The **RPL Set Comparison Tool** compares two RPL sets and shows you the differences between the sets. This allows you to see where items are different, what the specific differences are, and allows you to easily access the RPL set dialogs so that you can change one or both sets. This tool support merging by allowing you to copy a property's values from one set and replace the same value in another set. Further, it provides quick access to the RPL dialogs and the ability to copy and paste from the utility to the RPL editors.

A more general tool for comparing model files is described in "Model Comparison Tool" in *User Interface*. It also compares RPL sets that are embedded in the model file.

Accessing the Comparison Tool

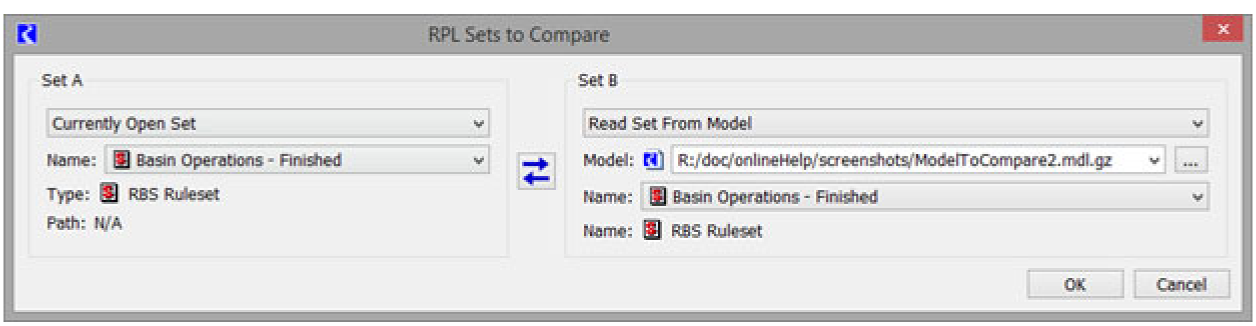
From the RiverWare workspace, use the menu: **Policy**, then **RPL Set Comparison Tool**

Or from any RPL set editor dialog select **Set**, then **Compare Set**

Selecting RPL Sets to Compare

The tool opens and shows a dialog where you specify the two sets to compare. The two sets are referred to as **Set A** and **Set B**. For each set, choose one of the following:

- **Currently Open Set**: Choose any of the sets that are open in the current model.
- **Read Set from File**: Choose a RPL set on the file system. Choose the type and file path. Use the **...** button to open a file chooser.
- **Read Set File from Git Repository**: Choose a RPL set from a Git repository on the file system. Select the files using the Git Repository Explorer. Then select the set type. See "Reading From a Git Repository" in *User Interface* for more information on this operation.
- **Read Set from Model**: Choose a RPL set that is embedded in another model file on the file system. First select the model file. Then select the name of the set.
- **Read Set from Model in Git Repository**: Choose a RPL set that is embedded in a model file that is part of a Git repository on the file system. Select the files using the Git Repository Explorer. Then select the Set type. See "Reading From a Git Repository" in *User Interface* for more information on this operation.



Use the swap button to switch the selection for model A with the selection for Model B.

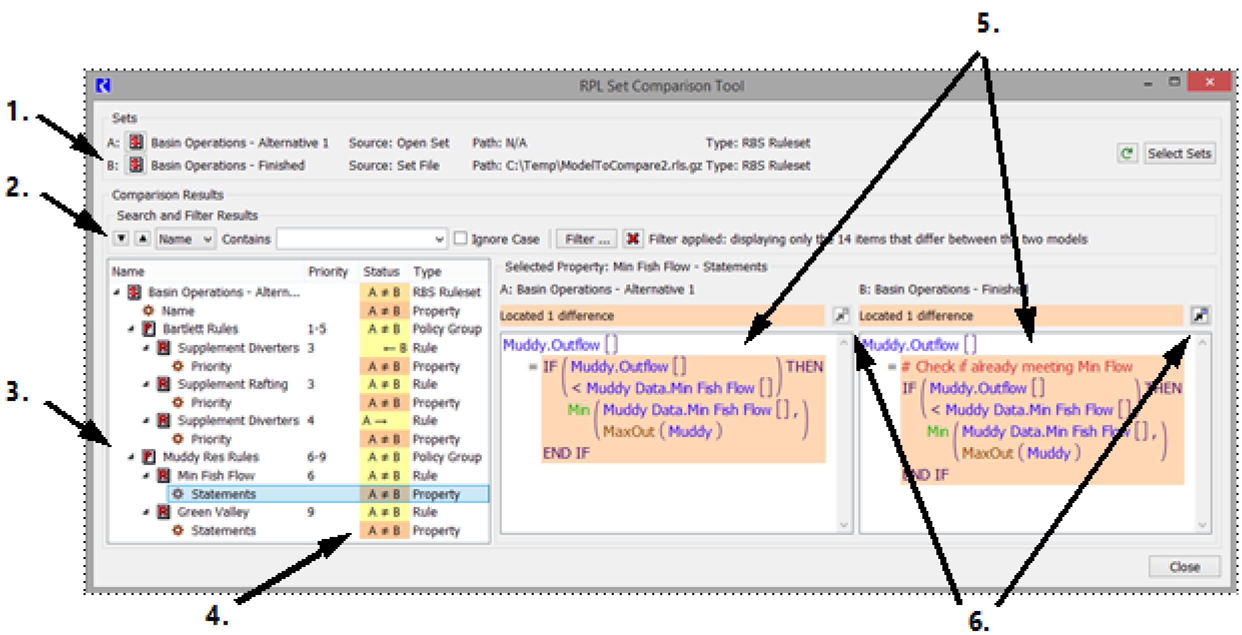
When finished, select **OK**. The main **RPL Set Comparison Tool** opens.

Tour of the RPL Set Comparison Tool

The RPL Set Comparison Tool contains the following areas, as shown in Figure 2.10 and explained by the numbered items:

1. The **Sets** panel shows the names, locations and type of the selected sets.
2. The **Search and Filter Results** panel allows for searching and filtering of the results.
3. The **Comparison Results** area shows a hierarchical view of the sets and the comparison results. This shows where there are differences between the two sets
4. The color coding and Status shows individual differences between the two models.
5. The **Selected Property** panel shows the values for the selected row for both Set A and Set B. This shows the differences for the selected item highlighted in orange.
6. The Copy Property operation allows you to copy the property's value from one set and replace it in the open set. Note, only replacing in open sets is supported. The appropriate buttons are enabled when this operation is supported.

Figure 2.10 Annotated Screenshot of the RPL Set Comparison Tool



The following sections describe these areas and how to use the tool.

Sets Panel

The Sets panel shows the sets that are being compared. It shows the set Name, Source, Path, and Type. Use the **Select Sets** button to choose alternative sets. See "Selecting RPL Sets to Compare" on page 27 for details on the dialog that is displayed.

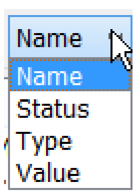
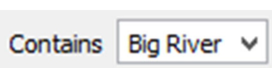
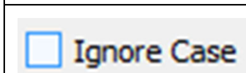
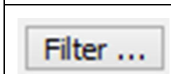
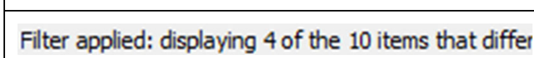
Recompute the differences between the two sets by clicking on the green circular arrow.  This is useful if you make changes to Set A or B.

Search and Filter Tools

You can search, in either direction, for specific content in any of the three columns (Name, Status, or Type). Use the drop-down menu to specify in which column to search. Type a string into the search field.

Table 2.4 describes the buttons in the Comparison Results panel.

Table 2.4 Search and Filter options shown in the Model Comparison Tool

Button	Text	Meaning
	Next Match	Select the next item that matches the search criteria.
	Previous Match	Select the previous item that matches the search criteria
	Search Column	Choose in which column you wish to search.
	Search Content	Type in the search criteria. Use the arrow menu to select previously entered items.
	Ignore Case	Check the box to ignore case when searching. When unchecked, case is considered.
	Filter...	Open a dialog where filters can be applied. This opens the Comparison Tool Display Filters dialog; see "Filter Settings" on page 30
	Cancel	Remove all filters.
	Filter Status	Text is shown noting that there are filters applied and the number of items shown.

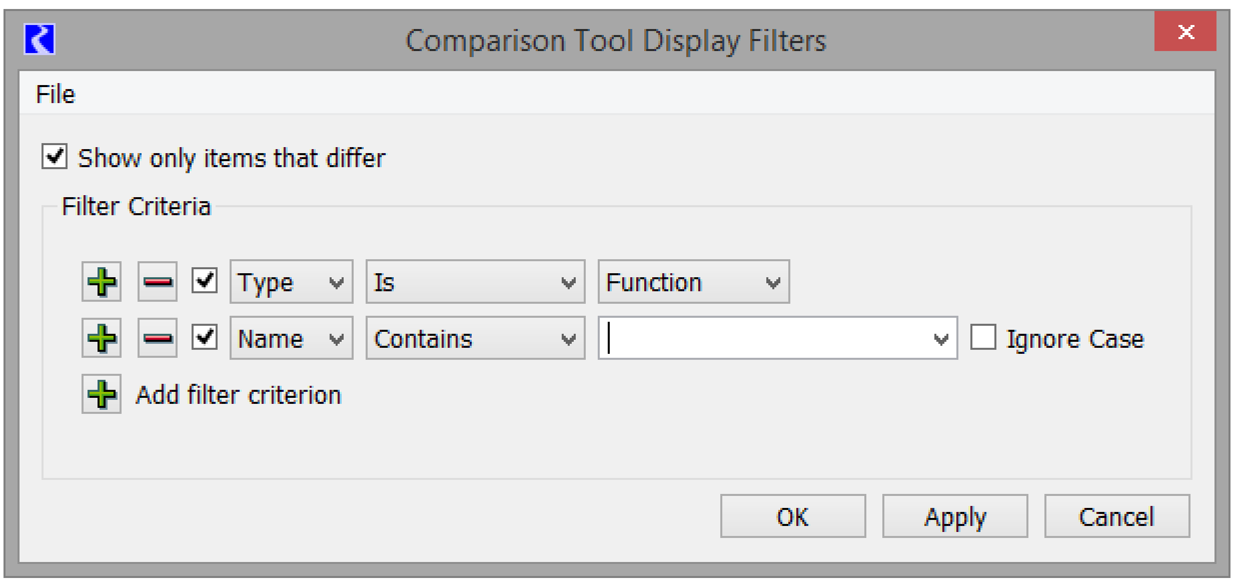
Filter Settings

When displaying results for a large set, there can be many differences. Use the **Filter** button to set up filters to only see desired items. As shown in Figure 2.11, the following types of filters can be applied:

- Show only items that differ (selected by default)
- Additional filters that can be configured. Add filter criteria using the button and remove using the button. For each filter criterion, options include
  - **Name**: Filter on the Name column to only show results containing the specified text. Choose to Ignore Case if desired.
  - **Status**: Filter on the Status. The menu shows the status values listed in Table 2.4.
  - **Type**: Filter on the Type column. The drop-down menu shows the Types that are available in the results.
  - **Value**: Filter on the values in the property panel.

When a filter is applied, the status in the Search and Filters status is updated. To remove a filter, select the red **X** in the RPL Set Comparison Tool.

Figure 2.11 Comparison Tool Display Filters



Custom Filter Sets

One RPL set comparison filter set can be saved in the model file as a “custom set”. This is useful when the same filter criteria are used frequently with a given model. These custom filter sets are accessed from the File menu:

- **Load Custom Filter Set**: updates the Comparison Tool Display Filters dialog with the filter set previously saved with the currently loaded model.
- **Save Current Filters as Custom Filter Set**: applies the current settings in the filters dialog as the new custom filter set in the currently loaded model.

Tip: There is one custom filter set for the Model Comparison Tool and one for the RPL Set Comparison tool.

You can also export/import filter sets as described in Export/Import Filter Sets, page 31.

Clearing Filters

Use the **File** and then **Clear Filter Set** menu to reset all filters in the Comparison Tool Display Filters dialog back to default values.

Export/Import Filter Sets

Filter sets can also be exported to and imported from separate files. This is useful when different sets of criteria are applicable in different circumstances or when a single set of criteria is useful across multiple models. Filter sets are exported and saved as an XML file.

Use the **File** and then **Export Filter Set** menu and then specify an XML file name/path to use. The current filters in the Comparison Tool Display Filters dialog are saved to the file.

Use the **File** and then **Import Filter Set** menu to import a previously exported set.

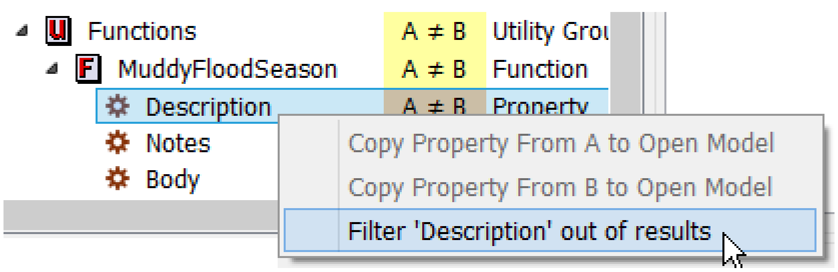
Note: Neither option has any impact on the custom filter sets saved in the model file.

Tip: Exported Filter Sets are specific to either the Model Comparison Tool or the RPL Set Comparison tool and are not interchangeable.

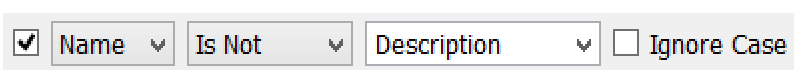
Quick Filters

Quickly filter an item out of the results tree using the right-click context menu on the results tree. For example, in Figure 2.12, you don’t want to see any descriptions in the results, so right-click and choose **Filter ‘Description’ out of results’**.

Figure 2.12 Screen shot of right-click context menu to filter item from results



This action creates a filter in one of the available filters in the Comparison Tool Display Filters:



For objects, you can quickly filter out objects of that name, all objects, or objects of that type (Reservoir, Reach, and so on)

Use the filter dialog, described in “Filter Settings” on page 30, to remove or edit the filter.

Comparison Results Panel

The **Comparison Results** panel shows a hierarchical view of the results of the comparison. The hierarchy is based on the structures of the two set. Columns include the **Name**, **Priority/Index**, **Status** and **Type**. The Set, Groups, and Rules/Goals/Methods show their particular icon. The **Status** column shows the results of the comparison. Table 2.5 lists the possible status and their meanings.

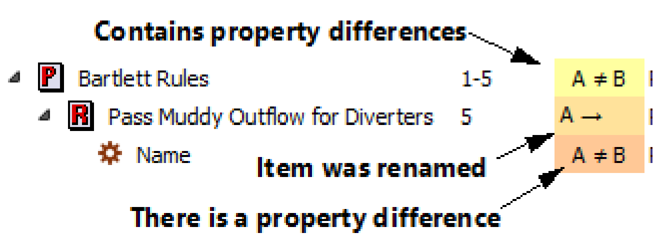
Table 2.5

Status	Meaning
A = B	The item is identical in Sets A and B.
A ≠ B	The item is different in Sets A and B.
A	The item exists in set A but not in Set B.
B	The item exists in Set B but not in Set A.
A ->	The item exists here in Set A and elsewhere in Set B and/or the item has been renamed.
<- B	The item exists here is Set B and elsewhere in Set A and/or the item has been renamed.

The **Type** column lists the type of that row, either a Set, Group, Rule, or Property. The comparison tool ultimately compares Properties of the items, not the items themselves. In the **Status** column, the color indicates the following meaning, This is also shown in Figure 2.13:

- Yellow indicates that the item *contains* properties and that within the item there are differences in properties.
- A light orange shading indicates the item's Name property has changed; that is, the item has been renamed. When an item has been renamed (not copied/pasted or moved), the item will show up twice in the list, once with the name used in set A (the status shows "A ->") and once with the name used in B (the status shows "<- B").
- The darker orange color indicates that the value of the property is different.

Figure 2.13 Color legend for the Status column



Double-click a row in the **Comparison Results** to open the RPL dialogs for that item.

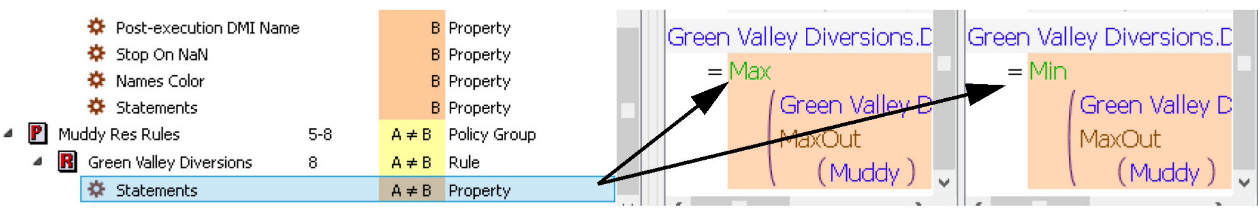
> **Note:** If the set was loaded from a RPL set file or another model file, the dialogs cannot be opened. If both sets are opened in the model file, then double-clicking a row will open two dialogs, one for each set.

> **Note:** The **RPL Set Comparison Tool** compares internal copies of the specified sets. If you make changes to one of the chosen sets, it is not immediately reflected in the comparison tool. Use the **Recompute Differences** button  to update the status in the tool.

Selected Property

The **Selected Property** panel shows the values for the selected row. If a yellow row (Rule, Set, Group, etc) is selected, a note is shown that "No Property Selected". Only properties are shown in these views. When an orange **Property** row is selected, the panel shows the values for that property. The information shown is based on the property type and can include text, colors, true/false, and RPL expressions. When a statement, function body, or execution constraint is selected, the RPL expression is shown for both sets A and B. For example, in Figure 2.14, a statement is selected and the RPL statements are shown. You can see that Set A has a Max function while Set B has a Min function. The total number of differences found is listed above each Selected Property panel. Multiple differences are highlighted whenever possible.

Figure 2.14



Following are known limitations related to the highlighting of differences:

- When two statements or expression differ only by comment, the tool highlights the entire statement/expression, not just the comment. Ideally the tool would highlight the individual differences in the comment text.
- The tool does not highlight differences in symbol declarations (E.g. NUMERIC num), either in the type or name, but the entire expression is highlighted
- Adding, deleting, or moving a statement within a block (or an item in a list expression) will usually lead the tool to overestimate the number of changes (i.e., every statement after the first difference will be considered different). This is not the case for insertions and deletions within textual properties.

Copy Property

Once you've compared sets and identified differences, sometimes you might want to "merge" the two sets together. The RPL Set Comparison Tool supports merging by allowing you to select a property and then chose to replace the value from one set into the value in the opened set. The operation is essentially:

```
Copy the value of the selected property (in A or B) and replace the same property in
the open Set (B or A)
```

Refreshing the comparison tool afterwards will lead to the relevant properties having the comparison status of A = B.

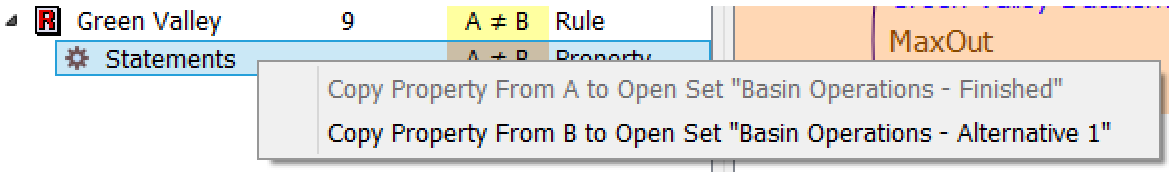
This operation is enabled only when the following conditions are met:

- The set that is "receiving" the value (whose properties would be modified by the operation) is the currently open set (specified by choosing the Currently Open Set option when selecting sets for comparison).
- A supported property is selected in the results tree view. Currently certain items are supported including RPL statements and RPL function bodies. See "Selecting Models to Compare" in *User Interface* for supported items.
- The property's comparison status is A≠ B.

With a property selected, there are two ways to initiate the Copy Property operation:

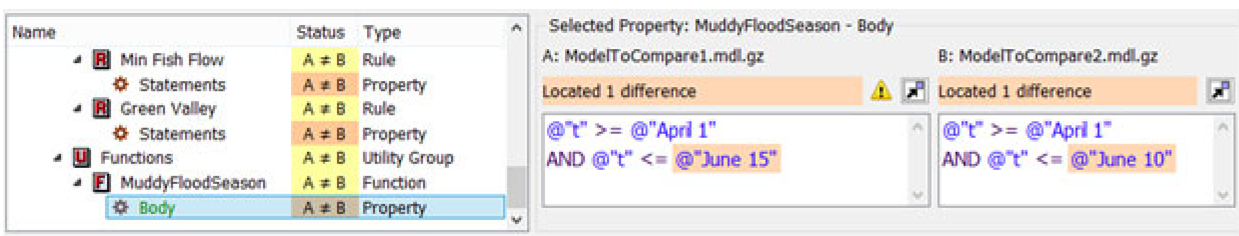
- Use the Copy Property buttons

- Right-click context menu on the property



The value in the opened set is replaced and the dialog is typically opened. **The value in the comparison's window does not changed.** It always shows the result of the last comparison. But the property text is now shown in green and the header shows a note that the property was modified in the opened model. Figure 2.15 shows a sample of the dialog box after a replacement has occurred.

Figure 2.15 Screenshot showing the selected property after a replacement has occurred. Note the green text and warning icon indicating the property has been replace.



> **Warning:** **The comparison tool and the copy property operation uses codes, called Universally Unique Identifiers (UUIDs), to track the items in the set. These were introduced starting in RiverWare 7.5. If you compare two models that diverged before this release, then the items in the two models do not have the same UUIDs and the comparison and copy properties operation may not work.**

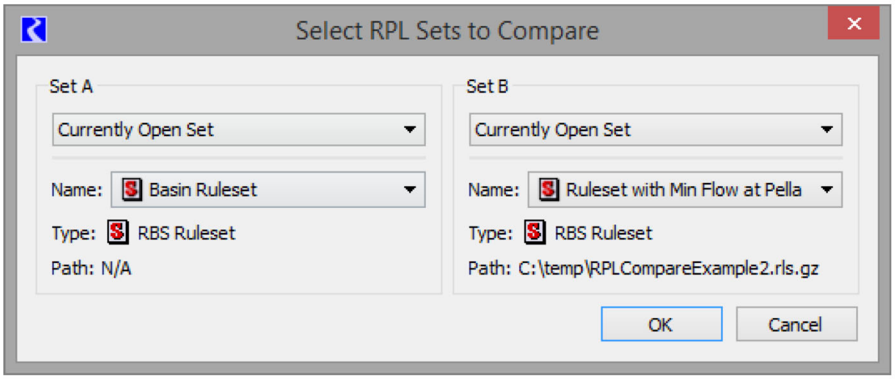
Example of Using the RPL Set Comparison Tool

Following is one use case example where the **RPL Set Comparison Tool** is very useful.

We have been developing a model and ruleset of our basin and need to add some proposed policy changes to the set. I am busy with other things, so I asked my co-worker to help implement the policy in the ruleset. She did this and sent me the new ruleset on her way out the door for vacation. She left the note that "I added the new rule and had to update a couple other things" but didn't describe what she changed. I also have been working on the model and now it is time to see what she did and merge it into my work. I use the following procedure.

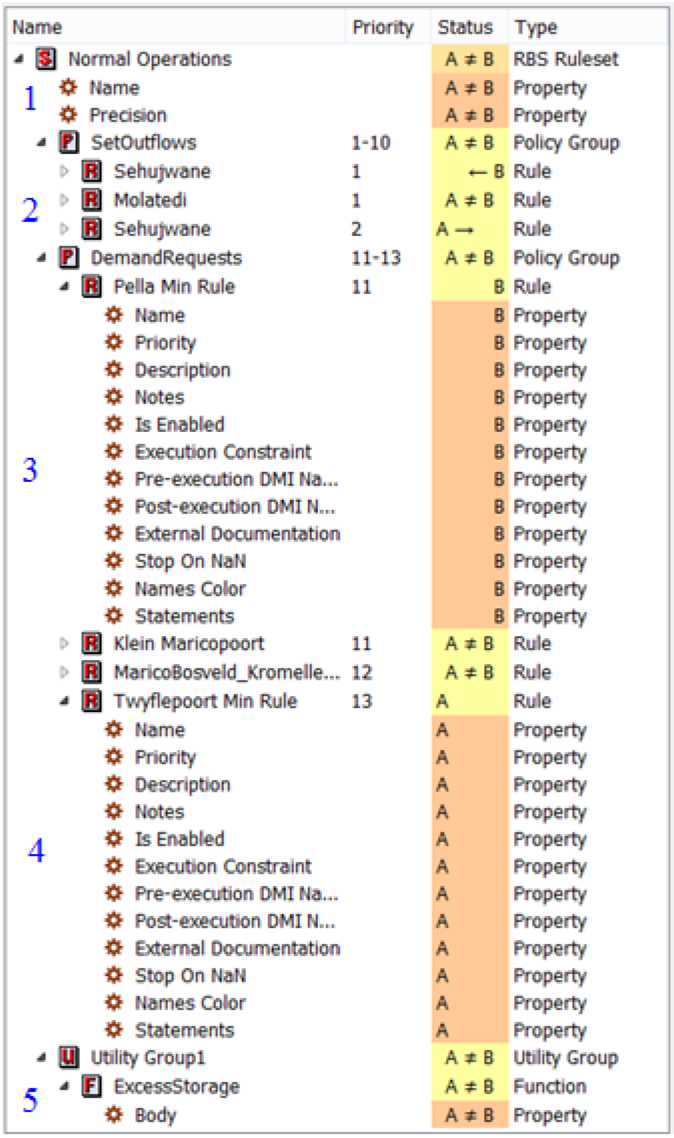
7. I open my model and load my ruleset. In this case, my ruleset is embedded in the model file; it is called **Basin Ruleset**.
8. I open (but don't load) my co-worker's modified ruleset. This is called **Ruleset with Min Flow at Pella**.
9. I use the menu: **Policy**, then **RPL Set Comparison Tool**

10. I then choose my embedded ruleset, **Basin Ruleset**, as Set A and my coworker's modified ruleset as Set B. I then select **OK**.



11. The RPL Set Comparison Tool opens and shows that there are differences between the two sets. I can use the **Next** and **Previous** buttons to step through the differences. Figure 2.16 shows the results.

Figure 2.16 Annotated screenshot of difference tree for sample RPL comparison



12. It looks like there are five main areas of differences. We'll go through these differences in Table 2.6. I select each row to see the property difference in the right hand panel.

Table 2.6

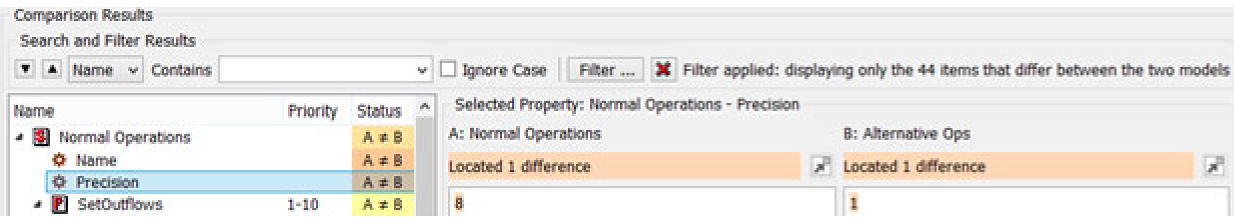
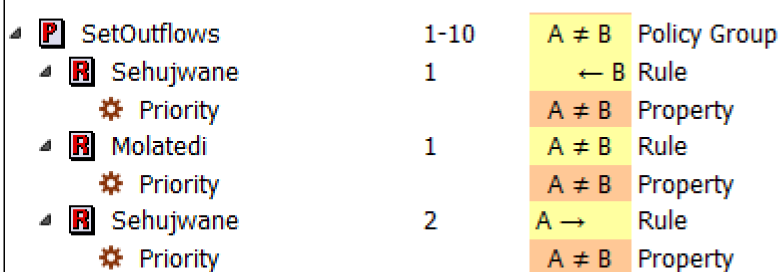
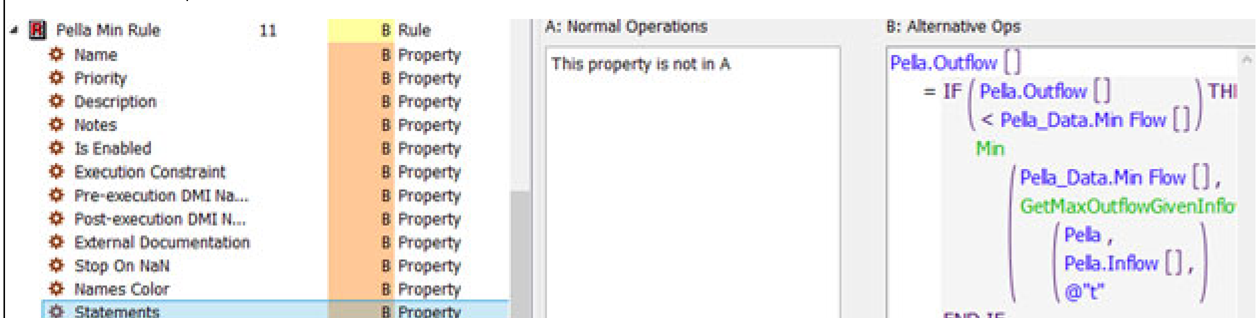
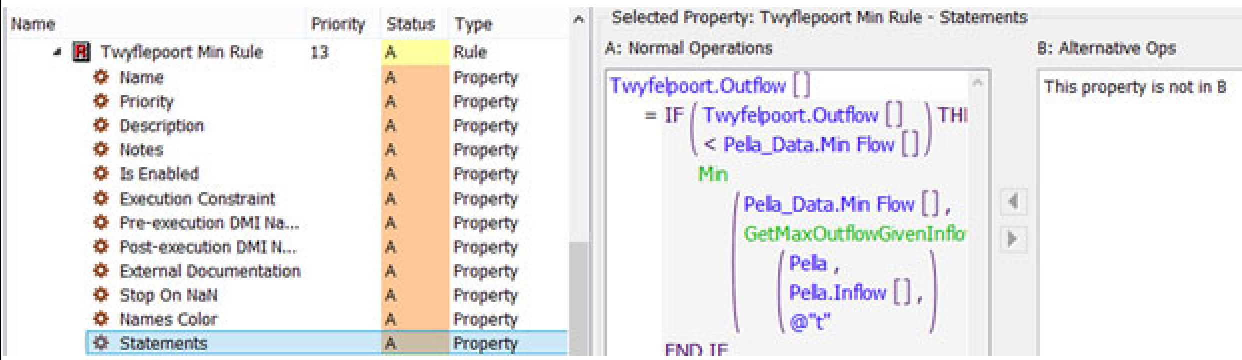
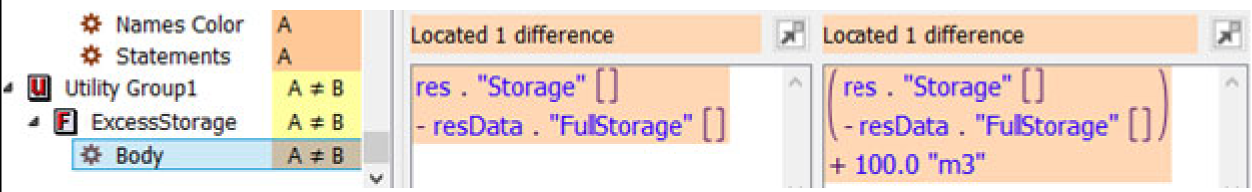
<table>
<tr><th>Difference</th><th>Description</th></tr>
<tr><td>1</td><td>The ruleset name and precision are different.</td></tr>
<tr><td colspan="2">
</td></tr>
<tr><td>2</td><td>Rule 1 and 2 have changed priority.In Set B, Sehujwane is priority 1. In Set A, Sehujwane is priority 2. Since it is strictly the priority order that has changed, there are no property differences, so the right panel doesn’t show any differences.</td></tr>
<tr><td colspan="2">
</td></tr>
<tr><td>3</td><td>Rule 11 is in Set B but not in Set A. The right panel shows the logic in Set B and lists that the property is not in Set A.</td></tr>
<tr><td colspan="2">
</td></tr>
<tr><td>4</td><td>Rule 10 is in Set A but not in Set B.</td></tr>
</table>

Table 2.6

Difference	Description
	
5	The ExcessStorage function has differences. It looks like my co-worker added 100 m3 tolerance for the comparison. It would be nice if she had documented why she did this.
	

13. Now I need to combine or merge her differences into my ruleset. First I'll select the SetOutflows row to open the policy group for both sets.
14. The priority order of the first two rules is correct in my set, so I'll leave it as is, with no changes.
15. I need to get the new rule that my coworker developed. Since the desired rule is in Set B, I open that set and right-click the **Pella Min Rule** and Copy. Then append it to that same group in Set A. I'll rearrange to make it rule 4.
16. The RPL Set Comparison tool hasn't changed even though I am editing the set. The comparison tool uses copies of both sets. I must select the **Recompute** button to get the changes.
17. Now I'll update the function that was changed. I click the ExcessStorage function Body row. I then click the Copy Property from B to Open Set button, to copy the value from Set B to Set A. The RPL Set Comparison tool doesn't change, but the two dialogs open, showing the 100m3 update in Set A.

In this way, I've looked at the differences and decided which ones I want to apply to my set. Although I had to do some manual and some Copy Property merging of the two sets, the **RPL Set Comparison Tool** really helped me to identify where there are differences and then present those differences in an organized fashion.

Exporting and Importing RPL Sets

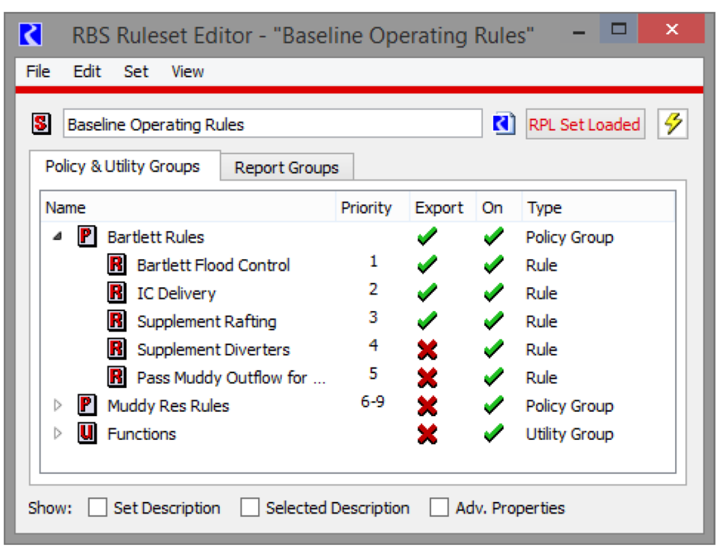
In this section we describe how to export all or a portion of a RPL set to a separate RPL set file, as well as how to import an existing RPL set file. The export and import mechanisms provide a way to incorporate all or some of one policy into another policy, and it is available for all application of RPL within RiverWare, including the following:

- Rule sets. Collections of rules and functions organized into policy and utility groups.
- Optimization goal sets. Collections of optimization goals and functions organized into policy and utility groups.
- The object-level accounting method set. A collection of methods and functions organized into predefined category groups and utility groups.
- The Expression Slot set. A collection of functions organized into utility groups. These functions may be called from an expression defining an expression slot.
- Initialization Rules. Collections of rules and functions organized into policy and utility groups.
- Iterative MRM RPL sets. Collections of rules and functions organized into policy and utility groups.
- Global Function Sets. Collections of functions organized into Global Utility Groups.

There are many situations in which export and import functionality is useful. For example, assume that you are writing a policy which could make use of some functions written as part of an existing policy. To avoid having to rewrite all these functions, you could open both sets and use the Copy/Paste mechanism to copy the functions in question from the existing policy to the new one. However, if you then modify the original functions, you might want to do the copy again to keep the functions identical, but you would have to first remove the copies of the functions before doing this. Furthermore, in the context of the object-level accounting method set it is not possible to have two such sets open simultaneously, so this mechanism would not work. The RPL set export and import functionality provides a simpler and more flexible method to implement sharing of policy elements. Alternatively, you could use Global Utility Groups; see “Global RPL Functions” on page 81 for details.

Export

The RPL set export mechanism allows you to save a portion of a RPL set as a separate RPL set file. The first step is to show the Export column of the RPL set editor. If this column is not currently shown (between the columns labeled Priority and On), then it can be shown by selecting **View**, then **Show Export Column**. Initially no item is selected for export, this is indicated by a red X in the Export column for each item.



Select the Export column of the items that you would like to export. This will convert the red X in that cell to a green check mark.

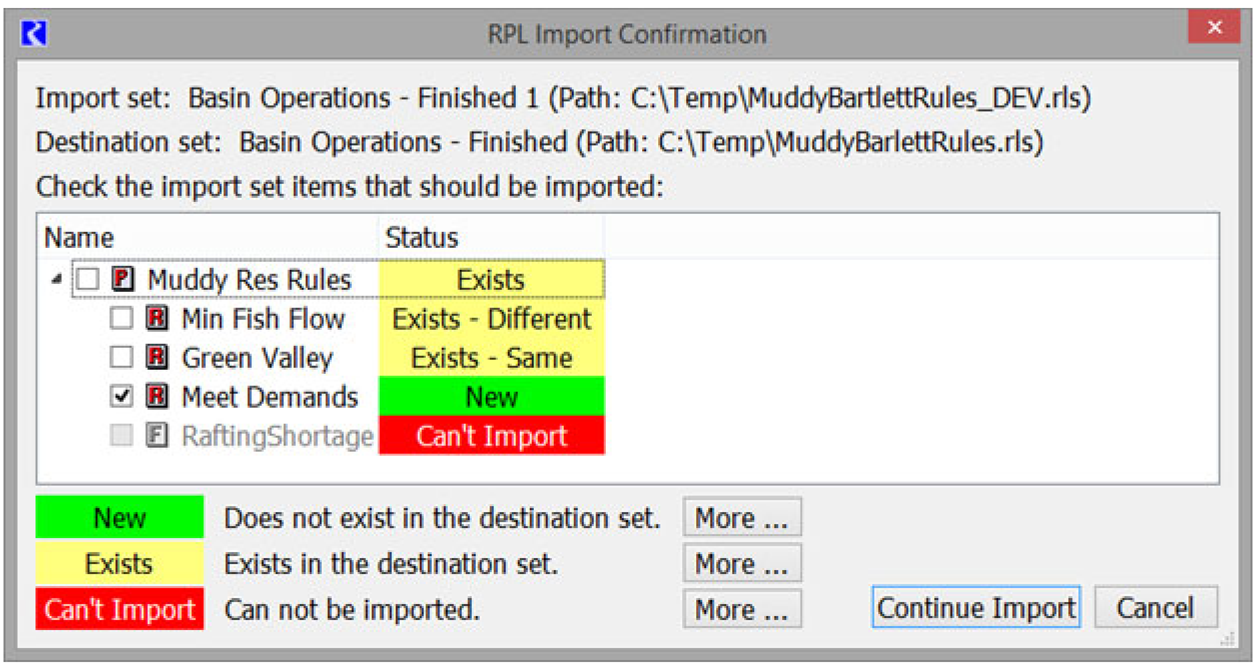
Note: Selecting a block or function for export automatically selects the containing group, and unselecting a group will clear all of its members.

Tip: To select multiple items, right-click and choose **Select Export For All in Group** or **Select Export For All in Set.**

Once the items to be exported have been identified in this way, the export operation is initiated by selecting **File**, then **Export Selected Items.** You will then be presented with a confirmation dialog summarizing the items to be exported If you choose to continue the export operation, a file chooser dialog appears, allowing you to specify the file to which the items will be exported. Once a file name has been selected, the file is created and a RPL set containing only the selected export items is written to that file.

Import

RPL set import allows you to incorporate all or some aspects of an existing RPL set policy into a set that you are editing. To initiate an import, select **File**, then **Import Set**. You will be presented with a file chooser dialog for specifying the name of a RPL set file typically exported as above. Once you have selected a file and selected the **Open** button, you are presented with the RPL Import Confirmation Dialog.



This dialog presents a listing of the import set. Each item is classified as having one of the following statuses:

- **New**: The imported group, block, or function does not exist in the destination set.
- **Existing**: The imported group, block, or function exists in the set in the destination set, i.e. the names match. There are two further classifications for blocks (rules, functions, etc):
  - Exists - Different - A comparison is made and the name and type are the same but the body or one or more properties do not match. In this case, use the RPL Set Comparison Tool to identify the differences.

– Exists - Same - A comparison is made and the name, body and properties of the item match.

  Note: In the comparison of whether the item is the same or different, priority and index are not compared as these are not carried through the export step.

- **Can't Import**: The group, block, or function begin imported can not be imported. There are several reason why this might be the case and a warning is posted to the Diagnostic Output Dialog for each item that can not be imported, explaining the problem. Possibilities include an attempt to import a function into a policy group when there is already an item of the same name in the global name space, e.g., a function of the same name exists in a utility group.

Before continuing with the import operation you should review the items being imported and indicate which items should in fact be imported by selecting the box to the left of its name in the list.

Select **Continue Import** to perform the import.

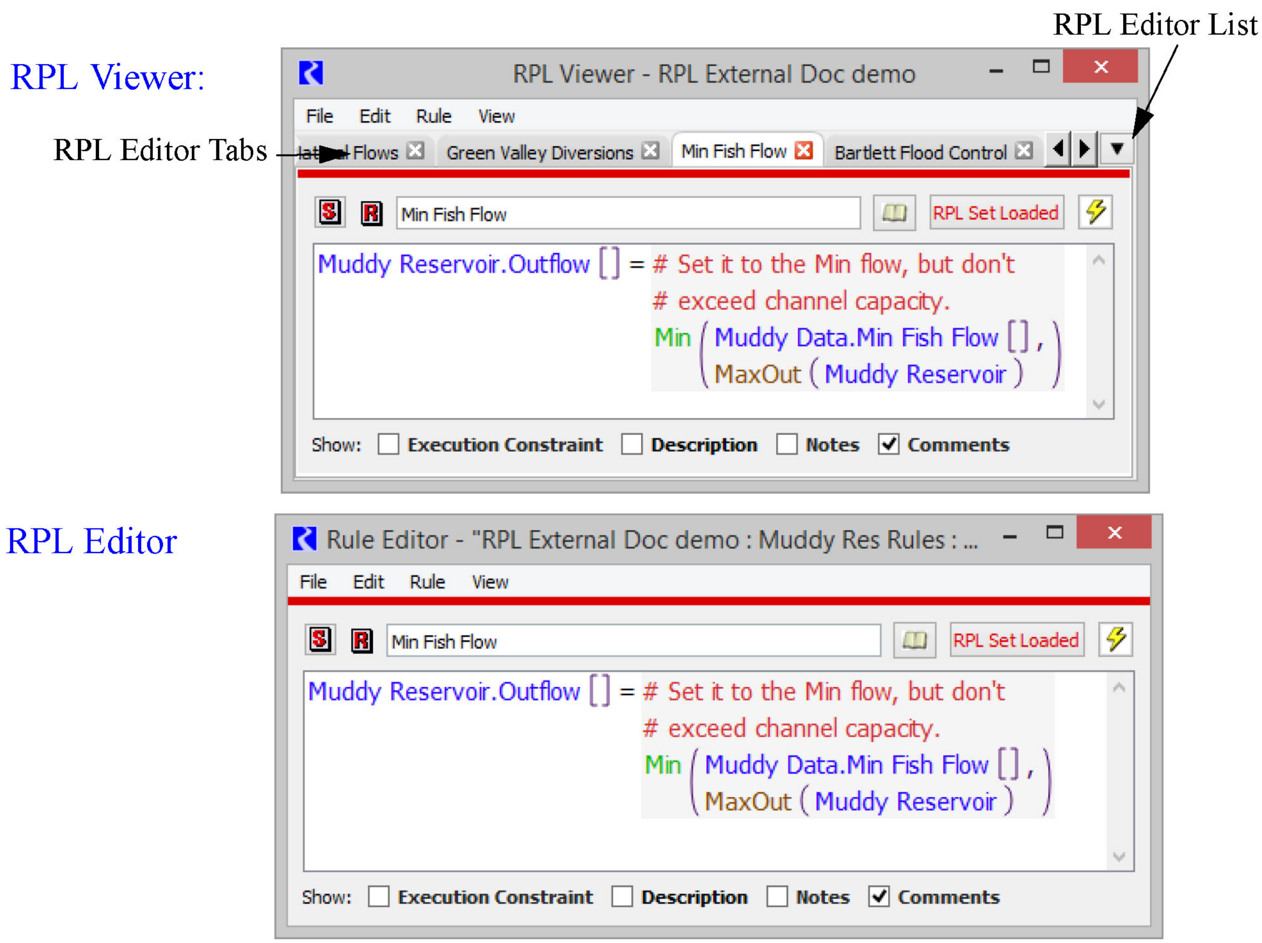
Importing a new group will append it to the list of groups; importing a new function or block into an existing group will append it to the groups of functions which already exist. Importing an already existing group will copy its description to the existing group. Copying an existing block or function will replace the existing block or function with the imported block or function.

Using the RPL Viewer and RPL Editor

There are two ways to view the RPL editor dialogs: either in the RPL Viewer or in the single RPL Editor.

When you open a function, rule, method, goal or any other block, the item will open as a new tab in the RPL Viewer. Opening subsequent RPL Items will add tabs to this RPL Viewer. This allows you to have one dialog where all RPL editors are shown. But, you can undock any item into its own RPL Editor. Figure 2.17 shows the **RPL Viewer** and the **RPL Editor** with the same rule shown. Notice the tabs in the viewer.

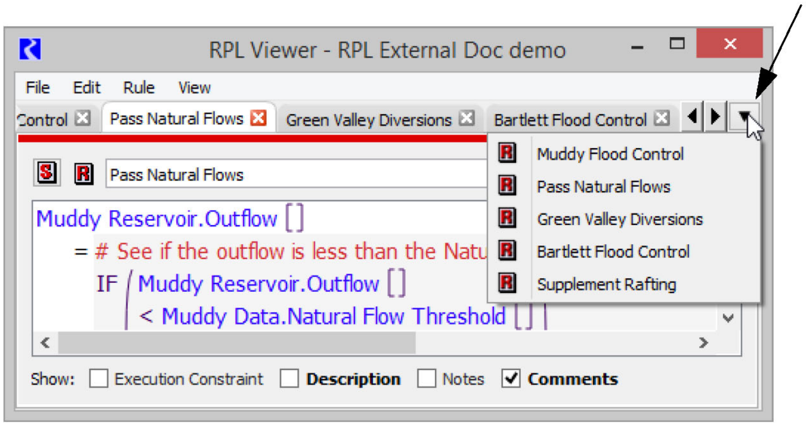
Figure 2.17 Screenshot of the RPL Viewer and RPL Editor



> **Note:** The menu bar options are the same between the two dialog boxes. They are dynamically updated in the viewer based on the tab selected.

The RPL Viewer is the default dialog used to look at a RPL editor. It is convenient as all RPL editors open in the one dialog. Select the tabs to switch between items. Or use the **RPL Editor List** menu on the right of the tabs. This is particularly useful if you have many items in the viewer. Tooltips on the tabs give the full name and priority, or index, where relevant.

RPL Editor List



Accessing the RPL Viewer

The RPL Viewer opens whenever you open a RPL item. You can also access a previously opened RPL Viewer from the workspace RPL Viewer button as shown in Figure 2.18. Select the desired set from the menu.

Figure 2.18 Screenshot of the RPL Viewer button on the RiverWare workspace.

Note: Closing the RPL Viewer removes that RPL Viewer from the button/menu on the workspace. Closing all tabs on a RPL viewer leaves it open but empty.

Arranging Tabs

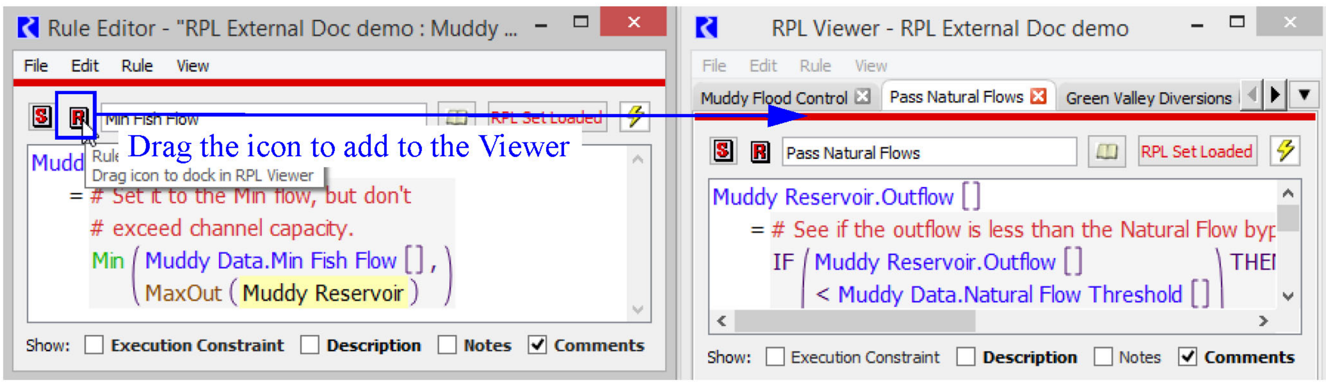
Drag the tabs left and right to rearrange. Use the left and right arrows to scroll to tabs that are not shown.

Undocking RPL Editors

To look at two editors at once, drag the tab off of the viewer to create a RPL Editor dialog for that item. Or use the **File**, then **Undock** menu.

Redocking RPL Editors

To move a RPL Editor dialog back onto the viewer, grab the R, F, G, Icon and drag it anywhere onto the viewer. A new tab will be created. Or use the **File**, then **Dock In RPL Viewer** menu. Also, from the viewer, you can use the **File**, then **Redock All** to move all RPL Editors to the viewer.



Closing a RPL Editor Tab

Use the red **X** on the tab to close a tab that is on the viewer.

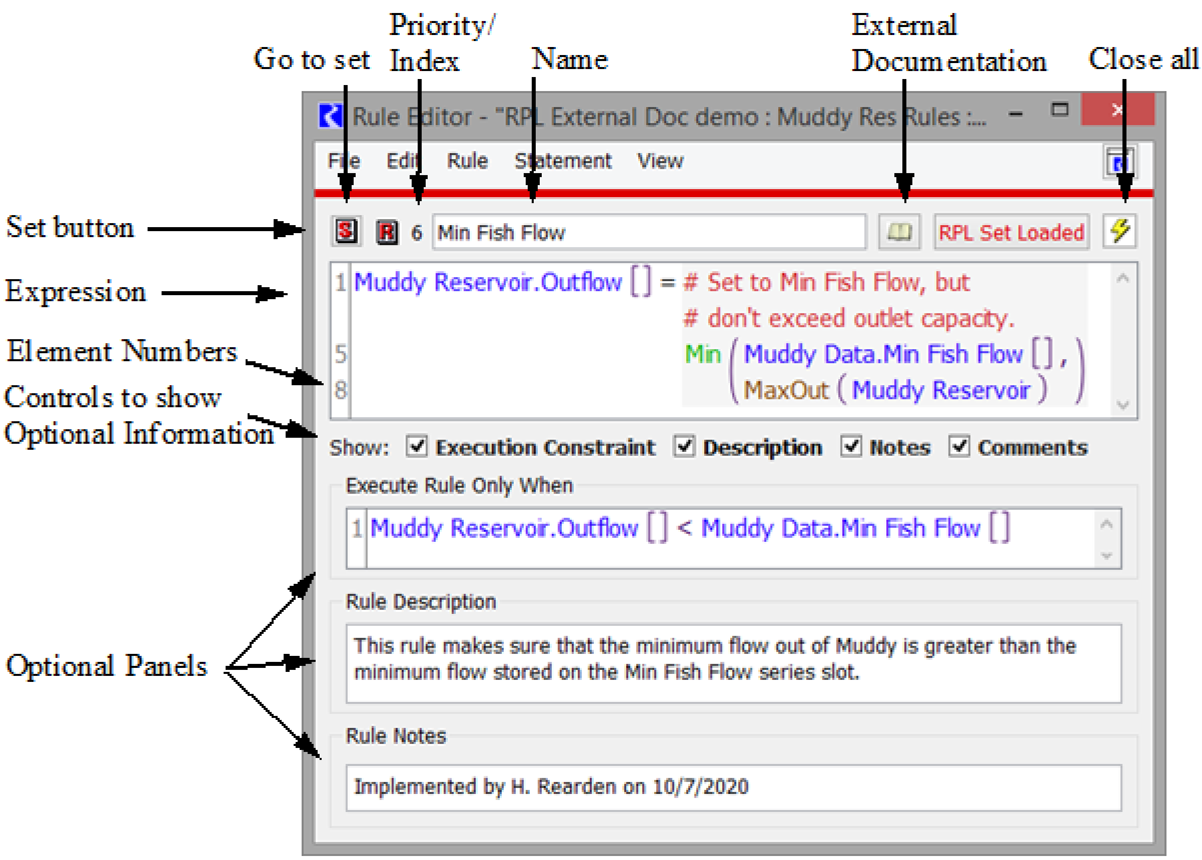
> **Note:** The RPL Viewer is transient; once it is closed, the configuration of items shown and their order is not saved.

Use the **File** and then **Clear Viewer** to remove all items from the viewer. This can be useful if trying to add an empty RPL Viewer to a Window Layout (“Window Layouts” in *User Interface*) or Multi-Window (“Multi-Windows” in *User Interface*).

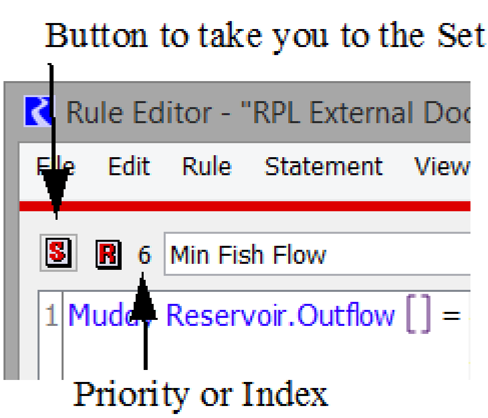
Working With RPL Dialogs

Whether you are looking at the editor or the viewer, the RPL dialogs consist of menus, name and buttons, the expression, and optional additional panels. Figure 2.19 illustrates, and these items are described in this section.

Figure 2.19 Annotated Screenshot of a RPL Editor



- **Type of Set**. The icon indicates the type of set to which this editor belongs.
- **Name**. The name field is where you specify the name.
- **External Documentation**. The button, when configured, opens the external documentation; see “RPL External Documentation” on page 131 for details.
- **Close All**. The button invokes a dialog in which you can close all open RPL dialogs except the top level editor.
- **Set Button**. Icon button allows you to show the containing set for each RPL dialog.



- **Priority / Index:** This area shows the priority or index. The tool tip indicates the value and type. See Table 2.2 for a list of which sets use priorities and which use index.
- **Expression:** This area contains the expression.
- **Element Numbers**: This column shows the first element on that line. See “Element Numbers” on page 93 for details.
- **Optional Information:** Optional information can be shown for various types of RPL dialogs. These are described in the following sections. If there is an entry in these panels (or a non-default value), the text label becomes bold.

Execution Constraint / Execute Block Only When

A block can be set up to execute only when a given condition is true. This functionality can be used to increase performance or it may be necessary to implement your RPL set correctly.

- From a RPL editor, select **View**, then **Show Execution Constraints** or select the **Execution Constraint** toggle.

An area is added to the bottom of the window. By default, the block executes only when **TRUE**. This means that the block will always execute when it comes up on the agenda. You may want this block to execute only when some other condition is true. The **TRUE** expression can be replaced with any logical structure that can be constructed from the palette

> **Note:** If you enter a boolean expression to replace the default TRUE, then clear this expression, an unspecified expression will remain. This unspecified expression will invalidate the rule. To prevent this, if you clear out this boolean, make sure you re-type TRUE to get the default behavior.

Descriptions

A special expanded area of the RPL dialog is used to enter a description of the block. The area is opened and closed by selecting **View**, then **Show Description** in the menu bar, or selecting **Description** at the bottom of the window.

> **Note:** If there is any description entered, the **Description** option will have bold text. Hover over the word Description to see up to 140 characters of the description as a tooltip. Descriptions can be optionally included in Model Reports.

Description can be included in Model Report RPL items (RPL Group, RPL Rule/Goal, RPL Set) with the **Show Descriptions** settings. See “RPL Group” in *Output Utilities and Data Visualization* for details.

Notes

Notes, like Descriptions, can be entered in a panel at the bottom of the dialog. Notes can be used when you have information that you want to enter that doesn’t belong in the **Description** field. For example, development notes or change logs could be entered in the Notes panel.

The area is opened and closed by toggling **View**, then **Show Notes** in the menu bar or selecting the **Notes** toggle at the bottom of the window.

> **Note:** If there is any note entered, the **Notes** toggle will have bold text. Hover over the word Notes to see up to 140 characters of the note as a tool tip.

Notes can be included in Model Report RPL items (RPL Group, RPL Rule/Goal, RPL Set) with the **Show Notes** settings. See "RPL Group" in *Output Utilities and Data Visualization* for details.

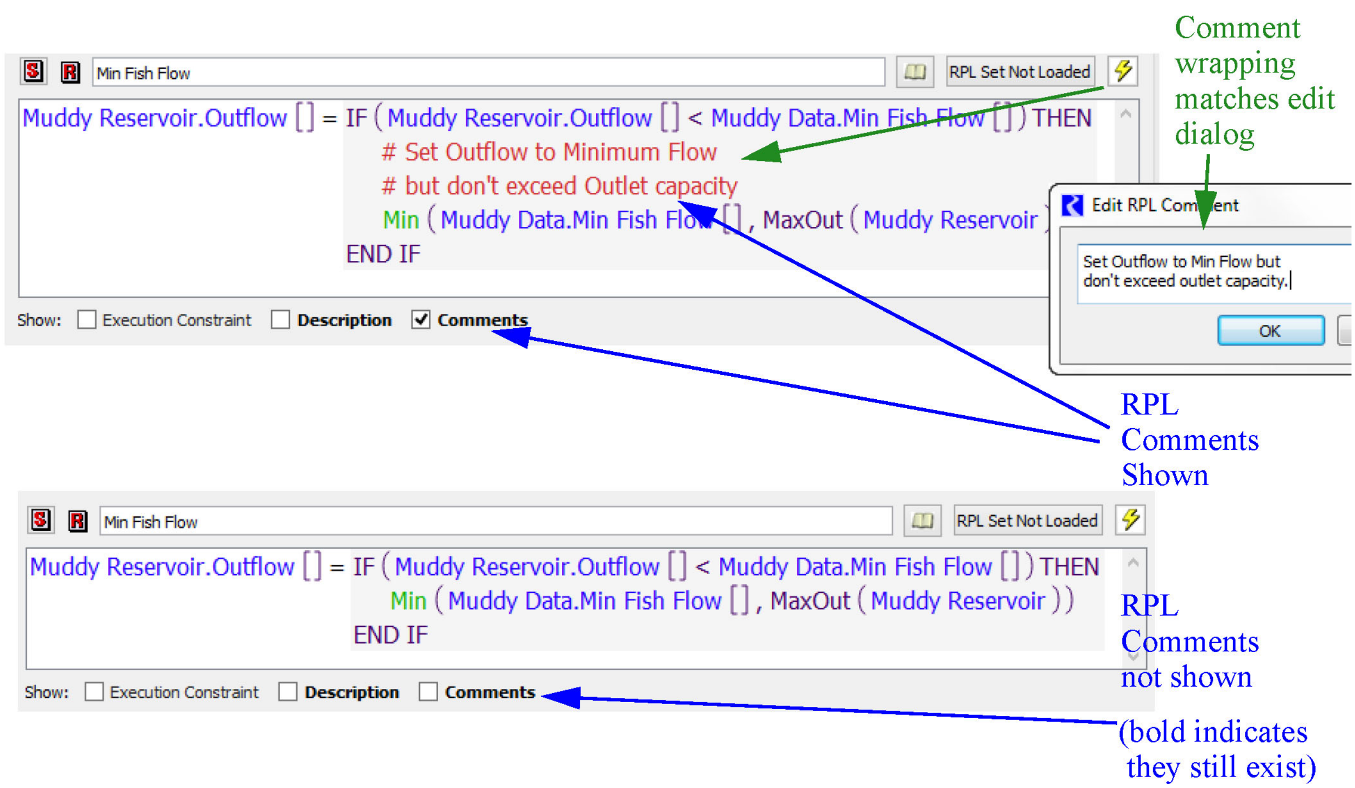
Comments

Inline Comments are added from the palette using the **Add Comment** button. A separate dialog is opened that allows you to type in a comment. In the RPL editor, the comment is displayed with # characters on the left, lines are wrapped as they were in the comment editor dialog. Double-clicking the comment reopens the edit dialog.

Inline RPL comments can be optionally shown/hidden using the **View**, then **Show Comments** menu or the **Comments** toggle.

> **Note:** If there are any comments defined, the **Comments** toggle will have bold text. Also, **Comments** are shown by default; if you do not want to see them, you must explicitly hide them. Figure 2.20 illustrates.

Figure 2.20



Executing DMIs From Blocks

RPL blocks can be used to execute DMIs or DMI groups. See "DMI User Interface" in *Data Management Interface (DMI)* for details.

To add a DMI, Select **Rule/Method**, then **Add Pre-execution DMI** or **Rule/Method**, then **Add Post-execution DMI**. Then, select the name of a DMI or DMI group that is defined in the model.

To remove a DMI, use the **Rule/Method**, then **Remove Pre/Post-execution DMI** menu.

A pre-execution DMI is executed as the first step of the execution, a post-execution DMI is executed as the last step of execution. Values imported by the pre-execution DMI are available for use by the statements. The order of execution for a rule is as follows:

1. If the Execution Constraint evaluates to TRUE, continue. Otherwise terminate the rule early.
2. Run the Pre-execution DMI.
3. Evaluate the body of the rule.
4. If the rule evaluation is successful, run the Post-execution DMI. If the rule is empty, terminates early, or is ineffective, the Post-execution DMI is not run.

To prevent unexpected solving of objects and/or conflicts between values, input DMIs executed from RPL blocks can only import values to *unlinked series slots on data objects*. Furthermore, values imported from a DMI executed from a RPL block are given the following flags:

- Control File-Executable DMIs. Imported values are given the INPUT (I) flag (priority zero). In long model runs, consider using the symbolic start_timestep and end_timestep control file keywords to limit the amount of data imported/exported.
- Database DMIs. Imported values are given the OUTPUT (O) flag and the priority of the rule executing the DMI.

An example application of this feature is executing an external water quality model at each timestep. In this example, the water quality model might take as input RiverWare values which reflect the current state of the system and return a recommended reservoir release value. At each timestep the block would first execute an output DMI to export current values from relevant RiverWare slot(s), e.g., inflow on a reach. An input DMI then runs the external water quality model (which reads the data from the output DMI) and then imports the resulting recommended reservoir release values.

> **Note:** You can turn on Diagnostics to see when rules and Rule DMIs execute. Use the **Rule Execution** and **DMI**, then **Rule DMI** diagnostic categories; see “Rulebased Simulation Diagnostic Groups” in *Debugging and Analysis* for details.

Stop On NaN

Typically, when a RPL block accesses a slot but the slot is NaN, the block terminates early but the run continues. Under certain conditions, you may wish to specify that a block does not terminate early but instead stops the run when a NaN is encountered. This could be useful for data checking rules; if the referenced slot value has not been specified, then you wish to stop the run and fix the data error.

To specify that a block should Stop On NaN, use the **Block**, then **Stop On NaN** menu. When enabled, a check mark is added next to the menu. Any time a slot expression accesses a NaN, the run will stop and a diagnostic will be posted that explains which block referenced a NaN.

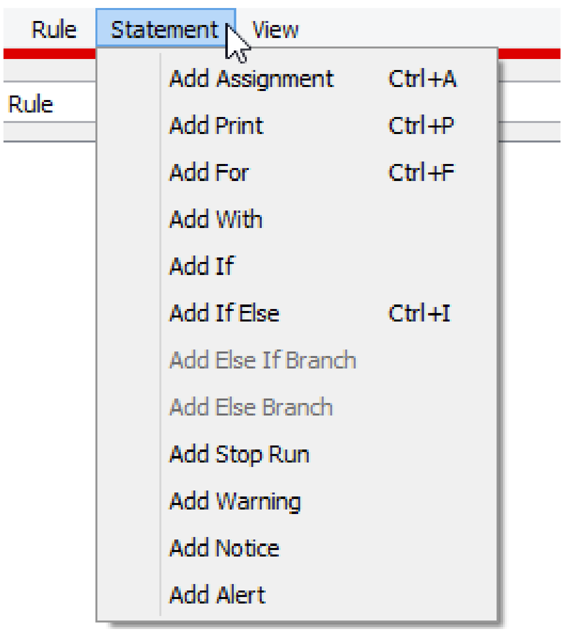
Note: In the context of initialization rules the run is not stopped immediately when an invalid value is encountered, but rather it is stopped only after all of the rules have executed. This allows RiverWare to report on all missing data with a single run.

RPL Statements

Blocks are the upper-level construct of a RPL set. For example, rules and accounting methods are blocks. Blocks are made up of statements which are different structures depending on the desired result. See “About RPL Sets” on page 9 for details

In the RPL set editor, open a block by double-clicking or right-clicking its icon. This will bring up the <block> Editor dialog**.** Rename the block by typing in a new name in the **Name** text field.

Blocks are constructed from statements at the top level, and statements are constructed from expressions. The following statements are available for RBS, Initialization and MRM rules and accounting methods. They can be selected from the **Statement** menu in the RPL Viewer or the individual rule or accounting method dialog**.**



Assignment

An assignment statement assigns to the slot on the left-hand side (LHS) of the equality the numeric value evaluated on the right-hand side (RHS) of the equality. The basic assignment is:

```
<numeric expr> = <numeric expr>
```

where the LHS <numeric expr> is a slot and the RHS <numeric expr> is any expression which evaluates to a value in the unit type of the LHS slot.

Note: RBS rules, MRM rules, and accounting methods can assign values on *series slots only*. Initialization Rules can assign values on series slots, table slots, scalar slots, and periodic slots; see “How to Use Initialization Rules” on page 78 for details.

For

Iterative loops can be very useful for computations and multiple assignments. For loops are available to make multiple slot assignments from similar logic and calculations. An index variable is assigned a new value for each iteration of the loop. The inside of the loop is one or more regular assignment statements which should use the index variable to perform a different assignments for each iteration of the loop. This variable may be used in both the LHS slot assignment and the RHS evaluation to affect slightly different behavior within each pass. The default For loop is:

```
FOR (NUMERIC index IN <list expr>) DO
  <numeric expr> = <numeric expr>
END FOR
```

where the number of loops/assignments is determined by the number of elements in the **<list expr>**, the **NUMERIC** label indicates the expression data type of the elements in the **<list expr>**, and the **index** is the variable name which will take on the value of each element for use inside the loop. All of these parts of the for statement may be modified.

Note: There is also a For expression on the palette; see “Conditional and Iterative Operations Buttons” on page 119 for details.

With

A With statement evaluates an expression and sets the result to a local variable with the given name and type. It then evaluates the contained statements, which may reference the variable. The default WITH syntax is

```
WITH (NUMERIC val = <numeric expr>) DO
  <numeric expr> = <numeric expr>
END WITH
```

Note: There is also a WITH expression on the palette; see “Conditional and Iterative Operations Buttons” on page 119 for details.

If

An If statement makes a statement conditional on a boolean expression without an ELSE.

```
IF(<boolean expr>) THEN
  <statement>
END IF
```

Note: There is also an IF expression on the palette; see “Conditional and Iterative Operations Buttons” on page 119 for details.

If Else

An If Else statement makes a statement conditional on a boolean expression with an ELSE.

```
IF(<boolean expr>) THEN
  <statement
```

```
ELSE
  <statement>
END IF
```

Note: There is also a IF ELSE expression on the palette; see “Conditional and Iterative Operations Buttons” on page 119 for details.

Else If Branch

Although not a statement by itself, you can add one or more ELSE IF branches to an IF or IF ELSE statement. The ELSE IF is available when the boolean condition or a consequence statement is selected (except when an ELSE consequence statement is selected). An ELSE IF branch is added after the selected branch.

```
IF(<boolean expr>) THEN
  <statement>
ELSE IF(<boolean expr>) THEN
  <statement>
END IF
```

Note: There is also a ELSE IF expression on the palette; see “Conditional and Iterative Operations Buttons” on page 119 for details.

Else Branch

Although not a statement by itself, you can add a single ELSE branch to an IF Statement or ELSE IF statement. This part of the statement will be evaluated when none of the other IF or ELSE IF boolean conditions are true.

```
IF(<boolean expr>) THEN
  <statement
ELSE
  <statement>
END IF
```

Note: There is also a ELSE expression on the palette; see “Conditional and Iterative Operations Buttons” on page 119 for details.

Print

A Print statement evaluates its expression and formats the result into a message. The blue message is displayed in the Diagnostics Output Window only when the Print Statements diagnostics group is enabled.

```
Print <expr>
```

where the `<expr>` is any expression or concatenated expressions which can be fully evaluated and represented as a string. See “Print Statements” in *Debugging and Analysis* for details.

Notice

A Notice statement posts a purple message to the Diagnostics Output Window regardless of diagnostics settings.

```
NOTICE <expr>
```

where the `<expr>` is any expression or concatenated expressions which can be fully evaluated and represented as a string. See “Notice, Warning, and Alert Messages” in *Debugging and Analysis* for details.

There is also a Notice expression on the palette; see “Miscellaneous Buttons” on page 125 for details.

Notice messages can be optionally configured in the Diagnostic Manager to automatically open and raise the Diagnostics Output Window to the front when they are posted. See “Raise Diagnostics Output Window” in *Debugging and Analysis* for details.

Warning

A Warning statement posts a message with a pink background to the Diagnostics Output Window regardless of diagnostics settings.

```
WARNING <expr>
```

where the `<expr>` is any expression or concatenated expressions which can be fully evaluated and represented as a string. See “Notice, Warning, and Alert Messages” in *Debugging and Analysis* for details.

There is also a Warning expression on the palette; see “Miscellaneous Buttons” on page 125 for details.

Warning messages can be optionally configured in the Diagnostic Manager to automatically open and raise the Diagnostics Output Window to the front when they are posted. See “Raise Diagnostics Output Window” in *Debugging and Analysis* for details.

Alert

An Alert statement posts a message with an orange background to the Diagnostics Output Window regardless of diagnostics settings.

```
ALERT <expr>
```

where the `<expr>` is any expression or concatenated expressions which can be fully evaluated and represented as a string. See “Notice, Warning, and Alert Messages” in *Debugging and Analysis* for details.

> **Note:** The Notice, Warning and Alert statements all have the same behavior. They simply post messages with different colors. This allows you to define messages with three different levels of severity.

There is also an Alert expression on the palette; see “Miscellaneous Buttons” on page 125 for details.

Alert messages can be optionally configured in the Diagnostic Manager to automatically open and raise the Diagnostics Output Window to the front when they are posted. See “Raise Diagnostics Output Window” in *Debugging and Analysis* for details.

Stop Run

A Stop Run statement aborts the run and posts the provided message as a diagnostic error message in red text. When executed from within an iterative MRM rule, this statement aborts the MRM run.

```
STOP RUN <expr>
```

where the `<expr>` is any expression or concatenated expressions which can be fully evaluated and represented as a string. See “Stop Run Statements” in *Debugging and Analysis* for details.

A message from a Stop Run statement will automatically open the Diagnostics Output Window and raise it to the front.

Execute Script

Available for Iterative MRM Rules only, an Execute Script statement executes the named script.

> **Caution:** This statement should be used with *extreme caution* as scripts can modify many parts of your model.

```
EXECUTE SCRIPT <string expr>
```

where `<string expr>` is the name of the script to be executed. See “Script Management” in *Automation Tools* for details about scripts.

Set Column

For a periodic or table slot, set the column label value. This statement is only available within Initialization rules, see “Initialization Rules Set” on page 76 for more information. Next, the statement can be used to set:

- Periodic slots numeric column headers. See “Numeric Headers” in *User Interface* for more information.
- Table slots numeric column headers. See “Table Slots” in *User Interface* for more information
- Periodic and table slot column labels.

The syntax is:

```
SET <slot expr> COLUMN <numeric expr> TO <expr>
```

The first <slot> is the periodic slot to set. The first <numeric> is the zero-based index of the column to set. The second <numeric> is the value to set. If you are setting a NUMERIC heading, the units must match the units of the numeric column headers. For example:

```
SET DataObject.PeriodicSlot COLUMN 2.0 TO 1000 “cfs”
```

When this statement sets values, the columns in the slot may rearrange as the column headers are always increasing.

The last <expr> can also be a STRING expression to modify the column label.

```
SET DataObject.Tablelot COLUMN 2.0 TO “Minimum”
```

Like other statements, the setting of the values are done at the end of the rule after all statements have been evaluated. All references to column indices by the Set Column statement should reference the periodic slot before the rule executed. Use of a FOR statement to loop over a set of columns is allowed, but you cannot use intermediate results in later parts of the loop; always refer to the slot as it exists at the start of the rule.

Set Row

For a periodic slot, set a row header datetime value. See “Periodic Slots” in *User Interface* for more information. This statement is only available within Initialization rules, See “Initialization Rules Set” on page 76 for more information.

The syntax is:

```
SET <slot expr> ROW <expr> TO <datetime expr>
```

The <slot expr> is the periodic slot to set. The first <expr> is either the zero-based index of the row to set or the datetime of the row to set. The <datetime expr> is the value to set. For example:

```
SET DataObject.PeriodicSlot ROW 2.0 TO @“January 21”
SET DataObject.PeriodicSlot ROW @“23:00 January 23” TO @“24:00 January 30”
```

The datetimes are typically partially specified datetimes and should match the components used in the periodic slot.

> **Note:** RPL uses 24:00 as the end of a day while irregular periodic slots use 0:00 on the start of the next day. For example 24:00 April 30 in RPL is the same as 0:00 May 1 on a periodic slot row. When writing RPL logic, continue to use 24:00 notation but realize this will map to 0:00 on irregular periodic slots.

When this statement sets values, the rows in the slot may rearrange as the row headers are always increasing.

Like other statements, the setting of the values are done at the end of the rule after all statements have been evaluated. All references to row indices by the Set Row statement should reference the periodic slot before the rule executed. Use a FOR statement to loop over a set of rows is allowed, but you cannot use intermediate results in later parts of the loop; always refer to the slot as it exists at the start of the rule.

Editing a RPL Expression

Expressions in the RPL editor are displayed as <expr>, <numeric expr>, <boolean expr>, <list expr>, <string expr>, <object expr>, <slot expr> or <datetime expr>. The type of the expression indicates the valid values that can go in that expression. See “Expression Data Types” on page 95 for details on each data type.

There are two ways to enter data into an unspecified expression, typing and using the palette. Any expression can by typed by first double-clicking then typing in the expression. There are many problems with this approach. First, you must know the exact syntax to be typed. Second, you must type all strings perfectly as typos in strings may not be caught on validation. A better approach is to use the palette to build expressions. You only need to type when they are specifying the inner most expression like exact numbers (100 cfs), boolean (true, false), datetimes (@“t”), or strings (“Entering Runoff season”). Following is a description of using the palette.

Using the Palette

See “RPL Palette” on page 105 for descriptions of the Palette buttons. Each of the buttons in the RPL Palette represents an expression. These expressions are convenient operations which evaluate to one of the expression data types just mentioned. Buttons on the RPL Palette are enabled and disabled dynamically. When an expression is highlighted in the Editor, the Palette buttons that satisfy the expected data type are enabled. Thus, you create expressions in the following manner: select an unspecified expression, select a palette button or function. Select another unspecified expression and select a palette button or function. Repeat this process until the expression is created. Then, if necessary, double-click any remaining unspecified expressions that cannot be created from the palette (like entering numbers) and type in the value.

Entering Values

To replace an unspecified expression with a literal value (like 10 “cfs” or TRUE) double-click the expression and type in the value. For NUMERIC data types, you may also enter a unit. If the unit has a “-” or “/” in it, you must include the quotes when typing the unit, like “acre-feet”. Without these special characters, you can type the units and no quotes. Here are some examples of correctly specified units: 10 cms, 100 “acre-ft”, 50. Illegal entries include 100 acre-ft/day (no quotes) and 100 “cfx” (no unit of that type). See “NUMERIC” on page 96 for details on entering units.

Another illegal entry is 1,000 “acre-ft”. This is because the comma (a thousands separator) is not allowed as an input. Instead, RiverWare gives the option of whether or not to display a comma as a thousands separator for all values throughout the model and RPL set. This option is selected or deselected by selecting the main **Workspace**, then **Show Commas in Numbers** option; see “Show Commas in Numbers” in *User Interface* for details.

> **Note:** After typing a literal value, you do not need to hit **Enter** to commit the change. Pressing Enter or any selection outside of the editor will signal completion of the editing.

See “Units in RPL” on page 126 and particularly “Use of Variable Length Time Units” on page 127 for more information on entering literal RPL values and their units.

Auto-Correct

When typing values, if the value you provide is not a valid replacement for the expression, RiverWare attempts to coerce the specified string into one that is valid. This auto-correction process is guided by the types which can legally replace the existing expression. RiverWare tries a series of variations on input, where each variation is an attempt to coerce the input into a different value type. Types are considered in the following order: DATETIME, OBJECT, SLOT, SRING, and LIST. If a valid auto-correction is found, it is used to replace the existing expression; if no valid auto-correction is found, an error notification is presented describing the problem with the input.

For example, consider the statement.

```
WITH (STRING val = <string expr>) DO
  PRINT "value: " CONCAT <expr>
END WITH
```

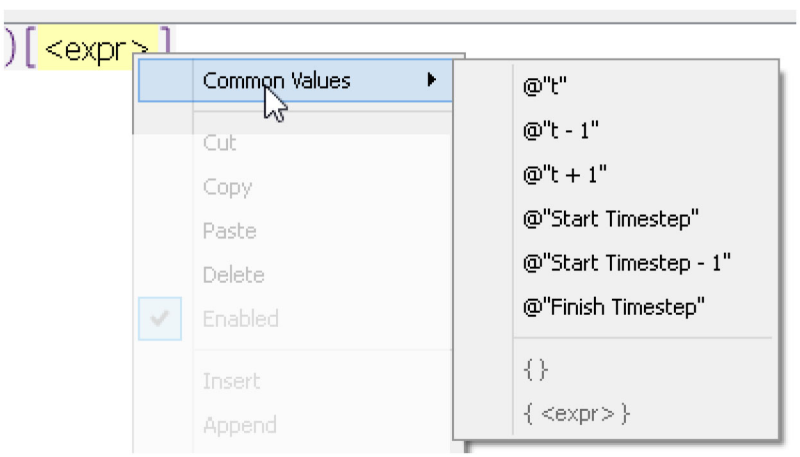
If the text t + 1 is entered as the variable value in the WITH expression, it is interpreted as a STRING "t + 1" because that is the only legal type for that expression; whereas the same text entered as the right-hand side of the

CONCAT expression is interpreted as the DATETIME @"t + 1", because all types are valid in that location and a DATETIME conversion is considered first.

Common Values

Common values are also available by right-clicking the selected expression and choosing the **Common Values** menu. These include common DATETIME values and list expressions.

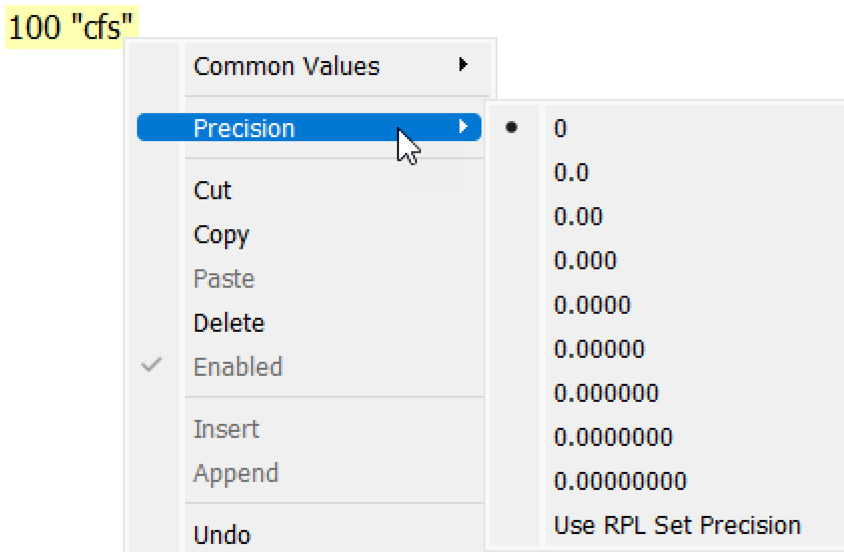
- @“t” - Current Timestep
- @“t-1” - Previous Timestep
- @“t+1” - Next Timestep
- @“Start Timestep” - First timestep in the run period
- @“Start Timestep - 1” - Initial timestep
- @“Finish Timestep” - Final Timestep in the run period
- { } - Empty list expression
- { <expr> } - List containing one empty expression.



Precision of Numeric Values

The default Set Precision is specified on the RPL set in the Advanced Properties frame as described in “Show Advanced Properties” on page 21. You can change the precision of any selected individual RPL numeric expression using the right-click **Precision** menu. Select from the nine options or select to use the set’s precision. For example in the screenshot, the selected expression, 100 cfs, has been configured to use 0 precision. To resume using the overall set precision, select the **Use RPL Set Precision** option.

Figure 2.21 Screenshot of Precision context menu.



Undo and Redo

When editing RPL expressions, undo and redo are available as follows:

- **Edit** menu on RPL dialogs
- Right-click context menu
- Keyboard shortcuts
  - Undo is ctrl-z
  - Redo is ctrl-shift-z
- **Expression** menu on Expression slots.

Any action that edits a RPL expressions can be undone or redone, including the following:

- Replacing the selected expression with a new expression by selecting a palette button
- Cutting or deleting the selected expression.
- Pasting an expression in the place of the selected expression.
- Adding or removing a statement from a block.
- Adding or removing a constraint from a function.
- Double-clicking an expression and changing it via an inline editor.

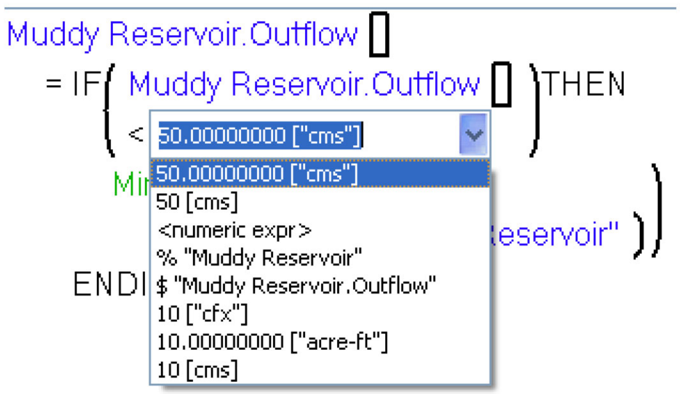
Undo and redo are on a per-dialog basis; the dialog must be selected before the undo/redo operation is performed. Also, the history of changes is preserved if a RPL dialog is closed and re-opened, but is not preserved if the set is closed. On expression slots, the history of changes is lost when the slot is closed.

The number of undos/redos is only limited to take you back to the original expression; there is no artificial limit imposed.

Finally, edits on the RPL set level cannot be undone including adding or deleting rules and re-arranging rule priorities.

Using the History

When you double-click an unspecified expression or literal value, a menu appears to the right of the inline editor. This menu tracks the history of past edits that have been used in this dialog you can then use the menu to choose a value from the list. The arrow keys can also be used to scroll through the list.



The history can also be used as a starting point if you type in an invalid expression. It provides an opportunity to correct the entry and try again without having to type it from scratch. For example, you wish to enter “Flood Season” into a string, but when you type, you only enter the string “Flood S” and hit Enter before finishing typing Season. RiverWare will not issue an error as “Flood S” is valid. You can then select “Flood S” from the pull-down history, and complete the desired text.

> **Note:** The saved history it is a per-dialog history only but does persist when the dialog is closed and reopened. It does not persist if the RPL set is closed.

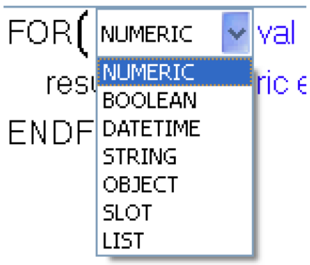
Data Types for Looping Variables

In the looping expressions, FOR, WHILE, WITH, SUM, AVE and in the FOR and WITH statements, you specify the data type of the looping variable. See “Expression Data Types” on page 95 for details on data types.

For example, the first line of a default FOR statement looks as follows:

```
FOR (NUMERIC val IN <list>) WITH NUMERIC result = <numeric> DO
```

You can change the type of the looping variable NUMERIC and the name of the variable. Menus provide an easy way to do this. You double-click the NUMERIC and are provided with a menu listing the 7 data types in RPL. A second menu is provided for the looping variable; this provides a history of all string values that have been previously entered in the dialog, similar to the history described in the previous section.



Renaming Looping Variables and Function Arguments

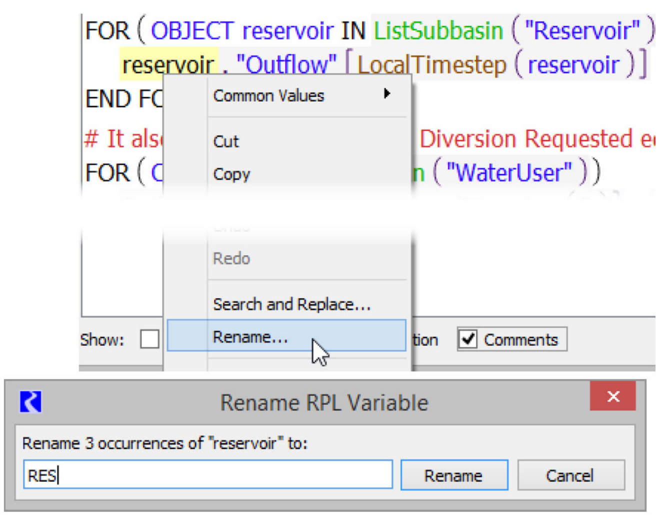
When a looping variable or function argument is selected, you can right-click and choose to **Rename** all of the occurrences of that variable or argument within the expression and the definition.

> **Note:** Variable names can be associated with statements (FOR and WITH), expressions (FOR, WITH, SUM and AVE), or functions (argument names).

To initiate a rename, use the right-click context menu and choose **Rename**. A dialog displays the number of occurrences of the name to be replaced, and allows you to type in the new name. Selecting **Rename** then makes the replacements.

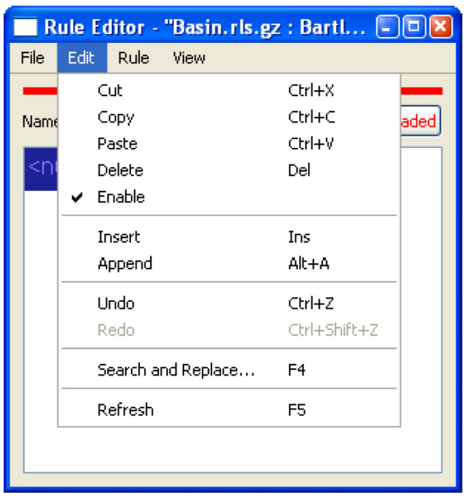
In Figure 2.22, three occurrences of the looping variable *reservoir* will be renamed RES.

Figure 2.22



RPL Shortcuts

If part of an expression can be used in another assignment, rule, or function, rather than repeating the above process for each new expression, highlight the portion of the expression that can be used elsewhere. From the menu bar, select **Edit**, then **Copy** or type **<Ctrl C>** to copy the highlighted portion. Right-clicking the highlighted portion will bring up a menu with the option to copy. In the location in which the expression is to be pasted, highlight the **<expr>** and select **Edit**, then **Paste <Ctrl V>**, or right-click and select **Paste** from the menu. This will paste the expression into the new location.



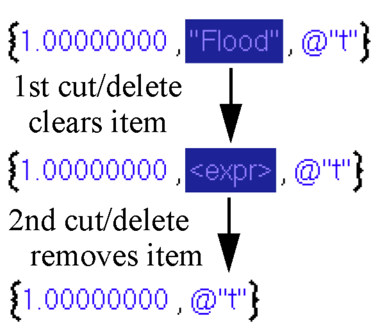
> **Tip:** Copying and pasting of RPL expressions is supported across opened RiverWare sessions. Copy an expression from one RPL set and paste it in another opened RPL set even in a separate RiverWare session.

When you copy an expression using **<Ctrl C>** or the **Edit** and then **Copy** menu, the expressions is also placed in the Clipboard tab of the palette. This allows you to use that expression or its subexpressions elsewhere. For more information, see "Clipboard Tab" on page 109.

Also, expressions can be deleted using the **Cut <Ctrl X>** or **Delete <delete>** operations. The **Cut** operation, puts the expression in the copy buffer, the **Delete** operation does not.

> **Note:** There is one exception to the cut/delete behavior. When deleting an item in a list, the first cut or delete operation removes the expression but leaves a blank expression. A second cut/delete removes the item from the list. Figure 2.23 shows an example.

Figure 2.23

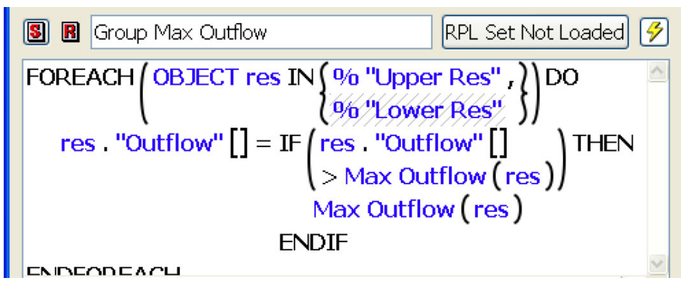


Statements may be cut, copied, pasted from the menu bar of the RPL Editor, using **<Ctrl C>,** or right-clicking and selecting **Copy** from the menu. Select **Paste** to paste over an existing statement, **Insert** to add it above the currently selected statement, or **Append** to add below the last statement.

Disabling an Item in a List or a Statement

On the RPL set editor, functions and RPL blocks can be disabled by selecting the green check mark turning it into a red X. Within a block, it is also possible to disable an individual item in a list or an entire statement.

- In the RPL editor, highlight the statement or item in the list that you wish to disable.
- Right-click the selected expression. Notice that there is a check mark next to the word **Enabled** indicating that the item is enabled.
- Select **Enabled** to toggle off the check mark thereby disabling the item. The disabled variable now shows up with cross hatching. The crosshatch color is defined in the set layout dialog; see "Colors" on page 88 for details. When the expression executes, it will skip the disabled item(s).



In general, disabling an assignment or an item in a list should only be used in the debugging and model building process. For example, you might want to have two assignment statements in one block. While debugging the model, it may be necessary to disable one assignment to ensure the other assignment is working correctly.

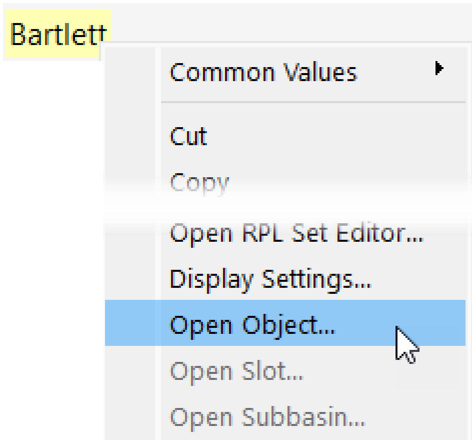
Opening Slots and Objects From RPL Dialogs

When the selected RPL expressions is a valid OBJECT or SLOT, the following right-click context menu selections open that object or slot dialog.

- **Open Object**
- **Open Slot**

  Note: This only works for expressions for which the workspace object can be determined without evaluation.

Figure 2.24 Right-click Context Menu showing Open Options



Opening Subbasins From RPL Dialogs

When the selected RPL expressions is a STRING, and the string exactly matches the name of a subbasin in the Subbasin Manager, the right-click context menu **Open Subbasin** becomes active as shown in Figure 2.25. Select the option to open the Subbasin Manager, scrolled to the named subbasin as shown in Figure 2.26.

Figure 2.25 Right-click Context Menu Showing Open Subbasin Option

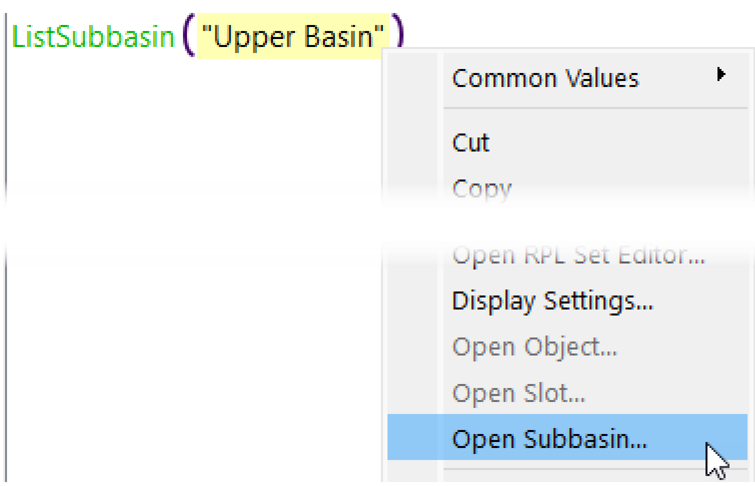
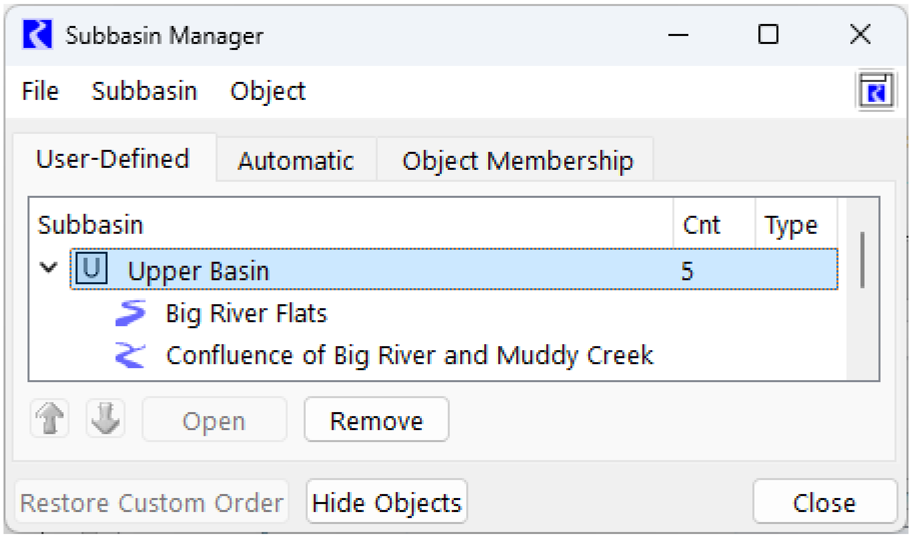
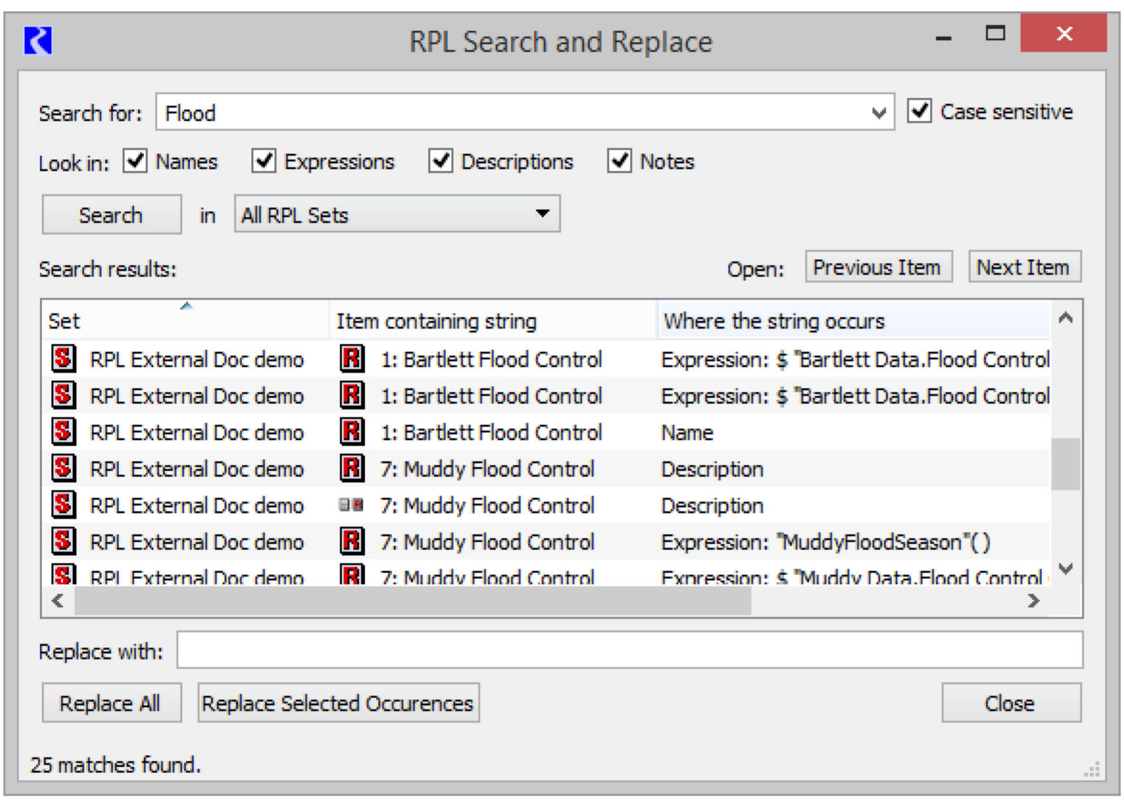


Figure 2.26 Resulting Subbasin Manager with Selected Subbasin Shown



RPL Search and Replace Dialog

The RPL search and replace dialog supports flexible replacement of strings within a single RPL set or all RPL sets. Within this dialog, you can search for all occurrences of a string and replace all or some of them with another string.



Accessing the Dialog

The RPL search and replace dialog is accessible in the following ways:

- From the main workspace, select **Policy**, then **RPL Search and Replace**
- Selecting **Search and Replace** from the **Edit** menu of the RPL editor and its children dialogs (the RPL Set, Function, and Rule/Method editors) or from the RPL Set analysis dialog.
- Pressing the **F4** key from any of the RPL editors or the analysis dialog.
- Right-clicking when editing any RPL expression and selecting **Search and Replace** from the resulting popup menu.

Searching for Occurrences of a String

To search for occurrences of a string, begin by typing the search string into **Search for:** field and then either hit **Enter** or select the **Search** button.

When you initiate a search, the search string is added to a *history* of searches. To re-select a previous string, select the triangle just to the right of the search string. This will present a menu containing previous search strings, allowing you to select one as the current search string.

Exactly where and how a search is conducted can be controlled by selecting various options.

- **Case sensitive**. When checked, RiverWare searches for strings which match the search string exactly, otherwise it will ignore case.
- Look in **Names**. When checked, RiverWare tries to match occurrences of the string within the names of RPL items in the set. This includes the names of the RPL set and its contained groups, rules/methods, and functions.
- Look in **Expressions**. When checked, RiverWare tries to match occurrences of the string within all expressions in the RPL set. Eligible sub-expressions within an expression include values of type DATETIME, NUMERIC, BOOLEAN, STRING, OBJECT, and SLOT, as well as function calls (the name of the function being called is eligible for a match)
- Look in **Descriptions**. When checked, the contents of all descriptions (set, group, and rule/method) are searched for matching strings.
- Look in **Notes**. When checked, the contents of all notes (set, group, and rule/method) are searched for matching strings.
- Search **in.** If the dialog is accessed from the workspace **Policy** menu, it starts with **All RPL Sets** selected. Otherwise it will be configured to search in the RPL set corresponding to the dialog through which it was accessed. The name of the set selected will be displayed in a box to the right of the **Search** button. You may switch from this set to any other open set or to all sets by selecting the set name and choosing a new set (or all sets) from the menu.

  Note: When you search in the **Expression Slot Functions Set** (or **All RPL Sets**), the expression slot expressions (i.e. the RPL expressions shown on the expression slot dialog) are searched along with the functions in the Expression Slot Functions Set. In this case, the parent is the object containing the expression slot.

The results of the search are displayed in the Search results table which contains one row for each string that matches the search string. Each row displays the name of the item containing the string (a set, function, group, rule/method, or expression slot) as well as where within that item the matching string was found (the object name, description, or one of its expressions). In the case of a match within an expression, the actually matching sub expression is also displayed.

Note: A search string can match multiple times within a given item. For example, when searching for the string “res”, a rule named “Forrest Reservoir Flood” will match twice.

Double-clicking a row of the Search results table will open an editor for the item containing the matching string corresponding to that row. If the match is within an expression, the matching sub-expression will be the selected expression in the editor and the dialog will be scrolled to that expression. Use the **Previous Item** and **Next Item** buttons to step through matches and opening/scrolling the dialog containing the match.

Tip: If you want to change the name of a function argument or looping variable (within a FOR or WITH), select the variable and use the right-click **Rename** menu. This utility will rename all instances of that variable within the expression or function. See “Renaming Looping Variables and Function Arguments” on page 59 for details on this utility.

Replacing Matching Strings With Another String

Once a search has been conducted, one can replace all or some of the matching strings with another string. First, type the replacement string into the field labeled **Replace with:**. To replace all of the occurrences then select the **Replace All** button. To replace only some of the matching strings, select those that you wish to replace in the **Search results** list (click selects a single row, **Ctrl-click** allows multiple independent selections, **Shift-click** selects a range of contiguous rows), then select the **Replace Selected Occurrences** button.

About RPL Functions

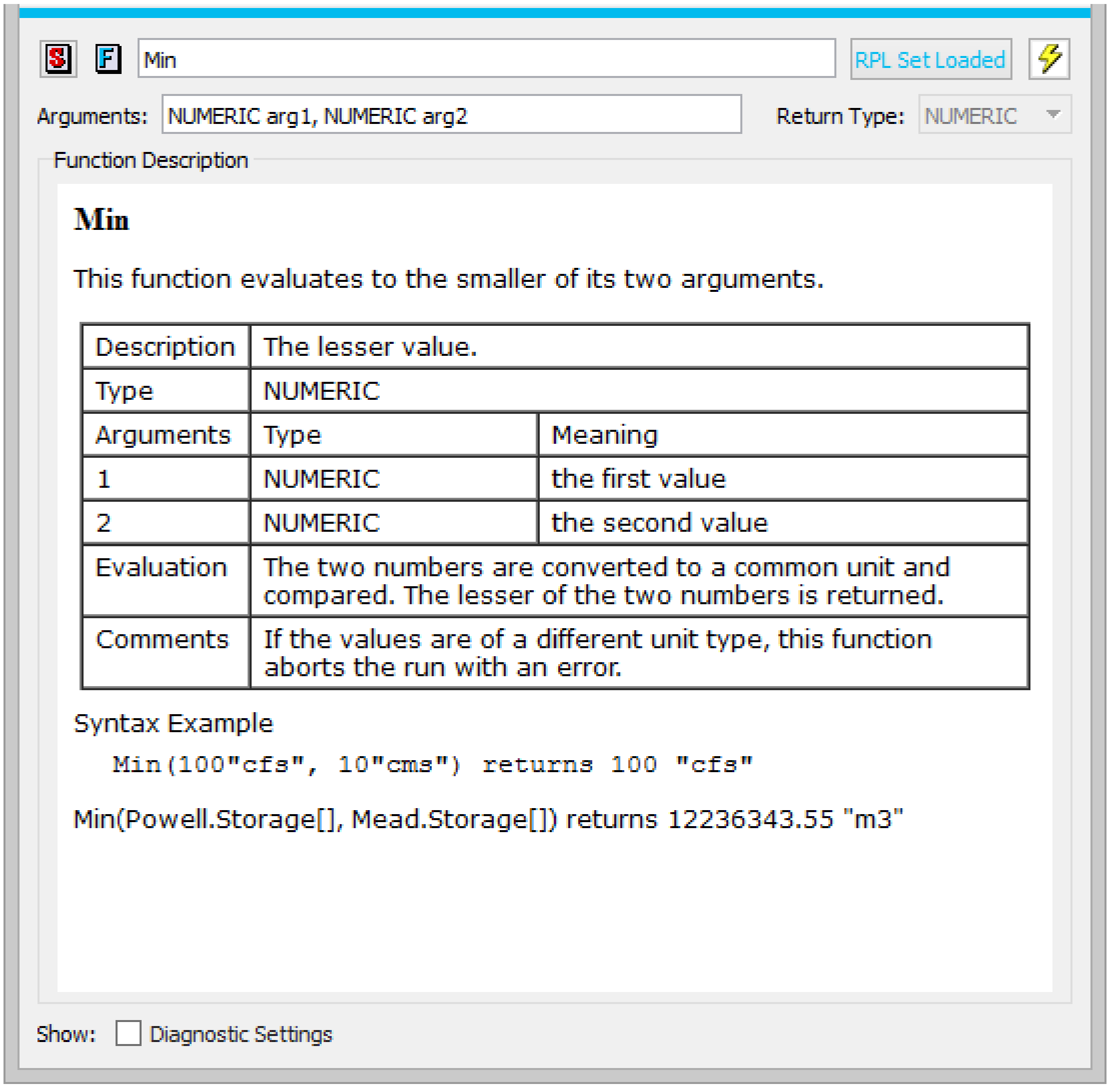
Following is a description of functions and their use in RPL sets.

Predefined Functions

Predefined functions are also useful in constructing rules, methods, and functions. Predefined functions are a set of mathematical, look-up, and mass balance routines that may be accessed from within any other expression. Predefined functions are coded into the RiverWare source code. Because of this, predefined functions cannot be modified. All predefined functions return their results in one of the standard data expression types; they can be substituted for any unspecified expression of that type. Predefined functions are accessed in the RPL Palette, “RPL Palette” on page 105, under the **Predefined Functions** tab at the top of the Palette dialog. Once added to a RPL expression, double-click the function to gets its editor. The documentation for the function is shown to help you specify the arguments and understand the evaluation.

See “RPL Predefined Functions” on page 147 for details on the predefined functions.

Figure 2.27



Writing a User-defined Function

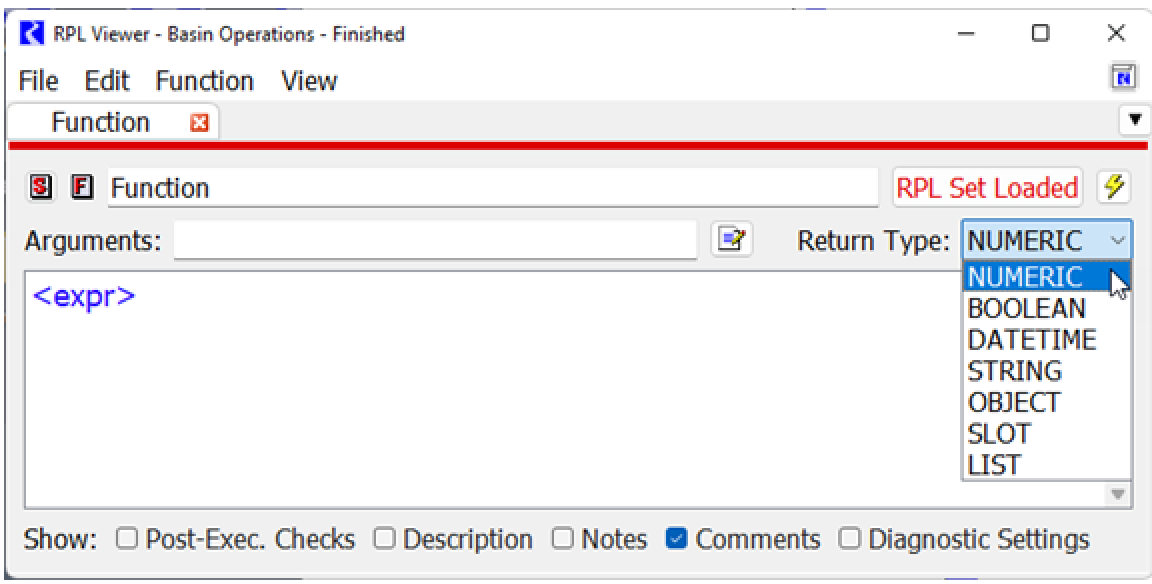
Functions are constructed in much the same way that blocks are constructed, using the RPL Palette. Expressions are simplified by using functions to perform logical and computational operations. To make a function flexible so that it may be used in a variety of situations, arguments are added to the function. Arguments permit the same function to behave differently, depending on from where it is called or by which object. For example, an internal function could be created which forecasts the evaporation from a reservoir based on the reservoir's surface area. This function could be of use at several reservoirs, but would have to know at which reservoir data to look. The reservoir for which the function should compute the evaporation could be an argument to the function.

Arguments to functions can be of any data expression type. There is no limit to the number of arguments a function may take, but the number of arguments is not dynamic with respect to block execution. The exact number and type of argument(s) is fixed in the function definition. Dynamic argument lists and default argument values are prohibited.

- Add a function to the desired Policy or Utility Group by selecting **Set**, then **Add Function.**
- Name the function and press <Enter> after naming.

  Note: If you are renaming a function that is called by other RPL expressions, a dialog asks if you want to rename calls to this function in the applicable RPL sets. Answering yes will update existing calls to the function with the new name. For the Expression Slot set, the Initialization Rules set, the Iterative MRM sets, the Object Level Accounting Method set, Optimization Goal sets, and Rulebased Simulation sets only function calls within the set are affected. For Global Function sets, calls in any of the above sets will be affected. Calls to a global function from DMIs, scripts, or other places outside of RPL sets are not renamed. Answering **No** continues the name edit but does not update any calls to the function. Answering **Cancel** stops the name change all together.

- In the field **Return Type**, select the data expression type you wish the function to return. For example, if NUMERIC is selected the function will return a numeric value to the block in which the function is called.



If the function is going to be general to other functions or blocks, adding an argument might be useful depending on the return type of the function. In the above graphic, the **Arguments: OBJECT res** allows the variable *res* to be used inside of the function anywhere an **<object expr>** is allowed. This will allow the function to look at different reservoirs' data without creating a separate function for each reservoir. If the function is going to be used only in a limited manner, arguments are not necessary.

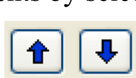
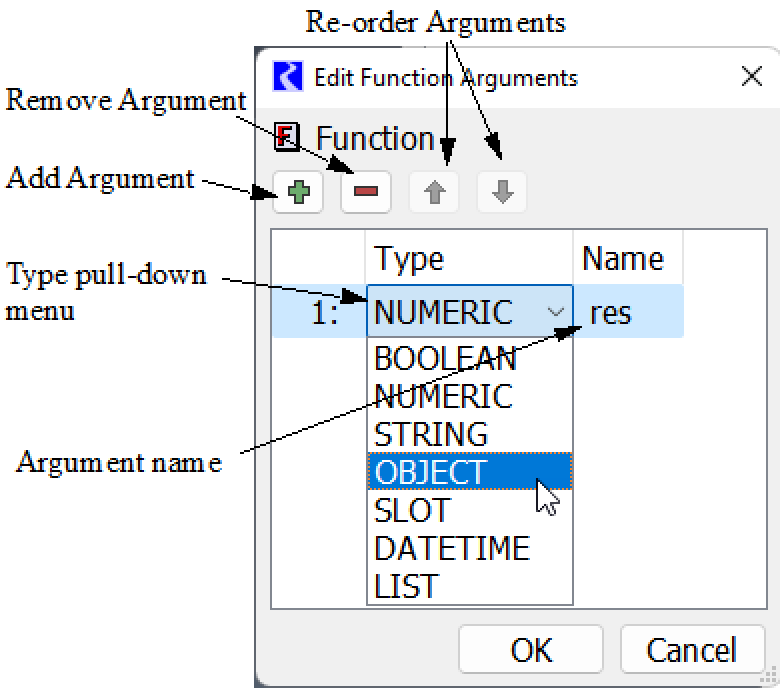
Arguments to functions can be entered in two ways, using the editor and by typing. The editor is accessed from the **Arguments** button  to the right of the Arguments line as shown to the right: Initially, the dialog is blank. You add arguments by selecting the green check button.  The red minus  is used to remove arguments and the arrows  are used to re-order arguments. The default type of an argument is NUMERIC. To change this, use the menu and select a different type. To change the name of the argument, double-click the name and type a new name. Select **OK** when finished. Figure 2.28 results in the argument shown below.

Figure 2.28 Annotated screenshot of the RPL Function Argument Editor

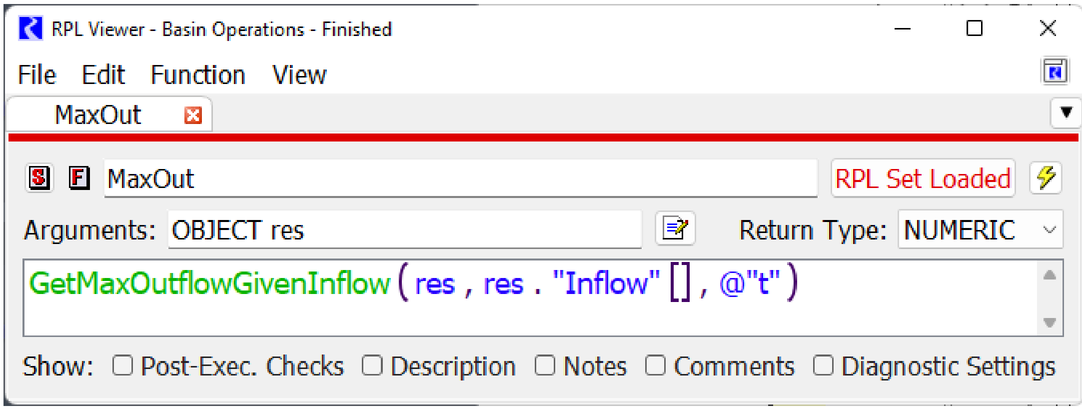


Typing was the original way to enter arguments. You type into the Arguments line using the syntax TYPE argument. Multiple arguments are separated by commas. For example, NUMERIC flow, LIST elevations, OBJECT res. The syntax, spelling, and capitalization must be exact or an error will be issued.

> **Note:** To change the name of a function argument, select the argument in the expression and use the right-click **Rename** option. This utility will rename all instances of that argument within the function's expression and definition. See "Renaming Looping Variables and Function Arguments" on page 59 for details on this utility.

Functions are constructed like blocks and use the RPL Palette to build arguments. Figure 2.29 shows a sample function constructed using predefined functions and RPL Palette buttons.

Figure 2.29 Screenshot of a completed RPL user defined function



The same short cut copy and paste abilities seen in blocks apply to functions as well. Any time an expression is used multiple times, whether within a function or between functions, time can be saved by copying and pasting that expression into the other locations in which it is used. As with blocks, it is wise to check the validity of an individual function after it is built and fix it rather than wait until the beginning of a run to check the validity of all blocks and functions.

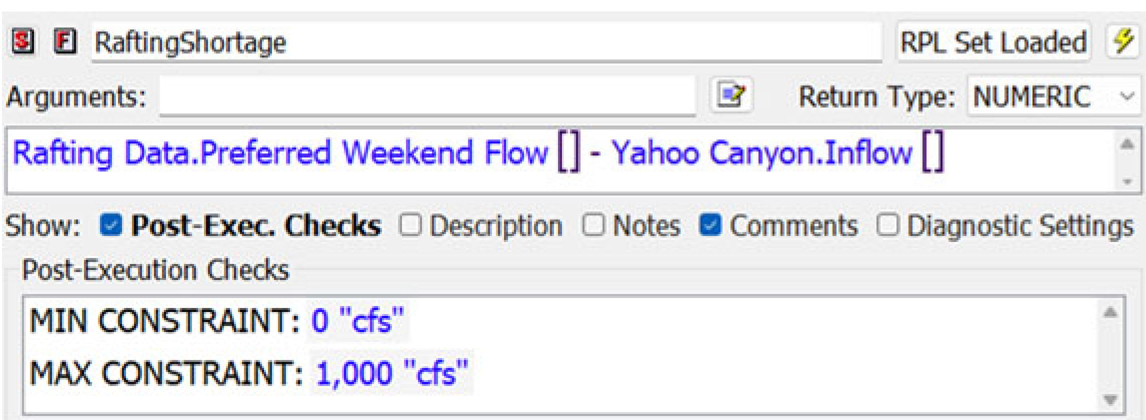
Constraints on Functions

Functions can be constrained to evaluate to either a minimum or maximum value, or configured to stop with an error if a minimum or maximum constraint has been violated.

From the Function Editor, Select **View**, then **Show Post-Execution Checks** or select the **Post-Exec Checks** toggle.

To add one or more constraint using the **Function** menu item.

- To add a minimum or maximum return value, select **Function**, then **Add Min Constraint** or **Add Max Constraint,** as shown in the figure**.** The palette is then used to set up the conditions of the selected constraint.



Caution: The Min and Max constraints must evaluate to a legal range or an infeasibility error will be issued.

- To stop the run if the function returns a value outside a range, select **Function**, then **Add Min Error** or **Add Max Error.** The palette is then used to set up the conditions of the selected constraint. If the computed values is larger than the Max Error value or smaller than the Min Error value, the RPL evaluation will be aborted with an error.

Tip: The Post-Execution Checks are executed in order, top to bottom.

Time-invariant Functions and Function Value Caching

The function editor **Edit** menu provides a toggle control labeled **Set Time Varying** which is set to on by default. Toggling this property off communicates to RiverWare that the function is guaranteed to evaluate to the same value each time it is evaluated, i.e., it is *time invariant*. If a function with no arguments is time invariant, then the first time the function is called within a run, the body will be evaluated and the result saved internally. For all subsequent calls of that function within the run, the cached value will be returned without further computation, reducing computation time.

Note: Functions with arguments will almost certainly not be time invariant; if a function has an argument, then presumably there are some argument values for which the function will evaluate to different values.

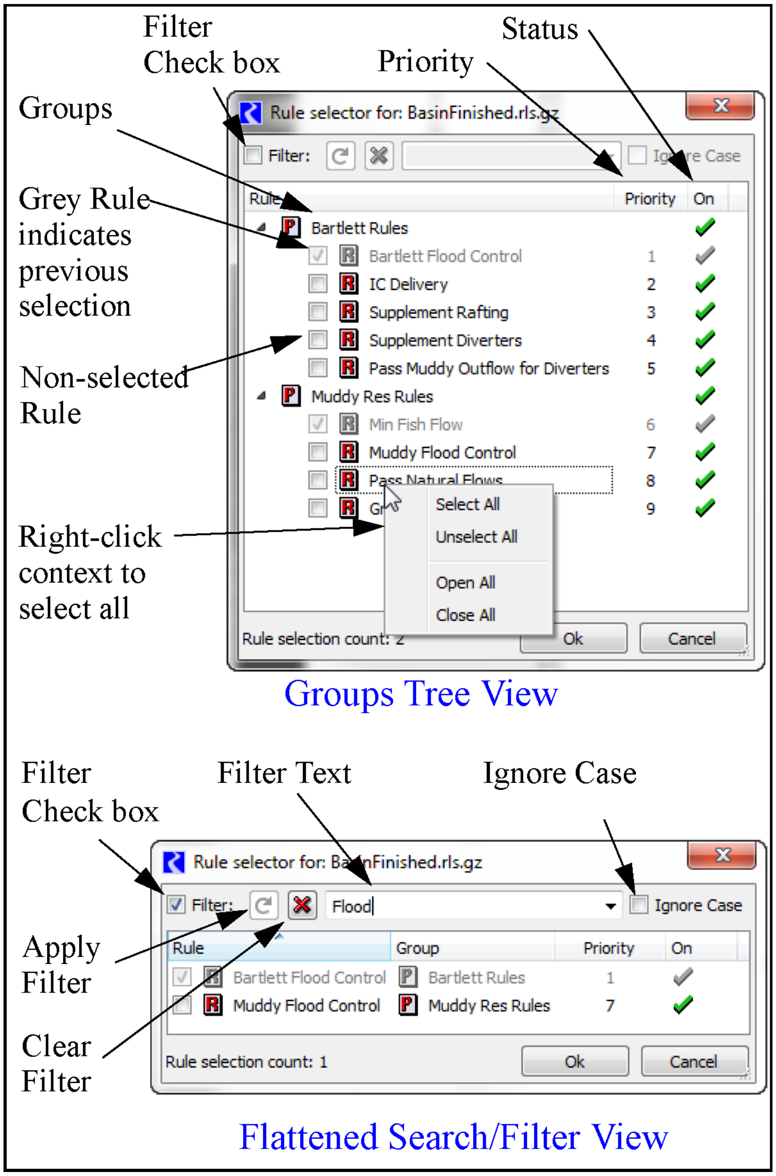
Note: Incorrect application of caching to a time varying function will lead to incorrect results, so we recommend that the time varying property be toggled off for a function only when it is definitely time invariant, run time is critical, and RPL set analysis has indicated that a significant portion of the run is spent evaluating the function.

Note: During block evaluation (e.g. within a rule or accounting method), the workspace remains unchanged (because RPL expressions evaluate without side effects), so it is safe to cache values for functions without arguments within a single block. RPL does this automatically for all functions without arguments; multiple evaluations of such a function within the same block execution will cause the function to evaluate only once.

Selecting RPL Items

The RPL Item selector dialog is used whenever groups, rules, or functions (i.e. RPL items) are to be selected. For example, in diagnostics, you may select the rules for which you want to print messages.

Figure 2.30



This dialog initially shows the groups and rules in a tree view. Also shown are the rule priority and rule status. In this dialog, you select the RPL items by selecting the boxes to add check marks next to one or more items. Depending on the context from which this dialog is called, you may be able to select one or many items. Items that have been previously selected for display are shown in grey and cannot be deselected. They can only be removed from the dialog from which this dialog was called.

Tip: All items can be selected by right-clicking and choosing **Select All**.

You can also select the **Filter** check box to show a flattened list of the rules. Then you can sort by column heading or filter by entering a text string at the top. Use the green arrow to refresh, red X to clear the filter, and the Ignore Case check box to do a case insensitive filter. Select RPL items by checking the boxes as described above.

Developing Efficient RPL Expressions

RPL expressions are built using the Palette; see "RPL Palette" on page 105 for details. Together with user and predefined functions, these provide all of the pieces necessary to create a simple or complex RPL expression; see "RPL Predefined Functions Reference" on page 148 for details. Typically, an analysis of your RPL set performance is necessary to locate slow or inefficient items. See "RPL Analysis Tool" in *Debugging and Analysis* for details on the tools, particularly the RPL Analysis Tool.

Following are some suggestions to writing efficient RPL sets in terms of performance.

- Use predefined functions or operators when available. In general, always prefer a built-in operator or predefined function to a user-defined function (or complex expression) which performs the same computation.
- Move complex logic to user defined functions. This not only makes the logic more readable and easier to debug, but it makes performance analysis easier. This is because the RPL set analysis tool reports the times for function calls but not for other parts of expressions (i.e., the granularity of the performance information is per function).
- Use WITH expressions to avoid re-evaluating expensive expressions.
- Make sure there is no unnecessary LIST processing or STRING manipulation.
- OBJECT and SLOT representations of workspace objects and slots are generally more efficient than string representations of them. For example, the expression

  ```
  GetSlot( (STRINGIFY reservoir) CONCAT "^" CONCAT account CONCAT ".Storage" )
  ```

  takes significantly longer to evaluate (and is more complex) than

  ```
  reservoir ^ ( account CONCAT ".Storage" )
  ```

  This also is slower and harder to read than

  ```
  reservoir ^ account . "Storage"
  ```

  When accessing accounting slots, the final option is preferred for performance and readability.
- For a large model, it is computationally expensive to obtain a SLOT given the full slot name (i.e., a STRING representation of the slot) because RiverWare must first break the string into its components, then look for an object with the appropriate name. Once the object is found, then for accounting slots RiverWare must search through the accounts. Finally a search is conducted for a slot on the object/account with the given slot name. Thus when referencing slots, one should take special care to apply the suggestions mentioned above.
- Make sure that time consuming functions like FloodControl() or the hypothetical simulation functions are not being called more than necessary.
- Use functions with no arguments to cache values. If a function has *no* arguments, then the first time it is executed in a block (a rule, accounting method, optimization goal, initialization rule, MRM rule or expression slot), the return value is cached. For the remainder of the block execution, the function need not be evaluated again, the cached value is used. Thus, if you have multiple assignments within one rule that call the same argument-less function, the function will be evaluated once and the value will be used for all function calls.
- If you are sure that a function with no arguments is always going to return the same value regardless of the timestep, set the **Set Time Varying** toggle to be off. This will lead to the function's first return value being reused throughout the run. See "Time-invariant Functions and Function Value Caching" on page 70 for details.

- MAPLIST is an efficient expression which operates on a list *and* returns a list, but if one wants to operate on a list and compute a single value, FOR expressions (or a variation of FOR, like SUM or AVE) are more efficient. E.g.

  ```
  Sum( MAPLIST (...))
  ```

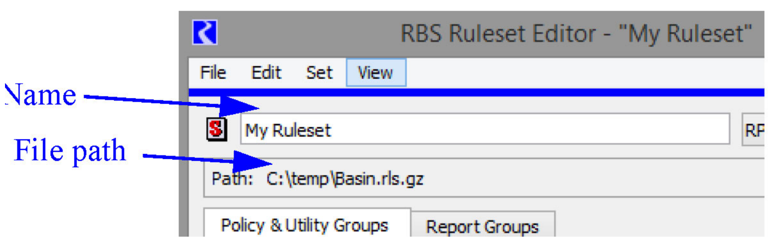
  is slower than

  ```
  FOR ( ... ) SUM
  ...
  END FOR
  ```

Example: Creating a New RBS Ruleset

This section presents an example of how to build a new Rulebased Simulation (RBS) ruleset. Similar actions can be used to build any of the other types of RPL sets.

1. Select **Policy**, then **Ruleset**, then **New** from the main menu bar. This will bring up a blank RPL Set Editor dialog.
2. Enter the desired Ruleset Name.
3. Save the ruleset to the desired directory by selecting **File**, then **Save** or **File**, then **Save As** from the menu bar of the Ruleset Editor dialog.



Adding Groups, Rules, and Functions

To begin the ruleset, use the following steps to add the desired number of policy groups and utility groups.

1. From the **Set** menu in the RPL Set Editor's main menu bar, select either **Add Policy Group** or **Add Utility Group.**
2. Name the Policy or utility Group by double-clicking its icon to open the Policy Group Editor or Utility Group Editor and replace the default name at the top with the desired name.

Policy groups are used to organize rules and functions. Utility groups are used to organize functions. Although an entire set can be created entirely within a single policy group, it is not recommended. Rules should be split up into different policy groups according to object, geography of the model, operation type, and/or priority of operations. Supporting functions which are common to several rules should be placed in utility groups. Functions within a policy group are only available to rules and functions within that policy group.

Use the following steps to add a rule or function.

1. Select the Policy or Utility Group so that it is highlighted.

2. From the **Set** menu, select **Add Rule** or **Add Function.**

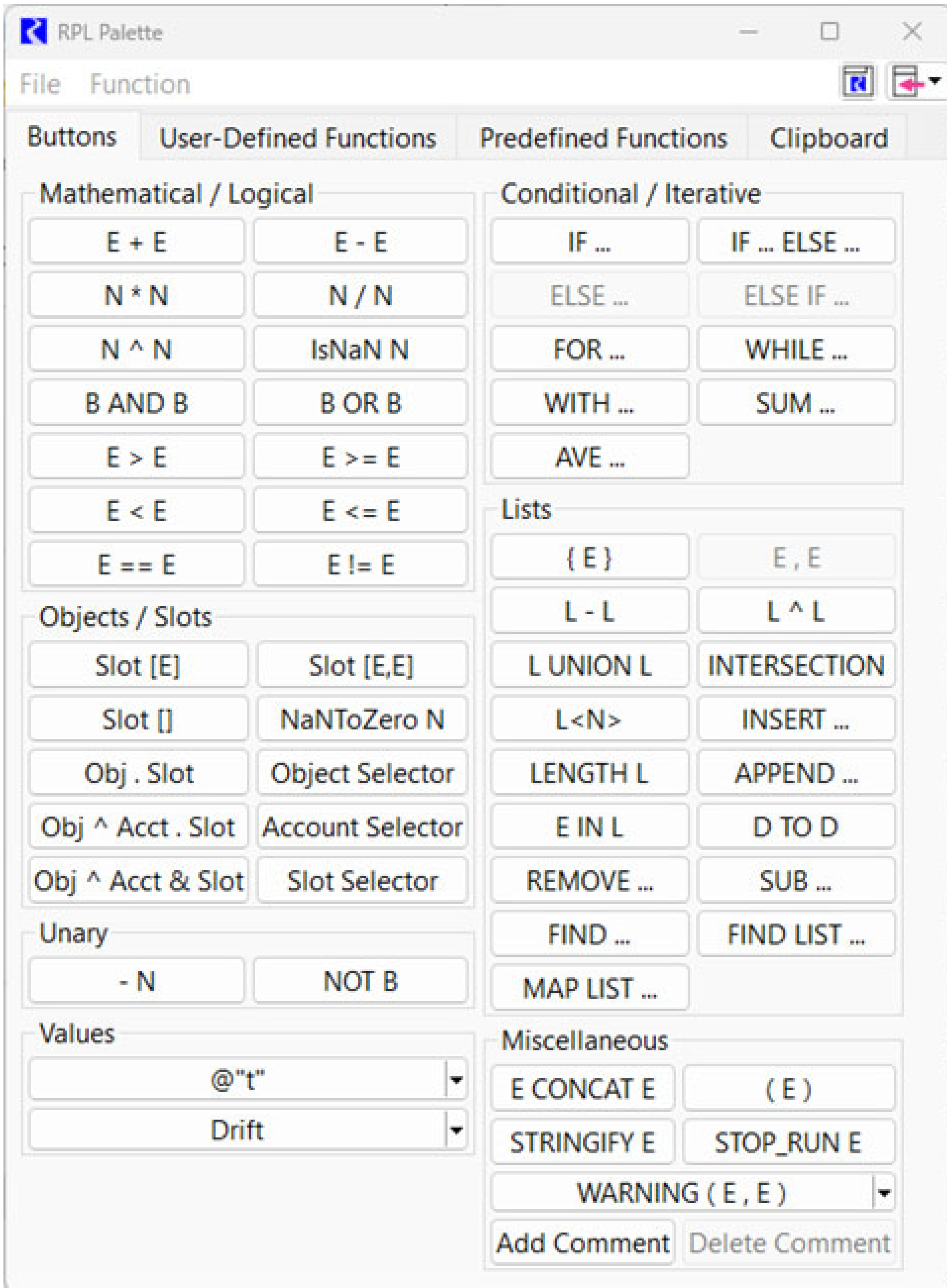
To view the elements in the policy or utility group, select the tree-view triangle left of the group name.

Unspecified expressions are portions of the rule that have not yet been set when building a rule. These expressions are represented as the expression data type.

To add an Assignment, from the **Statement** menu, select **Add Assignment**.

Expressions are built using the RPL Palette. The RPL Palette contains most of the expression elements needed to build a block, including logical operators, slot and object access expressions, flow operators, list operators, predefined and user made functions. See “RPL Palette” on page 105 for details. Depending on the data type of the expression highlighted in a block, the RPL Palette only enables buttons for expressions of that data type. See “Expression Data Types” on page 95 for details on the data types.

To open the RPL Palette, select **Rule**, then **Palette** from the menu bar of the Rule Editor dialog.



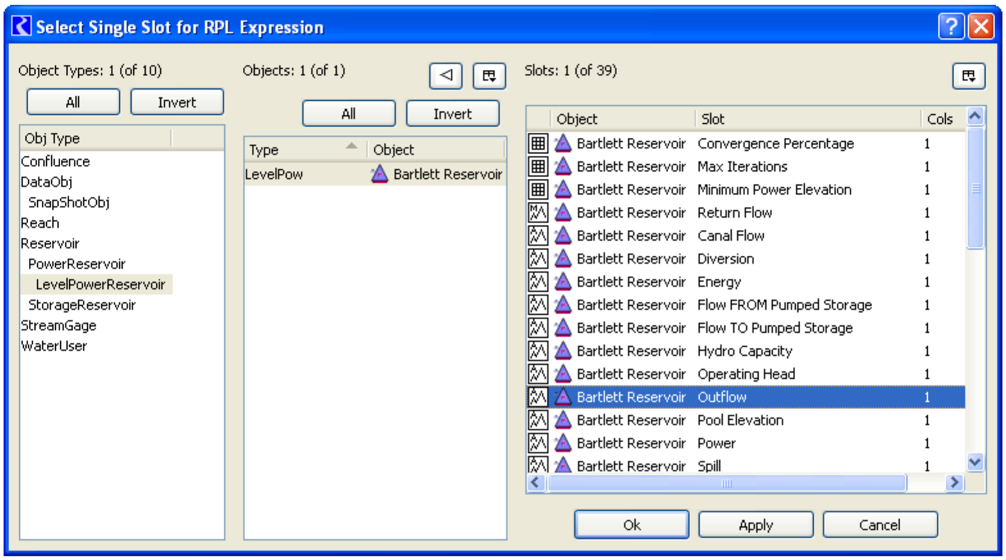
Use the following steps to finish building the rule.

1. Highlight the left **<numeric expr>**. This will be the slot to which the rule will assign a value.
2. In the **Objects/Slots** section of the RPL Palette, select the **Slot[]** button. This replaces the unspecified numeric expression with a series slot expression.

   The square braces following a slot expression refer to the timestep or the row and column of the slot. There are three slot expression possibilities:

- **Slot [E]** A series slot at the timestep indicated by **E**.
- **Slot [E, E]** A table slot at the row and column indicated by **E** and **E**, respectively. Table slot column references are zero based, i.e. the first column is referred to as column 0.
- **Slot [ ]** A series slot at the current controller timestep or a scalar slot value

3. Highlight the unspecified portion of the series slot expression, **<expr>**, and select the **Slot Selector** button in the **Objects/Slots** section of the RPL Palette. This will bring up the slot selector dialog.



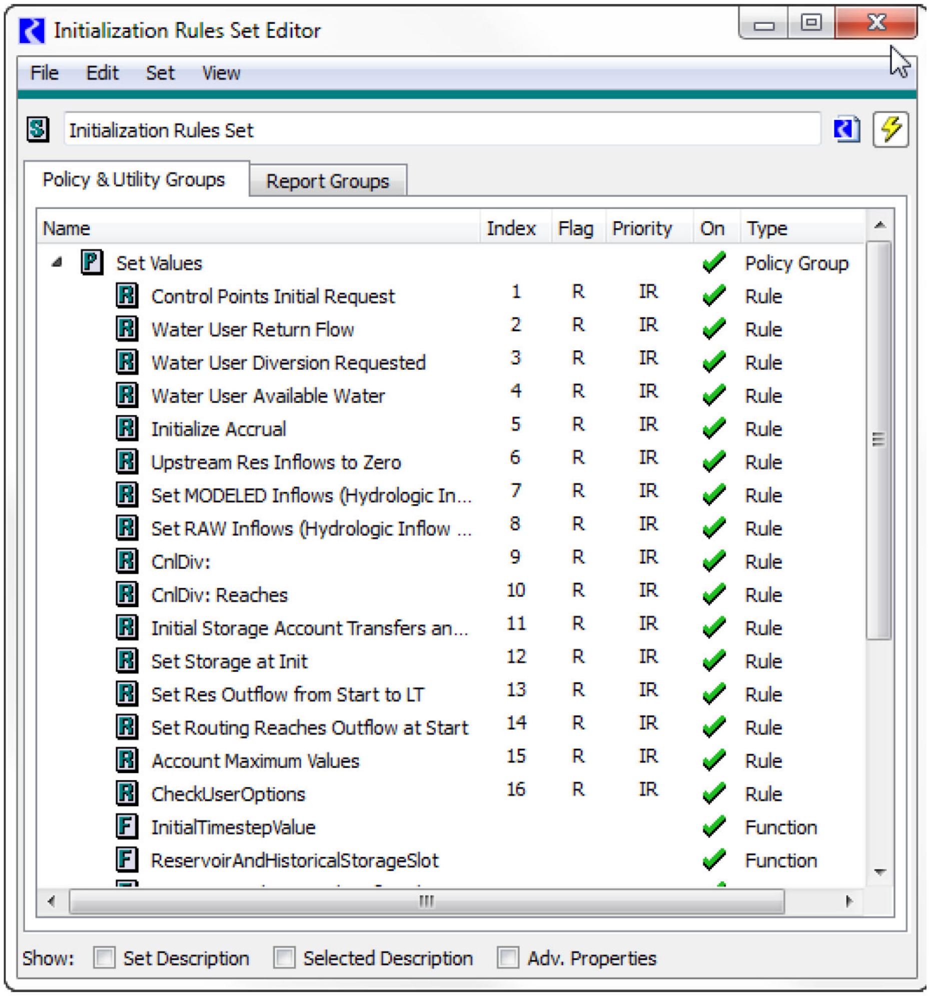
4. In the Single Slot Selector, select the appropriate **Object Type**, **Object**, and **Slot**. Then select **OK**.
5. Finish building the rule by building the right-hand side of the rule. The RHS should return a numeric value and units that are to be set on the slot specified on the left-hand side.

Initialization Rules Set

Initialization Rules are a set of RPL rules associated with the model which can be executed as part of run initialization to set up data for the run. Initialization Rules provide a complement to user inputs and execution of DMIs.

Unlike other rules which can only set series slots, initialization rules can be used to set Table Slots and Scalar Slots.

This section describes the applicability and use of Initialization Rules. See “RPL User Interface” on page 9 for details on the user interface for Initialization Rules and other RPL sets.



When to Use Initialization Rules

Each method for providing data to a simulation is suited to different modeling scenarios. For situations in which there are a small number of inputs that don’t change frequently, interactive setting of input values is often the simplest approach. However, when this process would be time-consuming or error-prone, it is preferable to automate the setting of user inputs. Input DMIs provide a flexible way of accomplishing this by providing mechanisms for communication between RiverWare and external programs (usually databases). If there is computational logic involved in this process, e.g., if the data need to be massaged to remove outliers, then this logic must somehow be embedded in the external DMI program. On the other hand, using initialization rules to accomplish the same task allows this logic to be viewed and edited from within RiverWare, often improving the clarity and ease of maintenance of the model.

Initialization Rules also differ from DMIs in that in the context of a RBS simulation, they can be used to provide default values which the policy can then override. To illustrate a scenario in which this might be useful, consider a run in which some quantity is generally assumed to be zero, but which can take on non-zero values under some conditions. One strategy for modeling this situation is to set the relevant values to zero using low priority rules with execution constraints that cause them to execute only on the first timestep. Alternatively, Initialization Rules accomplish this same behavior with less user effort (no execution constraints are necessary) and with the additional advantage that these rules are organized separately from the policy (if there is one). In fact, most RBS rules which are constrained to execute only on the first timestep are likely to be more appropriately included with the Initialization Rules.

Another important distinction between Initialization Rules and policy (rules) is that the former are executed before data checking at the beginning of the run. Thus, Initialization Rules can be used to set values such as initial **Storage** or **Pool Elevation** on a reservoir, whereas RBS rules execute only after data checking and so could not be used to give these slots reasonable values. Further, unlike other rules which can only set series slots, initialization rules can be used to set table and scalar slots.

How to Use Initialization Rules

This topic describes how to use initialization rules.

How do I access the Initialization Rule Set?

From the workspace, use the **Policy**, then **Initialization Rules RPL Se**t menu.

Where are they saved?

The set is saved in the model file. The Initialization Rules Set Editor dialog does have a **File**, then **Save Initialization Rules Set As** menu item to save the initialization rules to a file. A **File**, then **Replace Initialization Rules Set from File** menu item allows the initialization rules set to be replaced by the contents of a specified file. These menu items allow an initialization rules set to be moved between models via a file. However there is only a single instance of initialization rules in a model and this set is still saved and loaded with the model file. You can also use the **File**, then **Clear Set** to remove all items in the initialization set.

When are they executed?

The rules are executed during the initialization phase of Simulation and Rulebased simulation runs (including accounting) and Optimization runs. For Simulation/Rulebased, they are executed after values are cleared and inputs are registered but before beginning of run checks on all objects.

See “Simulation Controller” in *Solution Approaches* for details on the ordering for simulation.

See “Simulation/Rules Interaction” in *Solution Approaches* for rulebased simulation.

See “How the Accounting System Solves” in *Accounting* for accounting.

See “Initialization Rules” in *Optimization* for information on Initialization Rules in Optimization.

In what order are they executed?

The Initialization Rules are executed in Index order based on the specified Agenda Order. To see the order, select **View**, then **Show Advanced Properties.**

What do Initialization Rules set?

Initialization rules can set series slots, table slots, scalar slots, and periodic slots.

- For series slots, the current timestep is the start timestep.
- For table slots, specify the rows and columns using labels or a zero-based index.
- For scalar slots, use the Object.ScalarSlot[ ] syntax on the left side of the assignment statement.
- For periodic slots, use the Object.PeriodicSlot[row, column] or Object.PeriodicSlot[row] syntax on the left side of an assignment statement.

  The supported types for the row expression are DATETIME and NUMERIC. For a DATETIME value, an attempt is made to match the date with one of the rows of the periodic slot. The date must be partially specified and in the time, month, day, (and optional year) format, for example @ “23:00 July 1”.

  > **Tip:** RPL uses 24:00 as the end of a day while irregular periodic slots use 0:00 on the start of the next day. For example 24:00 April 30 in RPL is the same as 0:00 May 1 on a periodic slot. When writing RPL logic continue, to use 24:00 notation but realize this will map to 0:00 on the slot.

  If the row expression evaluates to a NUMERIC value, the value is truncated and used as a zero-based row index.

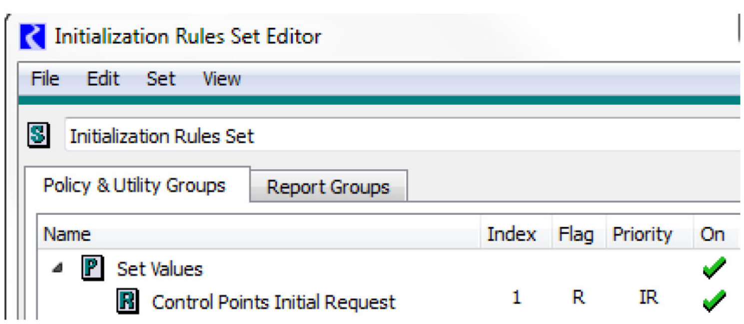
  The column expression, if present, must have NUMERIC type. The value is truncated and used as a zero-based column index. If the column expression is absent, the first column is assumed (column index 0).
- The row and column numeric headings can also be set by initialization rules using the Set Row and Set Column statements. See “Set Row” on page 54 and “Set Column” on page 53.

Do Initialization rules re-execute when dependent slots change?

No, initialization rules execute once and do not re-execute even if dependencies change.

Do Initialization Rules have Individual Priorities.

No, the Initialization rules are organized by **Index** as shown on the RPL set editor.



What Flag/Priority are the values given?

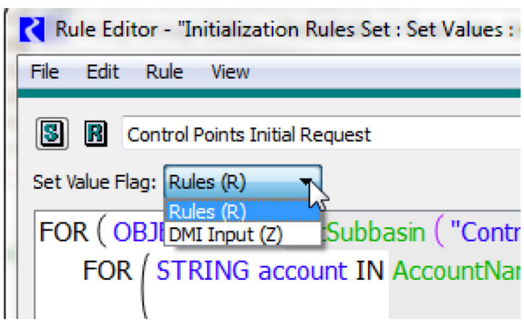
Series values set by Initialization Rules are given one of the following flags/priorities as configured through the **Set Value Flag** menu on each Initialization Rule:

- **Rule (R)**. Set the value with the R flag. All values set this way are given a very low priority (It is a large number) and the letters **IR** are shown indicating the Initialization Rule priority. R flagged values set by Initialization rules can be overwritten like other R flagged values.
- **DMI Input (Z)**. Set the value with the Z flag and give it a priority of zero (0). When simulating, this flag behaves identically to the INPUT (I) flag.

 Note: An Initialization Rule setting a value with the Z flag can overwrite an I or Z flagged value.

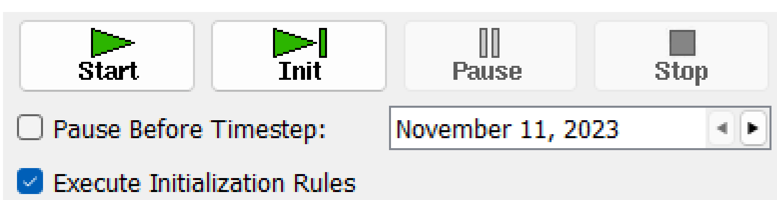
 Note: The values set by initialization rules are given the priority as described above. If these values are propagated or provide enough information for an object to dispatch, any subsequent values set will be given the controller priority, which would be zero (0) if they solve before any rules execute.

 Note: Table and Scalar slot values set by initialization rules are not give a priority or a flag. Choosing the R or Z flag from the menu shown above results in identical behavior for these types of slots.

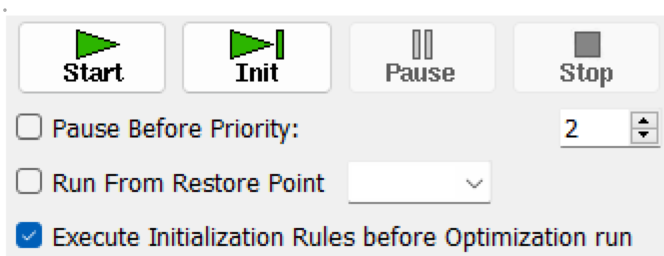


Can I disable the Initialization Rules?

When the Initialization Rules set contains at least one rule, then the Run Control Dialog displays a check box labeled **Execute Initialization Rules**, which controls whether or not to execute the Initialization runs during the initialization phase of runs. Clear this check box to disable the Initialization Rules.



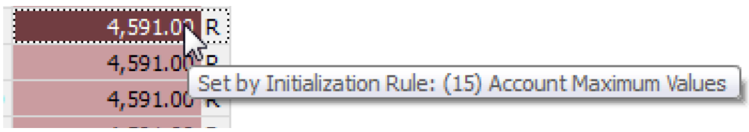
For an Optimization run, the toggle/flag is labeled **Execute Initialization rules before Optimization Run**.



Note, the toggles are separate for each controller. They can be on for one controller but off for another controller. See “Set Init. Rules Exec. Flag” in *Automation Tools* for a script action that sets these flags.

How do I know which Initialization rule set a slot value?

Tooltips on the series slot values display which Initialization Rule (or RBS rule or DMI) set that value. See the following figure. See "Series Slots" in *User Interface* for details on these tooltips.To open that rule, right-click and choose **Open Init Rule #.**



How do I debug Initialization Rules?

The RPL debugger can be used with Initialization Rules. Also, the following diagnostic categories in the Rulebased Simulation and Simulation settings dialog control diagnostics for the Initialization Rules:

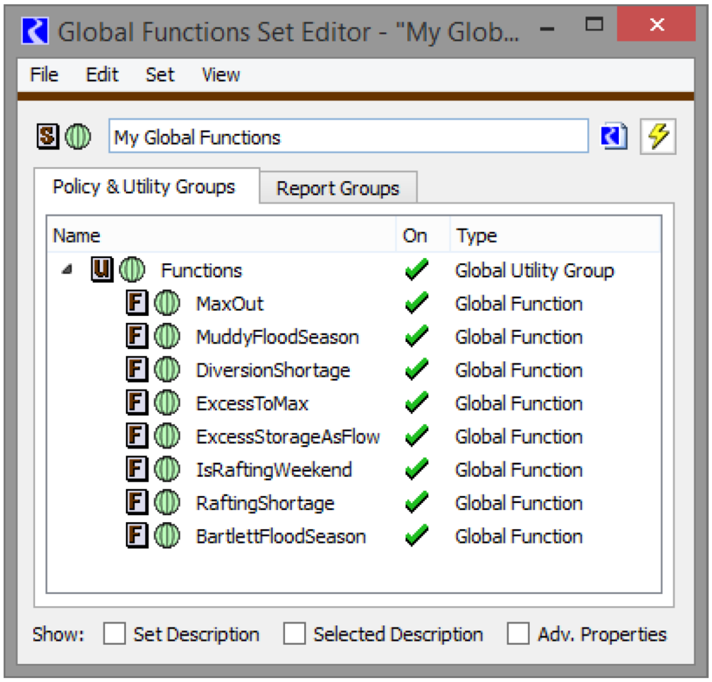
- Init. Rules Print Statements
- Init. Rules Rule Execution
- Init. Rules Function Execution

See "Types of RPL Debugging and Analysis" in *Debugging and Analysis* for details on debugging and analysis of RPL sets.

Global RPL Functions

Global functions allow you to define a function in one location and then use it in any of the RPL applications including Rulesets, Optimization Goal Sets, Object Level Accounting Method Set, Initialization Rules, MRM Rules, and RPL Expressions Slots.

Global RPL Functions exist within Global Functions Sets, organized in Global Utility Groups. Multiple Global Function Sets can be opened within a RiverWare session.



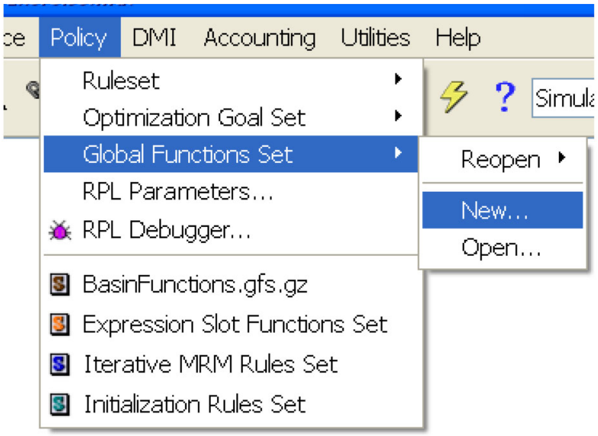
A RPL set is considered *global* when it is opened as a Global Set and any RPL Set file can be loaded as a Global Function RPL Set. Only the RPL Set's Utility Groups—and not its Policy Groups—are relevant in Global Function Sets.

Global RPL Functions can be called from RPL Functions and Rules (and other forms of RPL Blocks) in any open RPL Set. Global Functions can call other Global Functions, either in the same Global Function Set, or a different Global Function Set.

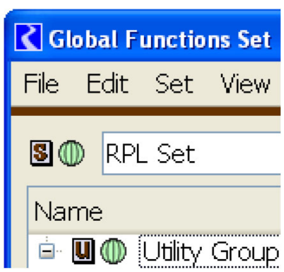
Creating a New Global RPL Function Set

A new Global RPL Function Set can be created from the RiverWare **Workspace** menu, with the following menu selection:

Policy, then **Global Function Set**, then **New**

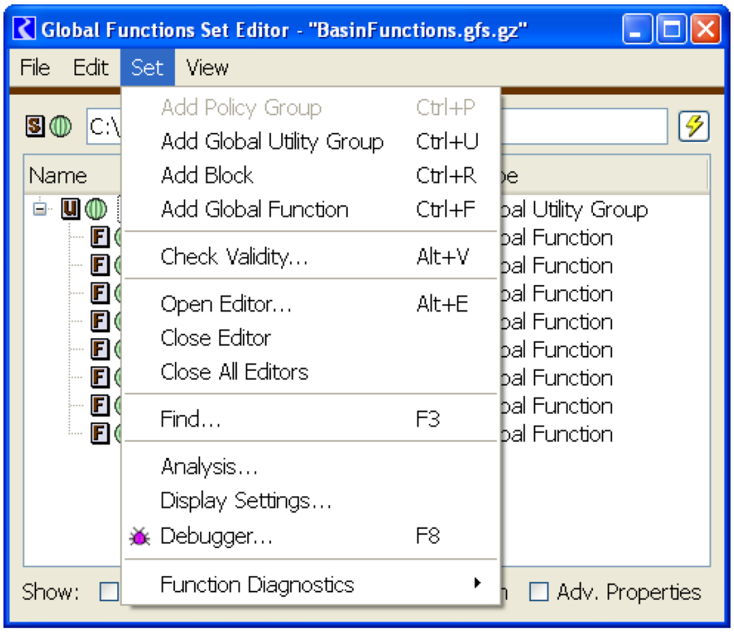


This opens up a Global Function Set editor. This is a RPL Set Editor for Global Function Sets, distinguished with a green globe icon in the upper-left area of the editor dialog.



As with all RPL Sets, a Global Utility Group must first be added before new user-defined RPL Functions can be defined. In a Global RPL Function Set, this is done from the RPL Set Editor's menu item, as follows:

Set, then **Add Global Utility Group**



Any function added to the new Global Utility Group is, by definition, a Global Function. A Global Function can be created in any of the following ways (not illustrated):

- From the RPL Set editor, with the desired Global Utility Group selected, using the **Set**, then **Add Global Function** (**Ctrl+F**) menu item.
- From the RPL Set editor, using the Context menu (right-click) on the desired Global Utility Group item, **Add**, then **Global Function**.
- From the Global Utility Group Editor, using the **Group**, then **Add Global Function** (**Ctrl+F**) menu item.
- From the Global Utility Group Editor, using the Context Menu (right-click) anywhere within the function list, **Add**, then **Global Function**.
- By Copying a Function from another RPL Group or RPL Set, and Appending into the Global Utility Group, using Context Menu operations.
- By Dragging a Function from another RPL Group or RPL Set into the Global Utility Group.

  Note: The Drag-and-drop functionality currently implemented in the RPL Set and RPL Group editors has some limitations, including the availability of that operation only if the target function list already has at least one function.

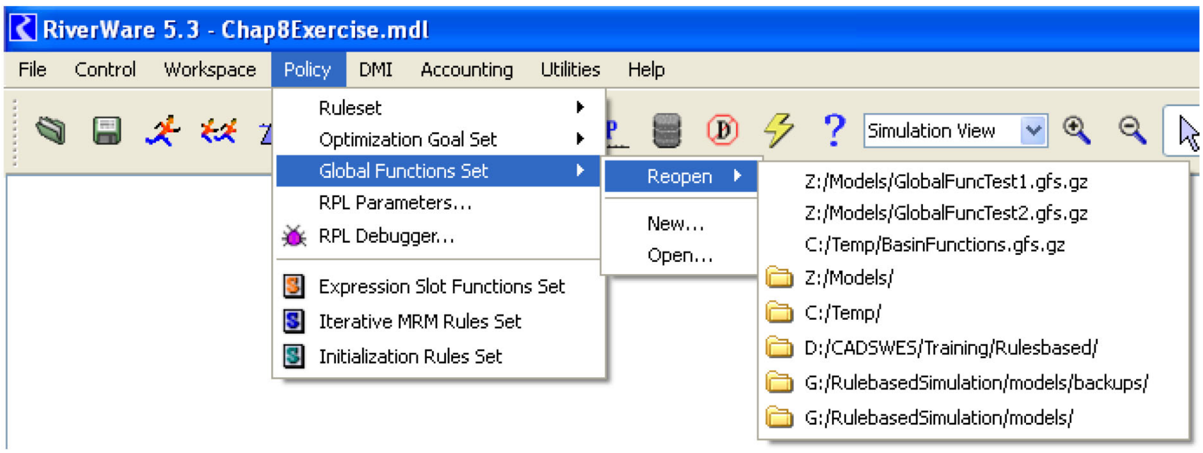
- By importing Utility Groups from a RPL Set file. This is done from the RPL Set editor, **File**, then **Import Set** menu item.

Only one instance of a function can occur within all loaded RPL sets and Global Function Sets. For example, you cannot have a function named GetMinimumFlow in a utility group in your ruleset and another named GetMinimumFlow in an open Global Function Set. An error will occur in this case.

Opening an Existing Global Function Set

The RiverWare Workspace **Policy** menu supports the opening of a RPL Set as a Global Function Set in ways similar to opening other RPL Set files.

Figure 2.31



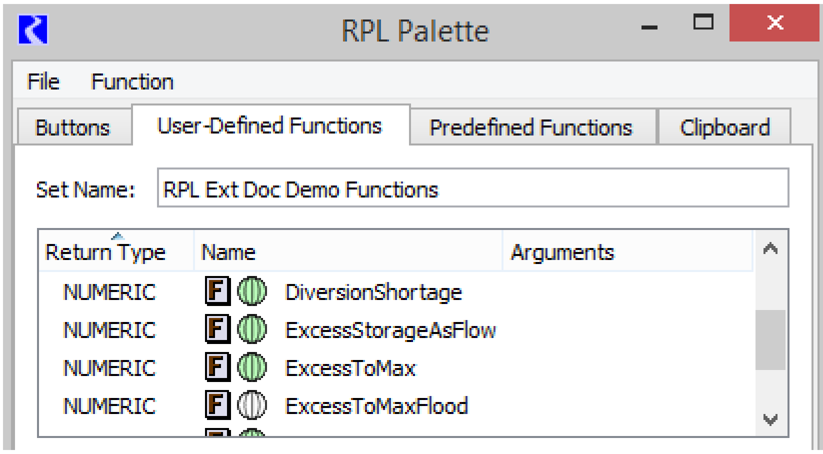
Any RPL Set file can be loaded as a Global Function Set, however only the Utility Groups within the RPL Sets loaded as Global Function RPL Sets are usable; rules (or other forms of RPL Blocks) within the set's Policy Groups are not. If a RPL Set file containing Policy Groups is opened as a Global Function Set, the warning dialog shown in Figure 2.32 appears, and a similar warning message is written to the RiverWare diagnostics output.

Figure 2.32

Note: It is important to open any Global Function Sets before trying to load a ruleset that uses functions in a global set.

Using Global RPL Functions

When editing a RPL Expression, any Global Function in any open Global RPL Set can be called. The standard way of adding a call to any user defined function is with the RPL Palette **User-Defined Functions** tab.



Within the User-Defined Function list, global functions are indicated with one of two globe icons:

- **White globe**. Indicates an *external* Global Function i.e. a global function in a RPL Set other than the set containing the RPL expression being edited.
- **Green globe**. Indicates a *local* Global Function i.e. a global function within the same RPL Set as the one containing the RPL expression being edited.

During evaluation of a Global RPL Function, the specific behavior is consistent with the caller's RPL Application. For example, "@t" (the current timestep) represents the Rulebased Simulation Controller's current timestep when the function is called from an Rule. But when called from a Series Slot with RPL Expression, "@t" (current timestep) represents a particular series timestep date/time.

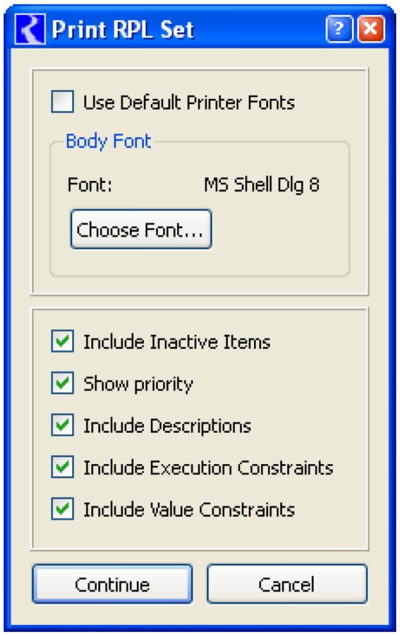
RPL Printing and Formatting

Blocks written in the RiverWare Policy Language can be complex, and with lengthy verbose object, slot, and function names, blocks can have very long lengths. RiverWare provides a mechanism to print blocks and dynamically format blocks to fit the width of desktop window or a printer page.

Printing

Rules and user defined accounting methods written in the RiverWare Policy Language can now be printed. A Printing dialog is available at each editor level from the **File**, then **Print** X menu. This provides you with the means to print an individual block from the Editor, an individual function from the Function Editor, an individual group from the Group Editor, or the entire RPL Set from the RPL Set Editor.

Using the print dialog, the block, function, group, or RPL set can be sent directly to the printer or to a file. You may select landscape or portrait printing. You may choose to include or exclude inactive items, descriptions, execution constraints, and value constraints. You may also choose the print font. A note on font selection. The font selection dialog presents a list of fonts that are available on computer console, these fonts may or *may not* all be available on the printer. If the font is not available on the printer, a default font will be chosen by the printer and the printout may be misaligned—formatting will be corrupted. Some experimentation may be necessary to determine which fonts an individual printer supports.



Formatting: Display Settings

Expressions written in the RiverWare Policy Language can be complex and very long. RiverWare provides a RPL Display Settings to control the fonts, colors, showing element numbers, and a dynamic formatting algorithm. This algorithm formats the blocks to fit the width of desktop window or a printer's page. The dialog is accessed through the **Settings Manager**.

Specify how so save the settings:

- Model File. This allows RPL logic to look the same from one user to the next.
- Settings File. This saves the settings on a per-user basis so one user can have different settings than another user.

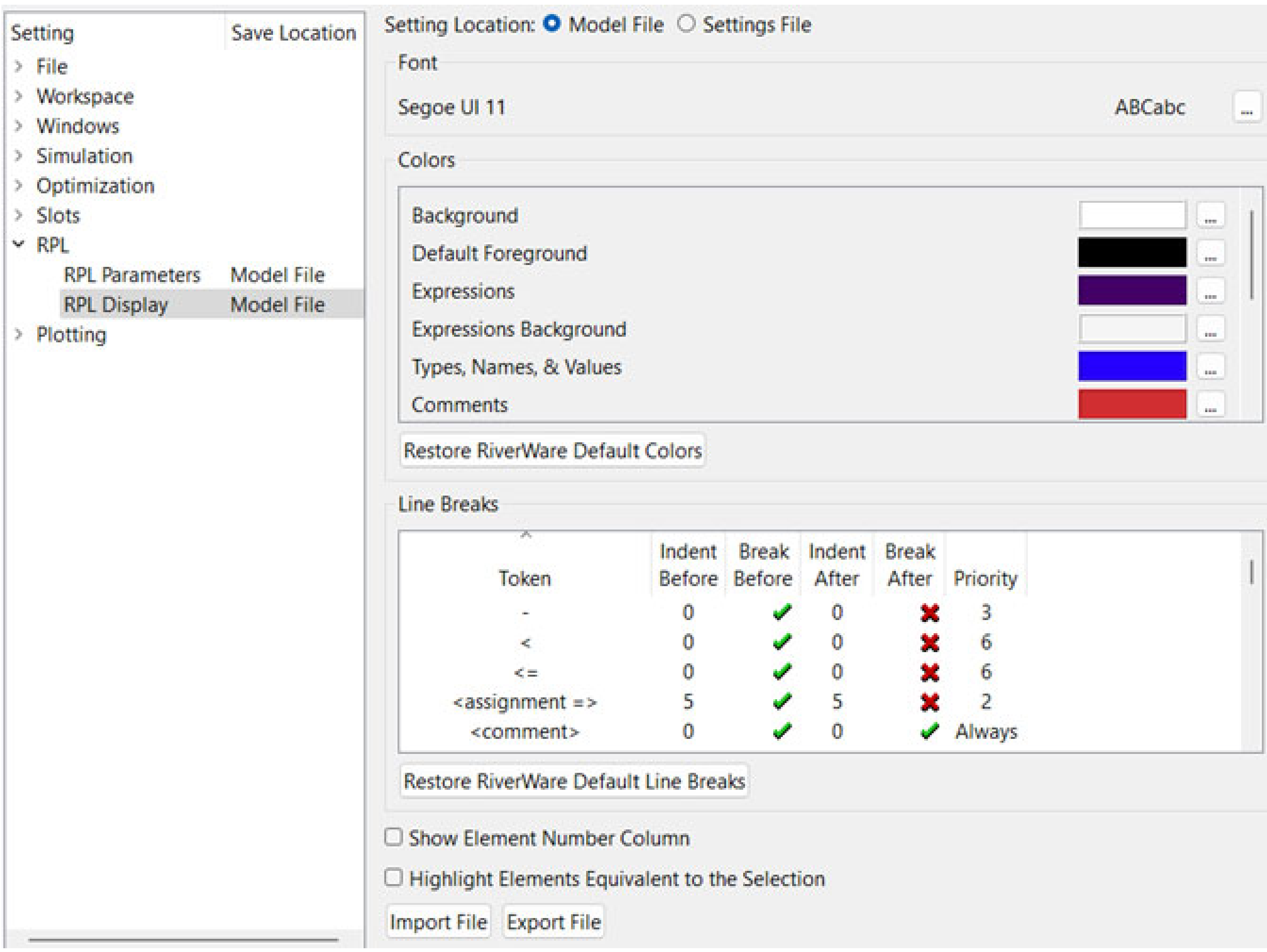
See "Save Location" in the Settings Manager section for more information.

Display settings may be imported and exported to/from a file through the buttons. Following are the options:

- **Import File.** Import the settings from a file that was exported from a RiverWare session.
- **Export File**. Export all of the settings to a file.

There are five areas, Font, Colors, Line Breaks, element numbers, and highlighting of equivalent expressions, shown in Figure 2.33 and described in the following sections.

Figure 2.33 RPL Display page in the Settings Manager



Font

The fonts used in RPL expressions can be changed by selecting the ... button then select the desired font, style, size, and effects.

Colors

The colors used in RPL set editors can be changed to fit your needs. You are able to specify the colors for items shown in Table 2.7. To change, select the ... buton, then select the desired color from the color chooser.

Table 2.7

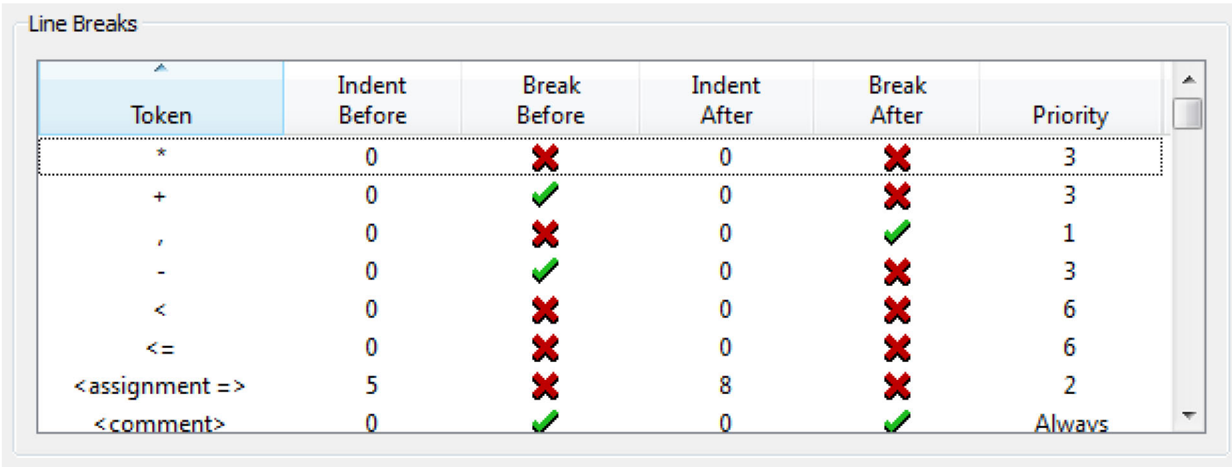
Item	Description
Background	The background color for statements and other areas (except expressions) that are not contained within the current selection.
Default Foreground	The color used to draw items in expressions or statements whose color is not governed by another color setting. For the current selection, this color is used for all items; outside of the current selection, this color is used to draw mathematical symbols, operators, and delimiters. **Note**: The selected expression’s foreground text is either white or black (dynamically computed) to contrast with the configured background color.
Expressions	The color for key words in expressions.
Expressions Background	The color used for the background of the bounding rectangle for expressions.
Types, Names, & Values	The color of types (e.g., NUMERIC), names of variables, and literal values.
Comments	The color of in-line comments.
User-Defined Functions	The names of user-defined functions.
Predefined Functions	The names of predefined functions.
Disabled Functions	The names of user-defined functions which are disabled (i.e., functions for which the red X appears in the On column of the functions RPL Set and Group editors).
Disabled Objects (Crosshatch)	The color of the crosshatching superimposed on disabled statements or expressions.
Selection Background	The background color for statements and expressions that are contained within the current selection.

Use the **Restore RiverWare Default Colors** to load the RiverWare default colors. This allows you to get back to the base starting colors.

Line Breaks

The formatting algorithm is designed to provide you with control over the formatting process. The formatting algorithm is a multi-pass algorithm. Each pass will attempt to find a position in the block where a line break can be placed to fit the block to the device width. The algorithm will cease, once the block can be rendered in the desired width, or no more positions where the block is allowed to be broken exist. You can define, through the RPL Display Settings dialog, the tokens (i.e., block positions) a line break can appear before or after, the indentation following the line break, and the priority (the pass) at which the line break should be inserted. Line breaks with a priority of zero will always be broken, while the maximum value will indicate that a line break at the token should never occur. Figure 2.34 illustrates.

Figure 2.34



A block is formatted using the following algorithm:

1. The formatting algorithm makes an initial pass, breaking the block at priority 0 tokens. All line break defined with priority 0 (i.e., Always) in the RPL Display Settings dialog will be formatted in this initial pass. This pass always occurs.
2. Next the formatting algorithm checks the length of block. If the block will fit in the width of device, the formatting terminates. If the block will not fit the width of the device, the priority is incremented, and line breaks are made at all tokens defined with priority 1.
3. Repeat step 2 through all priorities until the block fits in the device.

If the block exceeds the screen width after all line break positions have been exhausted, the window will be made scrollable. If the block exceeds the printer's page width after all line break positions have been exhausted, the excess will be printed on adjacent pages.

To edit any number in the dialog, select the number and then type in a new value. You can enter either "Always" or "0" for very high priority values. Enter "Never" or a large number (greater than about 40) for those items with low priority.

Example 2.1

Given the following line break settings

Token	Break Before	Break After	Priority
THEN	no	yes	Always (0)
END IF	yes	yes	Always (0)
+	yes	no	1
AND	yes	no	2

The block being fitted to the box

Before Formatting:

```
IF (SomeFunctionWithAVeryLongName() AND AnotherFunctionWithAVeryLongName) THEN Some-
ObjectSlotWithAVeryLongName + AnotherObjectSlotWithALongName END IF
```

After the initial pass (priority 0):

```
IF (SomeFunctionWithAVeryLongName() AND AnotherFunctionWithAVeryLongName) THEN
    SomeObjectSlotWithAVeryLongName + AnotherObjectSlotWithALongName
END IF
```

After the second pass (break on all priority 1 positions):

```
IF (SomeFunctionWithAVeryLongName() AND AnotherFunctionWithAVeryLongName) THEN
    SomeObjectSlotWithAVeryLongName
    + AnotherObjectSlotWithALongName
END IF
```

After the third pass (break on all priority 2 positions):

```
IF( SomeFunctionWithAVeryLongName()
    AND AnotherFunctionWithAVeryLongName() ) THEN
    SomeObjectSlotWithAVeryLongName
    + AnotherObjectSlotWithALongName
END IF
```

Rule Fits the box. Stop.

Notice that with this naive algorithm, had the object slot names been small, they would have still been broken at the “+” symbol, since, the “+” had a higher priority (1) than the AND and the block did not yet fit the box.

Before Fomatting:

```
IF (SomeFunctionWithAVeryLongName() AND AnotherFunctionWithAVeryLongName) THEN Object
Slot + AnotherObjectSlot END IF
```

After the initial pass (priority 0):

```
IF (SomeFunctionWithAVeryLongName() AND AnotherFunctionWithAVeryLongName) THEN
    ObjectSlot + AnotherObjectSlot
END IF
```

After the second pass (break on all priority 1 positions):

```
IF (SomeFunctionWithAVeryLongName() AND AnotherFunctionWithAVeryLongName) THEN
    ObjectSlot
    + AnotherObjectSlot
END IF
```

After the third pass (break on all priority 2 positions):

```
IF( SomeFunctionWithAVeryLongName()          ) THEN
  ( AND AnotherFunctionWithAVeryLongName()   )
    ObjectSlot
    + AnotherObjectSlot
END IF
```

Rule Fits the box. Stop.

Use the **Restore RiverWare Default Line Breaks** to load the RiverWare default line breaks. This allows you to get back to the base line breaks.

Element Numbers

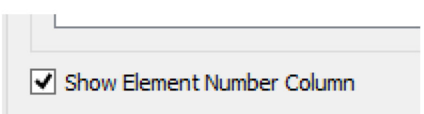
RPL Expressions can be long and it is sometimes hard to tell where you are within the RPL expression. To help with this, you can show **RPL Element Numbers** along the left hand side of the RPL frame. Notice in Figure 2.35, the column of numbers from 1 to 33.

Figure 2.35

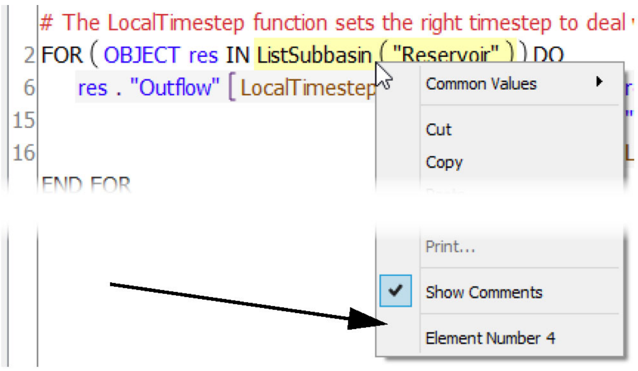
Element numbers are shown for the statements and expressions with which rules and functions are composed. As a result, they are neither continuous nor a constant increment, but are reflective of the number of elements on that line.

One can think of these like line numbers, but they are not line numbers as lines associated with a rule/goal/function vary with the formatting, which is adjusted to reflect the RPL display settings and the width of the display.

The display of Element Numbers is controlled by a check box in the **Set/Rule/Function/Goal**, then RPL Display Settings dialog. When display of the element numbers is enabled, a new column is shown along the left side of each RPL frame which displays, for each line, the number of the lowest numbered token beginning on that line.



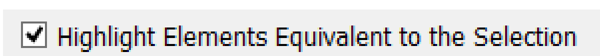
In addition, when this feature is enabled and there is a selection in a RPL frame, the right-click context menu displays the Element Number of the selected element.



Highlight Elements Equivalent to the Selection

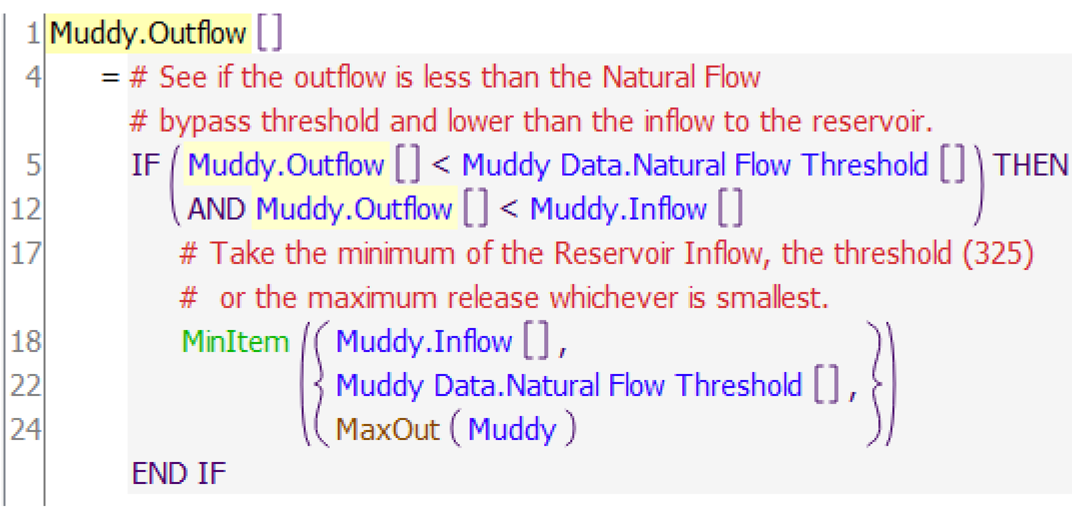
At the bottom of the RPL Display Setting dialog is a check box to Highlight Elements Equivalent to the Selection as shown in Figure 2.36.

Figure 2.36 Display setting to Highlight Elements Equivalent to the Selection



When this features is toggled on and a RPL element is selected, other elements in the same dialog which are semantically equivalent are highlighted in a paler shade of the selection color. See Figure 2.37 for an example. Muddy.Outflow, the first element, is selected. Then elements 5 and 12 are also highlighted in a paler shade of yellow.

Figure 2.37 RPL expression with highlighting



This functionality is particularly useful when searching for an expression, perhaps to refactor the expression into a more efficient WITH statement or expression. This highlighting provides a quick visual cue to indicate where there are identical or equivalent expressions.

Chapter 3
RPL Data Types and Palette

This section describes and presents examples of each of the Data Types used in RPL. Then the RPL palette buttons are described.

Topics

Expression Data Types

The RiverWare Policy Language language is composed of seven data types, such as numeric values, datetimes, text strings, and slots. The data types are the building blocks of RPL. They are the only types of data which expressions and functions are allowed to evaluate to and are the structure controlling what information can be entered in different parts of the RPL. The structure editor uses expression data types to enforce block and function validity as they are constructed. Attempting to enter an expression of one type where another type is expected, immediately results in an error.

Unspecified expressions are pieces of the RPL which have not yet been defined when creating a RPL set. Unspecified expressions are shown in the Rule and Function Editor as the name of the expression type, colored blue (by default), and surrounded by small angle braces (< expr >). An unspecified expression may be filled in directly with the appropriate data, or it may be replaced with a function which will evaluate to the appropriate data type. In most cases, unspecified expressions can be completed by using Palette buttons. This is the recommended approach. The complexity and syntactical requirements of some expression types, particularly object and slot, behooves you to use the built-in tools for formatting a correct expression.

Example 3.1

A rule contains an unspecified numeric expression: <numeric expr>

The expression may be filled in with an actual value and units by entering: `100.0 cfs`

or it could be replaced by an arithmetic operator which evaluates to a value and units by selecting the **N + N** palette button:

```
<numeric expr> + <numeric expr>
```

or it could be replaced by yet another palette selection which returns a value and units, such as a series slot lookup:

```
Lake Mead.Outflow []
```

In all of these cases, the original requirement that the expression evaluate to a number and units is satisfied.

Table 3.1 lists the expression data types. More complete descriptions follow.

Table 3.1

Expression Type	Unspecified Representation	Description	Examples
NUMERIC	`<numeric expr>`	A number and its units.	150 cfs 349.47926 “acre-feet/day” 1 NONE
BOOLEAN	`<boolean expr>`	True or False.	TRUE FALSE
DATETIME	`<datetime expr>`	An actual or symbolic date.	@“t”, @“t-1” @“Current Timestep” @“Previous Timestep” @“January 4, Current Year”
STRING	`<string expr>`	Quoted text and/or numbers.	“Entering spring runoff season.” “Minimum flow violated by 5%“
OBJECT	`<object expr>`	An object in the current model.	%“Havasu” %“Kentucky-Barkley Canal” %“SJBelowNavajo::ColoradoAg”
SLOT	`<slot expr>`	A slot on an object in the current model.	$“Lake Mead.Outflow” $“Kentucky.Pool Elevation” $“Rio Chama.Hydrologic Inflow”
LIST	`<list expr>`	A list of any mix of data types.	{ 1561 [“ft”], 1573 [“ft”], 1589 [“ft”] } { @“April 15, 1999” } { %“Powell”, 10 [“cfs”], TRUE } { { %“Mead”, TRUE }, { $“Inflow” } }

NUMERIC

The numeric expression data type is comprised of a number and its units. Numeric expressions may be typed directly into a block or function by double-clicking the unspecified <numeric expr> and typing a value and units into the textfield. As in Simulation, numbers are stored and evaluated internally with twelve significant figures of precision. When a numeric expression is typed directly into a block or function, the default display precision is two digits beyond the decimal point. The precision of displayed numbers can be modified in two ways:

- Change the **Precision** setting in the **Adv. Properties** portion of the RPL Set Editor. This affects values that are configured to use the Set Precision. See “Show Advanced Properties” on page 21 for more information.
- Change the precision of the selected numeric value using the right-click **Precision** context menu. Only the selected numeric expression is changed. See “Precision of Numeric Values” on page 56 for more information.

Units for numeric expressions must follow the value, separated by a single space. If no units are specified, the value is considered to be of a dimensionless unit type with units of NONE. When a numeric expression is typed directly into a block or function, the units may be entered without quotes if the unit name contains no special punctuation characters such as hyphens or forward slashes. When quotes are omitted, they will be filled in automatically.

Example 3.2

Entering numeric expressions with the display precision set to the default of 8, causes the following text to be shown in the editor.

Exact Text Typed Into Textfield	Resulting Display
150 cfs	150.00000000 “cfs”
0.000000004 “feet/day”	0.00000000 “feet/day”
0.5 NONE	0.50000000
248650 acre-feet	“parse error” due to hyphen, therefore you must type the quotes

Note: Variable length time units are not allowed in RPL literal values. For example, the following are not allowed: 10 “acre-feet/month”, 100 “m3/year”, 10 “months”, and 1 “year”. These values are ambiguous in terms of how many days to use. As a result, an error is posted if you enter these values or load a set with these values. See “Use of Variable Length Time Units” on page 127 for more information.

BOOLEAN

The boolean expression data type can be either true or false. The expressions may be entered directly in the editor by double-clicking the unspecified <boolean expr>, and typing into the textfield. The editor accepts lowercase, uppercase and capitalized spellings, but automatically converts these entries into an uppercase format.

Example 3.3

Entering boolean expressions (as shown below) cause the following text to be shown in the editor.

Exact Text Typed Into Textfield	Resulting Display
true	TRUE
False	FALSE
FALSE	FALSE
tRuE	“parse error”

DATETIME

The datetime expression type is used to represent moments in time. Datetimes may be entered directly by double-clicking an unspecified <datetime expr>, and typing into the resulting textfield. All datetime expressions begin with an @ symbol and are followed by the datetime specification in double quotes (“ and ”).

Datetime expressions may be specified symbolically. Symbolic representations of points in time are common in everyday language, but rare in programming languages. Symbolic representations include such datetimes as “next week," "20th day of month," and "April, next year." These types of datetime specifications *are* allowed in the RiverWare Policy Language. They facilitate the writing of RPL expressions by using date conventions to which you are more accustomed. They are evaluated by the RPL language, in the context in which they are used, to determine an exact moment in time. Slot access is still performed internally with exact datetime specification in number of seconds since New Year's, 1700 C.E.

Fully or Partially Specified Datetime Expressions

Datetime expressions fall into two categories and each may be used in the following instances or contexts:

Fully Specified

Fully specified datetimes are those which can be mapped directly to an instant in time, such as @"t" or @"June 4, 1986." Fully specified datetimes are required for all slot lookups, slot assignments and predefined function arguments.

Partially Specified

Partially specified datetimes are those which cannot be mapped to a specific instant in time, such as @"Current Month" or @"Tuesday." Partially specified datetimes are used in boolean comparisons. For example, a check for whether the current timestep is a Tuesday could be done with a fully specified datetime, @"t", and a partially specified datetime, @"Tuesday":

IF (@"t" == @"Tuesday") THEN ...

If you have a partially specified datetime that you want to convert into a fully specified datetime, use the CompletePartialDate predefined function; see "CompletePartialDate" on page 162 for details.

Formats

There are four general datetime formats. Within each format, there are specific components which the user can specify. Table 3.2 lists the four formats and components, which you can specify separately.

Table 3.2

Name	Time components (from lowest to highest resolution)	Components you can specify separately	Examples
1. Month, day, year	Year, Month, Day of Month, Hour, Minute, Second		@"August 23, 1997 4:00:00" @"7/22/1997 4:0000"
		Month	@"August", @"Month 8", @"Current Month"
		Day of Month	@"DayOfMonth 23", @"Previous DayOfMonth"
		Year	@"Year 1997", @"Current Year" , @"... Year"
		Time	@"4:12:00"
2. Timestep	Timesteps since the beginning of run		@"Timestep 12", @"t+2"
		Timestep	@"Max Timestep", @"t-1"

Table 3.2

Name	Time components (from lowest to highest resolution)	Components you can specify separately	Examples
3. Day, week, year	Year, Week of Year, Day of Week, Hour, Minute, Second		@“Monday Week 10, 1997 4:00:00” @“DayOfWeek 5 Week 12, 1998, 12:00:00”
		Day of Week	@“Monday”, @“DayOfWeek 7”, @“Next DayOfWeek”
		Day of Week and Time	@“Monday 4:12”, @“4:12 DayOfWeek 1”, @“4:12 Finish DayOfWeek”
4. Day, year	Year, Day of Year, Hour, Minute, Second		@“DayOfYear 227, 1997 4:00:00”
		Day of Year	@“DayOfYear 23”, @“Start DayOfYear”
		Day of Year and Time	@“DayOfYear 23 4:12”, @“4:12 Current DayOfYear”

Thus, for the first format, you can specify almost any component. For the second format, you can only specify the timestep and nothing else. So you *cannot* specify @“Timestep 12, Year 1997”; an error would return.

As you specify components, you only specify as much as is needed to uniquely reference the date. So, if in a boolean comparison you are comparing a date to see it is in 1997, you would enter (date == @“Year 1997”). But, if you want to compare if it is March of 1997, you enter @“March 1997” but do not include the word “Year” before 1997.

The list of symbolic specifiers include the following: Min, Max, Start, Finish, Previous, Current, and Next. Any of these specifiers can be used in the examples above.

If no time (hours, minutes, seconds) is specified at all, it defaults to 24:00:00. If a time is specified, it may appear either before or after the specification of the Month, Day, and Year. Times may be specified with or without seconds, if they are not specified then they default to 0 (e.g., both 17:23 and 17:23:00 refer to the same time of day).

> **Note:** Symbolic datetimes used in the RPL expression of a series expression slot are based on the slot timestep, which may be different from the run timestep.

Examples

Mixing of symbolic datetime elements results in an almost infinite number of datetime specifications. Table 3.3 lists examples for some of these acceptable fully specified expressions and their interpretations. The symbolic datetime elements shown can be combined into other forms not explicitly enumerated.

Table 3.4 lists partially specified datetime expressions.

> **Note:** These tables do not capture all the possible combinations of partially specified datetime expressions.

Table 3.3

Fully Specified Datetime Expression	Interpretation
@“4/01/1996 14:00:00”	April 1, 1996, 2 p.m.
@“April 1, 1996 14:00”	April 1, 1996, 2 p.m.
@“April 1, 1996 14:00:00”	April 1, 1996, 2 p.m.
@“t” or @“Current Timestep”	the current timestep
@“t-1” or @“Previous Timestep”	the previous timestep
@“t+1” or @“Next Timestep”	the next timestep
@“Start Timestep”	the begin timestep in the run or the begin timestep in an expression slot (in the evaluation range).
@“Finish Timestep”	the finish timestep in the run or the last timestep in an expression slot (in the evaluation range).
@“April 1, Current Year 14:00:00”	April 1 of the current year, 2 p.m.
@“April 1, 1996 Current Hour:00:00”	April 1, 1996 at the hour of the current timestep.
@“April Next DayOfMonth, 1996 Previous Hour:00:00”	Depends on the current timestep
@“7/22/1997 1:34:00 + 2 Days”	the date/time which is 48 hours after the given date (i.e., 7/24/97 1:34:00)
@“+ 25”	the date/time 25 timesteps beyond the current timestep
@“Next Timestep - 1”	Confusing way to refer to current timestep
@“2 hours before Current Timestep”	2 hours before current timestep

Table 3.4

Partially Specified Datetime Expression	Interpretation
@“Year 1997”	the year specified as 1997
@“Previous Year”	the year before the current year
@“Current Year”	the year specified as the year of the current timestep
@“Month 4”	the month specified as April
@“April”	the month specified as April
@“April 22”	the 22nd day in April
@“6:00 April 22”	6:00 AM on April 22
@“April 22 6:00”	6:00 AM on April 22
@“6:00”	6:00 AM
@“DayOfMonth 4”	the day of the month specified as the 4th
@“Next DayOfMonth”	the day of the month specified as the day after the day of the current timestep
@“Max DayOfMonth”	the day of the month that is the last day in the month
@“Monday”	the day of week specified as Monday

Table 3.4

Partially Specified Datetime Expression	Interpretation
@“WeekOfYear 2”	the week of year specified as the second
@“DayOfWeek 2”	the day of week specified as Monday
@“6:00 Previous DayOfWeek”	6:00 AM on the previous day of the week
@“DayOfYear 157”	the day of year specified as 157
@“Max DayOfYear”	the day corresponding to the last day of the year

Datetime Math

When performing addition or subtraction math on datetime values (for example, adding three timesteps to the current timestep), there are three approaches that can be used.

Math Within a Datetime Specification

Within a datetime literal, it is possible to add or subtract integral values from a base time. Thus, possible specifications for a datetime value three timesteps beyond the current timestep include the following:

- @“t + 3” or
- @“t + 3 timestep” or
- @“t + 3 day” (if the timestep is 1 Day) or
- @“t + 72 hour” (if the timestep is 1 Day)

The list of time units that are available in this context are (case *insensitive*):

- Minute
- Hour
- Day
- Week
- Month
- Year

Note: The date time math occurs within the quoted literal expression, which cannot include references to variables, that the integral increment is assumed to have units of “timestep” when the units are unspecified, and that the latter two specifications are less general because they assume that the model has a 1 Day timestep.

Note: Date time math in the RPL expression of a series expression slot uses the slot’s timestep, which may differ from the run timestep.

This approach to datetime math is probably the easiest to read, but is of course only useful when literal specification is possible. For example, when one would like to increment a base datetime by a number of timesteps which is itself the result of expression evaluation, then one of the alternative approaches to datetime math is required.

Mathematical Expressions Involving a Datetime Operand

A numeric value can be added to or subtracted from a datetime value. Continuing the example from above, the following expressions would evaluate to three timesteps beyond the current timestep in a model with a 1 Day timestep:

- @“t” + 72 “hour” or
- @“t” + 3 “day”

The list of time units that are available in this context are (case *sensitive*):

- min, minute
- hr, hour
- day
- week

Note: Within this context, “Month” and “Year” cannot be entered as they are not valid (See “Allow Literal RPL Values to Have Variable Length Time Units Setting” on page 129 for an exception). These literal values are ambiguous as they have a variable length. If you are trying to compute a flow by dividing a volume by 1 “month”, you should specify which month to use or the number of days to assume. For example, convert the volume to a flow using VolumeToFlow and specify the month or divide by GetDaysInMonth and specify the desired month. For non constant timestep lengths, the best approach is to stay in volumes as long as possible, and then convert to a flow when ready to set the value on a flow type slot.

Here are the math operations supported for datetime values:

- <DATETIME> + <NUMERIC> results in a <DATETIME>
- <NUMERIC> + <DATETIME> results in a <DATETIME>
- <DATETIME> - <NUMERIC> results in a <DATETIME>
- <DATETIME> - <DATETIME> results in a <NUMERIC> (with units of time).

In such expressions, the numeric value must have units of type Time.

Note: Timestep is not a legal unit of Time, so this type of approach could not be used when one would like to add some number of timesteps in a model whose timestep increment is not fixed (i.e., 1 Month or 1 Year timesteps).

The following operations are not supported as the result is undefined:

- <NUMERIC> - <DATETIME>, <DATETIME> + <DATETIME>

Note: Datetime math with partially specified datetime expressions is not supported. For example, the expression @”1:00” + 1 “hr” would cause the run to abort with an error message.

Using the OffsetDate function

The OffsetDate predefined function allows you to specify the increment and length of timestep to use for the addition or subtraction; see “OffsetDate” on page 258 for details. Thus, you could enter the following:

- OffsetDate(@ “t”, 1, “1 Days”)

This function is most useful for variable length timesteps (1 Month, 1 Year).

STRING

The string expression data type is any text surrounded by double quotes (“ and”). An unspecified string expression may be entered directly in expression by double-clicking the <string expr>, and typing in the string surrounded by double quotes. String expressions may contain any combination of letters, numbers and punctuation except double quotes.

OBJECT

The object expression data type is used to reference objects in the currently loaded model. An unspecified object expression may be completed by double-clicking the <object expr> and typing in the object name in double quotes and preceded by a percent symbol. When an object expression is typed directly into a block or function, the object name may be entered without quotes if the object name contains no spaces or special punctuation characters. When quotes are omitted, they will be filled in automatically.

It is highly recommended that you do not type object names directly into blocks and functions. The potential for error is great. An extra space or an incorrectly capitalized letter will invalidate the object expression. There is, however, a convenient and foolproof way to enter this data. Most commonly, an unspecified object expression is completed by using the Object Selector in the RPL Palette or by typing in a variable name whose type is an object expression.

Table 3.5 lists examples of object expressions entered directly into the Editor.

Table 3.5

Exact Text Typed Into Textfield	Resulting Display
%“Lake Mead”	Lake Mead
% DataObj	DataObj
% Havasu Diversion	“parse error” due to space

In contexts where an object expression is used as part of a complete slot specification, the object expression must be followed by a <string expression> indicating the slot.

Example 3.4

The Pool Elevation of Lake Powell could be filled into:

```
<object expr>.<string expr>
```

and result in the following expression:

```
Lake Powell.”Pool Elevation”
```

> **Note:** For Expression Slots and Object Level Account Methods, you can use the keyword ThisObject to access the containing object. For example, to get the slot named DailyTotals from this data object, you could create the expression shown in the image.

ThisObject . "DailyTotals" [ ]

SLOT

The slot expression data type is used to specify a specific slot on an object of the currently-loaded model. The expression must include the object on which the slot resides as well as the slot name itself. An unspecified slot expression may be completed by double-clicking the <slot expr>, typing in the object name, a dot, and the slot name, all in double quotes and preceded by a dollar symbol. When a slot expression is typed directly into a block or function, the object.slot name may be entered without quotes if neither of the names contain spaces or special punctuation characters. Regardless of how a slot expression is entered, the dollar sign and quotes are automatically omitted in the resulting display.

Again, it is highly recommended that you do not type object.slot names directly into blocks and functions. The same convenient and foolproof way to enter object data is available to enter object.slot data. An unspecified slot expression may be completed by using the Slot Selector in the RPL Palette or by typing in a variable name whose type is a slot expression. The Slot Selector is an interface similar to the one used elsewhere in RiverWare, which formats a selected object and slot correctly and inserts it into the slot expression.

Table 3.6 lists examples of slot expressions entered directly into the editor.

Table 3.6

Exact Text Typed Into Textfield	Resulting Display
$“Lake Mead.Outflow”	Lake Mead.Outflow
$ DataObj.target	DataObj.target
$ Havasu Diversion.Inflow	“parse error” due to space
$ “Havasu Diversion”.Inflow	“parse error”

LIST

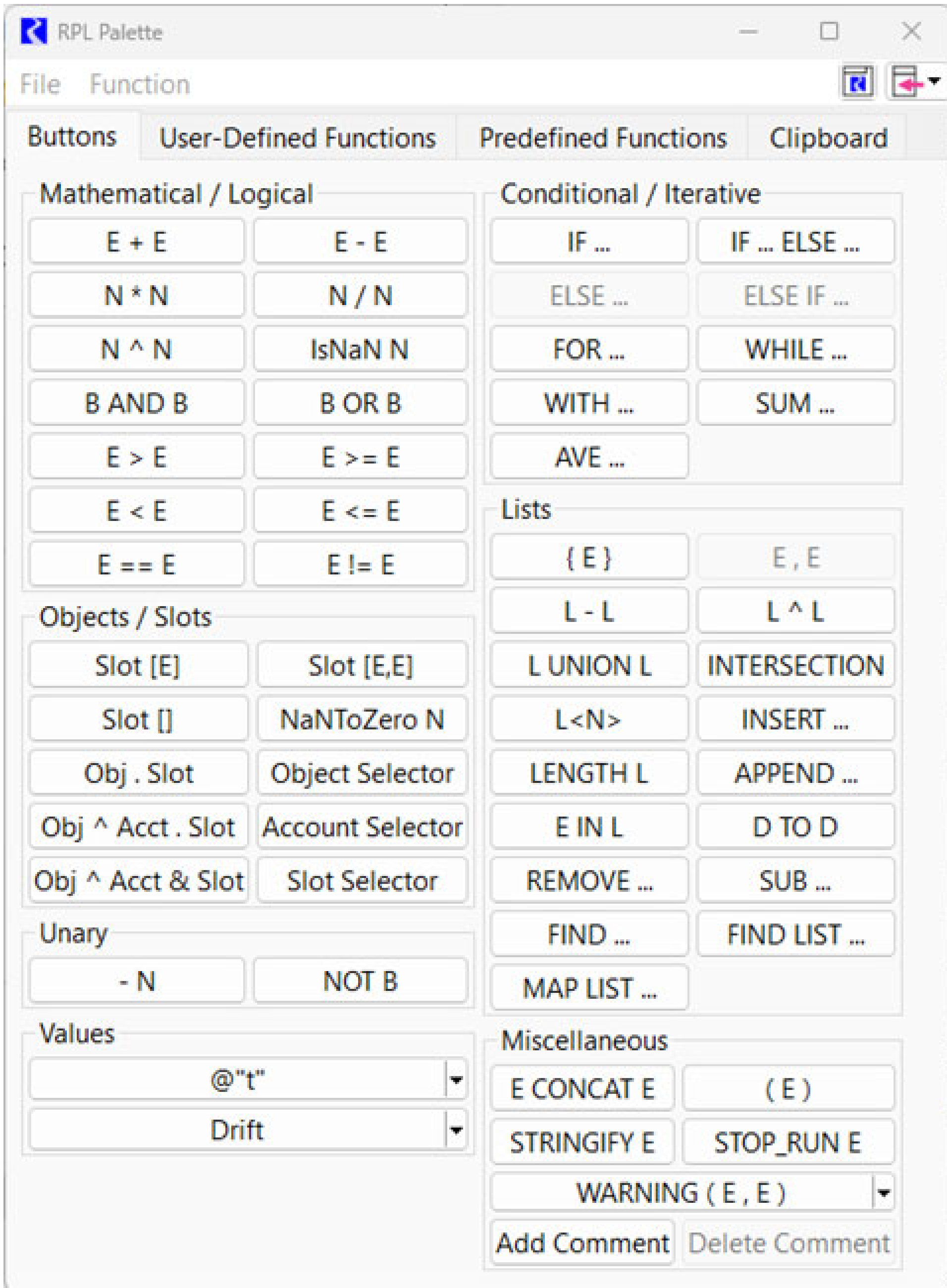
List expressions are ordered collections of other expression data types. Lists can contain zero or more elements. Elements may be of different types, and lists may even contain other lists. An unspecified list expression may be completed by double-clicking the <list expr> and typing in a comma-separated list of other expressions enclosed in curly braces ({ and }). As with object and slot expressions, it is highly recommended that you do not type list expressions directly into the Editor. The Palette contains a multitude of functions for creating and retrieving information from lists. Examples of valid lists expressions, before the expression language expands their elements, include:

```
{ 1561 [“ft”], 1573 [“ft”], 1589 [“ft”], 3800.05 [cfs] }
{ @“April 15, 1999” }
{ }
{ %“Powell”, 10 [“cfs”], True }
{ { %“Mead”, “Max Outflow”, TRUE }, { $“Inflow” }, False }
```

When an element of a list is read, its expression type must match the type expected by the reading function. If a function finds an object where a datetime is expected, the run is halted and an error is posted. The flexibility of lists is an advantage as long as the types of their contents are known, and they are used properly within blocks and functions. Because of their flexibility, however, the types of their elements cannot always be determined prior to a model run. For this reason, list type-checking is only done during execution, and errors in configuration are not caught until a run is in progress.

RPL Palette

The RPL Palette is the dialog from which the operators and functions are selected to build a block or user function. Using the Palette avoids having to type most expressions directly into the editor, thus limiting the potential for errors.



The Palette consist of four tabs, one for the buttons, one tab for **User-Defined Functions**, one tab for the **Predefined Functions**, and one tab for the **Clipboard** as follows.

Buttons Tab

On the **Palette Buttons** tab, each of the buttons represents an operation. The buttons use the following abbreviations:

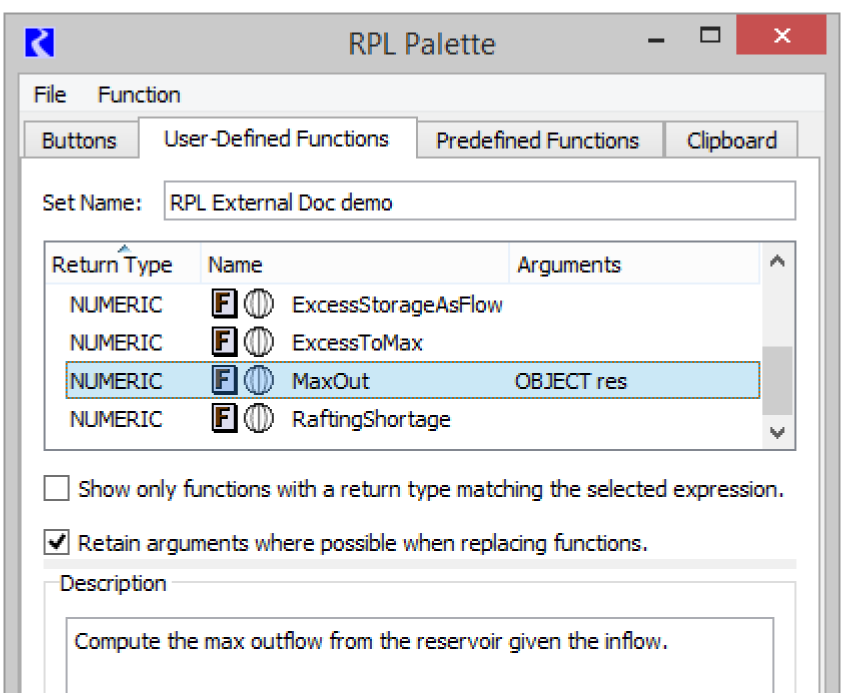
- B: BOOLEAN
- D: DATETIME
- E: Expression; can be more than one type
- L: LIST
- N: NUMERIC
- Obj: OBJECT
- Slot: SLOT

These buttons represent operations which evaluate to one of the expression data types mentioned above. Buttons on the RPL Palette are enabled and disabled dynamically. When an expression is highlighted in the Editor, the Palette buttons that satisfy the expected data type are enabled. See “RPL Palette Buttons” on page 109 for details.

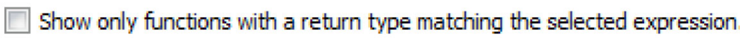
User-Defined Functions Tab

The **User-Defined Functions** tab shows available user functions in the RPL set or any opened Global Function sets. Select a function for use by double-clicking the function name to replace the selected expression with the function. Figure 3.1 illustrates this tab.

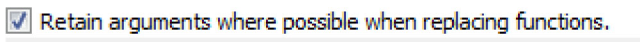
Figure 3.1 User-Defined functions tab of the RPL Palette



There is a toggle to show only functions which have the return type of the selected RPL expression:



In addition, there is a toggle to retain the function arguments, when possible, when replacing an existing function.



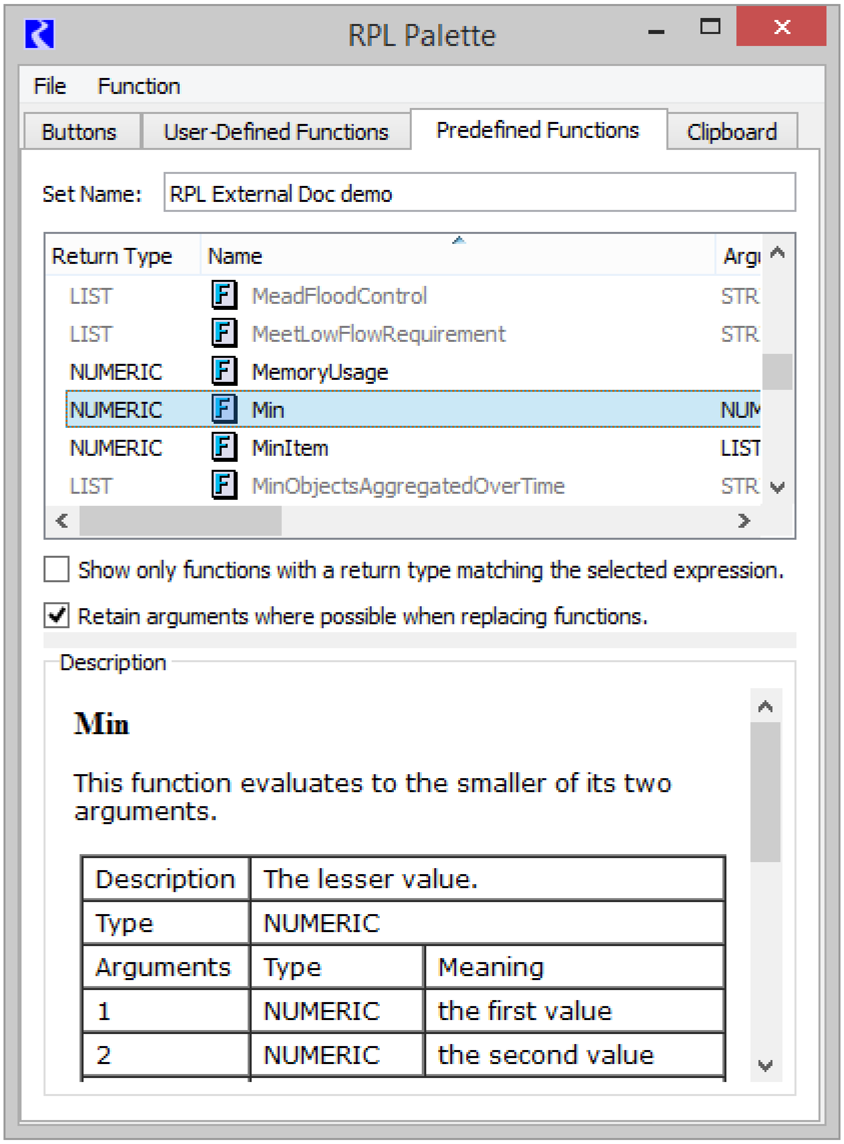
When checked, the newly selected function will retain arguments from left to right as long as the types match. If an argument's type does not match, or if there are not enough existing arguments, the remaining arguments in the new function will remain as empty expressions.

Use the **Function**, then **Show Description** menu to show the selected function's Description as specified on the function's RPL editor.

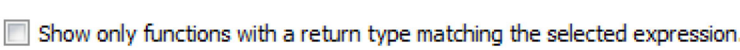
Predefined Functions Tab

This tab shows the list of predefined functions available in RiverWare. See "About RPL Functions" on page 65 for details. Shown are sortable columns for the Return Type, Name, and the list of Arguments for each function. Figure 3.2 illustrates this tab.

Figure 3.2 Screenshot of the User-Defined functions tab of the RPL Palette

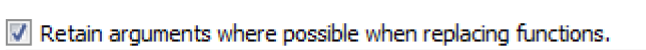


There is a toggle to show only functions which have the return type of the selected RPL expression:



When the check box is cleared, all predefined functions are shown even if no expression is selected.

This tab also has a toggle to retain function arguments. as possible, when replacing an existing function. The behavior is the same as described above for the user-defined function tab.



Use the **Function**, then **Show Description** menu to show the documentation for the selected function. See “RPL Predefined Functions” on page 147 for the same documentation.

Note: References are shown as blue but do not hyperlink elsewhere.

Included is a description, return type, list of the arguments and what they mean, description of evaluation, comments, syntax example and return example.

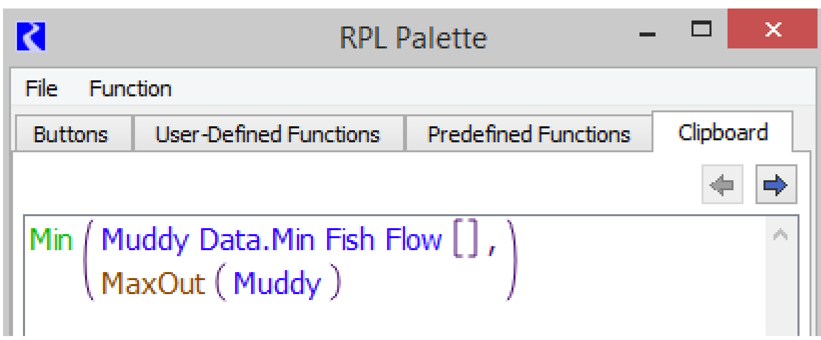
Clipboard Tab

The clipboard tab is a temporary location to hold copied expressions that you can then paste over other expressions. This is particularly useful in editing RPL expressions for readability or performance.

The clipboard maintains a list of the last 100 expressions copied since the workspace was last cleared. When you copy a RPL expression (except from the clipboard itself), a copy of that expression is added to the clipboard as the most recent expression. Use the Previous and Next buttons to view the entire copy history, and select and copy an expression or any subexpression in this history. Then, paste in other RPL locations to use that expression.

> **Tip:** Copy and paste works across RiverWare sessions allowing you to copy a RPL expression in one model and paste it in a RPL expression in another model. The Clipboard tab tracks the last copied expression in any currently open RiverWare session.

Figure 3.3 Screenshot of the Clipboard tab on the RPL Palette



The clipboard display is read-only, and supports the following operations:

- Copy (Ctrl+C) - places the selected expression in the copy buffer
- Previous (Alt+Left) - displays the previous copied expression
- Next (Alt+Right) - displays the following copied expression

> **Note:** The clipboard only tracks copied expressions. Other RPL items, for example, copied statements, function argument declarations, or function constraints are not tracked.

RPL Palette Buttons

This section describes the RPL Palette buttons and their operations.

Mathematical Operation Buttons

Button	Evaluates to	Unspecified Form and Description
E + E	NUMERIC or DATETIME	<expr> + <expr> Addition of two expressions. The two arguments can be numeric or fully specified datetime expressions. If numeric expressions are used, they must be of the same unit type. See “Mathematical Expressions Involving a Datetime Operand” on page 102 for details on using this operation with datetimes.
E - E	NUMERIC or DATETIME	<expr> - <expr> Subtraction of two expressions. The two arguments can be numeric or fully specified datetime expressions. If numeric expressions are used, they must be of the same unit type. See “Mathematical Expressions Involving a Datetime Operand” on page 102 for details on using this operation with datetimes.
N * N	NUMERIC	<numeric expr> x <numeric expr> Multiplication of two numeric expressions.
N / N	NUMERIC	<numeric expr> / <numeric expr> Division of two numeric expressions.
N ^ N	NUMERIC	<numeric expr> ^ <numeric expr> Exponentiation of one numeric expression (the base) to a power of another numeric expression (the exponent). The exponent is truncated to an integer. (For non-integer exponents, use the Exp RPL Predefined Function; see “Exp” on page 175.) The units of the base are raised to the power of the exponent.

Logical Operation Buttons

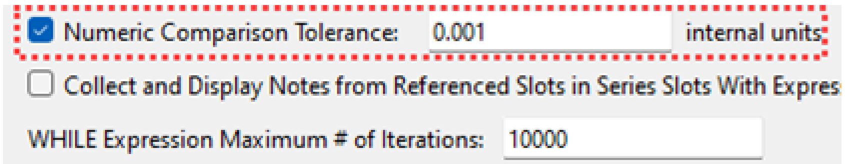
Button	Evaluates to	Unspecified Form and Description
IsNaN N	BOOLEAN	IsNaN <numeric expr> Logical validity check for numeric data types. The result is TRUE if the numeric expression evaluates to a NaN, i.e. Not a Number. The result is FALSE if the numeric expression has a value.
B AND B	BOOLEAN	<boolean expr> AND <boolean expr> Logical AND comparison of two expressions. The result is TRUE if both expressions are TRUE. The result is FALSE if one expression is TRUE and the other is FALSE. **Note:** If the first expression is FALSE, the entire AND is FALSE and the second expression is not evaluated.
B OR B	BOOLEAN	<boolean expr> OR <boolean expr> Logical OR comparison of two expressions. The result is TRUE if either expression is TRUE. Otherwise it is FALSE.

Button	Evaluates to	Unspecified Form and Description
E > E	BOOLEAN	<expr> > <expr> Greater than comparison. The result is TRUE if the first expression is strictly greater than the second expression. Otherwise it is FALSE. The two arguments can be numeric expressions of the same unit type or datetime expressions (fully or partially specified). Numeric expressions are automatically converted to common units before comparison. A datetime expression is considered greater if it is later in time. See below for more information on tolerance during the comparison.
E >= E	BOOLEAN	<expr> >= <expr> Greater than or equal comparison. The result is TRUE if the first expression is greater than or equal to the second expression. Otherwise it is FALSE. The two arguments can be numeric expressions of the same unit type or datetime expressions (fully or partially specified). Numeric expressions are automatically converted to common units before comparison. A datetime expression is considered greater if it is later in time. See below for more information on tolerance during the comparison.
E < E	BOOLEAN	<expr> < <expr> Less than comparison. The result is TRUE if the first expression is strictly less than the second expression. Otherwise it is FALSE. The two arguments can be numeric expressions of the same unit type or datetime expressions (fully or partially specified) Numeric expressions are automatically converted to common units before comparison. A datetime expression is considered less than if it is earlier in time. See below for more information on tolerance during the comparison.

Button	Evaluates to	Unspecified Form and Description
E <= E	BOOLEAN	`<expr> <= <expr>` Less than or equal to comparison. The result is TRUE if the first expression is less than or equal to the second expression. Otherwise it is FALSE. The two arguments can be numeric expressions of the same unit type or datetime expressions (fully or partially specified). Numeric expressions are automatically converted to common units before comparison. A datetime expression is considered less than if it is earlier in time. See below for more information on tolerance during the comparison.
E == E	BOOLEAN	`<expr> == <expr>` Equality comparison. The result is TRUE if the first expression is equal to the second expression. Otherwise it is FALSE. The two arguments can be numeric expressions (not necessarily of the same unit type), fully or partially specified datetime expressions, object expressions, slot expressions, boolean expressions, string expressions, or list expressions. Numeric expressions of the same unit type are automatically converted to common units before comparison. A fully specified datetime expression can be compared to a partially specified datetime expression; in this case, only the largest time unit is compared. See below for more information on tolerance during the comparison.
E != E	BOOLEAN	`<expr> != <expr>` Inequality comparison. The result is TRUE if the first expression is not equal to the second expression. Otherwise it is FALSE. The two arguments can be numeric expressions of the same unit type, fully or partially specified datetime expressions, object expressions, slot expressions, boolean expressions, string expressions, and list expressions. Numeric expressions are automatically converted to common units before comparison. Datetime expressions are considered greater as they are later in time. A fully specified datetime expression can be compared to a partially specified datetime expression; in this case, only the largest time unit is compared. See below for more information on tolerance during the comparison.

Setting Tolerance for Use in the Logical Comparison Operators

In the above operators, a comparison is made between two values. If the values are numeric, there is the possibility that due to unit conversion or other internal numerical computation, RiverWare could try to compare two numbers that are, in all practical purposes the same, yet are different in the comparison. For example, a RPL expression says (0.0 “cfs” == Flow[ ]) and the Flow[ ] evaluates to $1X10^{-13}$ cfs due to the addition of two double precision values. In this example, the statement will return FALSE but probably should return TRUE. To avoid this situation, you are able to define a tolerance value. From the main RiverWare workspace, select **Policy**, then **RPL Parameters** to open the RPL Parameter in the Settings Manager.



Select the **Numeric Comparison Tolerance** toggle. The box becomes active with a default value of zero. Enter a new value in the box to specify the tolerance. The tolerance represents an absolute value in standard units. Thus, in the example above, if the tolerance is 0.001, it represents that

Abs(0.0 “cfs” - Flow[]) <= 0.001 cms for the comparison to return TRUE. In other words, if two values are within the comparison tolerance of one another, then they are considered equal for the purposes of RPL comparison operations.

Table 3.7 provides details about how the comparison tolerance is applied for each of the RPL comparison operators.

Table 3.7

Operator	Application of Tolerance	RPL Example (Tolerance = 0.01)	Comparison With Tolerance Applied	Result
E == E	|A - B| <= Tolerance	5.00 cms == 5.00 cms	|5.00 cms - 5.00 cms| <= 0.01 cms	TRUE
		5.00 cms == 4.99 cms	|5.00 cms - 4.99 cms| <= 0.01 cms	TRUE
		4.98 cms == 5.00 cms	|4.98 cms - 5.00 cms| <= 0.01 cms	FALSE
E != E	|A - B| > Tolerance	5.00 cms != 5.00 cms	|5.00 cms - 5.00 cms| > 0.01 cms	FALSE
		5.00 cms != 4.99 cms	|5.00 cms - 4.99 cms| > 0.01 cms	FALSE
		4.98 cms != 5.00 cms	|4.98 cms - 5.00 cms| > 0.01 cms	TRUE
E > E	A > B + Tolerance	5.02 cms > 5.00 cms	5.02 cms > 5.01 cms	TRUE
		5.01 cms > 5.00 cms	5.01 cms > 5.01 cms	FALSE
		5.00 cms > 5.00 cms	5.00 cms > 5.01 cms	FALSE
E >= E	A + Tolerance >= B	5.01 cms >= 5.00 cms	5.02 cms >= 5.00 cms	TRUE
		4.99 cms >= 5.00 cms	5.00 cms >= 5.00 cms	TRUE
		4.98 cms >= 5.00 cms	4.99 cms >= 5.00 cms	FALSE
E < E	A + Tolerance < B	4.98 cms < 5.00 cms	4.99 cms < 5.00 cms	TRUE
		4.99 cms < 5.00 cms	5.00 cms < 5.00 cms	FALSE
		5.00 cms < 5.00 cms	5.01 cms < 5.00 cms	FALSE

Table 3.7

Operator	Application of Tolerance	RPL Example (Tolerance = 0.01)	Comparison With Tolerance Applied	Result
E <= E	A <= B + Tolerance	4.99 cms <= 5.00 cms	4.99 cms <= 5.01 cms	TRUE
		5.00 cms < = 4.99 cms	5.00 cms <= 5.00 cms	TRUE
		5.00 cms <= 4.98 cms	5.00 cms <= 4.99 cms	FALSE

The RPL Numeric Comparison Tolerance parameter is saved with the model file and is applied to all RPL sets used by the model including Rulesets, User Defined Accounting Method sets, Optimization Goalsets, Initialization Rules, Iterative MRM sets, Expression Slot sets and Global Functions sets.

Object and Slot Lookup and Assignment Buttons

Button	Evaluates to	Unspecified Form and Description
Slot [E]	NUMERIC	`<expr> [ <expr> ]` Series slot value at a particular timestep or slot value at a particular row. The first unspecified <expr> must be completed by an expression which evaluates to a specific slot on an object. The second <expr> must be a fully specified datetime which lies on an increment of the model run timestep or a row on the slot.
Slot [E, E]	NUMERIC	`<expr> [ <expr>, <expr> ]` Table slot value in a particular row and column. The first unspecified <expr> must be completed by an expression which evaluates to a specific table slot on an object. The two comma-separated <expr> are the row and column of the table slot, respectively. The row and column may each be specified as: • A **zero**-based numeric value with any units.The numeric is rounded down to the nearest integer in those units and the integer is used in the lookup, or • A string expression which matches the column or row label. OR Agg series slot or periodic slot value for a particular date and a particular column. The first unspecified <expr> must be completed by an expression that evaluates to a specific slot on an object. The two comma-separated <expr> are the date and column of the slot, respectively. The column may be specified as a zero-based numeric value with units of [NONE] or a string expression which matches the column label. See “Referencing Periodic Slots in RPL” in *User Interface* for details.
Slot [ ]	NUMERIC	`<expr> [  ]` Series slot value at the current timestep or scalar slot value. The unspecified <expr> must be completed by an expression which evaluates to a specific slot on an object.

Button	Evaluates to	Unspecified Form and Description

NaNToZero N	NUMERIC	NaNToZero <expr>
		This operator has a single numeric sub expression; if that sub expression is a lookup of an invalid value (NaN), then NaNToZero evaluates to 0.0 in the units of the lookup slot, otherwise it simply returns the value unchanged. This operator provides a simple and efficient way to treat a missing slot value as zero, a common requirement of water accounting policy.
		To illustrate, the following expression
		`NaNToZero "Res A.Outflow" [ ]`
		is equivalent to
		`IF ( IsNaN "Res A.Outflow" [ ] )`
		`   0.0 [<Res A.Outflow's units>]`
		`ELSE`
		`   "Res A.Outflow" [ ]`
		`END IF`
		Note: Unlike the IsNaN operator, this operator is only useful when the sub-expression is an object/slot lookup expression; if any other type of numeric sub expression encounters a NaN during evaluation, then evaluation of the entire containing expression will halt as usual. For example, if we assume that "Res A.Outflow" has units of cms but has no valid values, then the expression
		`NaNToZero ( "Res A.Outflow" [] ) + 0.0 [cms]`
		would evaluate to 0.0 [cms], but the expression
		`NaNToZero ( "Res A.Outflow" [] + 0.0 [cms] )`
		would not evaluate successfully; it would terminate when the invalid value on Res A was encountered.

Obj . Slot	SLOT	`<object expr> . <string expr>` Object and slot expression. This expression can be used to specify the object and slot required by the three slot lookup/assignment expressions above. The object expression must be an object in the model and the string expression must be the name of a slot, exactly as it appears on the object.
Obj ^ Acct . Slot	SLOT	`<OBJECT object> ^ <STRING account name> . <STRING slot name>` Object, account and accounting slot expression. This expression can be used to specify the object, account, and slot required by the three slot lookup/assignment expressions. The object expression must be an object in the model. The account is a string expression that returns the account name, for example “Contractor 1” The slot string expression must be the full name of an accounting slot, for example “Storage”. This ternary operator is preferred in terms of readability and performance over the Obj ^ Acct & Slot operator listed below.

Button	Evaluates to	Unspecified Form and Description
Obj ^ Acct & Slot	SLOT	`<object expr> ^ <string expr>` Object and accounting slot expression. This expression can be used to specify the object and slot required by the three slot lookup/assignment expressions above. The object expression must be an object in the model and the string expression must be the full name of an accounting slot, in the form: “account name.slot name”, for example “Contractor 1.Storage”. This operator was one of the original ways to specify an object, account and slot. Consider using the Obj ^ Acct . Slot ternary operator, described above, for improved performance and readability.
Object Selector	OBJECT	Invokes the object selector dialog. This button invokes the object selector to choose a single object in the model.
Account Selector	STRING	Invokes the account selector dialog. This button invokes the account selector to choose a single account as a string.
Slot Selector	SLOT or STRING	Invokes the slot selector dialog. This button invokes the slot selector to choose a slot. The slot can be on an object or on an account where applicable. In some contexts, the button evaluates to a string which represents the name of a slot, for example “Storage”.

Unary Operation Buttons

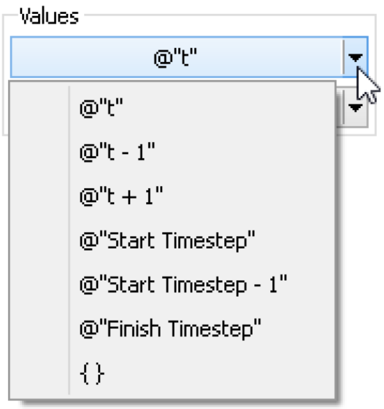
Button	Evaluates to	Unspecified Form and Description
- N	NUMERIC	- <numeric expr> Reverse the sign of a numeric expression. Magnitude and units are maintained, but a positive value becomes negative and a negative value becomes positive.
NOT B	BOOLEAN	NOT <boolean expr> Reverse the value of a boolean expression. TRUE becomes FALSE and FALSE becomes TRUE.

Values

The Values section is used to access common values and set flags on slots.

Buttons for Common Values

The common values buttons are used to access values that have traditionally been typed by the user. These include common DATETIME values and list expressions.



- @“t” - Current Timestep
- @“t-1” - Previous Timestep
- @“t+1” - Next Timestep
- @“Start Timestep” - First timestep in the run
- @“Start Timestep - 1” - Initial timestep
- @“Finish Timestep” - Final Timestep in the run period
- { } - Empty list expression

A similar list of common values can be found by right-clicking an expression and choosing the **Common Values** menu.

Buttons for Setting Flags on Slots

The following RPL operations can only be used to directly set a slot value. They cannot be passed to functions or referenced in block or function logic. They cannot replace any unspecified NUMERIC expression; they can only be used to set a slot.

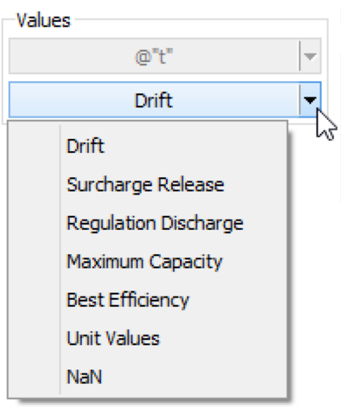


Table 3.8

Menu Button	Description	Notes
Drift	Used to set the DRIFT (D) flag on the Regulated Spill or Bypass slots on reservoir objects	The behavior of these operators is identical to the user setting the particular flag through the user interface. The only exception in this case is that the slot can be overwritten by a higher priority rule. When these operators are used, the slot will have both the R flag and the flag associated with the palette button selected. When the object dispatches, the slot will receive the appropriate value. The presence of the these flags will cause the slot to be considered as a known value (for choosing a dispatch method) even though it will not actually have a value until after the object dispatches.
Maximum Capacity	Used to set the MAX CAPACITY (M) flag on the Outflow or Energy slots on reservoir objects.	
Surcharge Release	Used to set the SURCHARGE RELEASE (S) flag on the Surcharge Release slot on reservoir objects.	
Best Efficiency	Used to set the BEST EFFICIENCY (B) flag on the Energy slot on reservoir objects.	
Regulation Discharge	Used to set the REGULATION_DISCHARGE (G) flag on the Reg Discharge Calculation slot on Control Point objects.	
Unit Values	Used to set the UNIT_VALUES (U) flag on Turbine Release or Energy slots on power reservoirs. This flag is used (with the Unit Power Table method) to specify that either Unit Turbine Release or Unit Energy will be specified.	

Conditional and Iterative Operations Buttons

Button	Evaluates to	Unspecified Form and Description

IF	any	IF (<boolean expr>) THEN
		   <expr>
		END IF
		Adds a conditional expression with NO else clause. If the boolean expression evaluates to TRUE, the expression on the second line is evaluated. Otherwise it is not, and the entire conditional statement does not evaluate to a value.

IF ... ELSE	any	IF (<boolean expr>) THEN
		   <expr>
		ELSE
		   <expr>
		END IF
		Adds a conditional expression including an else clause. If the boolean expression evaluates to TRUE, the expression on the second line is evaluated. If the boolean expression evaluates to FALSE, the expression on the fourth line is evaluated. The entire conditional statement evaluates to one of the two expressions.

ELSE	any	IF (<boolean expr>) THEN
		   <expr>
		ELSE
		   <expr>
		END IF
		Adds an Else branch to a conditional expression. It is generated by highlighting the either the boolean condition <boolean expr> or consequence expression <expr> of an IF expression or ELSE IF expression and selecting the **ELSE** button.
		If the boolean expression evaluates to FALSE, the expression on the fourth line is evaluated. The entire conditional evaluates to one of the two expressions.

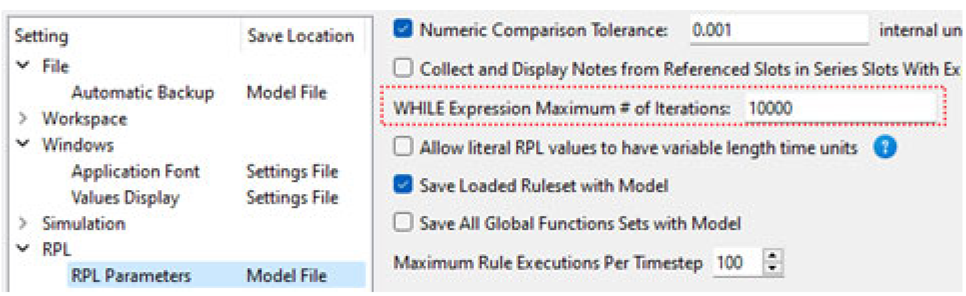
ELSE IF	any	IF (<boolean expr>) THEN
		   <expr>
		ELSE IF (<boolean expr>) THEN
		   <expr>
		ELSE IF (<boolean expr>) THEN
		   <expr>
		END IF
		Adds an ELSE IF branch to a conditional expression. It is generated by highlighting the boolean condition <boolean expr> or consequence <expr> of an IF or ELSE IF and selecting the **ELSE IF** button. The ELSE IF branch is added immediately after the branch containing the selection.
		In the example above, if the first boolean expression evaluates to FALSE, the boolean expression on the third line is evaluated. If that boolean is true, the expression on the fourth line is evaluated. You may add as many ELSE IF branches as needed.

Button	Evaluates to	Unspecified Form and Description

FOR	any	`FOR ( NUMERIC index IN <list expr> ) WITH NUMERIC result = <numeric expr> DO`
		`   result = <numeric expr>`
		`END FOR`
		Structure for looping over the items in a list. The looping variable, index, is set to the value of the next item in the list expression each time through the loop. The value variable, result, is initialized on the first line of the structure. Each time through the loop, the expression on the second line is evaluated and its value is set on the result variable. The previous value of the result variable may be used inside the expression on the second line. The items in the list and the result may be of any expression type even though the default structure uses the numeric type for the looping variable and the result. To change the type of the looping variable, index, and/or the value variable, result, double-click the **NUMERIC** element and enter the new expression type. The names of the looping and value variables may also be changed.
		There is also a FOR statement used to loop over a list and execute multiple statements. See "RPL Statements" on page 49 for details.

WITH	any	`WITH NUMERIC var = <numeric expr> DO`
		`   <expr>`
		`END WITH`
		Structure for defining a variable to be used multiple times within an expression. The variable, var, is set on the first line of the structure. This variable can be used as many times as desired within the expression(s) between the DO and END WITH. Because the variable is not recalculated each time it is used, this structure can make a block more efficient. The variable may be of any expression type even though the default structure uses the numeric type. To change the type of the variable, var, double-click the **NUMERIC** element and enter the new expression type. The name of the variable may also be changed.
		There is also a WITH statement used to set a temporary variable outside of multiple statements. See "RPL Statements" on page 49 for details.

Button	Evaluates to	Unspecified Form and Description

WHILE	any	WHILE (<boolean expr>) WITH NUMERIC result = <numeric expr> DO
		   result = <numeric expr>
		END WHILE
		Structure for looping as long as a condition is TRUE. The conditional boolean expression is evaluated prior to each loop. If the boolean expression is TRUE, the numeric expression on the second line is evaluated. If the boolean expression is FALSE, the structure stops looping and returns the value of the value variable. The value variable, result, is initialized on the first line of the structure. Each time through the loop, the expression on the second line is evaluated and its value is set on the result variable. The previous value of the result variable may be used inside the expression on the second line. The result may be of any expression type even though the default structure uses the numeric type. To change the type of the value variable, result, double-click the **NUMERIC** element and enter the new expression type. The name of the value variable may also be changed.
		When a WHILE expression is executed, it counts the number of times the body is evaluated, and fails with a message if the iteration count ever exceeds the value of the RPL parameter. The default max iterations is 10,000, but may be modified as needed in the RPL Parameters dialog. (From the main workspace, select **Policy**, then **RPL Parameters** to open the parameters in the Settings Manager).
		

SUM	NUMERIC	FOR (NUMERIC index IN <list expr>) SUM
		   <numeric expr>
		END FOR
		Specialized FOR loop structure for looping over the items in a list and summing a variable. For each item in the list, the looping variable, index, is set to the value of the next item in the list expression. Next, the expression on the second line is evaluated and its value is added to the result variable. The items in the list may be of any expression type even though the default structure uses the numeric type for the looping variable and the result. To change the type of the looping variable, index, double-click the **NUMERIC** element and enter the new expression type. The names of the looping variables may also be changed.

Button	Evaluates to	Unspecified Form and Description

AVE	NUMERIC	`FOR ( NUMERIC index IN <list expr> ) AVE`
		`   <numeric expr>`
		`END FOR`
		Specialized FOR loop structure for looping over the items in a list and averaging their values. For each item in the list, the looping variable, index, is set to the value of the next item in the list expression. Next, the expression on the second line is evaluated and its value is included in the total resulting average. The items in the list may be of any expression type even though the default structure uses the numeric type for the looping variable and the result. To change the type of the looping variable, index, double-click the **NUMERIC** element and enter the new expression type. The names of the looping variable may also be changed.

List Operation Buttons

Button	Evaluates to	Unspecified Form and Description
{ E }	LIST	`{ <expr> }` This button creates a list with a single item of an unspecified type.
E , E	any	`<expr>, <expr>` This button adds a new item of an unspecified type to an existing list. To use it, highlight one item and select the button. The new item is added immediately after the highlighted item in the list.
L - L	LIST	`<list expr> - <list expr>` The set difference expression takes two lists and returns a single list that contains the items that are in the left hand list but not in the right hand list. Duplicate items are removed.
L ^ L	LIST	`<list expr> ^ <list expr>` The set symmetric difference expression takes two lists and returns a single list that contains the items that are not in both lists. This is equivalent to the expression: (list1 U list2) - (list1 intersect list2). This is also equivalent to the expression: (list1 - list2) U (list2 - list1). Duplicate items are removed.
L UNION L	LIST	`<list expr> UNION <list expr>` The set union expression takes two lists and returns a single list that contains the items that are in either list or both lists. Duplicate items are removed.
INTERSECTION	LIST	`<list expr> INTERSECTION <list expr>` The set intersection expression takes two lists and returns a single list that contains the items that are in both lists. Duplicate items are removed.
L<N>	any	`<list expr>< <numeric expr> >` Evaluates to the expression located at the given numeric index in the given list. The index numeric expression is zero-based and has units of [NONE]. If the item at the given index in the list is not the expected type when the expression is evaluated or if there is no item at the given index in the list, the run aborts with an error.

Button	Evaluates to	Unspecified Form and Description
INSERT	LIST	`INSERT <expr> INTO <list expr>` Insertion of a new item of unspecified type into the first location of an existing list. If the list is initially empty, the new list contains only one item. If the list initially contains items, all items are shifted down to allow the new item to be inserted at the front of the list.
LENGTH L	NUMERIC	`LENGTH <list expr>` Determines the number of items in the list expression. The numeric value has units of [NONE].
APPEND	LIST	`APPEND <expr> ONTO <list expr>` Append a new item of unspecified type onto the end of an existing list. If the list is initially empty, the new list contains only one item. If the list initially contains items, the new item is appended as the last item of the list.
E IN L	BOOLEAN	`<expr> IN <list expr>` Tests for existence of the given expression as a member of the list expression. If the given expression is one of the items in the list, this function evaluates to TRUE. If the expression does not match any of the items of the list, the function is FALSE.
D TO D	LIST	`<datetime expr> TO <datetime expr>` Creates a list of datetimes beginning with the first datetime expression and ending on or before the second datetime expression. The datetime arguments must be fully specified. The interval between datetime items of the list is equal to the run timestep (or the slot’s timestep for an expression slot). If the second datetime argument does not correspond to a model run timestep, the last item of the list is the last model run timestep before the argument.
REMOVE	LIST	`REMOVE ITEM @INDEX <numeric expr> FROM <list expr>` Removes the item at a given index from an existing list. All of the items which followed the removed item in the list are shifted up. If there is no item at the given index when the expression is evaluated, the run aborts with an error.
SUB	LIST	`SUB <expr> FOR ITEM @INDEX <numeric expr> FROM <list expr>` Substitutes the item at a given index in an existing list with the given expression. If there is no item at the given index when the expression is evaluated, the run aborts with an error.

Button	Evaluates to	Unspecified Form and Description
FIND	NUMERIC	`FIND <expr> WITHIN <list expr>` This is used to find the index of a given item in a list. If the item is not contained in the list, -1 (negative one) is returned.

FIND LIST	LIST	`FIND LIST CONTAINING <expr> WITHIN <list expr>`
		Given a search value of any type and a list of lists, the Find List operator returns the first sublist in the input list that contains the search value.
		Example:
		`FIND LIST CONTAINING @“March 1” WITHIN {{@“February”, 2.0, TRUE},{@“March”, 3.0, TRUE},{@“April”, 4.0, FALSE}}`
		Returns:
		`{@“March”, 3.0, TRUE}`
		The following are additional details on this operator:
		• For each list item in the input list, the search value is compared with each value in turn; the search value can match any item within the list item. Note that the search value will not match any more deeply nested items.
		• The search value does not need to be the first item in the sublist; it can be anywhere in the sublist.
		• The *first* matched list is returned. There could be expressions that also match later in the list but they are not considered once the expression is found.
		• An item is “found” using the same logic as the equality operator. See E == E, page 112. For example, the expression above looks for @”March 1” and found the list containing March. In this DATETIME example, the months were compared.
		• All items in the input list must themselves be lists.

MAP LIST	LIST	`MAPLIST (NUMERIC index IN <list expr>) DO`
		`   <expr>`
		`END MAPLIST`
		This expression is used to take a list and perform some action to each element in the list thereby resulting in a new, modified list. This expression was developed primarily for performance reasons. Prior to the existence of this function, the user would use a FOR loop to rotate through the elements of a list, modify them, and create a new list with the modified elements. With the MAP LIST expression, this is much more efficient and more readable.

Miscellaneous Buttons

Button	Evaluates to	Unspecified Form and Description

E CONCAT E	LIST
	or
	STRING
		`<expr> CONCAT <expr>`
		Concatenates two expressions into one. If the two expressions are of type LIST, the resulting expression is a single list containing all of the items of the two original lists. The order of the items is preserved, with the first list's items before the second list's items. If the two expressions are of any other type, the resulting expression is a string made up by concatenating the STRINGIFY'd second expression onto the end of the STRINGIFY'd first expression. See the description of STRINGIFY below.
		Note: It is not necessary to explicitly STRINGIFY an expression before using it in a CONCAT expression. If the expression is not already a STRING, it will be automatically converted. Thus,
		`1000 CONCAT “cfs”`
		is equivalent to
		`(STRINGIFY 1000) CONCAT “cfs”`
		but the first expression is cleaner and more efficient

(E)	any	`( <expr> )` Adds parenthesis around any expression. Parenthesis can make complicated expressions more readable. Parenthesis may also be required to remove ambiguity in the order of expression evaluation.
STRINGIFY E	STRING	`STRINGIFY <expr>` Converts any expression into a string. The STRING representations of other expression types are: NUMERIC => “*number* [*units*]” or “*number*” if units are {NONE} DATETIME => “*hours*:*minutes month day*, *year*” or less, depending on model run timestep BOOLEAN => “TRUE” or “FALSE” OBJECT => “*object name*” SLOT => “*object name*.*slot name*” LIST => “{ *item*, *item*, *item* ... }”
STOP_RUN E	Aborts the Run	`STOP_RUN <expr>` This operator takes any expression type as an argument. When it is evaluated, it aborts the run with an error message which contains the argument as part of the message. If executed from within an iterative MRM rule, it aborts the MRM run.

Button	Evaluates to	Unspecified Form and Description

WARNING(E, E)
NOTICE(E, E)
ALERT(E, E)
	datatype of the second expression	`WARNING(<expr>,<expr>)`
		`NOTICE(<expr>,<expr>)`
		`ALERT(<expr>,<expr>)`
		When one of these expressions is executed, a message is posted to diagnostics for that particular expression (warning, notice, or alert). See “Notice, Warning, and Alert Expressions” in *Debugging and Analysis* for more information on these messages.
		The first argument of the operator is the content of the diagnostic message and the second argument is returned as the value of the expression.
		For example, when the following expression is evaluated, if the estimated flow is negative, a warning is posted to diagnostics and the expression evaluates to the estimated flow, a NUMERIC:
		WITH NUMERIC estimatedFlow = EstimateFlow() DO
		IF (estimatedFlow < 0.0 cms) THEN
		WARNING(“Estimated flow is negative”, estimatedFlow)
		ELSE
		estimatedFlow
		ENDIF
		END WITH

Add Comment	comment	`<comment>` Inserts a user specified in-line comment above the selected RPL expression or RPL statement. A separate dialog is opened that allows the user to type in a comment. In the RPL editor, the comment is displayed with # characters on the left, lines wrapped as they were in the comment editor dialog, and text in red (or user-specified comment color; see “Colors” on page 88). Double-clicking the comment reopens the edit dialog. For a given RPL dialog, the inline comments can be hidden or shown using the **View**, then **Show Comments** menu or by checking the **Show: Comments** toggle at the bottom of the dialog.
Delete Comment	Deletes comment	Deletes the selected comment. Comments can also be deleted by selecting the comment, then using the **Edit**, then **Delete** menu or pressing **Delete**.

Units in RPL

In Simulation, all computations are done in internal units, that is cms, m, m3, etc. In RPL, computations are performed in units based on the RPL logic, converting as necessary. For example, if you have an expression 10 cfs + Res.Outflow[ ], the first component will use the value in cfs, the second component will use the value from the slot in cms and then convert to cfs. Finally, it will compute the sum of the two values in cfs.

In terms of units, consider the following:

- You can use any units, even different than slot units.
- When a RPL expression accesses a value on a slot, the slot value is represented internally using a scale of 1.0 and the slots standard units.

- Unit types must always be consistent
- When units are not consistent one value is converted to the other (i.e. 20 cfs + 40 cms)

The unit syntax is “units” with the quotes. But quotes are not necessary unless a “ – ” or “ / ” is used, like “acre-ft”.

Unit Operators

Following are the operators available when specifying RPL units:

- “-” multiplication operator (ex. “acre-ft”)
- “/” division operator (ex. “m/s”)
- “^” raised to the power of". (ex. “m^2”)
- giga = 1e9 (ex. “giga-cfs”)
- mega = 1e6
- kilo = 1000
- pico = 1e-12

The complete list of prefix operators can be viewed in the Units List dialog; see Conversion Factors for RPL Units, page 127 for details.

Conversion Factors for RPL Units

The conversion factors for all RPL Units can be viewed in the Units List dialog. From the **Units** menu in the main RiverWare workspace, select **List Available Units.** Then in the resulting Units List dialog, select the **RPL Units** tab at the top. At the bottom left of the Units List dialog, there is a check box to **Show only units that are present on the workspace**. Uncheck the box to show all available units and their conversion factors.

> **Note:** The RPL Units list only shows the individual components that can be used to construct units in a RPL expression. It does not display separate definitions for units that are combinations of units that are already defined. For example, the flow unit of “acre-ft/day” can be used in a RPL expression, but it is not included in the RPL Units list because it is a combination of an area (acre), length (ft), and time (day) that are already defined. Similarly, “m3” is not listed separately as a volume unit because it is simply a length (m) raised to a power.

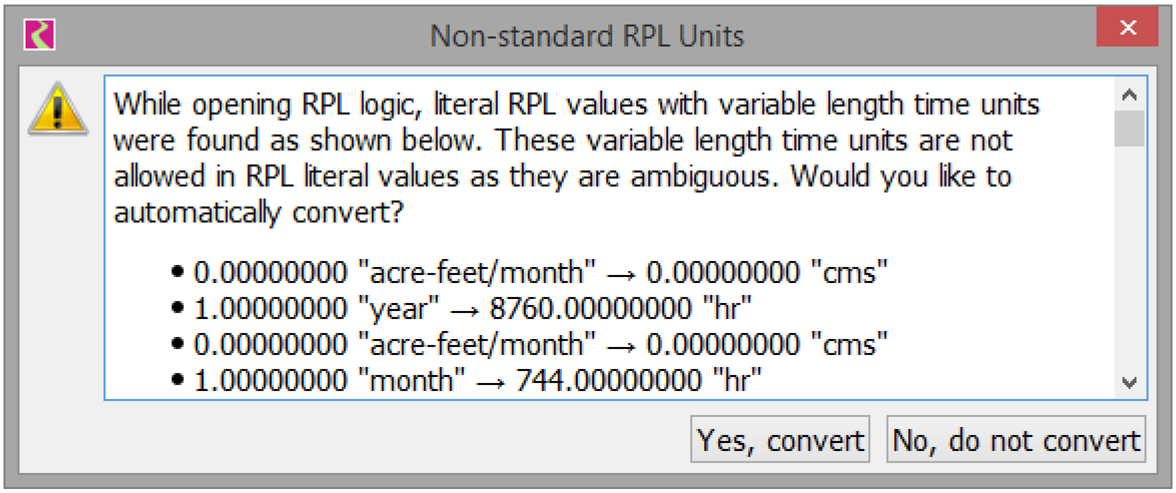
Use of Variable Length Time Units

Prior to RiverWare 9.0, literal RPL values with variable time units were allowed. These include year or month units such as: “acre-feet/month, “m3/year”, “month”, and “year”. In RPL literals, it was ambiguous as to how many days to use, so RPL always assumed 31 days in a month and 365 days in a year. In 8.6, use of these literal RPL units was removed as the RPL results could be incorrect due to the order of operation and how the values get assigned to slot values with variable length time units.

Because there are existing models that contain these unit types, a transition plan was implemented and a setting was created to revert to the previous behavior. These are described in the next two sections:

Transition

Starting in 9.0, when you open any RPL logic (opening a ruleset, loading a model with embedded sets or expression slots, etc.), the set is scanned for any of these unsupported literal RPL units. If any are found, a dialog is opened that asks if you want to automatically convert the values and units.



The conversions assume 31days per month and 365 days per year.

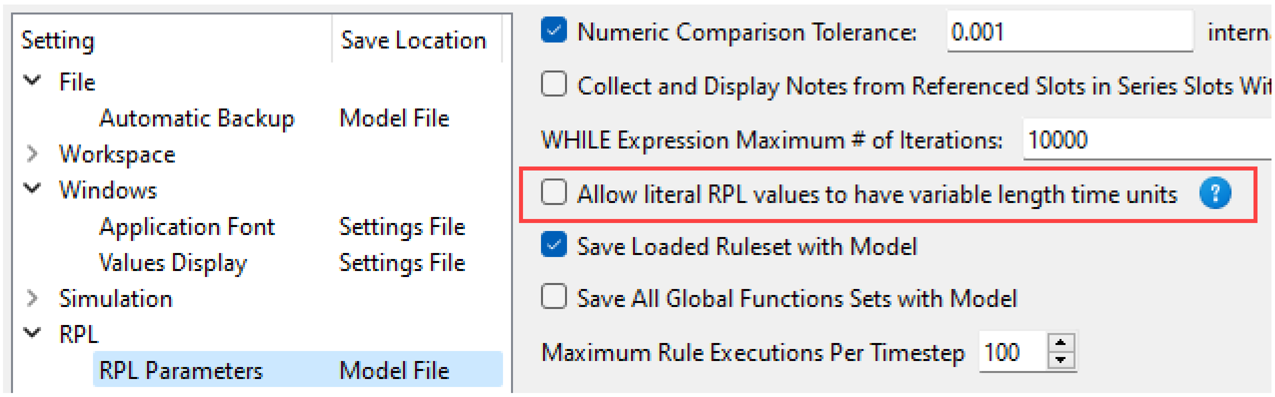
Tip: Each button has tool tips for more information.

Select **Yes, convert** to automatically convert the values. The RPL logic will be valid again. **In rare cases, results could differ from the earlier version.** Often the fix in RPL is to specify the specific month or year to use. For example, convert volume to a flow using VolumeToFlow and specify the month or divide by GetDaysInMonth and specify the desired month.

Select **No, do not convert** if you want to continue using the old approach or want to manually visit the locations and fix them yourself. You can use the Search and Replace tool to locate the month and year strings. See “RPL Search and Replace Dialog” on page 63 for more information. You will not be able to load or validate the RPL logic until you fix these literal values. The alternative is to enable the prior behavior as described in the next section.

Allow Literal RPL Values to Have Variable Length Time Units Setting

To continue using RPL logic with variable length time unit literal values, select the “No, do not convert” option above and then you must enable a setting. On the workspace, select **Policy** and then **RPL Parameters** to get directly to the appropriate page in the Settings Manager:



Check the box for **Allow literal RPL values to have variable length time units**. This setting is saved in the model file. The next time you open the RPL logic, you will not get the **Non-standard RPL Units** dialog. Also, you will be able to type in variable length literal RPL values. The behavior in terms of these values should be the same as in 8.5.

> **Warning: When loading a model with this setting enabled (checked), a brown warning message will be posted to diagnostics that it may lead to incorrect results due to the conversions. This will remind you to address the variable length time units and disable the setting.**

If you turn off this setting and then try to load a previously opened RPL set that contains the variable length units, you will get errors in the diagnostics. Use the Search and Replace to locate the problem expressions.

Chapter 4
RPL External Documentation

This section describes the RiverWare ability to link a RPL set to documentation that resides in a separate file viewable by commonly used viewing and editing programs.

> **Tip:** Model Reports are another and sometimes easier way to document your RPL sets. See "Model Report" in *Output Utilities and Data Visualization*.

Topics

About RPL External Documentation

The RiverWare RPL editors support an optional multiple-line text description, displayed in a plain-text editor box and stored in the RPL set file. Often this text description is not sufficient to adequately describe a complicated rule or policy set.

The purpose of a link to external documentation is to allow the user to develop more detailed documentation that resides in a separate application, such as Microsoft Word, Adobe PDF, or an HTML file. From the RPL editors, the user can click to go to the documentation file.

> **Note:** The external documentation support described in this document is distinct from the RPL set or block Description.

> **Note:** A RPL set consists of rules (or user-defined accounting methods) and functions organized into groups. In this section, the term *RPL object* refers generically to a RPL set component of any sort; that is, to the Set, Policy or Utility Group, Function, Rule, or Method.

External Documentation Features

Following is an overview of the capabilities of RPL external documentation.

- From a RPL dialog, the user is able to easily access external documentation for the RPL object associated with that dialog.
- The four supported types of documents are as follows:

- HTML
- Microsoft Word
- Adobe PDF
- Plain text

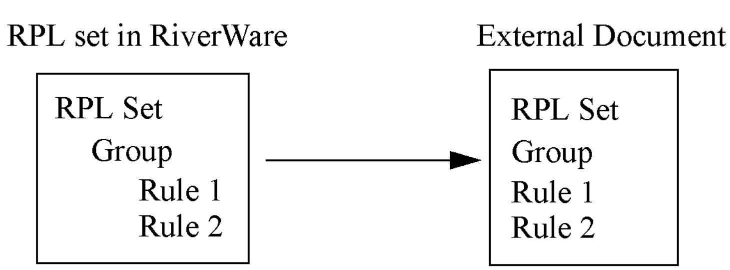
- The user is able to specify separate documents for viewing and editing
- The following modes of access are provided:
  - Edit. The user can view, create, and change the contents of the document.
  - View. The user can view but not change the contents of the document.
- The granularity of the documentation is flexible. Users can associate a separate document with each RPL object within a RPL set or they can document an entire RPL set with a single document.
- When using HTML, the application can open to the most relevant portion of the document. This applies in the following circumstances:
  - There is more than one RPL object described in the associated document, e.g., the documentation is a single file describing the entire ruleset, and,
  - The user is using standard named anchors within the documentation to describe the sections that pertain to each RPL objects.
- The user can create an HTML template of the RPL set that contains the description field for each object. The user can then expand the documentation using an HTML editor.

The following sections describe the external document link feature.

Document Structure

This section provides an overview of the possible document configurations, including some advantages and disadvantages to both. Following are possible setup configurations.

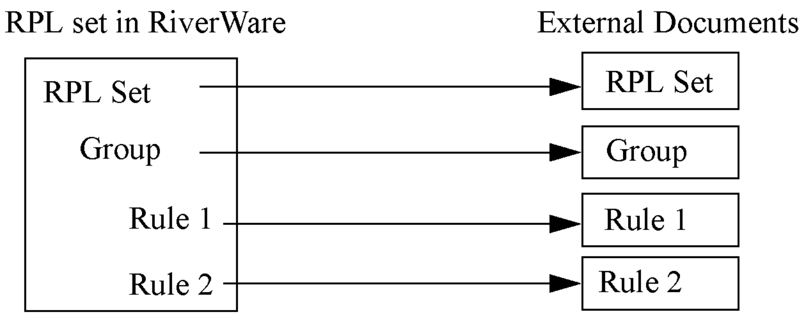
One Document that Describes the Entire Set



- **Advantages**. One document is typically easier to maintain and share. Also, the user only needs to configure the external link for the overall RPL set, not for each individual RPL object within the set.

- **Disadvantages**. Unless the viewing document is HTML, there is no way for the user to automatically open the document to the correct location. For example, if the user selects to view a rule document (say in PDF) they must still search for that rule within the PDF document. Also, multiple model users have to coordinate if they wish to develop documentation simultaneously.

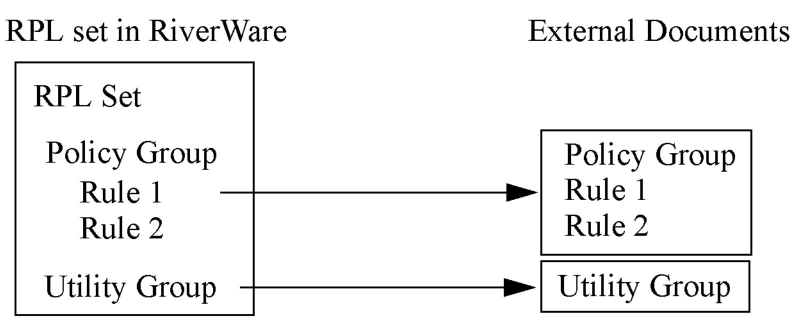
One Document for Each RPL Object in the Set



- **Advantages**. One document for each RPL object allows the user to easily view the external document specific to that object. For example, the user selects the **View** button for a rule and the document specific to that rule opens. Multiple users can more easily develop documentation simultaneously.
- **Disadvantage**. It is more time consuming to configure and maintain. When sharing, there are many more documents to pass around. If there are a large number of RPL objects, there will be a large number of documents.

Multiple Documents With One for Each Policy and Utility Group

This structure is a hybrid of the two above and has some of the advantages and disadvantages of both. This structure is useful because exceptions to the overall scheme are natural. For example, most of the set could be described by one document, while special objects could be described in a document of their own.

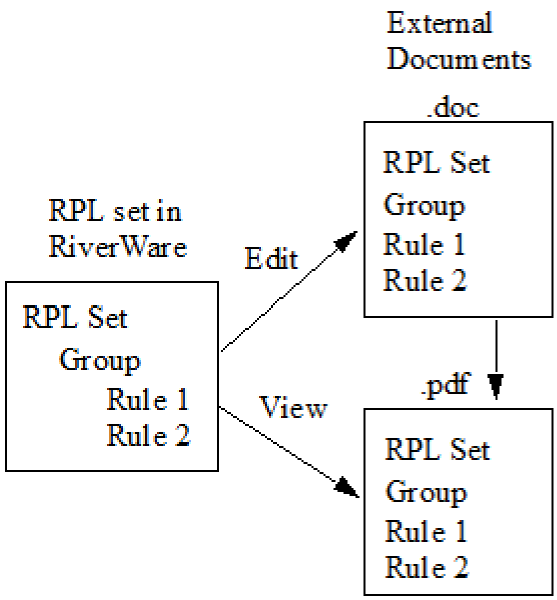


Combinations of the above could be envisioned. The choice of the structure to use depends on the format and the end use.

Separate View and Edit Documents

External documentation for a single object might also be found in two different files, one for editing and one for viewing. For example, documentation might be created in MS Word, saved in Word format for the purposes of future edits, and also saved to a PDF file for the purpose of viewing. Thus the utility allows the user to specify two locations, one for the purposes of viewing and another for editing. Figure 4.1 shows an example as two separate documents, one for viewing, one for editing. There is an arrow from the .doc file to the .pdf file showing how you would create the PDF from the DOC when ready.

Figure 4.1 Diagram using Separate View and Edit Documents



When the documentation for an object is split across many files, then it will often be convenient to the users for all the files to be located in the same directory. This organization is supported without requiring the user to provide redundant information (i.e., repeat the full path to the documentation for each object in the ruleset). At the same time, there is enough flexibility so as not to impose a specific organization on the documentation, either that there be a one-to-one correspondence between ruleset objects and documents or that multiple documents appear in the same directory.

Configuration and User Interface

The document structure is specified in the RPL set by supplying path names to a directory and then specifying file names within that directory. Further, because RPL objects are organized in a hierarchical organization (RPL sets contain policy and utility groups, policy groups contain rules and functions, utility groups contain functions) the hierarchical organization enables RPL objects to share information between objects. When configuration information is provided for an object, the containing objects inherit the configuration from their parent as their default configuration. The user can change the containing objects' configuration as necessary. Thus to specify a single directory in which all the documentation for a RPL set is contained, the user need only specify that value as the directory associated with the set; all groups, functions, and rules in that set will inherit this as their documentation directory.

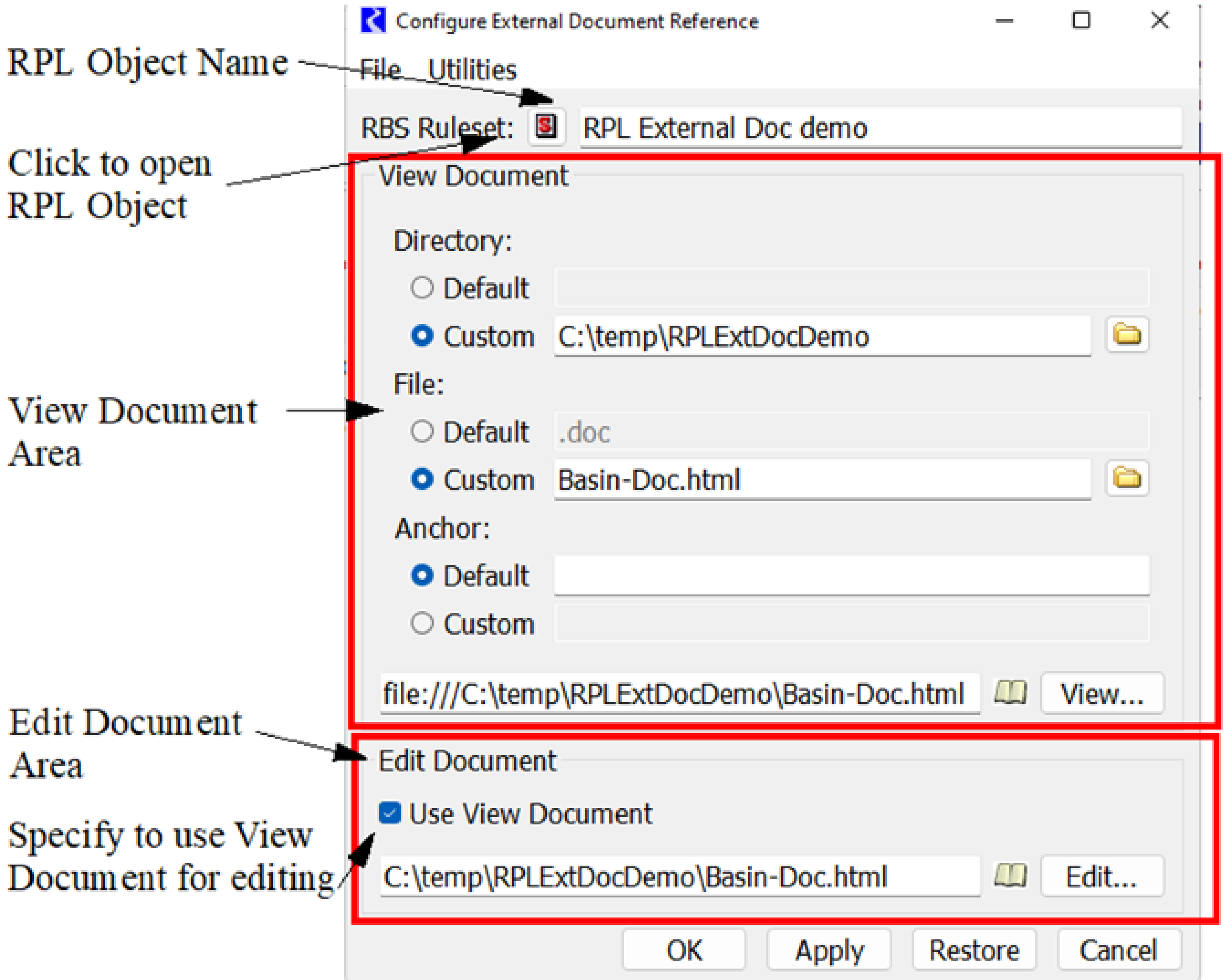
In addition to providing flexibility with respect to the organization of the documentation, this approach has the advantage that when documentation is moved (say from the modeler's file system to a stakeholder's file system), the ruleset will typically only need to be updated in one place to reflect this change. Environment variables can be used to further simplify moving RPL set documentation between machines. With the use of environment variables, the base directory of all RPL set documents within a users configuration can be set without any changes to the document link configuration within RiverWare. Any environment variable name (case sensitive with only alphanumeric characters and underscores) preceded with a dollar sign "$" is translated to the environment variable defined in the user session, outside of RiverWare. For example, an environment variable name `ALLUVIAL_RPL_DOC` is defined as `C:\Projects\docs`, then within the RPL external documentation link, `$ALLUVIAL_RPL_DOC\forecastDraft\` will refer to `C:\Projects\docs\forecastDraft`. This is specifically designed for portability of external RPL documentation across machines and platforms. Slashes can be forward or back slashes and redundant slashes are condensed.

In this documentation and in the user interface, we refer to the two parts of the location specification as the **Directory** and the **File** name. There is nothing that requires that the two parts correspond to a directory and file name in the local file system. Another possibility is that the two parts together specify the location of a document as a URL. In this case, the first part could be something like `http://www.WaterU.edu/Models`, which is not, strictly speaking, a directory.

> **Note:** The file name specification could be an absolute path specified relative to the directory specification; for example, "MyFunctions/FunctionA.htm".

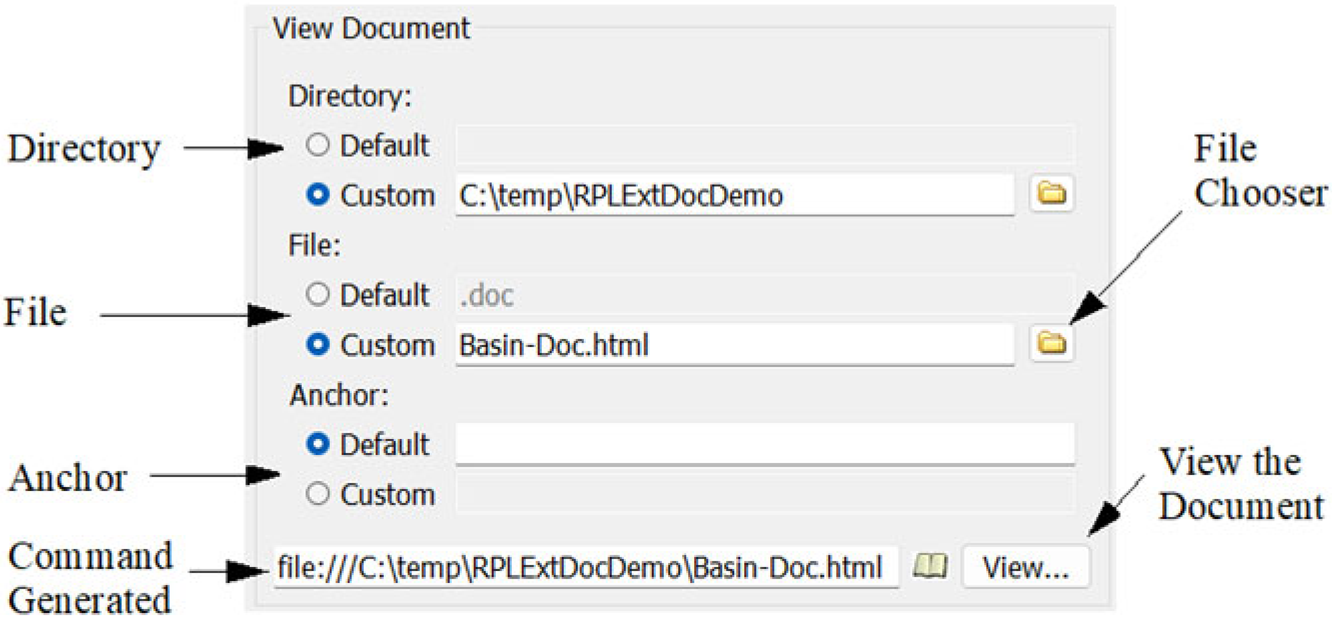
The configuration dialog is available from the RPL editor **View**, then **External Documentatio**n, then **Configure** menu. Figure 4.2 shows the dialog for the RPL Set.

Figure 4.2



In this dialog, the RPL object is listed at the top and there is a button to take you to the that RPL object. Then, there are two areas, one for the **View Document** and one for the **Edit Document**. If the **Use View Document** check box in the **Edit Document** area is checked, the remainder of the **Edit Documen**t area is hidden. This toggle specifies that the **View** and **Edit** documents are the same. The process of specifying a View and Edit document is the same, so it is only described for the View Document. Figure 4.3 shows the View Document dialog.

Figure 4.3



For the **Directory**, **File**, and **Anchor** (when using HTML), the user can specify to use the listed **Default** or select to use a **Custom** configuration. When **Custom** is first selected, the **Default** value is copied down from the **Default** field. For the **Custom** configurations, the user can either type in a value or use the File Chooser button to specify. Anchors are currently only supported for HTML files; see “HTML” on page 139 for details.

The default **Directory** is the directory containing the RPL set itself. The default **File** is the name of the RPL set with a modified filename extension. It is typically “.doc” or “.docx”, but will instead be “.html” if a file with the resulting name at the path actually exists.

For RPL objects (e.g. a group or rule) within a set, the default directory and file names are inherited from the containing RPL object, and the default anchor (when HTML is used) is based on the RPL object name.

> **Note:** Spaces and most punctuation are removed.

> **Note:** The default anchors for all the objects within a RPL set are reported in a table in the generated HTML template file; see “HTML” on page 139) for details.

At the bottom of the View area is the resulting command that is generated and will be passed to the operating system to start the application. This is the result of the configuration that has been defined. It is the chosen application followed by the command used by that application to open the specified file. Selecting the **View** (or **Edit)** button will execute the command.

As noted above, the Edit Document area is similar to the View Document area. At the bottom of the window are OK (apply and close), Apply, Restore to previously applied values, and Cancel buttons.

Once applied, the configuration settings are propagated to the default value of contained RPL Objects. Thus, the user only needs to specify as much information as necessary to fully configure the document. If the user is using the layout One Document that Describes the Entire Set, he/she only needs to configure the RPL set and all rules, groups and methods will be configured correctly; see “One Document that Describes the Entire Set” on page 132. If the user is using the layout One Document for Each RPL Object in the Set, he/she needs to make sure that each RPL object is configured correctly but using a consistent naming convention can be used to simplify the configuration; see “One Document for Each RPL Object in the Set” on page 133. If there are exceptions to the naming convention, the user would need to configure the RPL objects that have the exception.

Viewing and Editing Documents

This section first describes viewing and editing documents in general, then describes peculiarities and specifics for each type of supported document. Once configured, viewing can be done directly from the RPL Object Editor using one of the following methods:

- Select the **External Documentation** icon  . It is on the top of the dialog by the object name.

> **Note:** This icon is only shown if a document actually exists at the configured file path (including the default path if that is a correct configuration).

> **Note:** If the path leads to a web address, that is, `http://`, the icon is always shown. RiverWare does not attempt to verify the existence of such pages on the web.

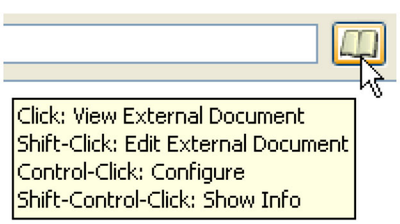
- Use the **View**, then **External Documentation**, then **View Document** menu.

Similarly, editing can be done directly from the RPL object using one of the following methods:

- **Shift-Click** the **External Documentation** icon 
- Use the **View**, then **External Documentation**, then **Edit Document** menu.

Hovering over the External Documentation icon provides tooltips on the use of this button, as follows:

- Select: View External Document
- Shift-Click: Edit External Document
- Control-Click: Configure Document
- Shift-Control-Click: Show Info



Although Anchors are only supported when viewing HTML documents, the actual name of the RPL Object is copied to the system clipboard; see “Anchors in HTML Files” on page 139 for details. This can then be used to paste into the **Search** field in the editor or viewer program. For example, if viewing a PDF, the user can click the external documentation icon  for a function, then press **Ctrl+F** to bring up the search field/dialog, then press **Ctrl+V** to paste the name of the function in the box.

The following sections describe the specific application recommended (and not recommended) to view and edit each type of document and then the mechanism to configure the application that is associated with a document type.

- Hyper Text Markup Language (.html)
- MS Word (.doc or .docx)
- Portable Document Format (.pdf)
- Text (.txt or other)

HTML

Hyper Text Markup Language (HTML) is a common format used by web pages. There are numerous tools to develop and view these types of files.

Viewing HTML Files

HTML can be viewed using any common browser.

Editing HTML Files

HTML can be edited using web development tools such as Adobe (MacroMedia) Dreamweaver. MS Word can also be used edit HTML files.

Anchors in HTML Files

When a document describes more than one object and the user requests to view the documentation for a particular object, the program, if configured correctly, can open to the most relevant location within the document. In order to accomplish this, there needs to be some sort of connection between locations within the document and objects in the set. In this document we will refer to these as **anchors**. In HTML, locations within a document are defined using named anchors.

RiverWare uses anchors to provide a mechanism for navigating to a particular location within a document. For a given object that inherits its document location from a parent object, by default RiverWare assumes the existences of an anchor for this object whose text is based on the object name. In particular, we assume that the anchor label is the object name, with illegal characters replaced with legal ones. The user may override the default value and edit it or specify that no anchor exists.

Typically, the user is responsible for inserting anchors into the external documentation; see “Generating a HTML Template File” on page 140 for a tool to help create a document with anchors. RiverWare doesn’t attempt to inspect the documentation, so care will be required on the part of the user to maintain anchors correctly. As with other sorts of references from an external document to the details of a ruleset, the potential for inconsistencies is large. For example, if the user changes the name of a function in a ruleset, then the name of the anchor for that function in the documentation would need to be changed as well. In addition, access programs will often not normally make anchor text visible, making it more challenging for the user to verify that the anchor labels are correct. To support the management of anchors, RiverWare supports copy/paste between fields containing object names and external programs.

For example, if the external document points to the following HTML file:

```
http://www.WaterU.edu/Models/Functions.html
```

and a function named Set Outputs is given the default anchor SetOutputs. RiverWare will then provide the following to the access program:

```
http://www.WaterU.edu/Models/Functions.html#SetOutputs
```

Generating a HTML Template File

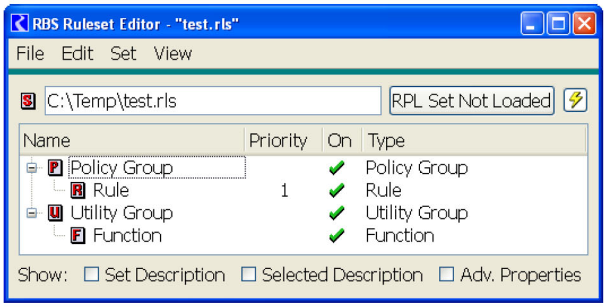
The External Documentation feature provides a utility to generate a template of the RPL objects in the set. This can be used as a starting point in which the user can fill in the missing pieces. It allows someone not too familiar with HTML to quickly generate a working document and all the associated anchors and formatting.

From the RPL set **Configure External Document Reference**, the user can select **Utilities**, then **Generating HTML Template File** to generate an HTML file that contains all of the RPL objects in the set. The file is created in the directory specified in the View Document configuration. It is given the name specified in the File field if it is an HTML file or the default name with a "-DocTemplate.html" appended if the File field contains a non-HTML file.

The HTML document also contains a table of contents that lists the type of RPL object, the name of that object (and a link to the referenced section) and the anchor that it uses. Shown in the body of the file is the RPL object name and any description (i.e. in the **View**, then **Show Description** in RPL editor) that existed in the RPL object at the time the template was created. Once the template is created, it can be viewed/edited using the view/edit button. The user can then fill in pieces as necessary and save the resulting file. Then there should be no need to re-generate this template again.

Figure 4.4 shows an example from a one group with one rule and one function. The text came from the description field in each object.

Figure 4.4



Template: C:/Temp/test-DocTemplate.html
RPL Set: C:\Temp\test.rls
Generated: 12:29 August 27, 2010

Type	Name	Anchor
RPL Set	test	test
Policy Group	Policy Group1	PolicyGroup1
Rule	Rule1	Rule1
RPL Group	Utility Group1	UtilityGroup1
User-defined Function	Function1	Function1

RPL Set: test

Full name: C:\Temp\test.rls

This is the RPL Set Description

Policy Group: Policy Group1

Policy Group1 is used for ...

Rule: Rule1

Rule 1 does ...

RPL Group: Utility Group1

Utility Group1 contains functions for ...

User-defined Function: Function1

Function1 has the following arguments... It does ...

Microsoft Word

Microsoft (MS) Word (.doc or .docx) files are commonly developed using Microsoft Word, but more recently can be viewed (and even edited) using many browsers.

Viewing Microsoft Word Files

Traditionally, DOC files could only be opened using Microsoft Word. Now, these can be viewed with certain browsers including Internet Explorer.

> **Note:** Choosing to view a .doc file with Microsoft Word does not automatically make the document non-editable. The file system must be used if the user wishes to make the document read-only.

Editing Microsoft Word Files

Microsoft Word is typically used to edit .doc files.

PDF

Adobe Portable Document Format (PDF) has become the standard document format when sharing read-only versions of documents. These documents can be viewed by the free Adobe Acrobat Reader and more recently by many browsers.

Viewing PDF Files

Viewing of PDF files is accomplished using a PDF Reader.

Editing PDF Files

Although limited editing of PDF can be done using the full version of Adobe Acrobat, this is not a supported editing tool. Otherwise, PDFs are read-only and cannot be edited. They are typically created from some other document editing application like MS Word or Adobe FrameMaker. We will not discuss editing PDF further.

Text Files

Text files can be viewed (and edited) by any text editor and many web browsers. No further information is provided.

Use Example

Following is an imaginary use example to show the process. Perhaps, we are a water agency and have a complex ruleset. We want to share this ruleset and documentation with stakeholders in the basin, but we do not want to let them have an easily editable copy of this document. Also, we already have some descriptions entered in the ruleset but no other documentation.

In this example, we want to view the document with our favorite web browser, and edit the document with our favorite HTML editor. Once the document is complete, we will post it to a web page so stakeholders can see it. As a result, we will use HTML as the type of document.

We set this up using the following procedure.

1. Make sure your Windows file associations are defined. We will use a web browser to view the HTML document.

   Because we are using HTML, we can use anchors to automatically open the HTML file to the right place.; see “Anchors in HTML Files” on page 139. As a result, we only need one document for the entire RPL set. Thus we are using one document that describes the entire set structure; see “One Document that Describes the Entire Set” on page 132. We do want to have separate documents for viewing and editing. While making changes to the ruleset and hence the Edit document, we will still have an official View document that stakeholders can use. See “Separate View and Edit Documents” on page 134 for a diagram of this structure. Hence, we will need to configure only the top level of the RPL set.

2. From the RPL set, we open the configuration menu using the **View**, then **External Documentatio**n, then **Configure** menu.

   Because we already have some descriptions in our RPL set and we wish to use HTML, we will use the automatically generated template as a starting point. First let’s specify a location to put the file, say `C:\temp\Docs`

3. In the View area, select the **Custom** toggle for the Directory and enter `C:\temp\Docs`

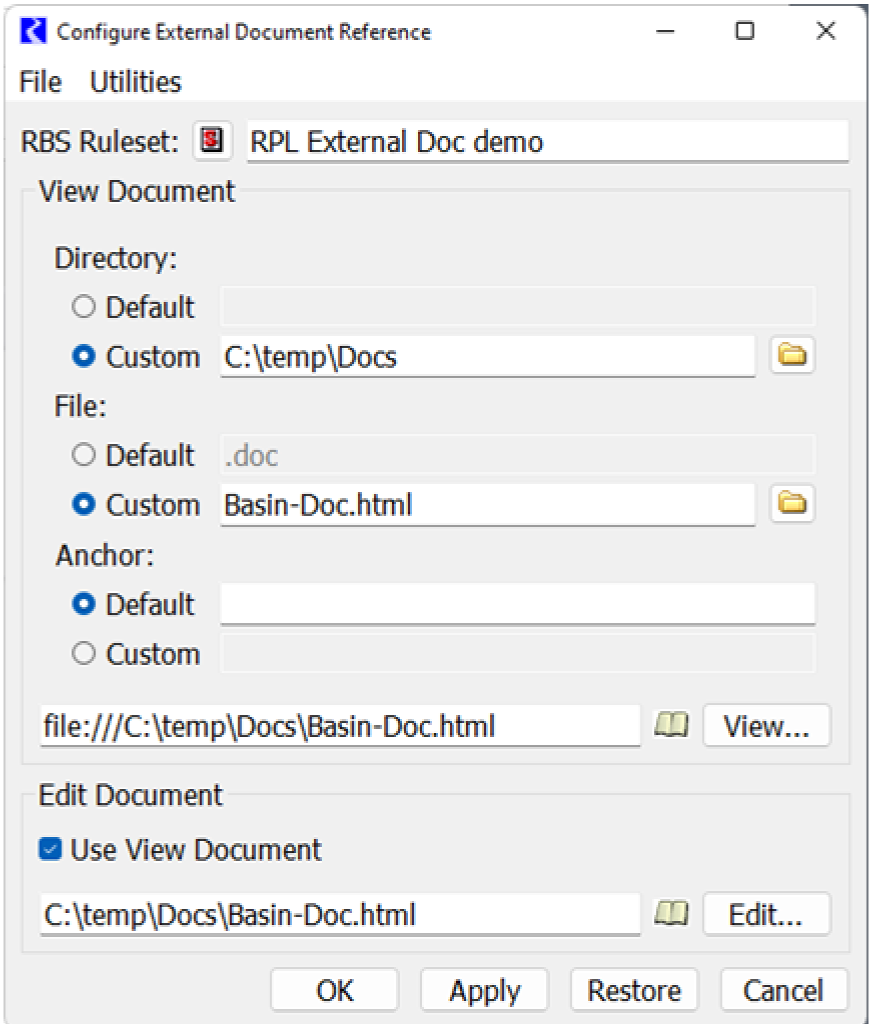
Now we will generate the template.

1. In the Configure dialog we choose **Utilities**, then **Generating HTML Template File.** It gives us a warning that a file named C:/Temp/docs/Basin-DocTemplate.html will be saved.

2. We do not like the name of this file (and to avoid overwriting our completed external document with an inadvertently generated template file in the future), we change both the file name and the name in the configuration. We change the file to Basin-Doc.html using file explorer. In the File area of the configuration, we change the name to Basin-Doc.html and select **Apply** when complete.

   For now we will leave the View document in the `C:\temp\Docs` folder. Once we are complete we will move this to a web server. Figure 4.5 shows our configuration.

Figure 4.5



3. Now select the **View** button and see the template file in a browser.

   We see that we have a pseudo table of contents with links, then each object is listed with the text that came from the description.

4. We can then edit this HTML file in our favorite HTML editor or even Word to add text, pictures, graphs, flowcharts, etc. We make sure to save it as the same name. We can edit the formats of the headings and text. When editing the headings, we make sure to use the **Format**, then **Styles and Formatting** menu to change all instances at once.

5. When editing is complete, we copy the final HTML to a web server.

6. Then we go back to the configuration menu for the RPL Set and change the View document. First we want to indicate that the Edit document is now different, so we toggle off the **Use View Document** option. We make sure the Edit document area is now correct.

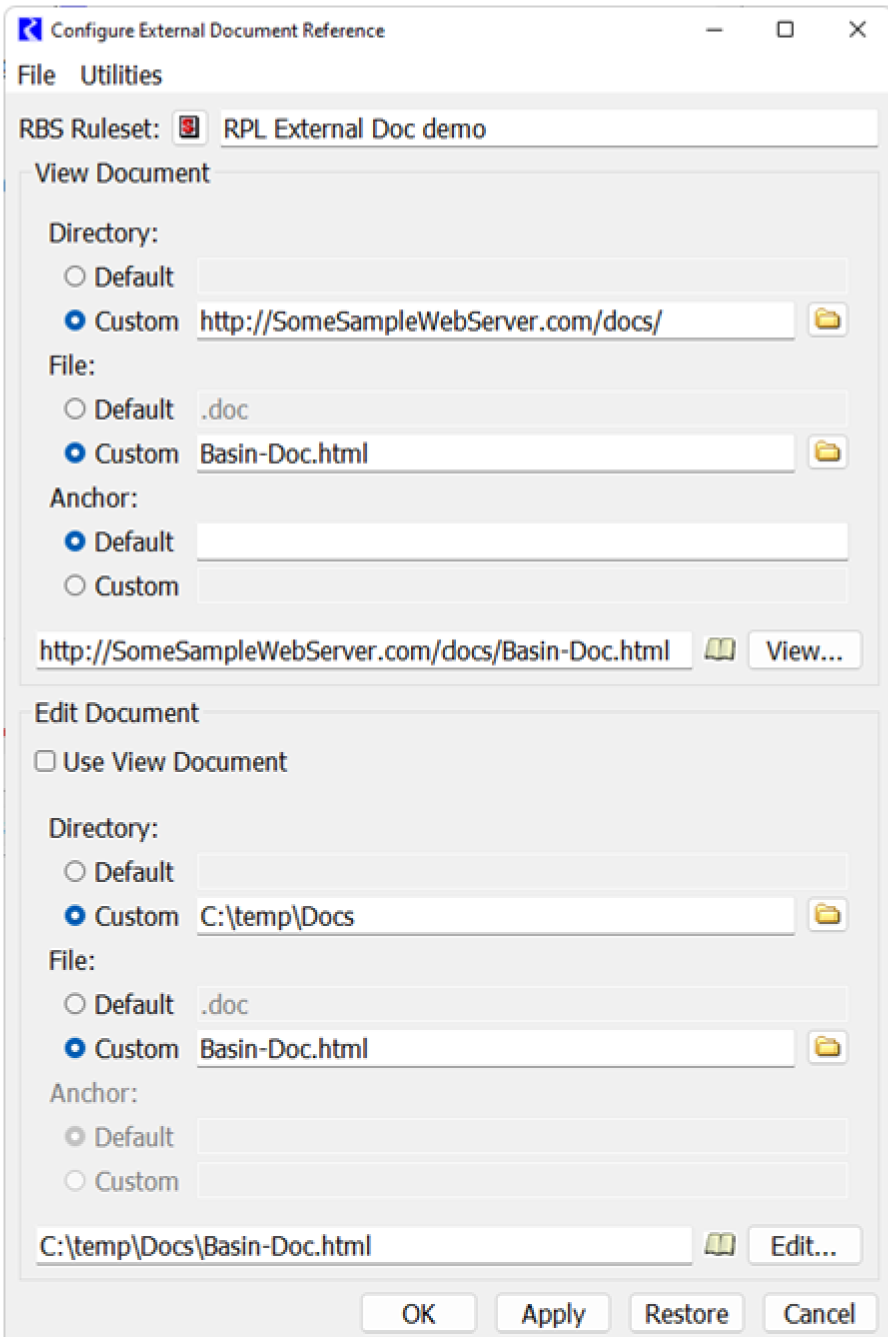
7. Now we want to change the View area to point the Custom Directory to a web server, such as:

   `http://SomeSampleWebServer.com/docs/`

Figure 4.6 shows the configuration for this setup.

If we select the external documentation icon for any RPL object, it will take us to this web page and automatically scroll to the section related to that object. We now share our model and ruleset with our stakeholders.

Figure 4.6



Chapter 5
RPL Predefined Functions

This section describes the predefined RiverWare functions which are available for use in RiverWare Policy Language (RPL) sets.

Topics

About RPL Predefined Functions

Predefined functions perform a wide range of calculations common to water management policy and its expression in a rule context. Some predefined functions provide rules with access to the same algorithms used in RiverWare simulation for mass balance calculations, flow/volume conversions, and table lookups. Other predefined functions are available for common mathematical operations, date/time manipulations, topographical evaluations, and some specialized river basin management calculations.

Predefined functions are used in RPL sets in the same way as user-defined custom functions. They are selected from the palette, and inserted into a rule, internal function, or other expression. Each predefined function is an expression that evaluates to one of the seven rules data types:

- NUMERIC
- BOOLEAN
- DATETIME
- OBJECT
- SLOT
- STRING
- LIST

Predefined functions may or may not have arguments. The computational algorithms and arguments to predefined functions may not be modified.

RPL Predefined Functions Reference

This section provides details about the RPL predefined functions. See “Hypothetical Simulations” for additional information about predefined functions used specifically for hypothetical simulations.

• Abs

This function evaluates to the absolute value of its single numeric argument.

Description	Absolute value operator	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	value to evaluate
Evaluation	Determines the absolute value of the numeric argument	
Comments		

Syntax Example

```
Abs(-11 "cfs") returns 11 "cfs"
```

Use Example

```
IF(Abs(res.Inflow[] - res.Inflow[@"Next Timestep"]) < 1 "cms") THEN TRUE
```

• AccountAttributes

Description	Given a string representing an account’s full name (object^account), returns a list containing the account’s attributes, i.e., the account’s water type, water owner, and account type.	
Type	LIST {STRING, STRING, STRING}	
Arguments	**Type**	**Meaning**
1	STRING	The name of the account.
Evaluation		
Comments		

Syntax Example

```
AccountAttributes("ResA^GoodWater")
```

Return Example

```
{"Intra-basin Transfer", "Big City", "StorageAccount"}
```

• AccountNameFromPriorityDate

This function evaluates to the name of the account having the specified priority date.

Description	This function returns the name of a single account having the specified priority date.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	DATETIME	The priority date.
Evaluation	The accounts in the system are examined and the one account having the indicated priority date is returned.	
Comments	Priority dates are a property of accounts. It is an error if no account has the specified priority date. It is an error if multiple accounts are found. See "AccountNamesFromPriorityDate" for a function that can be used when multiple accounts share a priority date.	

Syntax Example

```
AccountNameFromPriorityDate (@"12:00:00 August 12, 2004")
```

Return Example

```
"Account1"
```

• AccountNamesByAccountType

This function evaluates to the list of names of Accounts on the specified Object having the indicated Account type.

Description	This function returns a list of names of Accounts on a specified Object having the indicated Account type, sorted in ascending Account priority date order. Accounts which don't have a priority date are at the end of the list, sorted in ascending name order.	
Type	LIST {STRING}	
Arguments	**Type**	**Meaning**
1	OBJECT	The Object.
2	STRING	Account type name (currently, one of Diversion, Storage, Instream Flow or Passthrough) or ALL. The string is not case sensitive. "InstreamFlow is also a valid entry.
Evaluation	The set of Accounts on the Object are examined. The names of the Accounts having the specified account type are added to the returned list. If the Account type argument is ALL, then that attribute is ignored. The returned list will contain the names of ALL Accounts on the Object. The list is sorted as described above.	
Comments	Priority dates are properties of Accounts.	

Syntax Example

```
AccountNamesByAccountType ("Heron Reservoir", "Storage")
```

Return Example

```
{"Account1", "Account2"}
```

AccountNamesByWaterOwner

This function evaluates to the list of names of Accounts on the specified Object having the indicated WaterOwner.

Description	This function returns a list of names of Accounts on a specified Object having the indicated WaterOwner, sorted in ascending Account priority date order. Accounts which don't have a priority date are at the end of the list, sorted in ascending name order.	
Type	LIST {STRING}	
Arguments	**Type**	**Meaning**
1	OBJECT	The Object.
2	STRING	WaterOwner name or NONE or ALL
Evaluation	The set of Accounts on the Object are examined. The names of the Accounts having the specified WaterOwner are added to the returned list. If the WaterOwner argument is NONE, then only Accounts having the default (unassigned) WaterOwner are included in the returned list. If the WaterOwner argument is ALL, then that attribute is ignored. The returned list will contain the names of ALL Accounts on the Object. The list is sorted as described above.	
Comments	WaterOwners and priority dates are properties of Accounts.	

Syntax Example

```
AccountNamesByWaterOwner (%"Heron Reservoir", "Contractor2")
```

Return Example

```
{"Account1","Account2"}
```

AccountNamesByWaterType

This function evaluates to the list of names of Accounts on the specified Object having the indicated WaterType.

Description	This function returns a list of names of Accounts on a specified Object having the indicated WaterType, sorted in ascending Account priority date order. Accounts which don't have a priority date are at the end of the list, sorted in ascending name order.	
Type	LIST {STRING}	
Arguments	**Type**	**Meaning**
1	OBJECT	The Object.
2	STRING	WaterType name or NONE or ALL
Evaluation	The set of Accounts on the Object are examined. The names of the Accounts having the specified WaterType are added to the returned list. If the WaterType argument is NONE, then only Accounts having the default (unassigned) WaterType are included in the returned list. If the WaterType argument is ALL, then that attribute is ignored. The returned list will contain the names of ALL Accounts on the Object. The list is sorted as described above.	
Comments	WaterTypes and priority dates are properties of Accounts.	

Syntax Example

```
AccountNamesByWaterType (%"Heron Reservoir", "SanJuan")
```

Return Example

```
{"Account3","Account4"}
```

• AccountNamesFromObjReleaseDestination, AccountNamesFromObjReleaseDestinationIntra

This function evaluates to the list of names of Accounts on the specified Object having outflow Supplies of the given ReleaseType and Destination.

Description	This function returns a list of names of Accounts on a specified Object where the attributes of the outflow Supplies of the Accounts match the given ReleaseType and Destination. The list is sorted in ascending Account priority date order; Accounts which don't have a priority date are at the end of the list, sorted in ascending name order.	
Type	LIST {STRING}	
Arguments	**Type**	**Meaning**
1	OBJECT	The Object.
2	STRING	ReleaseType name or NONE or ALL
3	STRING	Destination name or NONE or ALL
Evaluation	The set of Accounts on the Object are examined. The outflow Supplies on those Accounts are then examined. The names of the Accounts with Supplies that meet the following requirements are added to the returned list: • Link a different downstream Object, and • Have the indicated ReleaseType, and • Have the indicated Destination If the ReleaseType argument or the Destination argument is NONE, then only Supplies having the default (unassigned) attribute of that type are considered. If the ReleaseType argument or the Destination argument is ALL, then that Supply attribute is ignored. The list is sorted as described above. The Intra version of the function will only look at transfer supplies that are within the object.	
Comments	ReleaseTypes and Destinations are properties of Supplies; priority dates are properties of Accounts.	

Syntax Example

```
AccountNamesFromObjReleaseDestination (%"Heron Reservoir",
  "Account Fill", "Abiquiu")
```

Return Example

```
{"DownstreamAcct1", "NaturalFlowAccount"}
```

• AccountNamesFromPriorityDate

This function evaluates to a list of accounts having the specified priority date.

Description	This function returns a list of the accounts having the specified priority date. This is useful when the model allows accounts to share priority dates.	
Type	LIST of STRING	
Arguments	**Type**	**Meaning**
1	DATETIME	The priority date.
Evaluation	The accounts in the system are examined and the accounts having the indicated priority date are returned as a list of STRINGS.	
Comments	Priority dates are a property of accounts. If there are no accounts with the specified priority date, an empty list is returned. See also "AccountNameFromPriorityDate" for a similar function that returns a STRING instead of a list.	

Syntax Example

```
AccountNamesFromPriorityDate (@"12:00:00 November 2, 2006")
```

Return Example

```
{"Ash", "Willow"}
```

• AccountPriorityDate

This function evaluates to the priority date of the Account, on the specified Object, having the specified name.

Description	This function returns the priority date of the Account, on the specified object, having the specified name.	
Type	DATETIME	
Arguments	**Type**	**Meaning**
1	OBJECT	The Object
2	STRING	The Account name
Evaluation	The Object's accounts are examined. If an Account exists with the specified name its priority date is returned.	
Comments	Priority dates are a property of Accounts. It's an error if either the Object doesn't have an Account with the specified name or the Account doesn't have a priority date.	

Syntax Example

```
AccountPriorityDate (%"Reservoir1", "NaturalFlowAcct")
```

Return Example

```
@"February 23, 1902"
```

• AggregateSeriesSlot

This function aggregates slot values over time using the specified function; returned is a list of aggregated values and the timestep associated with each.

Description	For the specified slot, compute the aggregated values using the specified function at the specified timestep size.	
Type	LIST of {DATETIME, NUMERIC}	
Arguments	**Type**	**Meaning**
1	SLOT	The slot to aggregate. This slot has the smaller timestep size.
2	DATETIME	Begin timestep of the returned (larger timestep) values
3	DATETIME	End timestep of the returned (larger timestep) values
4	STRING	Timestep to which to aggregate, "6 hours", "1 days", etc. This is the size of the larger timestep.
5	STRING	Aggregation function or filter to use. It must be one of the following: "SUM", "AVG", "MIN", "MAX", "FIRST", "LAST".
6	STRING	How to treat NaN values in the slot that is being aggregated. It must be one of the following: • “ERROR”: The function aborts the run and posts an explanation to diagnostics. • “TERMINATE”: If any value in the smaller timestep aggregation interval is NaN, the calling expression (for example, the rule) will terminate early. • “IGNORE”: If at least one value in the smaller timestep interval is valid, the interval is aggregated and the NaN value is ignored. If all values in the aggregation interval are NaN, the calling expression will terminate early as above.
Evaluation	For the specified slot, compute the aggregated values using the specified function at the specified timestep size.	

Comments	This function can be used to aggregate a series slot to return a list of larger timestep values. A use example from an initialization rule is shown:
	FOR (LIST pair IN AggregateSeriesSlot (DataObj1.6hr Flows ,
	@"Start Timestep" ,
	@"Finish Timestep" ,
	"1 days" ,
	"AVG" ,
	"IGNORE")) DO
	DataObj1.1DayFlows [pair ⟨ 0 ⟩] = pair ⟨ 1 ⟩
	END FOR	

Syntax Example

```
AggregateSeriesSlot( BigRes.Inflow_Hourly,
   @"Start Timestep", @"Finish Timestep",
   "1 days", "AVG", "IGNORE" )
```

Return Example

```
{ {@"December 31, 1999", 10 cfs} ,
   {@"January 1, 2000", 20cfs} }
```

• AnnualEventCount

This function analyzes a slot's value over some number of years, counting the occurrence of certain *events*.

Description	Return the number of events which occurred on a slot in a given period.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	a slot
2	DATETIME	analysis period start date
3	DATETIME	analysis period end date
4	DATETIME	event period start date
5	DATETIME	event period end date
6	NUMERIC	value threshold
7	BOOLEAN	value threshold is upper bound
8	NUMERIC	event threshold
9	BOOLEAN	event threshold is upper bound
Evaluation	See the on-line documentation for AnnualEventStats, which performs identical computation, but returns more information. This function returns only the number of events which occurred in the analysis period.	
Comments		

Syntax Example

```
AnnualEventCount($ "Lottawatta Reservoir.Outflow",
  @"24:00:00 February 28, 1994",
  @"24:00:00 January 31, 2005",
  @"24:00:00 May 31",
  @"24:00:00 August 31",
  100.0, TRUE, 2.0, TRUE)
```

Return Example

```
102.0000
```

• AnnualEventLastOccurrence

This function analyzes a slot's value over some number of years, noting the last occurrence of a certain type of event.

Description	Return the number of event periods which occurred after the last event on a slot.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	a slot
2	DATETIME	analysis period start date
3	DATETIME	analysis period end date

4	DATETIME	event period start date
5	DATETIME	event period end date
6	NUMERIC	value threshold
7	BOOLEAN	value threshold is upper bound
8	NUMERIC	event threshold
9	BOOLEAN	event threshold is upper bound
Evaluation	See the on-line documentation for AnnualEventStats, which performs identical computation, but returns more information. This function returns only the number of event periods which occurred after the last event. If no events occurred, then this is the number of event periods.	
Comments		

Syntax Example

```
AnnualEventLastOccurrence ($ "Lottawatta Reservoir.Outflow",
  @"24:00:00 February 28, 1994",
  @"24:00:00 January 31, 2005",
  @"24:00:00 May 31",
  @"24:00:00 August 31",
  100.0, TRUE, 2.0, TRUE)
```

Return Example

```
2.00000
```

• AnnualEventStats

This function analyzes a slot's value over some number of years, noting the occurrence of certain *events*.

Description	Collects and returns statistics on annual events occurring on a slot.	
Type	LIST	
Arguments	**Type**	**Meaning**
1	SLOT	a slot
2	DATETIME	analysis period start date
3	DATETIME	analysis period end date
4	DATETIME	event period start date
5	DATETIME	event period end date
6	NUMERIC	value threshold
7	BOOLEAN	value threshold is upper bound
8	NUMERIC	event threshold

9	BOOLEAN	event threshold is upper bound
Evaluation	The analysis period start and end dates define the period during which the analysis will be performed. Within the analysis period, only the timesteps which occur on or between the day and month of the event period start and end dates are considered. Each of these periods within the analysis period is called an event period. At each event period, an event can either occur or not. An event is defined by the value threshold and comparison type and the subevent count threshold and comparison type. At each timestep within an event analysis period, the slot's value is compared to the threshold value. If the value threshold is an upper bound and the slot's value is greater than the value threshold, then a subevent is said to have occurred at that timestep; similarly, if the value comparison is a lower bound and the slot's value is less than the value threshold, then a subevent is said to have occurred. After the subevents within an event analysis period have been noted, then they are counted up and compared to the subevent count threshold. If the subevent count threshold is an upper bound and the number of subevents which occurred in an event analysis period is greater than the subevent count threshold, then an event is said to have occurred, and similarly, if the subevent count comparison is a lower bound and the number of subevents which occurred in an event analysis period is less than the subevent count threshold, then an event is said to have occurred. The return list contains the following items (listed in order): • The total number of event periods. • The number of events which occurred. The number of event periods which occurred after the last event. If no events occurred, then this is the number of event periods.	
Comments	As defined above, the first and last event periods might be of shorter duration than the other event periods. For example, if the analysis period is July 1, 1980 through June 30, 1989 and the event period is May 1 through September 30, then the first event period will be July 1, 1980 through September 30, 1980; subsequent event periods will be from May 1 through September 30, until the last event period, which will be from May 1, 1989, through June 30, 1989. If the event period contains the end of February, then event periods during leap years will also have a different duration. It is an error for the start or end date of the event period to be February 29, which does not exist in each year. Event periods can span year boundaries. For example, if the event period begin is December and the event period end is January, then each event period will be from December of one year to January of the next. One can leave the year field of the event period start or end date unspecified, if one is using a format which contains that component, such as the month/day/year format. E.g., one could specify the event start as @"6:00 May 1". The year component of the event period start and end date is ignored whether or not it is specified. Any missing value in the slot's series is treated as a non-subevent. The comparison with the value threshold is done to within 0.01% of the threshold's value. That is, values which are within 0.01% of the threshold's value are considered to have exceeded the threshold.	

Syntax Example

```
AnnualEventStats($ "Lottawatta Reservoir.Outflow",
  @"24:00:00 February 28, 1994",
  @"24:00:00 January 31, 2005",
  @"24:00:00 May 31",
  @"24:00:00 August 31",
  100.0, TRUE, 2.0, TRUE )
```

Note: This call will determine how often outflow from Lottawatta Reservoir exceeded 100 cfs more than two times between May and August in an eleven-year period starting in 1994.

Return Example

```
{11.00, 3.00, 2.00}
```

There were eleven event periods. In three of these, the flow exceeded 100 cfs more than two times. There were four event periods after the last event in 2001—the summers of 2001, 2002, 2003, and 2004.

• AvgObjectsAggregatedOverTime

This function returns a single numeric value obtained by averaging several objects' aggregated slot values. The objects' slot values may be aggregated as a SUM, AVG, MIN, or MAX over a specified time range.

Description	Aggregates several objects' values, each of which is the result of aggregating a slot's values over time.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation function (SUM, AVG, MIN, or MAX)
4	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
5	BOOLEAN	time conversion option (TRUE or FALSE)
6	DATETIME	start date
7	DATETIME	end date
Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, each slot's values are aggregated according to the aggregation function argument over the time range of the datetime arguments. During each of these slot aggregations, any values which do not satisfy the aggregation filter argument are ignored. Finally, all of the object's aggregated slot values are averaged.	
Mathematical Expression	$\overline{\forall_{(obj\ \text{in Subbasin})}[\forall_{(t\ \text{from}\ start\ \text{to}\ end)}[AggFunction_{(obj)}(obj.slotname)]]}$	
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, RiverWare aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.	

Syntax Example

```
AvgObjectsAggregatedOverTime("upper basin",
   "Inflow", "MAX", "ALL", TRUE,
   @"October, Previous Year",
   @"September, Current Year")
```

Return Example

```
52623.32 "cms"
```

AvgObjectsAtEachTimestep

This function evaluates to a list. Each item of the list is a list comprised of the datetime at which the average was performed, and the value of the average.

Description	Average several object's slot values, for each timestep in a range.	
Type	LIST{LIST{DATETIME, NUMERIC}}	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
4	BOOLEAN	time conversion option (TRUE or FALSE)
5	DATETIME	start date
6	DATETIME	end date
Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values to be averaged are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, all of the object's slot values are averaged, yielding one value for each timestep in the time range of the datetime arguments. The function returns a list of two items, where the first and second items of the inner lists are the datetime and the average value, respectively.	
Mathematical Expression	$\forall_{(t \text{ from } start \text{ to } end)}[\{t, \overline{\forall_{(obj \text{ in Subbasin})}[obj.slotname]}\}]$	
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, RiverWare aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.	

Syntax Example

```
AvgObjectsAtEachTimestep("upper basin",
  "Storage", "ALL", FALSE
  @"October, Previous Year",
  @"September, Current Year")
```

Return Example

For a model with a 1 Month timestep, the above function would return something like:

```
{ { 24:00 October 31, 1996, 1233232.2 "m3" },
  { 24:00 November 30, 1996, 1067478.3 "m3" }, ...
  { 24:00 September 30, 1997, 1563456.7 "m3" } }
```

• AvgTimestepsAggregatedOverObjects

This function evaluates to a single numeric value. This value is the average, over time, of values resulting from aggregating several objects slot values at each timestep.

Description	Aggregate over a timeseries of values, each of which is the result of aggregating several objects' slot values.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation function (SUM, AVG, MIN, or MAX)
4	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
5	BOOLEAN	time conversion option (TRUE or FALSE)
6	DATETIME	start datetime
7	DATETIME	end datetime
Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, all of the objects' slot values are aggregated according to the aggregation function argument for each timestep in the time range of the datetime arguments. During each of these slot aggregations, any values which do not satisfy the aggregation filter argument are ignored. Finally, the timeseries of object aggregated slot values are averaged.	
Mathematical Expression	$\overline{\forall_{(t \text{ from } start \text{ to } end)}[\forall_{(obj \text{ in Subbasin})}[AggFunction_{(t)}(obj.slotname)]]}$	
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, RiverWare aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.	

Syntax Example

```
AvgTimeStepsAggregatedOverObjects ("upper basin",
   "Storage", "MAX", "ALL", FALSE,
   @"October, Previous Year",
   @"September, Current Year")
```

Return Example

```
230000 "m3"
```

• AvgTimestepsForEachObject

This function evaluates to a list. Each item of the list is a list comprised of the object name and the average value of the slot on that object for the time range specified.

Description	Average a slot's values over a time range, for each object in a subbasin.	
Type	LIST {LIST {OBJECT, NUMERIC}}	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
4	BOOLEAN	time conversion option (TRUE or FALSE)
5	DATETIME	start datetime
6	DATETIME	end datetime
Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. For each object, the slot's values are averaged over every timestep in the range of the datetime arguments. Any values which do not satisfy the aggregation filter argument are ignored during the calculation. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are first multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME.	
Mathematical Expression	$\forall_{(obj \text{ in Subbasin})}[\{obj, \overline{\forall_{(t \text{ from } start \text{ to } end)}[obj.slotname]}\}]$	
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, this function aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, this function also aborts RiverWare with an error.	

Syntax Example

```
AvgTimestepsForEachObject("upper basin",
  "Storage", "ALL", TRUE,
  @"October, Previous Year",
  @"September, Current Year")
```

Return Example

For a model with a 1 Month, the above function would return something like:

```
{ { %"Res1", 1233232.2 "m3" },
  { %"Res2", 1067478.3 "m3" },
  { %"Res3", 1997, 1563456.7 "m3" } }
```

• Ceiling

This function rounds a numeric value up to the nearest multiple of a numeric factor.

Description	The ceiling numeric operation, to a multiple of a factor.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the value

2	NUMERIC	the factor
Evaluation	Converts the value into the units of the factor, then returns the smallest integral multiple of the factor which is not less than the converted value. The returned value has the units of the factor.	
Comments	If the scalar portion of the factor is 1.0, then this function simply returns the ceiling of the value expressed in the units of the factor. If the two arguments are of a different unit type, this function aborts the run with an error. See also “RoundToFactor” and “Floor”.	

Syntax Example

```
Ceiling("Dry Reservoir.Pool Elevation"[], 100.0 "ft")
```

Return Example

```
400 "ft"
```

• ColumnLabel

Description	Returns the labels associated with column of a table slot or aggregate series slot.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	SLOT	A table slot or agg series slot.
2	NUMERIC	The column index (0-based).
Evaluation	Returns the label of the column of the slot which has the given index.	
Comments	It is an error to provide an illegal index (e.g., an index of 4 with a table which has only 4 columns). If the column index is legal but there is no label for that column, then the empty string is returned: " ".	

Syntax Example

```
ColumnLabel(DataObjA.CoeffTable, 2)
```

Return Example

```
"Coefficient 3"
```

• ColumnLabels

Description	Returns a list containing the column labels of a given table slot or agg. series slot, in order.	
Type	LIST of STRING values	
Arguments	**Type**	**Meaning**
1	SLOT	A table slot or agg. series slot
Evaluation	Returns the column labels of the given slot.	
Comments	It is an error if the input slot has a type other than table slot or agg. series slot. For each column, if no label exists the empty string is returned.	

Syntax Example

```
ColumnLabels(DeepLake.Elevation Volume Table)
```

Return Example

```
{“Pool Elevation”, “Storage”}
```

• CompletePartialDate

Description	Fill in the missing components of a partially specified date/time.	
Type	DATETIME	
Arguments	**Type**	**Meaning**
1	DATETIME	a partially specified date/time.
2	DATETIME	a source date/time, used to complete the other date.
Evaluation	Fills in the missing components of a partially specified date value. The missing component values are taken from the second parameter, a date value which, if not fully specified, should have at least the components which are missing from the date which is being completed. See the following syntax examples. See “PreviousDate” and “NextDate” for details on related functions.	
Comments	The behavior is not defined if the resulting date is not valid; for example, if the day of month is not valid for the month and year.	

Syntax Example

```
CompletePartialDate(@"March", @"t")
```

Return Example

Assuming @”t” is the second day of some month in 1994:

```
24:00 March 2, 1994
```

• ComputeReservoirDiversions

Description	Used to meet multiple water user demands using multiple reservoir diversions
Type	LIST{LIST {SLOT, NUMERIC, OBJECT}}

<table>
<tr><th>Arguments</th><th>Type</th><th>Meaning</th></tr>
<tr><td>1</td><td>STRING</td><td>The computational subbasin used for the calculations</td></tr>
<tr><td>Evaluation</td><td colspan="2">Returns a LIST of slot, value triplets. Each triplet is a LIST that contains a slot (at index zero) and the value to set on that slot (at index one). The slot, value triplets computed by this function are for the subslots on the Supply From Reservoirs slot on each Water User object and the Incoming Available Water slot on each Water User object.
For each Water User in the specified subbasin:
• A list of supply reservoirs is generated by following the links to the Supply From Reservoirs slot
• The list of reservoirs is ranked by Operating Level in descending order.
• Each reservoir makes a diversion to meet the Water User’s Diversion Requested value. This value is limited by: the Maximum Delivery Rate specified on the Water User object that applies to the current reservoir, the Max Diversion specified on the Diversion object that applies to the current reservoir, and the amount of water remaining in the conservation pool.
• If the Limit by Reservoir Level method is selected (on the Water User object) a diversion cannot be made if the Demand Reservoir is in the flood pool or has a higher operating level than the supply reservoir.
• Each reservoir is visited until the Diversion Requested is met or there are no reservoirs left to consider.
• The function returns each subslot on each Supply From Reservoirs slot and the associated value. Also the Incoming Available Water slot on each Water User is returned with the value to be set on that slot. The Incoming Available Water is the sum of all the Supply From Reservoirs subslot values</td></tr>
<tr><td>Comments</td><td colspan="2">The computational subbasin specified as the argument to this function must contain all the objects relevant to these calculations (Water Users, Diversion Objects, Reservoirs, etc.)
The computational subbasin must have a method selected in the Diversions from Reservoirs category. Please consult the help file for the Computational Subbasin object (under Simulation Objects) for more details on this method category.
The use of this function requires a specific configuration of objects and method selections. The schematic diagram below displays the required object and link configurations.
See “Reservoir Diversions” in USACE-SWD Modeling Techniques for details on using this function for USACE-SWD.</td></tr>
</table>

Syntax Example

If Diversion Basin contains two reservoirs and the WU1 and WU2 water users connected to those reservoirs:

```
ComputeReservoirDiversions("Diversion Basin")
```

Return Example

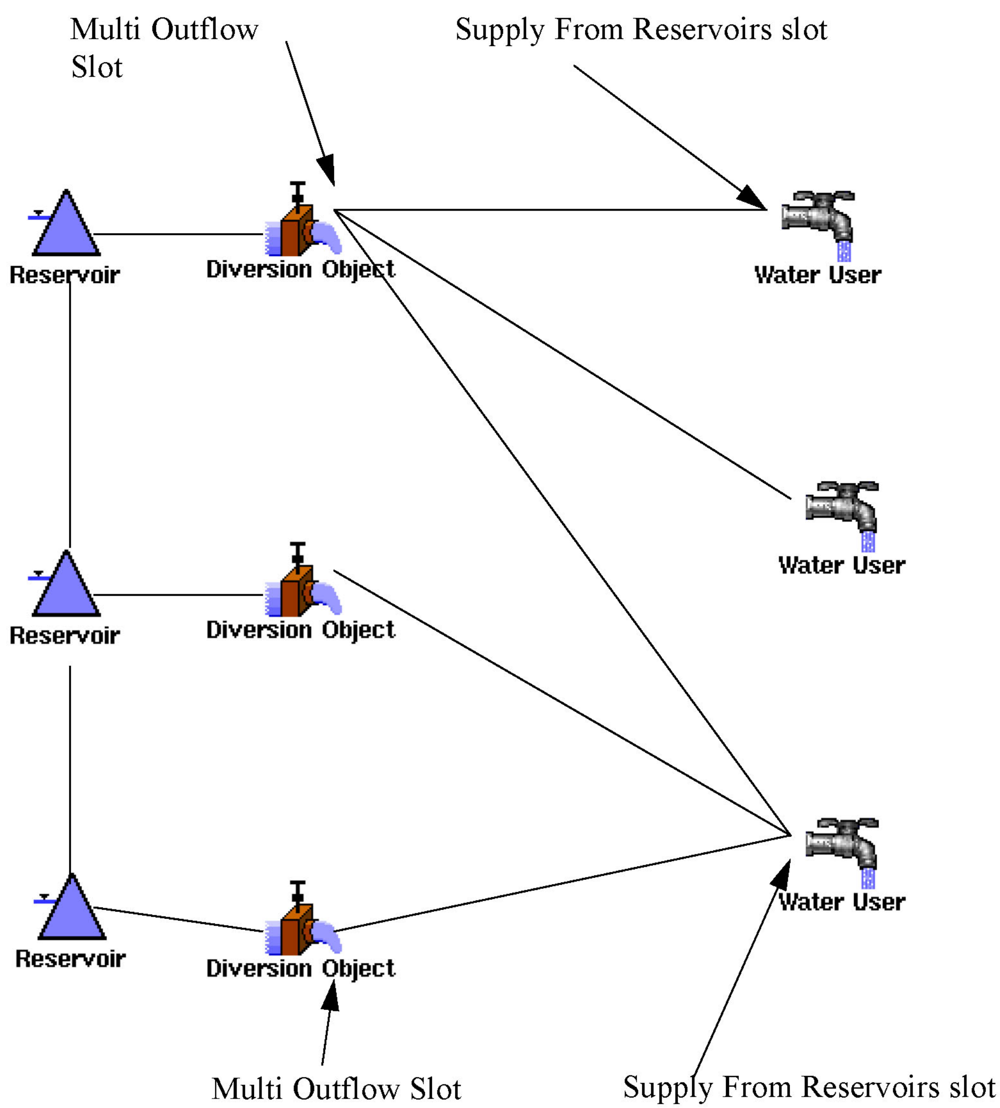
```
{ {"WU1.Supply From Reservoirs.WU1_Divert__dot__Multi Outflow", 2.26534773 "cms", "WU1"},
  {"WU1.Incoming Available Water", 2.26534773 "cms", "WU1"},
  {"WU2.Supply From Reservoirs.WU2_Divert__dot__Multi Outflow", 0.67960432 "cms", "WU2"},
  {"WU2.Incoming Available Water", 0.67960432 "cms", "WU2"}  }
```

Use Example

```
FOR EACH ( LIST result IN ComputeReservoirDiversions("Diversion Basin")) DO
  result<0> [] = result<1>
END FOR EACH
```

In Figure 5.1, the Diversion slot on each reservoir is linked to the Diversion slot on the Diversion Object. The demands are represented by the Water User objects. The Supply From Reservoirs slot on each Water User is linked to the Multi Outflow slot on each Diversion Object that can act as a supply for that demand. The rule sets the values on the Supply From Reservoirs slots. These propagate to the Multi Outflow slots on the connected Diversion Objects. The Diversion objects solve for their Diversion slot. The Diversion values are passed to the Diversion slot on the Reservoir object and the water is removed from the Reservoir. On each reservoir, the Conservation and Flood Pools method in the Operating Levels category should be selected to instantiate the Bottom of Conservation Pool slot.

Figure 5.1 Schematic Diagram for ComputeReservoirDiversions Function



• DateMax

This function returns the later of two dates.

Description	Compare two dates and return that which is chronologically greater.	
Type	DATETIME	
Arguments	**Type**	**Meaning**
1	DATETIME	a date
2	DATETIME	another date
Evaluation	The two dates are resolved and compared, the one which is chronologically greater is returned.	
Comments		

Syntax Example

```
DateMax(@"t", @"January 1, 2001")
```

Return Example

- If current timestep is March 2, 2002:
  `@"24:00 March 2, 2002"`
- If current timestep is May 3, 1999:
  `@"24:00 January 1, 2001"`

• DateMin

This function returns the earlier of two dates.

Description	Compare two dates and return that which is chronologically lesser.	
Type	DATETIME	
Arguments	**Type**	**Meaning**
1	DATETIME	a date
2	DATETIME	another date
Evaluation	The two dates are resolved and compared, the one which is chronologically lesser is returned.	
Comments		

Syntax Example

```
DateMin(@"t", @"January 1, 2001")
```

Return Example

- If current timestep is May 2, 2002:
  `@"24:00 January 1, 2001"`
- If current timestep is May 3, 1999:
  `@"24:00 May 3, 1999"`

DatesInPeriod

Description	Given a periodic slot and a date, this function returns an ordered list of dates representing the beginning time of each interval which begins in the specific period containing the input (reference) date.	
Type	LIST {DATETIME}	
Arguments	**Type**	**Meaning**
1	SLOT	a periodic slot
2	DATETIME	a reference date
Evaluation	Every periodic slot has a period associated with it and this period is divided into intervals. Intervals are either regular (e.g., Days) or irregular (e.g., the beginning of one interval might be 8:00 July 3 of each period). One can map a period (divided into intervals) onto a time line, leading to several specific periods (divided into specific intervals). For example, the period "Year" maps onto specific periods corresponding to each year, such as the specific period which is the year 2003. Providing a reference date serves to indicate a specific period, and this function returns the dates corresponding to the beginning of each time interval which begins in that specific period.	
Comments	When the beginning of an interval occurs exactly at a period boundary (e.g., an interval beginning at "0:00 January 1" with an 1 Year period), then we consider that interval to begin in the period occurring after midnight, not the one before. Not all time intervals (rows) defined in a Periodic Slot will correspond to intervals in a specific period. For example, for a period of Month, an interval might be defined which begins at "12:00 Day 30". This interval does not exist in all months and so for example if the reference date is 12:00 February 1, 2003, then the list returned by this function would not include the date 12:00 February 30, 2003.	

Syntax Example

```
DatesInPeriod(TableA.AvePrecipitation, @"January 1, 2001")
```

Return Example

If TableA.AvePrecipitation has three rows: 0:00 January 1; 6:00 June 15; and 24:00 September 1:

```
{ @"24:00 December 31, 2000", @"6:00 June 15, 2001", @"24:00 September 1, 2001"}
```

DateToNumber

Description	Given a Date/Time value, returns that date encoded as a numeric value of the type used by slots to containing date/time values.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	DATETIME	The date/time value to encode as a numeric value.
Comments	Slots representing date/time values have unit type DateTime. The date/time value need not be fully specified, but the return value should only be assigned to a slot with appropriate units. For example, if the value @"January 1" should only be assigned to a slot with units "MonthAndDay".	

Syntax Example

```
DateToNumber(@"t")
```

Return Example

`6508706400.00`

Note: This is equivalent to “06:00 April 3, 2006” (FullDateTime).

Use Example

This function should be used in conjunction with dates on series slots (see “DateTime Values in Slots” in *User Interface*) and the NumberToDate function (see “NumberToDate”). For a use example, see “Access to DateTime Values Using RPL” in *User Interface*.

Destinations

This function evaluates to the list of user-defined Destinations.

Description	This function returns a list of the names of all Destinations defined in the Water Accounting System Configuration.
Type	LIST {STRING}
Arguments	none
Evaluation	
Comments	Destinations are properties of Supplies. The returned list does not include the default (NONE) Destination.

Syntax Example

`Destinations()`

Return Example

`{"FarmerA", "City1", "City2"}`

DestinationsFromObjectReleaseType

This function evaluates to the list of Destinations which represent outflows from an Object of a specified Release Type.

Description	This function returns a list of unique names of Destination Type of Supplies which represent outflows from a specified Object, and which have the indicated Release Type.	
Type	LIST {STRING}	
Arguments	**Type**	**Meaning**
1	OBJECT	The Object.

2	STRING	Release Type name or NONE or ALL
Evaluation	The set of Accounts on the Object are examined. The outflow Supplies on those Accounts which link a different downstream Object and which have the indicated Release Type are considered. The names of the Destination Types of those Supplies are added to the returned list -- but any given Destination Type name will appear on the list only once. If the Release Type argument is NONE, then only Supplies having the default (unassigned) Release Type are considered. If the Release Type argument is ALL, then that attribute is ignored when considering Supplies.	
Comments	Destination Type and Release Types are properties of Supplies. The returned list can include the default (NONE) Destination Type. Supplies which represent *internal flows* between two Accounts on the Object are not considered.	

Syntax Example

```
DestinationsFromObjectReleaseType(%"Big Reservoir", "Account Fill")
```

Return Example

```
{"FarmerA", "City2"}
```

• DispatchCount

Description	Returns the number of dispatch method executions that have occurred since the beginning of the current run.
Type	NUMERIC
Arguments	none
Comments	Returns the number of dispatch method executions that have occurred since the beginning of the current run, if called during a dispatching run (Simulation or Rulebased Simulation). Otherwise, returns the total number of dispatch executions in the previous dispatching run.

Syntax Example

```
DispatchCount()
```

Return Example

```
12,345
```

• DispatchEndDate

This function returns the last timestep in the model for which dispatching is allowed.

Description	The last dispatch timestep.
Type	DATETIME

Arguments	none
Evaluation	Returns the DATETIME that is the last timestep at which the current controller allows dispatching. If this function is called from a context in which the current controller does not have a last dispatch timestep (i.e. optimization), then the end date of the run is returned.
Comments	The Number of Post-Run Dispatch Timesteps is set on the Run Control Parameters dialog for Simulation or Rulebased Simulation. See “Number of Post-Run Dispatch Timesteps (Simulation) and Number of Post-Run Dispatch Timesteps (RBS)” in *User Interface* for details on changing the Number of Post-Run Dispatch Timesteps.

Syntax Example

```
DispatchEndDate()
```

Return Example

If the Run Finish is March 19, 2011 (timestep of 1 Day) and the Number of Post-Run Dispatch Timesteps is 3:

```
24:00 March 22, 2011
```

• DispatchTime

Description	Returns the accumulated time spent executing dispatch methods since the beginning of the current run.
Type	NUMERIC
Arguments	none
Comments	Returns the accumulated time spent executing dispatch methods since the beginning of the current run, if called during a dispatching run (Simulation or Rulebased Simulation). Otherwise, returns the total time spent executing dispatch methods in the previous dispatching run

Syntax Example

```
DispatchTime()
```

Return Example

```
67.8 seconds
```

• Div

This function computes the integer division of two numbers.

Description	Integer division of two numbers.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the numerator
2	NUMERIC	the units to which to convert the numerator
3	NUMERIC	the denominator

4	NUMERIC	the units to which to convert the denominator
Evaluation	Converts numerator and denominator into the specified units, then returns the integral division of the converted values, where integral division of x and y is defined as: $\mathrm{Div}(x, y) \equiv \left\lfloor \frac{\lfloor x \rfloor}{\lfloor y \rfloor} \right\rfloor$	
Comments	If the denominator is equal to zero, this function aborts the run with an error. Each of the units arguments must have units which are compatible with the value with which they are associated, otherwise the run is aborted with an error. This function does not use the scalar portion of either of the units arguments.	

Syntax Example

```
Div(10.5 "m", 0.0 "ft", 2.4 "sec", 0.0 "sec")
```

Return Example

```
17.00 "0.304800 m/s"
```

• ElevationToArea

These function performs a lookup in a Reservoir object's Elevation Area Table based on a given elevation and evaluates to the corresponding area.

Description	Find the surface area corresponding to a reservoir's elevation.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	reservoir object
2	NUMERIC	pool elevation
Evaluation	The pool elevation argument is looked up in the Pool Elevation column, of the Elevation Area Table, of the reservoir object argument, to determine the Surface Area. If the exact elevation is not in the table, the lookup performs a linear interpolation between the two nearest bounding elevations and their corresponding surface areas.	
Mathematical Expression	$area = area_{(lesser)} + \frac{area_{(greater)} - area_{(lesser)}}{elev_{(greater)} - elev_{(lesser)}}(elevation - elev_{(lesser)})$	
Comments	If the object is not a reservoir, or the reservoir does not have an Elevation Area Table, the function aborts the run with an error (CRSSEvaporationCalc, DailyEvaporationCalc, PanAndIceEvaporation, or InputEvaporation must be selected as the Evaporation and Precipitation Category selected Method. This function will issue an error if the Time Varying Elevation Area method is selected (see "Time Varying Elevation Area" in *Objects and Methods*). Instead, use the ElevationToAreaAtDate function (see "ElevationToAreaAtDate").	

Syntax Example

```
ElevationToArea(%"Lake Mead", 1210.03 "ft")
```

Return Example

```
634547087.2 “m2”
```

• ElevationToAreaAtDate

This function performs a lookup in the Reservoir object's Elevation Area Table or **Elevation Area Table Time Varying** based on a given elevation and datetime and evaluates to the corresponding surface area. This function must be used when the Time Varying Elevation Area method is selected. Otherwise, the ElevationToArea function can be used and no DATETIME argument is required. See "Elevation Area Table Time Varying" in *Objects and Methods* and "Time Varying Elevation Area" in *Objects and Methods* for details.

Description	Find the surface area corresponding to a reservoir's elevation.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	reservoir object
2	NUMERIC	pool elevation
3	DATETIME	the datetime at which to do the conversion
Evaluation	On the specified reservoir object argument, if the Time Varying Elevation Area method is selected (see "Time Varying Elevation Area" in *Objects and Methods*), the function will reference the Elevation Area Table Time Varying table (see "Elevation Area Table Time Varying" in *Objects and Methods*). The function will select the appropriate column to use based on the datetime argument. On timesteps that exactly match a modification date, the previous column is used. The relationship changes only at the end of that timestep and is taken into account when the reservoir dispatches. For this algorithm the previous column relationship is used. Otherwise, the Elevation Area Table is used and the datetime is ignored. Then, the pool elevation argument is looked up in the Pool Elevation column to determine the Surface Area from the appropriate Surface Area column. If the exact elevation is not in the table, the lookup performs a linear interpolation between the two nearest bounding elevations and their corresponding surface areas.	
Mathematical Expression	$area = area_{(lesser)} + \frac{area_{(greater)} - area_{(lesser)}}{elev_{(greater)} - elev_{(lesser)}}(elevation - elev_{(lesser)})$	
Comments	If the object is not a reservoir, or the reservoir does not have an Elevation Area Table or Elevation Area Table Time Varying, the function aborts the run with an error (i.e. a method must be selected in the Evaporation and Precipitation Category).	

Syntax Example

```
ElevationToAreaAtDate(%"Lake Mead", 1210.03 "ft", @”t”)
```

Return Example

```
634547087.2 “m2”
```

• ElevationToMaxRegulatedSpill

This function performs a lookup in a Reservoir object's Regulated Spill Table based on a given elevation and evaluates to the corresponding maximum regulated spill.

Description	Find the maximum regulated spill at a given reservoir elevation.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	reservoir object
2	NUMERIC	pool elevation
3	DATETIME	datetime context for unit conversions
Evaluation	The pool elevation argument is looked up in the Pool Elevation column, of the Regulated Spill Table, of the reservoir object argument, to determine the Max Regulated Spill. If the exact elevation is not in the table, the lookup performs a linear interpolation between the two nearest bounding elevations and their corresponding maximum regulated spills.	
Mathematical Expression	$max\ spill = max\ spill_{(lesser)} + \frac{max\ spill_{(greater)} - max\ spill_{(lesser)}}{elev_{(greater)} - elev_{(lesser)}}(elevation - elev_{(lesser)})$	
Comments	If the object is not a reservoir, or the reservoir does not have a Regulated Spill Table, the function aborts the run with an error (Regulated; Regulated and Unregulated; Regulated and Bypass; Regulated, Bypass and Unregulated; or Bypass, Regulated and Unregulated must be the Spill category selected method).	

Syntax Example

```
ElevationToMaxRegulatedSpill(%"Lake Mead", 1210.03 "ft", @"t")
```

Return Example

```
1783.25 “cms”
```

• ElevationToStorage

This function performs a lookup in a Reservoir object’s Elevation Volume Table based on a given elevation and evaluates to the corresponding storage.

Description	Find the reservoir storage at a given elevation.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	reservoir object
2	NUMERIC	pool elevation

Evaluation	The pool elevation argument is looked up in the Pool Elevation column, of the Elevation Volume Table, of the reservoir object argument to determine the Storage. If the exact elevation is not in the table, the lookup performs a linear interpolation between the two nearest bounding elevations and their corresponding storage values.
Mathematical Expression	$storage = storage_{(lesser)} + \frac{storage_{(greater)} - storage_{(lesser)}}{elev_{(greater)} - elev_{(lesser)}}(elevation - elev_{(lesser)})$
Comments	If the object is not a reservoir, the function aborts the run with an error. If the reservoir is a Slope Power Reservoir, the calculation is based only on level storage and does not include any wedge storage effects. This function will issue an error if the Time Varying Elevation Volume method is selected (see “Time Varying Elevation Volume” in *Objects and Methods*). Instead, use the ElevationToStorageAtDate function described next.

Syntax Example

```
ElevationToStorage(%"Lake Mead", 1210.03 "ft")
```

Return Example

```
2212323.233 "m3"
```

• ElevationToStorageAtDate

This function performs a lookup in the Reservoir object’s Elevation Volume Table or Elevation Volume Table Time Varying based on a given elevation and datetime and evaluates to the corresponding volume. This function must be used when the Time Varying Elevation Volume method is selected. Otherwise, the ElevationToStorage function can be used and no DATETIME argument is required. See “Elevation Volume Table Time Varying” in *Objects and Methods* and “Time Varying Elevation Volume” in *Objects and Methods*.

Description	Find the volume corresponding to a reservoir’s elevation.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	reservoir object
2	NUMERIC	pool elevation
3	DATETIME	the datetime at which to do the conversion

Evaluation	On the specified reservoir object argument, if the Time Varying Elevation Volume method is selected, the function will reference the Elevation Volume Table Time Varying table. The function will select the appropriate column to use based on the datetime argument. On timesteps that exactly match a modification date, the previous column is used. The relationship changes at the end of that timestep and is taken into account when the reservoir dispatches. For this algorithm, the previous column relationship is used. Otherwise, the Elevation Volume Table is used and the datetime is ignored. Then, the pool elevation argument is looked up in the Pool Elevation column to determine the Volume from the appropriate column. If the exact elevation is not in the table, the lookup performs a linear interpolation between the two nearest bounding elevations and their corresponding surface areas.
Mathematical Expression	$storage = storage_{(lesser)} + \frac{storage_{(greater)} - storage_{(lesser)}}{elev_{(greater)} - elev_{(lesser)}}(elevation - elev_{(lesser)})$
Comments	If the object is not a reservoir, or the reservoir does not have an Elevation Volume Table or Elevation Volume Table Time Varying, the function aborts the run with an error.

Syntax Example

```
ElevationToStorageAtDate(%"Lake Mead", 1210.03 "ft", @"t")
```

Return Example

```
634547087.2 "m3"
```

• ElevationToUnregulatedSpill

This function performs a lookup in a Reservoir object's Unregulated Spill Table based on a given elevation and evaluates to the corresponding unregulated spill.

Description	Find the unregulated spill at a given reservoir elevation.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	reservoir object
2	NUMERIC	pool elevation
3	DATETIME	datetime context for unit conversions

Evaluation	The pool elevation argument is looked up in the Pool Elevation column, of the Unregulated Spill Table, of the reservoir object argument, to determine the Unregulated Spill. If the exact elevation is not in the table, the lookup performs a linear interpolation between the two nearest bounding elevations and their corresponding unregulated spills.
Mathematical Expression	$unreg\ spill = unreg\ spill_{(lesser)} + \frac{unreg\ spill_{(greater)} - unreg\ spill_{(lesser)}}{elev_{(greater)} - elev_{(lesser)}} \times (elevation - elev_{(lesser)})$
Comments	If the object is not a reservoir, or the reservoir does not have an Unregulated Spill Table, the function aborts the run with an error (Unregulated; Regulated and Unregulated; Regulated, Bypass and Unregulated; or Bypass, Regulated and Unregulated must be the Spill selected method).

Syntax Example

```
ElevationToUnregulatedSpill(%"Lake Mead", 1210.03 "ft", @"t")
```

Return Example

```
1212.25 "cms"
```

• Exp

Description	Exponentiation of a dimensionless quantity.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the operand
2	NUMERIC	the exponent
Evaluation	Returns the result of exponentiating the operand to the power given by the exponent. The return value is dimensionless (has no units).	
Comments	The exponent is not restricted to being an integer (as with the "^" operator), but it is an error for the operand to have units.	

Syntax Example

```
Exp(16.0, 0.5)
```

Use Example

```
4.0000 "None"
```

• FilterByObjType

This function evaluates to a list of objects containing objects from the original list which match the specified types.

Description	Filter a list of objects to include only objects of the specified types.	
Type	LIST {OBJECT}	
Arguments	**Type**	**Meaning**

1	LIST	list of objects
2	LIST	list of object types to include, where each object type is expressed as a STRING.
Evaluation	The list of object types to include is parsed and mapped to RiverWare object types. Then, the list of objects is evaluated in order, and each object which is one of the requested object types is added to the returned list. ***The spellings and capitalization of objects can be found in the Subbasin Manager under the Automatic tab.***	
Mathematical Expression	$\{OBJECT\} = \{OBJECT\} \cap \{OBJECT\ TYPE\}$	
Comments	The order of objects is preserved from the argument object list to the returned object list. The list arguments may contain any number of items. If either of the arguments is an empty list, the function evaluates to an empty list.	

Syntax Example

```
FilterByObjType( {%"Lake Mead", %"Lake Powell", %"Virgin River"},
{"LevelPowerReservoir"} )
```

Syntax Example

```
{%"Lake Mead",%"Lake Powell"}
```

FlattenList

Description	This function takes a list and replaces any lists contained within that list with the individual items from those lists.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST	the list to be flattened
Evaluation	For each item in the input list, if the item is not a list, it is appended to the answer list, if it is a list, then it is flattened and then all of its items are appended to the answer list in turn.	
Comments		

Syntax Example

```
FlattenList( {1, {2, 3}, {{4}} } )
```

Return Example

```
{1, 2, 3, 4}
```

FloodControl

This function invokes the selected Flood Control method on a computational subbasin; see “Flood Control” in *Objects and Methods*.

Description	Invokes computational subbasin’s selected Flood Control method.	
Type	LIST { LIST { SLOT, NUMERIC, OBJECT } }	
Arguments	**Type**	**Meaning**

1	STRING	the name of the computational subbasin
Evaluation	Runs the selected Flood Control method on the subbasin. Returns a list of { slot, value, object } sets. For each reservoir in the subbasin, three sets may be returned: one for the Outflow slot, one for the Flood Control Release slot on the reservoir, and one for the Target Balance Level on the reservoir.	
Comments	The calling rule is expected to make the assignments of the values to the slots. Typically, this function should be called only once per timestep. To constrain this, use the following as an execution constraint: NOT(HasRuleFiredSucessfully("This Rule")) See “Flood Control” in *USACE-SWD Modeling Techniques* for details on using this function for USACE-SWD.	

Syntax Example

```
FloodControl("Flood Basin")
```

where “Flood Basin” contains Res1 and Res2.

Return Example

```
{ {"Res1.Outflow", 6344.32 "cfs", "Res1"},
  {"Res1.Flood Control Release", 6344.32 "cfs", "res1"},
  {"Res1.Target Balance Level", 8.32, "res1"},
  {"Res2.Outflow", 3243.02 "cfs", "Res2"},
  {"Res2.Flood Control Release", 2312.20 "cfs", "Res2"},
  {"Res2.Target Balance Level", 8.32, "Res2"} }
```

Use Example

```
FOREACH(LIST triplet IN FloodControl("Flood Basin")) DO
  (triplet<0>)[] = triplet<1>
ENDFOREACH
```

• Floor

This function rounds a numeric value down to the nearest multiple of a numeric factor.

Description	The floor numeric operation, to a multiple of a factor.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the value
2	NUMERIC	the factor
Evaluation	Converts the value into the units of the factor, then returns the largest integral multiple of the factor which is not greater than the converted value. The returned value has the units of the factor.	
Comments	If the scalar portion of the factor is 1.0, then this function simply returns the floor of the value expressed in the units of the factor. If the two arguments are of a different unit type, this function aborts the run with an error. See also “Ceiling” and “RoundToFactor”.	

Syntax Example

```
Floor("Wet Reservoir.Pool Elevation"[], 100.0 "ft")
```

Return Example

If Wet Reservoir.Pool Elevation[] is 5343.35ft:

```
5300.0 ft
```

FlowToVolume

This function evaluates to the volume of water corresponding to a flow over a timestep.

Description	The volume of water resulting from a steady flow over a timestep.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	constant flow to be converted
2	DATETIME	timestep over which to convert
Evaluation	The number of seconds in the timestep of the datetime argument is determined. Then, the flow argument is multiplied by this number of seconds. Returns value in units of volume.	
Mathematical Expression	$volume = flow \times \Delta t_{(current\ timestep)}$	
Comments	If the flow argument is entered in units containing a "/month" component, it is scaled to reflect the length of the month indicated by the timestep argument before being multiplied by this timestep length. When called from a series expression slot, the flow to volume conversion uses the run timestep size, which may differ from the expression slot timestep size. (This is different from the VolumeToFlow function, which uses the expression slot timestep size for the conversion.)	

Syntax Example

```
FlowToVolume(Lake Powell.Inflow[], @"t")
```

Return Example

```
6155584.04 "m3"
```

Fraction

This function returns the fractional remainder after dividing two numbers.

Description	The fractional remainder after division.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the numerator

2	NUMERIC	the denominator
Evaluation	Converts the numerator into the units of the denominator, divides the result by the denominator, then returns the fractional portion of the division. In other words: $\mathrm{Fraction}(x, f) \equiv \frac{x}{f} - \mathrm{Floor}(x, f)$ The returned value has the units of *factor*.	
Comments	If the scalar portion of the denominator is 1.0, then this function simply returns the fractional portion of the first argument when it is expressed in the units of the denominator. If the values are of a different unit type, this function aborts the run with an error.	

Syntax Example

```
Fraction("Whitewater Creek.Inflow"[], 1.0 "cms")
```

Return Example

If Whitewater Creek.Inflow is 134.3 cfs:

```
0.80295250 "cms"
```

Get3DTableVals

Description	Return the contents of a Table Slot that is structured for 3D table interpolation.	
Type	LIST{LIST {NUMERIC LIST{NUMERIC} LIST{NUMERIC}}}	
Arguments	**Type**	**Meaning**
1	SLOT	the table slot whose values are to be returned.
2	NUMERIC	z column index (zero-based)
3	NUMERIC	x column index (zero-based)
4	NUMERIC	y column index (zero-based)
Evaluation	Returns the contents of a 3D table as a list of the table values associated with successive z value. For each distinct z value in the table slot, the returned list contains a sublist with the following values: 1. Current z value 2. List of x values associated with the current z value 3. List of y values associated with the current z value	
Comments	Units are not required for row and column indices and, if provided, will be ignored. In the context of rulebased simulation, if one of the slot’s values is NaN, the function exits the rule with an early termination.	

Syntax Example

```
Get3DTableVals( Wet Reservoir.Plant Power Table, 0, 1, 2 )
```

Operating Head (m)	Turbine Release (cms)	Power (HP)
320	0.00	0
320	120.32	470
340	5.00	10
340	127.32	500

Return Example

```
{ {320.00 "m", {0.00 "cms", 120.32 "cms"}, {0 "HP", 470 "HP"} },
  {340.00 "m", {5.00 "cms", 127.32 "cms"}, {10 "HP", 500 "HP"} } }
```

• Get3DTableValsSkipNaN

Description	Return the contents of a Table Slot that is structured for 3D table interpolation.	
Type	LIST{LIST {NUMERIC LIST{NUMERIC} LIST{NUMERIC}}}	
Arguments	**Type**	**Meaning**
1	SLOT	the table slot whose values are to be returned.
2	NUMERIC	z column index (zero-based)
3	NUMERIC	x column index (zero-based)
4	NUMERIC	y column index (zero-based)
Evaluation	Returns the contents of a 3D table as a list of the table values associated with successive z value. For each distinct z value in the table slot, the returned list contains a sublist with the following values: 1. The current z value 2. The list of x values associated with the current z value 3. The list of y values associated with the current z value	
Comments	Units are not required for row and column indices and, if provided, will be ignored. In the context of rulebased simulation, if one of the slot’s values is NaN, all values in that row and rows below it are ignored.	

Syntax Example

```
Get3DTableValsSkipNaN( Wet Reservoir.Plant Power Table, 0, 1, 2 )
```

Operating Head (m)	Turbine Release (cms)	Power (HP)
320	0.00	0
320	120.32	470
340	5.00	10
340	127.32	500
NaN	NaN	NaN

Return Example

```
{ {320.00 "m", {0.00 "cms", 120.32 "cms"}, {0 "HP", 470 "HP"} },
  {340.00 "m", {5.00 "cms", 127.32 "cms"}, {10 "HP", 500 "HP"} } }
```

GetAccountFromSlot

Description	Return the name of a slot's account.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	SLOT	The slot whose account is returned.
Comments	It is an error if the slot is not on an account.	

Syntax Example

```
GetAccountFromSlot($"ResA^Municipal.Inflow")
```

Return Example

```
"Municipal"
```

GetAllNamedBasins

This function evaluates to a list containing the names of the user defined subbasins in a model.

Description	The names of all user defined subbasins in the current model, expressed as strings.
Type	LIST {STRING}
Arguments	none
Evaluation	The function first retrieves a list of all defined subbasins in the model, then filters out any automatic subbasins (object type basins generated by RiverWare).
Comments	If there are no user defined subbasins in the model, this function evaluates to an empty list.

Syntax Example

```
GetAllNamedBasins()
```

Return Example

```
{"Upper", "Flood Control", "Lower"}
```

GetColMapVal

Description	Get a column map value from a periodic slot given a date and a value. This is the inverse of the way values are usually accessed in periodic slots with column maps (i.e., given a date and column map value, find the corresponding slot value).	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	The periodic slot to be accessed.

2	DATETIME	The date to be used to index into the time dimension of the Periodic Slot (its row map).
3	NUMERIC	The value to use for the lookup, having the same type of units as the values in the periodic slot itself.
Evaluation	If the default access method for the table is lookup, then we first find the row whose associated time interval contains the input date. We then find the two consecutive values in that row whose values bracket the input value. We then find the column map values associated with these two values, and return a value interpolated between them according to where the input value falls between its two bracketing values. If the default access method is interpolation then the procedure described above is followed for the row whose time interval follows the given date, and the return value is interpolated between the values found for the two rows.	
Comments	The input slot must be a periodic slot with a column map, the numeric value must have units compatible with the units of the periodic slot, for the relevant time intervals, the slot values must be either a monotonically non-decreasing or monotonically non-increasing function of the column map values, and the input value must fall within the domain of that function. If there are multiple possible return values, i.e., if the input value appears for multiple columns, then the largest column map value is returned.	

Syntax Example

```
GetColMapVal(Meander Res.Operating Level Table, @"t", 1.0)
```

Return Example

```
2.323
```

• GetColumnIndex

Description	The index of the table slot or agg. series slot column whose name matches a string.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the table slot or agg. series slot in which to find a column
2	STRING	the name of the column to match
Evaluation	The labels of the slot columns are compared to the string argument until a match is found.	
Comments	Slot column and row indices are zero based and have units of type [NONE]. If the specified slot is not a table slot or agg. series slot, or the specified string is not the label of a column on the slot, this function aborts the run with an error. If several columns of the slot match the string argument, this function evaluates to the index of the left-most matching column. The matching process treats sequences of white space characters as a blank; for example, the input string " ab c " will match a column with the label "ab c." This allows the method to match labels that are displayed on multiple lines because they contain a carriage return character.	

Syntax Example

```
GetColumnIndex(RiverData.Minimum Flow, "Dolores")
```

Return Example

```
0.000
```

GetDate

Description	Interpret a string as a date.	
Type	DATETIME	
Arguments	Type	Meaning
1	STRING	Textual representation of a date/time.
Evaluation	Returns the date which corresponds to the input text. Legal text is the same as is legal for symbolic date/times. For example, the expression: GetDate("January 1, Current Year") is exactly equivalent to the expression: @"January 1, Current Year".	

Syntax Example

```
GetDate("January 20, 1996")
```

Return Example

```
@"24:00 January 20, 1996"
```

GetDates

This function evaluates to a list of datetimes; from a start datetime to an end datetime, with a given interval separating the dates.

Description	Generate a list of datetimes between two datetimes at a given interval.	
Type	LIST {DATETIME}	
Arguments	Type	Meaning
1	DATETIME	starting datetime
2	DATETIME	ending datetime
3	STRING	string representation of a datetime interval expressed as an integer, a space, and a time unit.

Evaluation	The starting datetime and ending datetime; which may be specified symbolically, are converted into actual datetimes. The string representation of the interval is resolved into a time length. Then, a list is created beginning with the starting datetime. The time length is added to each previous datetime in the list until the resulting datetime is later than the ending datetime.
Mathematical Expression	$\{DATETIME\} = \{start \le end, (datetime_{(n-1)} + interval) \le end, \ldots\}$
Comments	If the ending date is before the starting date, the function evaluates to an empty list. If the ending date is equal to the starting date, or if the time interval is larger than the interval between the starting and ending dates, the function evaluates to a list which only contains the start date. The accepted datetime interval units are: • hours or Hours • days or Days • weeks or Weeks • months or Months • years or Years

Syntax Example

```
GetDates(@"January 20, 1996", @"January Max DayOfMonth, 1996", "6 Hours"}
```

Return Example

```
{@"24:00 January 20, 1996", @"06:00 January 21, 1996", @"12:00 January 21, 1996", ...}
```

• GetDatesCentered

This function evaluates to a list of datetimes, centered around a given date.

Description	Generate a list of datetimes separated by a given interval, and centered at a given date. If desired, dates not within the run duration are filtered out of the list.	
Type	LIST {DATETIME}	
Arguments	**Type**	**Meaning**
1	DATETIME	center datetime
2	NUMERIC	number of dates to return in the list
3	STRING	string representation of a datetime interval expressed as an integer, a space, and a time unit
3	BOOLEAN	whether to limit return dates to those within the run

Evaluation	The center datetime, which may be specified symbolically, is converted into an actual datetime. The string representation of the interval is resolved into a time length. Then a list is created with the given number of dates, each separated by the given time interval. The center date is always included in this list, with an equal number of dates appearing before and after it (in the case of an odd number of dates). If their are an even number of dates, then there is one more date appearing before the center date than appear after. After the list has been created, if the user has specified that they only want dates within the run duration, then all other dates are filtered out of the return list.
Mathematical Expression	$\{DATETIME\} = \left\{ d + -\left\lceil \frac{n}{2} \right\rceil \text{offset},\ d + -\left(\left\lceil \frac{n}{2} \right\rceil - 1 \right) \text{offset},\ \ldots,\ d + \left\lfloor \frac{n}{2} \right\rfloor \text{offset} \right\}$
Comments	The accepted datetime interval units are: • hours or Hours • days or Days • weeks or Weeks • months or Months • years or Years

Syntax Example

```
GetDatesCentered(@"January 20, 1996", 3, "6 Hours", true}
```

Return Example

```
{@"18:00 January 20, 1996", @"24:00 January 20, 1996", @"06:00 January 21, 1996"}
```

• GetDayOfMonth

This function evaluates to a number which represents the day of the month of the given datetime in units of time.

Description	The day of the month as a unit of time.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	DATETIME	the datetime whose day of month to determine
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime. Then, the day of the month in which the datetime is, is determined. This function requires that the datetime be at least partially specified with a valid month and day, E.g. @"January 1" or @"Current Month 23" will work.	
Comments	When displayed, the return value will be displayed according to the active unit scheme’s time unit type rule. For example, if the active unit scheme displays Time values as Hours, then the return value for @”January 2” will be displayed as 48 “hours”. To convert the return value into a dimensionless value representing the number of days, divide it by 1 “day”. As elsewhere in RiverWare 24:00 hours is considered to be the day which is ending, and 00:00 hours is considered to be the day which is just beginning.	

Syntax Example

```
GetDayOfMonth(@"February 23, 1996")
```

Return Example

`23.0 “day” or 553 “hour”`

• GetDayOfYear

This function evaluates to a number which represents the day of the year of the given datetime.

Description	The day of the year as a one-based integer in units of time.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	DATETIME	the datetime whose day of the year to determine
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime. Then, the day of the year in which the datetime is contained, is determined. This function requires that the specified datetime resolve to a fully specified datetime or an error will occur.	
Comments	When displayed, the return value will be displayed according to the active unit scheme’s time unit type rule. For example, if the active unit scheme displays Time values as Hours, then the return value for @”January 2” will be displayed as 48 “hours”. To convert the return value into a dimensionless value representing the number of days, divide it by 1 “day”. As elsewhere in RiverWare, 24:00 hours is considered to be the day which is ending, and 00:00 hours is considered to be the day which is just beginning.	

Syntax Example

```
GetDayOfYear(@"February 23, 1996")
```

Return Example

`54.0 “day” or 1296 “hour”`

• GetDaysInMonth

This function evaluates to the number of days in the month of the given datetime.

Description	The number of days in the month in units of time.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**

1	DATETIME	the datetime of any time within the month
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime. Then, the number of days in the month in which the datetime is contained, is determined. This function requires that the specified datetime resolve to a fully specified datetime or an error will occur.	
Comments	When displayed, the return value will be displayed according to the active unit scheme's time unit type rule. For example, if the active unit scheme displays Time values as Hours, then the return value for @"January 2" will be displayed as 744 "hours". To convert the return value into a dimensionless value representing the number of days, divide it by 1 "day". As elsewhere in RiverWare, 24:00 hours is considered to be the day which is ending, and 00:00 hours is considered to be the day which is just beginning.	

Syntax Example

```
GetDaysInMonth(@"February 23, 1996")
```

Return Example

```
29.0 “day” or 696 “hour”
```

• GetDisplayVal

Description	This function takes a series or periodic slot and a date and returns the value of the slot at the given date, in units based on the display scale and units for that slot.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the series or periodic slot whose value is to be returned
2	DATETIME	the datetime of the value to be returned
Evaluation		
Comments	The function returns an error and aborts the run if the input slot is not a series or periodic slot, if the date is not fully specified, or if the date is not contained in the series.	

Syntax Example

```
GetDisplayVal(MyReservoir.Outflow, @"24:00 February 23, 1996")
```

Return Example

```
3.03012926 "1000 * cfs"
```

• GetDisplayValByCol

Description	This function takes an agg series slot or periodic slot, a date, and a column, and returns the value of the slot in the given column and at the given date, in units based on the display scale and units for that slot.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the series or periodic slot whose value is to be returned

2	DATETIME	the datetime of the value to be returned
3	NUMERIC	the column, interpreted as a 0-based integral index
Evaluation	The function returns and error and aborts the run if the input slot is not of an appropriate type, if the date is not fully specified, or if the date is not contained in the series.	

Syntax Example

```
GetDisplayValByCol(MyData.Flow, @"February 23, 1996", 1.0)
```

Return Example

```
3.03012926 "1000*cfs"
```

• GetElementName

Given an element in an aggregate object, this function returns its name.

Description	Return the name of an element in an aggregate object, without the name of the object's name prepended.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	OBJECT	the element of an aggregate object (e.g., a WaterUser within an AggDiversionSite) whose name is to be returned.
Evaluation	The function returns the name of the element object.	
Comments	This function returns only the name of the element itself, without the name of the parent aggregate object. If the full name is desired, then one may use the built-in RPL operation STRINGIFY. If the object argument is not an element of an aggregate object, then the run is aborted with an error.	

Syntax Example

```
GetElementname(% "Below Abiquiu Diversions:Chamita")
```

Return Example

```
"Chamita"
```

• GetEnsembleTraceValue

Given a keyword name for trace metadata when using an ensemble, return the keyword value for the current trace executing in a run.

Description	Return the value for a trace keyword for the current trace executing in a run.	
Type	STRING	
Arguments	**Type**	**Meaning**

1	STRING	the name of a trace metadata keyword.
Evaluation	The function returns the value for the trace keyword for the currently executing run.	
Comments	If the function is called outside of a run or if the trace metadata keyword cannot be found, then the function fails. This function would typically be called during a multiple run when input ensembles are used in the MRM configuration. If a single trace is configured for an ensemble dataset to use outside of a multiple run and a DMI with this dataset is invoked during a single run, the metadata for that trace would also be available to query.	

Syntax Example

```
GetEnsembleTraceValue("Hydrology")
```

Return Example

```
"Dry"
```

• GetEnsembleValue

Given a keyword name for ensemble metadata when using an ensemble, return the keyword value for the metadata for that run.

Description	Return the value for an ensemble keyword for the current run.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	STRING	the name of an ensemble metadata keyword.
Evaluation	The function returns the value for the ensemble keyword for the currently executing run.	
Comments	If the function is called outside of a run or if the ensemble metadata keyword cannot be found, then the function fails. This function would typically be called during a multiple run when input ensembles are used in the MRM configuration. If a single trace is configured for an ensemble dataset to use outside of a multiple run and a DMI with this dataset is invoked during a single run, the metadata for that ensemble would also be available to query.	

Syntax Example

```
GetEnsembleValue("Hydrology")
```

Return Example

```
"Historical"
```

• GetEnsembleSeriesValue

This function returns the value for the appropriate trace's slot at the given timestep from the ensemble data set.

Description	Get the value for the slot at the specified timestep for the given trace from the ensemble data set.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**

1	OBJECT	The ensemble data set. This must be an data set of type ensemble; that is, the data must have been imported from an RDF file.
2	SLOT	The simulation slot for which to get the trace value. Note, this is the workspace slot not the data set slot.
3	DATETIME	The datetime at which to get the value.
4	NUMERIC	The trace number.
Evaluation	On the given ensemble data set, the function locates the appropriate trace for the given slot. Then it returns the slot's value at the given datetime.	
Comments	For more information on Ensemble Data Sets and the Data Analysis Tool, see "Data Analysis Tool (DAT)" in *Output Utilities and Data Visualization*.	

Syntax Example

```
GetEnsembleSeriesValue(%"MRM_Output", "BigRes.Outflow", @"April 26, 2008", 9.0)
```

This call assume there is an ensemble data set named MRM_Output that was imported from an RDF file. The ensemble data set contains at least 10 traces (10 slots) for BigRes.Outflow and the run includes April 26, 2008.

Return Example

```
1102.06 cfs
```

• GetJulianDate

This function evaluates to the Julian date of the given datetime.

Description	The Julian date of the timestep in units of NONE.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	DATETIME	the datetime whose Julian date to evaluate to
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime. Then, the Julian date of this timestep is determined. This function requires that the specified datetime resolve to a fully specified datetime or an error will occur.	
Comments	Julian Dates are represented as the number of days from noon GMT on January 1, 4713 B.C. (47120101 12:00 P.M. GMT). Julian Dates in RiverWare also include the decimal fraction of the day down to 0.00001, the equivalent of 1 second.	

Syntax Example

```
GetJulianDate(@"14:31:59 February 23, 1996")
```

Return Example

```
2450137.10554398
```

• GetLinkedObjs

Description	Given a slot, returns a list of the Objects which contain the slots to which the input slot is linked.	
Type	LIST {OBJECT}	
Arguments	**Type**	**Meaning**
1	SLOT	the slots whose links are to be explored
Evaluation	For each slot to which the input slot is linked, we determine if that slot is managed by a Objects; if so, it is added to the return list. Thus, an empty list is returned if the slot is not linked to any slots on a Objects.	
Comments	It is considered an error if the input slot is not a Series Slot.	

Syntax Example

```
GetLinkedObjs("Res A.Inflow")
```

Return Example

```
{%"Reach 1", %"Reach 2"}
```

• GetLowerBound

Description	Returns the lower bound for the specified series slot	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the slot whose bound is to be returned.
Evaluation		
Comments	It is considered an error if the specified slot is not a Series Slot with a valid lower bound. The lower bound is specified in the slot configuration.	

Syntax Example

```
GetLowerBound(“Res A.Power”)
```

Return Example

```
0.0 MW
```

• GetLowerBoundByCol

Description	Returns the lower bound for the column of the specified agg series slot.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the agg series slot whose bound is to be returned.

2	NUMERIC	the column index (0-based).
Evaluation		
Comments	It is considered an error if the input slot is not an Agg Series Slot with a valid lower bound for the given column. The lower bound is specified in the slot configuration.	

Syntax Example

```
GetLowerBoundByCol("Res A.Hydro Block Use", 3)
```

Return Example

```
0.0 MWH
```

• GetMaxOutflowGivenHW

This function evaluates to the maximum Outflow from a StorageReservoir, LevelPowerReservoir, or SlopePowerReservoir with the given Pool Elevation at the specified timestep.

Description	The maximum combined outflow of a reservoir, including outlet works or turbine release, and any possible regulated, unregulated, and/or bypass spills.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the reservoir object for which to calculate
2	NUMERIC	the given pool elevation; at the end of the specified timestep
3	DATETIME	the timestep at which to calculate

Evaluation	• **Unit Generator Power**: The turbine release is the sum of the maximum releases for each available turbine, as specified in the Generators Available and Limit table. Each turbine's maximum release is determined from a table interpolation in the Full Generator Flow table using the operating head as the lookup in the appropriate unit type's Head for Type n column. If the average pool elevation is less than the Minimum Power Elevation, the turbine release is zero. • **Peak Power Equation with Off Peak Spill**: The turbine release is the peak release over the entire timestep. This is calculated by iterating the selected Tailwater method and operating head calculation with a table interpolation in the Operating Head vs. Generator Capacity table. • **Peak Power and Peak and Base**: The turbine release is the peak flow over the entire timestep. This is calculated by iterating the selected Tailwater method and operating head calculation with a table interpolation in the Best Generator Flow table using the operating head as the lookup in the Head for Type 1 column. • **LCRPowerCalc**: Because this power method has no turbine release limit, a maximum outflow cannot be calculated. RiverWare issues an error message and aborts the execution of this rule. • **Unregulated Spill** (if an Unregulated Spill method is selected on the object): The maximum unregulated spill is determined from a table interpolation in the Unregulated Spill Table using the average pool elevation as the Pool Elevation. • **Regulated Spill** (if a Regulated Spill method is selected on the object): If the Regulated Spill slot is specified by the user (I, Z or R flag), the specified value is used as the maximum regulated spill. Otherwise the maximum regulated spill is determined from a table interpolation in the Regulated Spill Table using the average pool elevation in the Pool Elevation column. **Note:** If the MonthlySpill method is selected, the result of GetMaxOutflowGivenInflow is the value in the Maximum Controlled Release table slot. • **Bypass** (if a Bypass Spill method is selected on the object): If the Bypass slot is specified by the user (I, Z or R flag), the specified value is used as the maximum bypass. Otherwise the maximum bypass is determined from a table interpolation in the Bypass Table using the average pool elevation. All of the individual outflows are then summed to calculate the maximum outflow. • **Spill**: If the Spill slot is specified by the user (I, Z or R flag), the specified value is used as the maximum spill. The individual outflows are then summed to calculate the maximum outflow.
Mathematical Expression	$Outflow_{max} = release_{max} \text{ or } turbine\ release_{max} + unregulated\ spill_{max} + regulated\ spill_{max} + bypass_{max}$
Comments	The Tailwater Base Value is automatically added as a dependency to the calling rule. **Caution:** The Pool Elevation or Storage slot must have a valid value on the timestep before the timestep passed in on Argument 4. Otherwise, an error will be issued.

Syntax Example

```
GetMaxOutflowGivenHW(%"Glen Canyon Dam", 3704 "ft", @"June 3, 1983"}
```

Return Example

```
1283.7047 "cms"
```

• GetMaxOutflowGivenInflow

This function evaluates to the maximum Outflow from a StorageReservoir, LevelPowerReservoir, or SlopePowerReservoir with the given Inflow at the specified timestep.This function takes into account all side flows, sinks and sources. The inflow argument should be the inflow that would go into the Inflow slot on the reservoir. Since the function already considers Hydrologic Inflow, the hydrologic inflow value should NOT be included in the inflow argument.

Description	The maximum combined outflow of a reservoir, including outlet works or turbine release, and any possible regulated, unregulated, and/or bypass spills.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the reservoir object for which to calculate
2	NUMERIC	the inflow over the timestep, often the reservoir’s inflow slot value
3	DATETIME	the timestep at which to calculate

Evaluation	A convergence algorithm is used in this function; see “Reservoir Convergence” in *Objects and Methods* for details. The function iterates to convergence by computing the end of timestep storage and pool elevation, the average pool elevation over the timestep, and the following outflows: • **Release** (if the object is a StorageReservoir): The maximum release is determined from a table interpolation in the Max Release table using the average pool elevation as the lookup in the Pool Elevation column. • **Turbine Release** (if the object is a LevelPowerReservoir or SlopePowerReservoir): The maximum turbine release is determined based on the selected Power method. This calculation is iterative, since the maximum outflow impacts the reservoir tailwater elevation and operating head, which affect the maximum turbine release. The selected Tailwater method is used to determine the tailwater elevation. • **No Power Turbine Flow**: The turbine release is determined from a table interpolation in the Max Flow Through Turbines table using the average pool elevation as the lookup in the Reservoir Elevation column. • **Plant Power Coefficient**: The turbine release is determined from a table interpolation in the Max Turbine Q table using the operating head as the lookup in the Operating Head column. If the average pool elevation is less than the Minimum Power Elevation, the turbine release is zero. • **Plant Efficiency Curve**: The turbine release is determined from a table interpolation in the Auto Max Turbine Q table using the operating head as the lookup in the Operating Head column. If the average pool elevation is less than the Minimum Power Elevation, the turbine release is zero. • **Unit Generator Power**: The turbine release is the sum of the maximum releases for each available turbine, as specified in the Generators Available and Limit table. Each turbine’s maximum release is determined from a table interpolation in the Full Generator Flow table using the operating head as the lookup in the appropriate unit type’s Head for Type n column. If the average pool elevation is less than the Minimum Power Elevation, the turbine release is zero. • **Peak Power Equation with Off Peak Spill**: The turbine release is the peak release over the entire timestep. This is calculated by iterating the selected Tailwater method and operating head calculation with a table interpolation in the Operating Head vs. Generator Capacity table. • **Peak Power** and **Peak and Base**: The turbine release is the peak flow over the entire timestep. This is calculated by iterating the selected Tailwater method and operating head calculation with a table interpolation in the Best Generator Flow table using the operating head as the lookup in the Head for Type 1 column. • **LCRPowerCalc**: Because this power method has no turbine release limit, a maximum outflow cannot be calculated. RiverWare issues an error message and aborts the execution of this rule.

	• **Unregulated Spill** (if an Unregulated Spill method is selected on the object): The maximum unregulated spill is determined from a table interpolation in the Unregulated Spill Table using the average pool elevation as the Pool Elevation. • **Regulated Spill** (if a Regulated Spill method is selected on the object): If the Regulated Spill slot is specified by the user (I, Z or R flag), the specified value is used as the maximum regulated spill. Otherwise the maximum regulated spill is determined from a table interpolation in the Regulated Spill Table using the average pool elevation in the Pool Elevation column. **Note:** If the MonthlySpill method is selected, the result of GetMaxOutflowGivenInflow is the value in the Maximum Controlled Release table slot. • **Bypass** (if a Bypass Spill method is selected on the object): If the Bypass slot is specified by the user (I, Z or R flag), the specified value is used as the maximum bypass. Otherwise the maximum bypass is determined from a table interpolation in the Bypass Table using the average pool elevation. in the Pool Elevation column. • **Spill**: If the Spill slot is specified by the user (I, Z or R flag), the specified value is used as the maximum spill. Once the iteration has converged on an ending storage and pool elevation, all of the individual outflows are summed to calculate the maximum outflow
Mathematical Expression	$Outflow_{max} = release_{max}\ or\ turbine\ release_{max} + unregulated\ spill_{max} + regulated\ spill_{max} + bypass_{max}$
Comments	This function takes into account the following sources and sinks automatically, and thus they should not be included in the inflow value for Argument 2. • The Evaporation and Precipitation category selected Method. • The Bank Storage category selected Method. • The Seepage category selected Method. • Side inflows including: Inflow 2 (Slope Power Reservoir only), Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. These slots in addition to Tailwater Base Value are automatically added as dependencies to the calling rule. **Caution:** The Pool Elevation or Storage slot must have a valid value on the timestep before the timestep passed in on Argument 4. Otherwise, an error will be issued.

Syntax Example

```
GetMaxOutflowGivenInflow(%"Hoover Dam", 68651 "cfs", @"June, 1983}
```

Return Example

```
1283.7047 "cms"
```

• GetMaxOutflowGivenStorage

This function evaluates to the maximum Outflow from a StorageReservoir, LevelPowerReservoir, or SlopePowerReservoir with the given Storage at the specified timestep.

Description	The maximum combined outflow of a reservoir, including outlet works or turbine release, and any possible regulated, unregulated, and/or bypass spills.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the reservoir object for which to calculate
2	NUMERIC	the given storage, at the end of the specified timestep
3	DATETIME	the timestep at which to calculate

Evaluation	The ending pool elevation is determined from the ending storage in argument 2 and is averaged with the previous timestep's ending Pool Elevation to yield an average pool elevation over the timestep. The average pool elevation is then used to compute the following outflows: • **Release** (if the object is a StorageReservoir): The maximum release is determined from a table interpolation in the Max Release table using the average pool elevation as the lookup in the Pool Elevation column. • **Turbine Release** (if the object is a LevelPowerReservoir or SlopePowerReservoir): The maximum turbine release is determined based on the selected Power method. This calculation is iterative, since the maximum outflow impacts the reservoir tailwater elevation and operating head, which affect the maximum turbine release. The selected Tailwater method is used to determine the tailwater elevation. • **No Power Turbine Flow**: The turbine release is determined from a table interpolation in the Max Flow Through Turbines table using the average pool elevation as the lookup in the Reservoir Elevation column. • **Plant Power Coefficient**: The turbine release is determined from a table interpolation in the Max Turbine Q table using the operating head as the lookup in the Operating Head column. If the average pool elevation is less than the Minimum Power Elevation, the turbine release is zero. • **Plant Efficiency Curve**: The turbine release is determined from a table interpolation in the Auto Max Turbine Q table using the operating head as the lookup in the Operating Head column. If the average pool elevation is less than the Minimum Power Elevation, the turbine release is zero. • **Unit Generator Power**: The turbine release is the sum of the maximum releases for each available turbine, as specified in the Generators Available and Limit table. Each turbine's maximum release is determined from a table interpolation in the Full Generator Flow table using the operating head as the lookup in the appropriate unit type's Head for Type n column. If the average pool elevation is less than the Minimum Power Elevation, the turbine release is zero. • **Peak Power Equation with Off Peak Spill**: The turbine release is the peak release over the entire timestep. This is calculated by iterating the selected Tailwater method and operating head calculation with a table interpolation in the Operating Head vs. Generator Capacity table. • **Peak Power** and **Peak and Base**: The turbine release is the peak flow over the entire timestep. This is calculated by iterating the selected Tailwater method and operating head calculation with a table interpolation in the Best Generator Flow table using the operating head as the lookup in the Head for Type 1 column. • **LCRPowerCalc**: Because this power method has no turbine release limit, a maximum outflow cannot be calculated. RiverWare issues an error message and aborts the execution of this rule. • **Unregulated Spill** (if an Unregulated Spill method is selected on the object): The maximum unregulated spill is determined from a table interpolation in the Unregulated Spill Table using the average pool elevation as the Pool Elevation. • **Regulated Spill** (if a Regulated Spill method is selected on the object): If the Regulated Spill slot is specified by the user (I, Z or R flag), the specified value is used as the maximum regulated spill. Otherwise the maximum regulated spill is determined from a table interpolation in the Regulated Spill Table using the average pool elevation in the Pool Elevation column. **Note:** If the MonthlySpill method is selected, the result of GetMaxOutflowGivenInflow is the value in the Maximum Controlled Release table slot.

Evaluation (continued)	• **Bypass** (if a Bypass Spill method is selected on the object): If the Bypass slot is specified by the user (I, Z or R flag), the specified value is used as the maximum bypass. Otherwise the maximum bypass is determined from a table interpolation in the Bypass Table using the average pool elevation. All of the individual outflows are then summed to calculate the maximum outflow. • **Spill**: If the Spill slot is specified by the user (I, Z or R flag), the specified value is used as the maximum spill. The individual outflows are summed to calculate the maximum outflow.
Mathematical Expression	$Outflow_{max} = release_{max} \text{ or } turbine\ release_{max} + unregulated\ spill_{max} + regulated\ spill_{max} + bypass_{max}$
Comments	The Tailwater Base Value is automatically added as a dependency to the calling rule. This function will issue an error if the Time Varying Elevation Volume method is selected (see “Time Varying Elevation Volume” in *Objects and Methods*) and the specified timestep is a modification date on the table. **Caution:** The Pool Elevation or Storage slot must have a valid value on the timestep before the timestep passed in on Argument 4. Otherwise, an error will be issued.

Syntax Example

```
GetMaxOutflowGivenStorage(%"Hoover Dam", 17321.400 "1000 acre-feet", @"May, 1998")
```

Return Example

```
1283.7047 "cms"
```

• GetMaxReleaseGivenInflow

This function evaluates to the maximum Release, or Turbine Release from a StorageReservoir, LevelPowerReservoir, or SlopePowerReservoir with the given Inflow at the specified timestep.

Description	The maximum release of a reservoir, through outlet works or turbine release.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the reservoir object for which to calculate
2	NUMERIC	the inflow over the timestep, often the reservoir’s inflow slot value
3	DATETIME	the timestep at which to calculate

Evaluation	A convergence algorithm is used in this function; see “Reservoir Convergence” in *Objects and Methods* for details. The function iterates to convergence by computing the end of timestep storage and pool elevation, the average pool elevation over the timestep, and the release: • **Release** (if the object is a StorageReservoir): The maximum release is determined from a table interpolation in the Max Release table using the average pool elevation as the lookup in the Pool Elevation column. • **Turbine Release** (if the object is a LevelPowerReservoir or SlopePowerReservoir): The maximum turbine release is determined based on the selected Power method. This calculation is iterative, since the maximum outflow impacts the reservoir tailwater elevation and operating head, which affect the maximum turbine release. The selected Tailwater method is used to determine the tailwater elevation. • **No Power Turbine Flow**: The turbine release is determined from a table interpolation in the Max Flow Through Turbines table using the average pool elevation as the lookup in the Reservoir Elevation column. • **Plant Power Coefficient**: The turbine release is determined from a table interpolation in the Max Turbine Q table using the operating head as the lookup in the Operating Head column. If the average pool elevation is less than the Minimum Power Elevation, the turbine release is zero. • **Plant Efficiency Curve**: The turbine release is determined from a table interpolation in the Auto Max Turbine Q table using the operating head as the lookup in the Operating Head column. If the average pool elevation is less than the Minimum Power Elevation, the turbine release is zero. • **Unit Generator Power**: The turbine release is the sum of the maximum releases for each available turbine, as specified in the Generators Available and Limit table. Each turbine’s maximum release is determined from a table interpolation in the Full Generator Flow table using the operating head as the lookup in the appropriate unit type’s Head for Type n column. If the average pool elevation is less than the Minimum Power Elevation, the turbine release is zero. • **Peak Power** and **Peak and Base**: The turbine release is the peak flow over the entire timestep. This is calculated by iterating the selected Tailwater method and operating head calculation with a table interpolation in the Best Generator Flow table using the operating head as the lookup in the Head for Type 1 column. • **Peak Power Equation with Off Peak Spill**: The turbine release is the peak release over the entire timestep. This is calculated by iterating the selected Tailwater method and operating head calculation with a table interpolation in the Operating Head vs. Generator Capacity table. • **LCRPowerCalc**: Because this power Method has no turbine release limit, a maximum Outflow cannot be calculated. RiverWare issues an error message and aborts the execution of this rule.
Mathematical Expression	$Release_{max} = release_{max} \text{ or } turbine\ release_{max}$
Comments	This function takes into account the following sources and sinks automatically, and thus they should not be included in the inflow value for Argument 2. • The Evaporation and Precipitation category selected Method. • The Bank Storage category selected Method. • The Seepage category selected Method. • Side inflows including: Inflow 2 (Slope Power Reservoir only), Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. These slots in addition to Tailwater Base Value are automatically added as dependencies to the calling rule. **Caution:** The Pool Elevation or Storage slot must have a valid value on the timestep before the timestep passed in on Argument 4. Otherwise, an error will be issued.

Syntax Example

```
GetMaxReleaseGivenInflow(%"Hoover Dam", 68651 "cfs", @"June, 1983")
```

Return Example

```
1283.7047 "cms"
```

• GetMinSpillGivenInflowRelease

This function evaluates to the minimum spill from a StorageReservoir, LevelPowerReservoir, or SlopePowerReservoir with the given inflow and release at the specified timestep.

Description	The minimum required spill through unregulated and regulated spillways.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the reservoir object for which to calculate
2	NUMERIC	the inflow over the timestep, often the reservoir’s inflow slot valueep
3	NUMERIC	the average release over the timestep
4	DATETIME	the timestep at which to calculate

Evaluation	This function calls the getMinSpillGivenInflowRelease() function on the given reservoir object at the given timestep, and provides it with the inflow and release over the timestep. A convergence algorithm is used in this function; see “Reservoir Convergence” in *Objects and Methods* for details. The function iterates to convergence by computing the end of timestep storage and pool elevation, the average pool elevation over the timestep, and the spill: • **unregulated spill**: calculated from the Unregulated Spill Table based on the average Pool Elevation. See the spill method for more details on how this is computed • **regulated and bypass spills**: assumed to be zero unless input by the user, set by a rule, or if if Regulated Spill includes Closed Gate Overflow; if input or set by a rule, that specified value is used; if Closed Gate Overflow is included, it is calculated based on the updated average pool elevation at each iteration (see “Closed Gate Overflow” in *Objects and Methods* for details on how Closed Gate Overflow is calculated). • **outflow**: sum of the calculated spill and the release specified in the function. • **pool elevation**: solved for by mass balance using the specified inflow and calculated outflow.
Mathematical Expressions	$spill_{min} = unregulated\ spill + regulated\ spill(\text{ if input, rules or closed gate overflow}) + bypass\ spill(\text{ if input or rules})$ $outflow_{min} = spill_{min} + release$
Comments	This function takes into account the following sources and sinks automatically, and thus they should not be included in the inflow value for Argument 2. • The Evaporation and Precipitation category selected Method. • The Bank Storage category selected Method. • The Seepage category selected Method. • Side inflows including: Inflow 2 (Slope Power Reservoir only), Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. These slots in addition to the previous timestep’s Pool Elevation and Storage are automatically added as dependencies to the calling rule. Since the function evaluation depends on these slots, any change to their values at the indicated timestep, may impact the function result. Caution should be used when calling this function if Regulated Spill or Bypass is an input or has been set by a rule. If the inflow and/or release given to the function, combined with the specified Regulated Spill and/or specified Bypass results in a max Regulated Spill or Bypass capacity that is less than the specified value, the function will issue an error. **Caution:** The Pool Elevation or Storage slot must have a valid value on the timestep before the timestep passed in on Argument 4. Otherwise, an error will be issued.

Syntax Example

```
GetMinSpillGivenInflowRelease(%"Hoover Dam", Hoover Dam.Inflow[], 0.0 "cfs", @"t")
```

Return Example

```
1283.7047 "cms"
```

• GetMonth

This function evaluates to the integer number of the month of the given datetime.

Description	The month number, base 1, in units of NONE.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	DATETIME	the datetime of any time within the month
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime. Then, the number of the month in which the datetime is contained, is determined. This function requires that the specified datetime resolve to at least a partially specified datetime in the “Month day, year” format with the month specified.	
Comments	As elsewhere in RiverWare, 24:00 hours is considered to be the day which is ending, and 00:00 hours is considered to be the day which is just beginning.	

Syntax Example

```
GetMonth(@"February 23, 1996")
```

Return Example

```
2.000
```

GetMonthAsString

This function evaluates to the string name of the month of the given datetime.

Description	The month name.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	DATETIME	the datetime of any time within the month
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime. Then, the name of the month in which the datetime is in, is determined. This function requires that the specified datetime resolve to at least a partially specified datetime in the “Month day, year” format with the month specified.	
Comments	As elsewhere in RiverWare, 24:00 hours is considered to be the day which is ending, and 00:00 hours is considered to be the day which is just beginning.	

Syntax Example

```
GetMonthAsString(@"February 23, 1996")
```

Return Example

```
"February"
```

GetNumbers

This function evaluates to a sequence of values in a given range with a given offset.

Description	Returns a sequence of values in a given range with a given offset.	
Type	LIST	
Arguments	**Type**	**Meaning**
1	NUMERIC	the start value
1	NUMERIC	the end value
1	NUMERIC	the offset
Evaluation	The end value and offset are converted into the units of the start value. A list is created whose first item is the start value, the second item is the start value plus the offset, and so on, until the next value to be added to the list would not be in the range defined by the start and end value.	
Comments	The units of all values must be compatible. If the offset is positive and the start value is greater than the end value, the return list is empty; similarly, if the offset is negative and the start value is less than the end value, the return list is empty.	

Syntax Example

```
GetNumbers(0.0 "cfs", 10 "cms", 1 "cfs")
```

Return Example

```
{0.0 "cfs", 1.0 "cfs", 2.0 "cfs", ... , 352.0 "cfs", 353.0 "cfs"}
```

• GetObject

This function looks for an object on the global workspace with a given name and returns that object, if it exists.

Description	Return the object with a given name.	
Type	OBJECT	
Arguments	**Type**	**Meaning**
1	STRING	the name of the object for which to search.
Evaluation	The function returns the object with the given name, if it exists.	
Comments	If no object with the given name exists on the global workspace, then the run is aborted with an error.	

Syntax Example

```
GetObject("Heron Reservoir")
```

Return Example

```
%"Heron Reservoir"
```

• GetObjectDebt

This function evaluates to the sum of the debts to all accounting exchanges which may be paid by supplies on the given object. If there are no exchange paybacks on the given object, the debt is zero.

Description	The total debt which can be paid by this object.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the object who's debt to calculate
2	DATETIME	the timestep at which to calculate the debt
Evaluation	The function loops over all supplies, on all accounts, on the given object. If a supply is an exchange payback source, the value of its debt slot at the given timestep (if known) is added to the cumulative debt of the object.	
Mathematical Expression	$NUMERIC = \sum_{payback\ source\ supplies_{(object)}} Debt_{(payback,\ timestep)}$	
Comments	If the debt slot of a payback supply at the given timestep is not known, its debt is assumed to be zero. If there are no payback supplies on the given object, the total debt is zero.	

Syntax Example

```
GetObjectDebt(%"Heron Reservoir", @"t")
```

Return Example

```
1.823 "m3"
```

GetObjectFromSlot

Description	Return a slot's parent object.	
Type	OBJECT	
Arguments	**Type**	**Meaning**
1	SLOT	The slot whose object is returned.
Comments	It is an error if the slot is not on a Simulation Object or an account (which is on an Object).	

Syntax Example

```
GetObjectFromSlot($"ResA^Municipal.Inflow")
```

Return Example

```
%"ResA"
```

GetPaybackDebt

This function evaluates to the value of the debt slot of the given exchange payback source at the given timestep.

Description	The debt at a payback source.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	STRING	the payback source supply whose debt to calculate

2	DATETIME	the timestep at which to calculate the debt
Evaluation	The function begins by verifying that the string argument is an accounting supply, and that this supply is a payback source. If so, the function evaluates to the value of the payback's debt slot at the given timestep.	
Mathematical Expression	$NUMERIC = Debt_{(payback,\, timestep)}$	
Comments	If the string argument is not a valid supply, or the supply is not a payback source, this function aborts the run with an error. If the debt slot of the payback for which this supply is a source does not contain a value at the given timestep, the function evaluates to zero.	

Syntax Example

```
GetPaybackDebt("Heron SanJuan to WillowAndRioChama SanJuan.Supply", @"t")
```

Return Example

```
1.823 "m3"
```

• GetRowIndex

This function evaluates to the index of the row with the given name in a table slot.

Description	The index of the table slot row whose name matches a string.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the table slot in which to find a row
2	STRING	the name of the row to match
Evaluation	The labels of the slot rows are compared to the string argument until a match is found.	
Comments	Table row and column indices are zero based and have units of type [NONE]. If the specified slot is not a table slot or the specified string is not the label of a row on the slot, this function aborts the run with an error. If several rows of the table slot match the string argument, this function evaluates to the index of the topmost matching row.	

Syntax Example

```
GetRowIndex(RiverData.Minimum Flow,"Dolores")
```

Return Example

```
1.00000
```

• GetRowIndexByDate

Description	Given a slot with rows indexed by date, this function returns the 0-based index corresponding to a given date.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**

1	SLOT	the slot in which to find a row
2	DATETIME	date of the row to match
Comments	The value -1 is returned if the given date is not within the date range of the slot. This function is applicable to the following types of slot: • Series Slots • Table Series Slots • Periodic Slot It is considered an error if the slot is not indexed by date (i.e, not one of these types).	

Syntax Example

```
GetRowIndexByDate(DeepReservoir.Inflow, @"t")
```

Return Example

```
5.00000
```

• GetRunCycleIndex

Description	Returns the 1-based index of the current cycle through the timesteps in the run time range, in units of NONE.
Type	NUMERIC
Arguments	none
Comments	Rulebased simulations can cycle through the run timesteps more than once each run (see “Run Cycles” in *Solution Approaches*). This function provides access to the current run cycle, which can be used, for example, within execution constraints to control the cycle on which a rule should execute. If called from outside of a run or when the controller is not Rulebased Simulation or Inline Rulebased Simulation and Accounting, the behavior of this function is undefined.

Syntax Example

```
GetRunCycleIndex()
```

Return Example

```
2.0000
```

• GetRunIndex

This function evaluates to the number of the model run. It is commonly used within a Multiple Run Management ruleset to determine the run which is currently executing. See also “GetTraceNumber” for a similar function that returns the trace number for MRM runs that start on different traces, such as in distributed MRM.

Description	The number of the currently executing model run, base 1, in units of NONE.
Type	NUMERIC
Arguments	none
Comments	If the current run is not a Multiple Run Management run, this function evaluates to 1. If called from within a pre-MRM run rule of an iterative MRM run, this function evaluates to 1. If called from within a post-run rule of an iterative MRM, this function evaluates to the index of the run which just completed.

Syntax Example

```
GetRunIndex()
```

Return Example

```
3.0000
```

• GetSelectedUserMethod

Description	Given an object and a user method category name, return the name of the selected method.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	OBJECT	the simulation object
2	STRING	name of the user method category
Comments	An error is issued if the object or the category name is not found.	

Syntax Example

```
GetSelectedUserMethod(DeepReservoir, "Power")
```

Return Example

```
"Peak Power"
```

• GetSeriesSlots

Description	Returns a list of all of the visible Series Slots on an object.	
Type	LIST{SLOT}	
Arguments	**Type**	**Meaning**
1	OBJECT	The object whose Series Slots are to be returned.
Comments	If no object with the given name exists on the global workspace, then the run is aborted with an error.	

Syntax Example

```
GetSeriesSlot(%"My Data Object")
```

Return Example

```
{$"My Data Object.Series 1", $"My Data Object.Series 2"}
```

• GetSlot

This function looks for a slot on the global workspace with a given name and returns that slot, if it exists.

Description	Return the slot with a given name.	
Type	SLOT	
Arguments	**Type**	**Meaning**

1	STRING	the name of the slot for which to search.
Evaluation	The function returns the slot with the given name, if it exists.	
Comments	If no slot with the given name exists on the global workspace, then the run is aborted with an error.	

Syntax Example

```
GetSlot("Heron Reservoir.Inflow")
```

returns:

```
$"Heron Reservoir.Inflow"
```

```
GetSlot("Abiquiu Reservoir^RioGrande.Inflow")
```

returns:

```
$"Abiquiu Reservoir^RioGrande.Inflow"
```

• GetSlotName

Description	Return the slot name portion of a slot's full name.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	SLOT	The slot whose name is returned.
Comments	If the slot is a column of an aggregate series slot and if it is the first column, then the agg series slot name is returned. If the input slot is a column of an aggregate series slot other than the first column, the column label is returned. For example, assume that data.Agg is a two column aggregate series slot series slot with columns labeled "zero" and "one": • GetSlotName(data.Agg) = "Agg" • GetSlotName(data.Agg.0) = "Agg" • GetSlotName(data.Agg.zero) = "Agg" • GetSlotName(data.Agg.1) = "one" • GetSlotName(data.Agg.one) = "one" See also "GetSlotNameAndCol", which behaves differently when the input slot is an aggregate series slot.	

Syntax Example

```
GetSlotName($"ResA^Municipal.Inflow")
```

Return Example

```
"Inflow"
```

GetSlotNameAndCol

Description	Given a slot, return the slot name portion of the full name, combined with the column label when the input slot is an aggregate series slot or a column of an aggregate series slot.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	SLOT	The desired aggregate series slot.
Comments	For example, assume that data.Agg is a two column aggregate series slot with columns labeled “zero” and “one”: • GetSlotNameAndCol(data.Agg) = “Agg.zero” • GetSlotNameAndCol(data.Agg.0) = “Agg.zero” • GetSlotNameAndCol(data.Agg.zero) = “Agg.zero” • GetSlotNameAndCol(data.Agg.1) = “Agg.one” • GetSlotNameAndCol(data.Agg.one) =”Agg.one” See also “GetSlotName”, which behaves differently when the input slot is an agg. series slot.	
Notes		If called on a slot that is not an aggregate series slot, the slot name is returned.

Syntax Example

```
GetSlotName($"DataObj.Aggregate.Inflow")
```

Return Example

```
"Aggregate.Inflow"
```

GetSlotVals, GetSlotValsNaNToZero

Description	This function evaluates to a list composed of the values of a given series slot within a time range. GetSlotVals can also be used on a periodic slot, while GetSlotValsNaNToZero, cannot.	
Type	LIST{NUMERIC}	
Arguments	**Type**	**Meaning**
1	SLOT	the series (or periodic slot) whose values to get
2	DATETIME	start datetime
3	DATETIME	end datetime
Evaluation	A list is generated by looking up each value in the given slot, beginning with the start datetime, and ending with the end datetime. All slot values in the range are returned, regardless of the slot data’s timestep resolution vis-a-vis that of the run control.	
Mathematical Expression	$\{NUMERIC\} = \{slot_{(start\ datetime)}, \ldots, slot_{(end\ datetime)}\}$	
Comments	If the start datetime or end datetime does not match one of the slot’s values’, or if the start datetime is after the end datetime, this function aborts the run with an error. For GetSlotVals, if one of the slot values within the desired time range is a NaN, the function exits the rule with an early termination. For GetSlotValsNaNToZero, it converts any NaNs into zero. For periodic slots and GetSlotVals, the dates used are those within the range and falling on a run timestep; the column used is the first (column 0).	

Syntax Example

```
GetSlotVals(Dolores.Inflow, @"t", @"September 31, Current Year")
GetSlotValsNaNToZero(Mead.Seepage, @"Start Timestep", @"t")
```

Return Example

```
{ 1.43"cms", 2.12 "cms" }
```

• GetSlotValsByCol, GetSlotValsByColNaNToZero

Description	This function evaluates to a list composed of the values in a column of a given Agg Series Slots (or for GetSlotValsByCol, it could be a periodic slot) within a time range.	
Type	LIST{NUMERIC}	
Arguments	**Type**	**Meaning**
1	SLOT	the agg series slot (or for GetSlotValsByCol, it could be a periodic slot) whose values to get
2	DATETIME	start datetime
3	DATETIME	end datetime
4	NUMERIC	the column (interpreted as a 0-based integral index)
Evaluation	A list is generated by looking up each value in the given column of the slot, beginning with the start datetime, and ending with the end datetime. All slot values in the range are returned, regardless of the slot data's timestep resolution vis-a-vis that of the run control.	
Mathematical Expression	$\{NUMERIC\} = \{slot_{(start\ datetime)}, \ldots, slot_{(end\ datetime)}\}$	
Comments	If the slot is an Agg Series Slot and the start datetime or end datetime does not match one of the slot's values', or if the start datetime is after the end datetime, this function aborts the run with an error. For GetSlotValsByCol, if one of the slot values within the desired time range is a NaN, the function exits the rule with an early termination. For GetSlotValsByColNaNToZero, it converts any NaNs into zero. For periodic slots and GetSlotValsByCol, the dates used are those within the range and falling on a run timestep.	

Syntax Example

```
GetSlotValsByCol(WaterUser1.Periodic Diversion Request,
  @"t", @"September 31,Current Year", 3)
GetSlotValsByColNaNToZero(WaterUser1.IrrigatedAreaByCrop,
  @"t", @"September 31,Current Year", 3)
```

Return Example

```
{ 1.43"cms", 2.12 "cms", 2.54 "cms", 2.2 "cms"}
```

• GetTableColumnVals, GetTableColumnValsSkipNaN

This function evaluates to a list. Each item of the list is the value of the given table slot, in the given column, from the given start row, to the given end row.

Description	All of the values of a table slot column between two rows.	
Type	LIST{NUMERIC}	
Arguments	**Type**	**Meaning**
1	SLOT	the table slot whose values to get
2	NUMERIC	column
3	NUMERIC	start row
4	NUMERIC	end row
Evaluation	A list is generated by looking up each value in the given column of the given table slot beginning with the start row, and ending with the end row (inclusive). Rows and columns are numbered beginning with zero.	
Mathematical Expression	$\{NUMERIC\} = \{slot_{(start\ row,\ column)}, ..., slot_{(end\ row,\ column)}\}$	
Comments	Units are not required for row and column indices and, if provided, will be ignored. If the column, start row, or end row does not exist in the slot, or if the start row is greater than the end row, this function aborts the run with an error. For the GetTableColumnVals function, if one of the slot values within the desired time range is a NaN, the function exits the rule with an early termination. For the GetTableColumnValsSkipNaN variation of this function, these values are just omitted from the return list.	

Syntax Example

```
GetTableColumnVals(Chickamauga Data.Power Coeffs, 0, 0, 1)
GetTableColumnValsSkipNaN(Chickamauga Data.Power Coeffs, 0, 0, 1)
```

Return Example

```
{ 1.43"cms", 2.12 "cms"}
```

• GetTableRowVals, GetTableRowValsSkipNaN

This function evaluates to a list. Each item of the list is the value of the given table slot, in the given row, from the given start column, to the given end column.

Description	All the values of a table slot column between two columns.	
Type	LIST{NUMERIC}	
Arguments	**Type**	**Meaning**
1	SLOT	the table slot whose values to get
2	NUMERIC	row
3	NUMERIC	start column
4	NUMERIC	end column

Evaluation	A list is generated by looking up each value in the given row, of the given table slot beginning with the start column, and ending with the end column (inclusive). Rows and columns are numbered beginning with zero.	
Mathematical Expression	$\{NUMERIC\} = \{slot_{(row,\, start\ column)}, \ldots, slot_{(row,\, end\ column)}\}$	
Comments	Units are not required for row and column indices and, if provided, will be ignored. If the row, start column, or end column do not exist in the slot, or if the start column is greater than the end column, this function aborts the run with an error. For the GetTableRowVals function, if one of the slot values within the desired time range is a NaN, the function exits the rule with an early termination. For the GetTableRowValsSkipNaN variation of this function, these values are just omitted from the return list.	

Syntax Example

```
GetTableRowVals(Chickamauga Data.Power Coeffs, 0, 0, 1)
GetTableRowValsSkipNaN(Chickamauga Data.Power Coeffs, 0, 0, 1)
```

Return Example

```
{2.54 "cms", 2.2 "cms"}
```

• GetTextSlotValueAsString

This function gets the value off of a Text Series Slot at the specified datetime and returns a string.

Description	Returns the Text Series Slot value as a STRING	
Type	STRING	
Arguments	**Type**	**Meaning**
1	SLOT	The text series slot
2	DATETIME	The timestep at which to lookup
Evaluation	Text Series Slots store text as a number encoded on the slot. See “Text Series Slots” in *User Interface*. This function returns the value of the specified Text Series Slot at the specified date as a STRING. An error will be issued if the slot is not a Text Series Slot or the DATETIME is not fully specified. If the slot does not have a value at the date then the function returns an invalid value, which will cause the calling block to terminate early.	
Comments	See “TextSlotNumericToString” for a similar function. See “StringToTextSlotNumeric” for the opposite conversion function.	

Syntax Example

```
GetTextSlowValueAsString(DataObject1.MyTextSlot, @”Start Timestep”)
```

Return Example

```
“Start of Flood Season”
```

• GetTimestep

This function evaluates to the length of the timestep ending on the given datetime.

Description	The length of a timestep, in units of "sec".	
Type	NUMERIC	
Arguments	Type	Meaning
1	DATETIME	the datetime of the end of the timestep
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime. Then, the length of the timestep within which this time is, is determined. This function requires that the specified datetime resolve to a fully specified datetime or an error will occur.	
Comments	If the given datetime corresponds to the moment when one timestep ends and another begins, this function evaluates to the length of the timestep which is ending. As elsewhere in RiverWare, 24:00 hours is considered to be the day which is ending, and 00:00 hours is considered to be the day which is just beginning.	

Syntax Example

```
GetTimestep(@"February 23, 1996")
```

Return Example

- In a model with a 6 Hour timestep:
  `21600 "sec"`
- In a model with a 1 Day timestep:
  `86400 "sec"`
- In a model with a 1 Month timestep:
  `2505600.0 "sec"`

GetTraceNumber

This function evaluates to the trace number of the model run. It is commonly used within a Multiple Run Management ruleset to determine the run which is currently executing. This differs from “GetRunIndex” which always returns 1...N, while GetTraceNumber can start at a defined starting trace number which may not be 1 if the value is changed or if you are using Distributed MRM or the Select Years input mode. This function is most useful for debugging concurrent multiple runs.

Description	The number of the currently executing trace run in units of NONE.

Type	NUMERIC
Arguments	none
Comments	If the current run is not a Multiple Run Management run, this function evaluates to 1. If called from a rule within concurrent MRM, if the multiple run is configured to begin with trace M then GetTraceNumber() returns traces M..(M+N-1) (with the exception of using the Select Years input mode). See "Input DMIs" in *Solution Approaches* for more information. For distributed multiple runs, as described in "Distributed Concurrent Runs" in *Solution Approaches*, each multiple run is configured to begin with different trace numbers so, for example 80 traces distributed across 4 RiverWare instances, GetTraceNumber() would return 1..20, 21..40, 41..60 and 61..80 for the 4 RiverWare instances (with the exception of using the Select Years input mode). If used with the MRM Select Years input mode, GetTraceNumber() returns the selected year that identifies each trace, whether using interactive or distributed MRM. See "Select Years" in *Solution Approaches* for more information. If called from within a pre-MRM run rule of an iterative MRM run, this function evaluates to 1. If called from within a post-run rule of an iterative MRM, this function evaluates to the index of the run which just completed.

Syntax Example

```
GetTraceNumber()
```

Return Example

```
21.0
```

• GetUpperBound

Description	Returns the upper bound for the given series slot.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the slot whose bound is to be returned.
Evaluation		
Comments	It is considered an error if the specified slot is not a series slot with a valid upper bound. The upper bound is specified in the slot configuration under the view menu.	

Syntax Example

```
GetUpperBound("Res A.Power")
```

Return Example

```
400.0 MW
```

• GetUpperBoundByCol

Description	Returns the upper bound for the specified column of the given aggregate series slot.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**

1	SLOT	the agg slot whose bound is to be returned.
2	NUMERIC	the column index (0-based).
Evaluation		
Comments	It is considered an error if the input slot is not an Agg. Series Slot with a valid upper bound for the given column. The upper bound is specified in the slot configuration under the view menu.	

Syntax Example

```
GetUpperBoundByCol("Res A.Hydro Block Use", 3)
```

Return Example

```
200.0 MWH
```

GetYear

This function evaluates to the year of the given datetime.

Description	The year, expressed as a number in units of NONE.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	DATETIME	the datetime
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime. Then, the year within which this time is, is determined. This function requires that the datetime be at least partially specified with a valid year, E.g. @"Year 2010" or @"Current Year" will work.	
Comments	As elsewhere in RiverWare, 24:00 hours is considered to be the day which is ending, and 00:00 hours is considered to be the day which is just beginning.	

Syntax Example

```
GetYear(@"24:00 December 31, 1999")
```

Return Example

```
1999.0
```

GetYearAsString

This function evaluates to the year of the given datetime as a string.

Description	The year as a string.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	DATETIME	the datetime
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime and the year is returned as a string. This function requires that the datetime be at least partially specified with a valid year, E.g. @"Year 2010" or @"Current Year" will work.	
Comments	As elsewhere in RiverWare, 24:00 hours is considered to be the day which is ending, and 00:00 hours is considered to be the day which is just beginning.	

Syntax Example

```
GetYearAsString(@"February 23, 1996")
```

Return Example

```
"1996"
```

HasFlag

This function returns whether the specified slot on the given timestep has the specified flag.

Description	Use to test the flag status of a slot at a given datetime.	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
1	SLOT	The slot to test
2	DATETIME	The datetime at which to test
3	STRING	The flag to test. The flag is specified by string, which can be the single letter version of the flag or the full flag name: • O: Output • I: Input • R: Rules • B: Best Efficiency • M: Maximum Capacity • i: Iterative MRM • Z: DMI Input • U: Unit Values • T: Target • TB: Target-Begin • tb: Target-Begin-RiverWare • S: Surcharge Release • G: Regulation Discharge • D: Drift • m: Method • A: Account • P: Propagated
Evaluation	The function returns whether or not the slot has the flag at that date. If the slot does not have a value at the date then the function returns that an invalid value was found (which will cause the calling block to terminate early).	
Comments	An error is returned if the slot is not a Series Slot or if the date is not a legal timestep for that slot. **Note:** See “IsInput”, which is a more limited function used to test for the I or Z flag. Unlike the HasFlag function, IsInput returns FALSE when the slot does not have a value.	

Syntax Example

```
HasFlag($"Mead.Outflow", @"24:00 December 31, 1999", "R")
HasFlag($"Powell.Storage", @"24:00 March 31, 2006", "Target")
```

Return Example

```
TRUE or FALSE
```

• HasRuleFiredSuccessfully

Description	Returns whether or not the rule with a given name has successfully executed (with or without effect) during the current timestep of a rulebased simulation.	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
1	STRING	The name of the rule. The special string "Current Rule", "This Rule", or "Current Method" (not case sensitive; can specify with or without a space) is interpreted as a reference to the currently executing rule.
Evaluation	This function returns TRUE if either: • The rule finished successfully (that is, at least one assignment is attempted and none fail), or • The rule finished ineffectively (that is, the rule is evaluated but the logic within the execution constraint or within the body of the rule decides no assignment is necessary or the rule attempts assignment, but priority is junior so no assignment is made). The function returns FALSE if either: • The rule has not yet fired, or • The rule has fired but terminated early (the rule encountered a NaN in a slot value). **Note:** As mentioned above, if the input name is "Current Rule" or "This Rule" or "Current Method" (with or without a space), then this is taken to be a reference to the currently executing rule. Using the structure NOT HasRuleFiredSuccessfully("This Rule") will cause the that rule to only execute successfully once.	
Comments	HasRuleFiredSuccessfully behaves as follows for the various RPL Sets: • Rulebased Simulation Rules—has the rule fired in the current timestep, as described above. • Initialization Rules—has the rule fired in the current single run. • Iterative MRM Rules—has the rule fired in the current MRM iteration (single run). • Global Functions—sets the behavior for the caller • Other—not applicable; aborts with an error message.	

Syntax Example

```
HasRuleFiredSuccessfully("Smith Flood Control")
HasRuleFiredSuccessfully("Current Rule")
HasRuleFiredSuccessfully("ThisRule")
```

Return Example

```
TRUE or FALSE
```

• HydropowerRelease

This function calculates the additional outflow necessary to satisfy an unmet load (energy requirement) while not causing additional downstream flooding.

Description	Calculates the additional outflow necessary to meet an unmet load, if any exists. The function limits the additional release to ensure that additional downstream flooding does not occur.	
Type	LIST { LIST { SLOT, NUMERIC, OBJECT } }	
Arguments	**Type**	**Meaning**

1	STRING	the name of the computational subbasin

Evaluation

The function does the following:

1. Prioritizes the power reservoirs in the basin according to the relative Load shortage using the equation below.

$$Shortage = \frac{Load - Energy}{Load}$$

If Load is unknown because the Seasonal Load Time method is selected, it is calculated. The calculated shortage then is a value less than one. The reservoirs with the highest values are first, the lowest reservoirs last.

2. Runs the selected Additional Hydropower Release method on the each power reservoir in the subbasin to calculate the proposed additional release required to meet the load within outflow constraints.

3. In order of priority, hypothetically makes the additional release, visits downstream control points until it reaches a tandem reservoir or the end of the subbasin, whichever comes first. If the release causes (additional) flooding at a control point, it reduces the release until flooding is not caused or the release becomes zero. Resulting releases are then hypothetically routed to downstream control points to make adjustments to their available space hydrographs.

A control point's available space hydrograph (in units of flow projected into the future based on the routing coefficients on the control point) is calculated as:

$$\text{Available Space} = \text{Regulation Discharge} - (\text{Inflow} + \text{forecasted Local Inflow})$$

Inflow includes the value of the Inflow slot (at the time of the last dispatch) and the additional inflow resulting from the hypothetical additional releases from upstream reservoirs. It also contains the proposed flood control release hydrograph from the last pass of the flood control method.

> **Note:** If the Releases Not Constrained by Flooding method is selected in the Hydropower Flooding Exception category on the control point (see "Releases Not Constrained by Flooding" in *Objects and Methods*), the control point is ignored, i.e. flooding is allowed at that control point.

Once all power reservoirs have been visited (in priority order), the HydropowerRelease function returns to the calling rule. For each reservoir in the subbasin, three triplets may be returned:

- Additional Power Release: This is the additional release to meet the load. If the proposed release is positive, but the additional power release was constrained to zero, the triplet {reservoir.Additional Power Release, 0.0, reservoir} will be returned. If the proposed release is zero, the Additional Power Release (of zero) triplet will not be returned.
- Outflow: If the new Outflow is the same as the existing Outflow, no Outflow triplet is returned because the value of the Outflow slot will not change as a direct result of this rule. Otherwise the value for outflow is the existing Outflow plus the additional power release.
- Load: the Load triplet is only returned if the Seasonal Load Time method (see "Seasonal Load Time" in *Objects and Methods*) is selected on the reservoir.

The calling rule is expected to make the assignments of the values to the slots.

Comments

This function is dependent on having executed the predefined function FloodControl() on a computational subbasin using the Operating Level Balancing method. This flood control method operation sets up the network topology and necessary data. HydropowerRelease requires that the reservoirs in the subbasin have already dispatched and have valid values in the Regulation Discharge, Outflow, Energy, and Load slots.

See "HydropowerRelease Function—Detailed Description of Logic" in *USACE-SWD Modeling Techniques* for details on using this function for USACE-SWD.

Syntax Example

```
HydropowerRelease("Flood Basin")
```

where “Flood Basin” contains Res1 and Res2.

Return Example

```
{ {"Res1.Additional Hydropower Release", 63.32 "cms", "res1"},
  {"Res1.Outflow", 63.32 "cms", "Res1"},
  {"Res2.Additional Hydropower Release", 23.20 "cms", "Res2"}
  {"Res2.Outflow", 32.02 "cms", "Res2"},
  {"Res2.Load", 2.1 "MWH", "Res2"} }
```

Use Example

```
FOREACH (LIST triplet IN HydropowerRelease("Flood Basin")) DO
  ( triplet<0> )[] = triplet<1>
ENDFOREACH
```

HypLimitSim

This function finds a value which, when set on a given slot, will lead to another slot achieving but not exceeding a limiting value within a given time frame.

Description	Given a control slot and a limit slot, limit date/time, and limit value, this function uses hypothetical simulation (see description of the predefined function HypSim) to find a value *x* such that if the control slot were set to *x* at all timesteps in the range [current date, target date], then the limit slot’s maximum value in this range would equal the target value. If the value *x* exceeds the physical constraint for that slot at a particular timestep (max outflow on a reservoir for example), then the constrained value is used instead of the *x* value for that timestep.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	STRING	The name of the Subbasin over which to perform the hypothetical simulations. This should include the objects on which the control and limit slot exist as well as all other objects necessary to compute the limit slot’s value. If there is no Subbasin with the given name, the string is taken to be the name of an object and a temporary Subbasin is created containing only that object. An error will be issued if this subbasin contains a Data Object.
2	SLOT	The control slot, the slot with which you desire to control the target slot’s value.
3	NUMERIC	The minimum control slot value. A value less than this is not considered a legal return value.
4	NUMERIC	The maximum control slot value. A value greater than this is not considered a legal return value.
5	LIST { LIST { SLOT, NUMERIC, DATETIME } }	Fixed values the user would like to set in each hypothetical simulation. Each item in the list is a list itself containing a slot, the value to set, and the timestep at which to set it.
6	SLOT	The limit slot, the slot whose value you would like to attain a certain value.

7	DATETIME	The end limit date/time, the end of the time range during which you are concerned with the limit slot's value.
8	NUMERIC	The limit value, the value which you would like the limit slot to achieve but not exceed during the limiting time range.
9	NUMERIC	The tolerance or desired accuracy of the returned value. If the function is successful, it indicates that setting the control slot to the returned value will lead to a maximum limit slot value which differs by no more than the tolerance from the desired limit value.
10	NUMERIC	The maximum number of iterations of hypothetical simulations allowed. If this number is reached without finding an return value within the tolerance, then the function fails.
11	NUMERIC	The minimum number of timesteps before and after the current timestep which might be involved in the simulation. As part of hypothetical simulation RiverWare makes copies of the objects in the subbasin and this input is used to determine how much data should be copied from each object. One can usually set this value to 0 and RiverWare will use a heuristic to determine the range over which to copy data. If this function fails because there was not enough data on some object, then input a higher value.
Evaluation	The implementation of this function uses an iterative algorithm (the bisection algorithm) which performs an hypothetical simulation of the subbasin at each iteration.	
Comments	RiverWare assumes that the target value range (computed using the minimum and maximum control slot values) includes the target value itself. For example, if the control slot minimum of 100 cfs leads to a simulated target value of 100 cfs. the control slot maximum of 1000 cfs leads to a simulated target slot value of 200 cfs, and the target value is 300 cfs, then the function would fail because the target value is not in the range implied by the input control slot minimum and maximum values (100-200 cfs). Mathematically, this is the assumption that limit slot's value is a monotonic function of the control slot's value. See "Hypothetical Simulations"; all comments mentioned there apply here as well.	

Syntax Example

```
HypLimitSim("upper basin", Navajo.Outflow, 10 "cfs", 1000 "cfs",
  { {Navajo.Outflow, 1000 "cfs", @"t"} },
  Powell.Inflow, @"t", 100 "cfs", 0.1 "cfs", 10)
```

Return Example

```
18.34 "cms"
```

• HypLimitSimWithStatus

This function finds a value which, when set on a given slot, will lead to another slot achieving but not exceeding a limiting within a given time frame. If a value satisfying this criterion is not found, then an attempt is made to find a value that comes close to doing so.

Description	Given a control slot and a limit slot, limit end date/time, and limit value, this function uses hypothetical simulation (see description of the predefined function HypSim) to find a value x such that if the control slot were set to x at all timesteps in the range [current date, end limit date], then the limit slot's maximum value in this range would equal the target value. If the value x exceeds the physical constraint for that slot at a particular timestep (max outflow on a reservoir for example), then the constrained value is used instead of the x value for that timestep. A four-item list is returned. The first item in the list is a boolean TRUE value if a satisfying control slot value was found, FALSE otherwise. If the first item is TRUE, then the second item is the satisfying control slot value, otherwise this value is as close as the function could get to finding such a value. The third item is a list of the control slot values used in the solution. These values will all be the same as the second item, except if some of the values were constrained due to physical limitations. The fourth item is a list of the limit slot values that correspond to the control slot values given in the previous list. **Note:** This function is similar to HypLimitSim. The only difference is that HypLimitSim fails if it can not find a satisfying control slot value, whereas this function does not fail, rather it still returns a value, along with the indication that this value does not achieve the limit and the additional information discussed above.	
Type	LIST {BOOLEAN, NUMERIC, LIST, LIST}	
Arguments	**Type**	**Meaning**
1	STRING	The name of the Subbasin over which to perform the hypothetical simulations. This should include the objects on which the control and limit slot exist as well as all other objects necessary to compute the limit slot's value. If there is no Subbasin with the given name, the string is taken to be the name of an object and a temporary Subbasin is created containing only that object. An error will be issued if this subbasin contains a Data Object.
2	SLOT	The control slot, the slot with which you desire to control the target slot's value.
3	NUMERIC	The minimum control slot value. A value less than this is not considered a legal return value.
4	NUMERIC	The maximum control slot value. A value greater than this is not considered a legal return value.
5	LIST { LIST { SLOT, NUMERIC, DATETIME } }	Fixed values the user would like to set in each hypothetical simulation. Each item in the list is a list itself containing a slot, the value to set, and the timestep at which to set it.
6	SLOT	The limit slot, the slot whose value you would like to attain a certain value.
7	DATETIME	The end limit date/time, the end of the time range during which you are concerned with the limit slot's value.
8	NUMERIC	The limit value, the value which you would like the limit slot to achieve but not exceed during the limiting time range.
9	NUMERIC	The tolerance or desired accuracy of the returned value. If the function is successful, it indicates that setting the control slot to the returned value will lead to a maximum limit slot value which differs by no more than the tolerance from the desired limit value.
10	NUMERIC	The maximum number of iterations of hypothetical simulations allowed. If this number is reached without finding an return value within the tolerance, then the function fails.

11	NUMERIC	The minimum number of timesteps before and after the current timestep which might be involved in the simulation. As part of hypothetical simulation RiverWare makes copies of the objects in the subbasin and this input is used to determine how much data should be copied from each object. One can usually set this value to 0 and RiverWare will use a heuristic to determine the range over which to copy data. If this function fails because there was not enough data on some object, then input a higher value.
Evaluation	There are two conditions in which that function will fail but this function will return false as the first item in the return list: • The minimum and maximum control slot values lead to a range of limit values which does not include the input limit value. In this case the value returned is the minimum or maximum control slot value, which ever leads to a limit slot value closest to the input limit value. • The tolerance is not achieved within the iteration limit. In this case, the value returned is the current best guess. If Hypothetical Simulation diagnostics are turned on, then if HypLimitSimWithStatus can not find a satisfying control value, a diagnostic will be posted describing why it failed to do so.	
Comments	See also documentation for HypLimitSim for more details; the differences between these two functions are how problems are dealt with, this function is more flexible (as described in the Evaluation section). See "Hypothetical Simulations"; all comments mentioned there apply here as well.	

Syntax Example

```
HypLimitSimWithStatus("upper basin", Navajo.Outflow, 10 "cfs", 1000 "cfs",
  { {Navajo.Outflow, 1000 "cfs", @"t"},
  {FlamingGorge.Outflow, 1000 "cfs", @"t"} },
  Powell.Inflow, @"t", 100 "cfs", 0.1 "cfs", 10)
```

Return Example

```
{TRUE, 2.3 "cms", {2.3 "cms", 2.28 "cms"}, {2.83 "cms", 2.83 "cms"} }
```

HypSim

This function performs a hypothetical simulation with user specified values and returns user-requested result values from the simulation.

Description	Hypothetically simulate a portion of the workspace with user input values and return requested result values.	
Type	LIST {NUMERIC}	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name over which to perform the hypothetical simulation. If there is no Subbasin with the given name, the string is taken to be the name of an object and a temporary Subbasin is created containing only that object. An error will be issued if this subbasin contains a Data Object.
2	LIST { LIST { SLOT, NUMERIC, DATETIME } }	Fixed values the user would like to set in each hypothetical simulation. Each item in the list is a list itself containing a slot, the value to set, and the timestep at which to set it.

3	LIST { LIST { SLOT, DATETIME } }	Outputs to get back from the simulation - each output must specify the slot and the timestep from which to return a value
4	NUMERIC	The minimum number of timesteps before and after the current timestep which might be involved in the simulation. As part of hypothetical simulation RiverWare makes copies of the objects in the subbasin and this input is used to determine how much data should be copied from each object. One can usually set this value to 0 and RiverWare will use a heuristic to determine the range over which to copy data. If this function fails because there was not enough data on some object, then input a higher value.
Evaluation	When function is executed, a hypothetical simulation is initiated where the fixed values are set on the specified slots, the portion of the workspace specified by the Subbasin is simulated, and the values of the output slots are returned as a list.	
Comments	This simulation is hypothetical in that none of the actual values on any objects in the workspace will be modified by hypothetical simulation within a rule. HypSim does not support accounting or optimization functionality. **Note:** HypSim was originally named HypotheticalSimulation. See “Hypothetical Simulations”; all comments mentioned there apply here as well.	

Syntax Example

```
HypSim( "upper basin",
  { {Navajo.Outflow, 1000 "cfs", @"t"},
    {FlamingGorge.Outflow, 1000 "cfs", @"t"} },
  { {GreenColorado.Outflow, @"t + 1 Timesteps"},
    {SanJuanColorado.Outflow, @"t + 2 Timesteps"} }, 5.0 )
```

Return Example

```
{2.83 "cms", 2.86 "cms"}
```

• HypTargetSim

This function finds a value which, when set on a given slot, will lead to a desired value on another slot.

Description	Given a control slot and a target slot, target date/time, and target value, this function uses hypothetical simulation (see description of the predefined function HypSim) to find a value x such that if the control slot were set to x at all timesteps in the range [current date, target date], then the target slot’s value would equal the target value. If the value x exceeds the physical constraint for that slot at a particular timestep (max outflow on a reservoir for example), then the constrained value is used instead of the x value for that timestep.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	STRING	The name of the Subbasin over which to perform the hypothetical simulations. This should include the objects on which the control and target slot exist as well as all other objects necessary to compute the target slot’s value. If there is no Subbasin with the given name, the string is taken to be the name of an object and a temporary Subbasin is created containing only that object. An error will be issued if this subbasin contains a Data Object.

2	SLOT	The control slot, the slot with which you desire to control the target slot's value.
3	NUMERIC	The minimum control slot value. A value less than this is not considered a legal return value.
4	NUMERIC	The maximum control slot value. A value greater than this is not considered a legal return value.
5	LIST { LIST { SLOT, NUMERIC, DATETIME } }	Fixed values the user would like to set in each hypothetical simulation. Each item in the list is a list itself containing a slot, the value to set, and the timestep at which to set it.
6	SLOT	The target slot, the slot whose value you would like to attain a certain value.
7	DATETIME	The target date/time, the timestep at which you would like the target slot to attain the desired value.
8	NUMERIC	The target value, the value which you would like the target slot to achieve at the target date/time.
9	NUMERIC	The tolerance or desired accuracy of the returned value. If the function is successful, it indicates that setting the control slot to the returned value will lead to a value which differs by no more than the tolerance from the desired target value.
10	NUMERIC	The maximum number of iterations of hypothetical simulations allowed. If this number is reached without finding an return value within the tolerance, then the function fails.

11	NUMERIC	The minimum number of timesteps before and after the current timestep which might be involved in the simulation. As part of hypothetical simulation RiverWare makes copies of the objects in the subbasin and this input is used to determine how much data should be copied from each object. One can usually set this value to 0 and RiverWare will use a heuristic to determine the range over which to copy data. If this function fails because there was not enough data on some object, then input a higher value.
Evaluation	In a sense, HypTargetSim is the inverse of HypSim. In particular, *HypTargetSim(basin, control, min, max, targetSlot, targetDate, targetValue, tolerance, maxIterations) = x* implies that *HypSim(basin,{{targetSlot, t, x}, ..., {targetSlot, targetDate, x}}, {{controlSlot, targetDate}} = targetValue* The implementation of this function uses an iterative algorithm (the bisection algorithm) which performs an hypothetical simulation of the subbasin at each iteration.	
Comments	In general, the function assumes that the target value range (computed using the minimum and maximum control slot values) includes the target value itself. For example, if the control slot minimum of 100 cfs leads to a simulated target value of 100 cfs. the control slot maximum of 1000 cfs leads to a simulated target slot value of 200 cfs, and the target value is 300 cfs, then the function would fail because the target value is not in the range (100-200 cfs) implied by the input control slot minimum and maximum values. Mathematically, this is the assumption that the target slot's value is an always a monotonic function of the control slot's value. There is an exception that relaxes the constraint that the control slot values must lead to target values that bound the solution. If either the minimum or maximum control slot values fails to lead to a valid target value because the simulation fails to dispatch successfully with that control value, the search algorithm will effectively treat that bound as infinite. The search will then continue until either the interval becomes bounded and a satisfying control value is found, or the maximum iteration limit is reached. **Note:** HypTargetSim was originally named HypotheticalTargetSimulation. See "Hypothetical Simulations"; all comments mentioned there apply here as well.	

Syntax Example

```
HypTargetSim( "upper basin", Navajo.Outflow, 10 "cfs", 1000 "cfs",
   { {Navajo.Outflow, 1000 "cfs", @"t"} },
   Powell.Inflow, @"t", 100 "cfs", 0.1 "cfs", 10 )
```

Return Example

```
23.4 "cms"
```

• HypTargetSimWithStatus

This function finds a value which, when set on a given slot, will lead to a desired value on another slot. If a value satisfying this criterion is not found, then an attempt is made to find a value that comes close to doing so.

Description	Given a control slot and a target slot, target date/time, and target value, this function uses hypothetical simulation (see description of the predefined function HypSim) to find a value x such that if the control slot were set to x at all timesteps in the range [current date, target date], then the target slot's value would equal the target value. If the value x exceeds the physical constraint for that slot at a particular timestep (max outflow on a reservoir for example), then the constrained value is used instead of the x value for that timestep. A three-item list is returned. The first item in the list is a boolean TRUE value if a satisfying control slot value was found, FALSE otherwise. If the first item is TRUE, then the second item is the satisfying control slot value, otherwise this value is as close as the function could get to finding such a value. The third item is a list of the control slot values used in the solution. These values will all be the same as the second item, except if some of the values were constrained due to physical limitations. **Note:** This function is similar to HypTargetSim. The only difference is that HypTargetSim fails if it can not find a satisfying control slot value, whereas this function does not fail, rather it still returns a value, along with the indication that this value does not achieve the target and the list of control slot values.	
Type	LIST {BOOLEAN, NUMERIC, LIST}	
Arguments	**Type**	**Meaning**
1	STRING	The name of the Subbasin over which to perform the hypothetical simulations. This should include the objects on which the control and target slot exist as well as all other objects necessary to compute the target slot's value. If there is no Subbasin with the given name, the string is taken to be the name of an object and a temporary Subbasin is created containing only that object. An error will be issued if this subbasin contains a Data Object.
2	SLOT	The control slot, the slot with which you desire to control the target slot's value.
3	NUMERIC	The minimum control slot value. A value less than this is not considered a legal return value.
4	NUMERIC	The maximum control slot value. A value greater than this is not considered a legal return value.
5	LIST { LIST { SLOT, NUMERIC, DATETIME } }	Fixed values the user would like to set in each hypothetical simulation. Each item in the list is a list itself containing a slot, the value to set, and the timestep at which to set it.
6	SLOT	The target slot, the slot whose value you would like to attain a certain value.
7	DATETIME	The target date/time, the timestep at which you would like the target slot to attain the desired value.
8	NUMERIC	The target value, the value which you would like the target slot to achieve at the target date/time.
9	NUMERIC	The tolerance or desired accuracy of the returned value. If the function is successful, it indicates that setting the control slot to the returned value will lead to a value which differs by no more than the tolerance from the desired target value.
10	NUMERIC	The maximum number of iterations of hypothetical simulations allowed. If this number is reached without finding an return value within the tolerance, then the function fails.

11	NUMERIC	The minimum number of timesteps before and after the current timestep which might be involved in the simulation. As part of hypothetical simulation RiverWare makes copies of the objects in the subbasin and this input is used to determine how much data should be copied from each object. One can usually set this value to 0 and RiverWare will use a heuristic to determine the range over which to copy data. If this function fails because there was not enough data on some object, then input a higher value.
Evaluation	There are two conditions in which that function will fail but this function will return false as the first item in the return list: • The minimum and maximum control slot values lead to a range of target values which does not include the input target value. In this case the value returned is the minimum or maximum control slot value, which ever leads to a target value closest to the input target value. • The tolerance is not achieved within the iteration limit. In this case, the value returned is the current best guess. If Hypothetical Simulation diagnostics are turned on, then if HypTargetSimWithStatus can not find a satisfying control value, a diagnostic will be posted describing why it failed to do so.	
Comments	See also documentation for HypTargetSim for more details; the differences between these two functions are how problems are dealt with, this function is more flexible (as described in the Evaluation section). See “Hypothetical Simulations”; all comments mentioned there apply here as well.	

Syntax Example

```
HypTargetSimWithStatus( "upper basin", Navajo.Outflow, 10 "cfs", 1000 "cfs",
  { {Navajo.Outflow, 1000 "cfs", @"t"},
    {FlamingGorge.Outflow, 1000 "cfs", @"t"} },
  Powell.Inflow, @"t", 100 "cfs", 0.1 "cfs", 10 )
```

Return Example

```
{TRUE, 2.3 "cms", {2.3 "cms", 2.28 "cms"} }
```

• IntegerToString

Description	Returns a string representation of a numeric value interpreted as an integer.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	NUMERIC	a value
Evaluation	Given a numeric value, IntegerToString returns a string representation of that value rounded to the nearest integer and with the units removed.	
Comments	This function uses the RPL units of the specified NUMERIC. In the example, below, the RPL units are cfs as it is a literal value. But, if you reference a slot value, it will return the string using the relevant RPL units, which are often, but not always, internal units, cms, m, and so on. For more flexibility with units, see “IntegerWithUnitsToString”.	

Syntax Example

```
IntegerToString(123.456 “cfs”)
```

Return Example

“123”

• IntegerWithUnitsToString

Description	Returns a string representation of a numeric value interpreted as an integer after the specified value is converted to the given units.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	NUMERIC	a value to convert and truncate
2	NUMERIC	a value whose units are those to which the first value should be converted
Evaluation	Given a numeric value, IntegerWithUnitsToString converts that value to the display units of a second input value, and then returns a string representation of the converted value with the fractional part of the value and the units removed.	
Comments	It is an error if the two numeric input values do not have compatible units (i.e., are not of the same unit type). For the second argument, only its display units are involved in the computation; the scalar portion of this value is ignored. This value should typically be specified as a literal value, not the result of computation or lookup on a slot, so that the units of this value will be known with certainty.	

Syntax Example

```
IntegerWithUnitsToString(Res.Inflow[], 1.0 “cfs”)
```

Return Example

If the current value on the slot Res.Inflow is 123.9 cfs:

“123”

• IsControllerOpt

Description	Returns true if and only if the current controller is Optimization.	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
Evaluation		
Comments		

Syntax Example

```
IsControllerOpt()
```

Return Example

```
TRUE or FALSE
```

IsControllerRBS

Description	Returns true if and only if the current controller is Rulebased Simulation (RBS) or Inline Rulebased Simulation and Accounting.	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
Evaluation		
Comments		

Syntax Example

```
IsControllerRBS()
```

Return Example

```
TRUE or FALSE
```

IsControllerSim

Description	Returns true if and only if the current controller is Simulation or Inline Simulation and Accounting.	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
Evaluation		
Comments		

Syntax Example

```
IsControllerSim()
```

Return Example

```
TRUE or FALSE
```

IsEven

This function returns whether or not a given number is even.

Description	Returns true if and only if the input value (rounded down) is even.	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
1	NUMERIC	a value
2	NUMERIC	the units in which to determine evenness.
Evaluation	Converts the value into the desired units, rounds down to the nearest integer, then returns whether or not this value is even.	
Comments		

Syntax Example

```
IsEven(Puddle.Inflow[], 1.0 "cfs")
```

Return Example

```
TRUE or FALSE
```

IsInput

This function evaluates whether there is an input value on the given slot at the given datetime.

Description	The input status of a slot at a given datetime.	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
1	SLOT	the series slot
2	DATETIME	the datetime
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime. Then, the flag of the value in the series slot at that time is compared with the user input flags. If the flag is an I or Z, true is returned. An “i” is considered a user input within an iterative MRM run, but not a user input within a single run or outside of a run. The R flag is not considered an input. Also, IsInput returns false if the slot[datetime] is NaN. But, this function does NOT terminate the executing rule if the value at the given datetime is a NaN. See also “HasFlag”, which is a similar but more general function.	

Syntax Example

```
IsInput(SanJuanData.Mexico Call, @"t")
```

Return Example

```
TRUE or FALSE
```

IsOdd

This function returns whether or not a given number is odd.

Description	Returns true if and only if the input value (rounded down) is odd.	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
1	NUMERIC	a value
2	NUMERIC	the units in which to determine oddness.
Evaluation	Converts the value into the desired units, rounds down to the nearest integer, then returns whether or not this value is odd.	
Comments		

Syntax Example

```
IsOdd(Puddle.Inflow[], 1.0 "cfs")
```

Return Example

```
TRUE or FALSE
```

LeapYear

This function evaluates whether a given date occurs in a leap year.

Description	Whether or not a given datetime is in a leap year (containing 366 days instead of 365).	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
1	DATETIME	the datetime
Evaluation	The datetime argument; which may be specified symbolically, is converted into an actual datetime. If the date of this year is a leap year, true is returned, otherwise false.	
Comments		

Syntax Example

```
LeapYear(@"t")
```

Return Example

```
TRUE or FALSE
```

ListDownstreamObjects

Creates a list of linked downstream objects that are between two specified objects.

Description	This function evaluates to a list of linked downstream objects that are between two specified objects, inclusive.	
Type	LIST {OBJECT}	
Arguments	**Type**	**Meaning**
1	OBJECT	The upstream object at which to start the search
2	OBJECT	The downstream object at which to finish the search
Evaluation	ListDownstreamObjects searches links downstream from the first object until it finds the second object. It returns a list of all objects that it finds, inclusive of the specified objects.	
Comments	Notes / Limitations to this function: • To find the next downstream object, the function traverses links from main channel outflow slots. These slots typically include Outflows from each object; the table below shows the main channel outflow slot for each object • If a Bifurcation or Pipe Junction has more than one outflow link, an error is issued. • Object 1 must be upstream of Object 2. That is, if Object 2 is not found in the search, an error is issued. • For Aggregate Reaches and Agg Distribution canals, only the elements are returned, not the aggregate (but each element includes the agg name, E.g. %“AggReach:Element1”).	

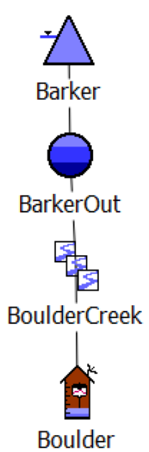
Object Type	Main channel outflow slots
Agg Diversion Site	Total Outflow
Agg Distribution Canal	Total Return Flow
Agg Reach Confluence Control Point Distribution Canal Groundwater Inline Power Inline Pump Pipeline Reach Reservoir	Outflow
Bifurcation	Outflow1, Outflow2
Diversion Object,	NONE
Pipe Junction	Flow 1, Flow 2
Stream Gage	Gage Outflow
Water User	NONE

Syntax Example

```
ListDownstreamObjects(%”Barker”, %”Boulder”)
```

Return Example

```
{ %”Barker”, %”BarkerOut”, %”BoulderCreek:Routing”,
  %”BoulderCreek:Locals”, %”Boulder” }
```



• ListSlotSet

This function evaluates to a list of the slots in a given Slot Set.

Description	The slots in the given Slot Set.	
Type	LIST {SLOTS}	
Arguments	**Type**	**Meaning**
1	STRING	The name of a Slot Set defined in the model.
Evaluation	The Slot Sets in the model are searched for a match to the given string. Then, a list is generated from the member slots of that set.	
Comments	Both static and dynamic Slot Sets may be referenced. Dynamic Slot Sets are resolved when the function is called. Member slots are listed in the order in which they appear in the Slot Set Manager dialog (from the Workspace menu, select **Workspace**, then **Slots**, then **Slot Set Management**). If there is no Slot Set with the given name in the model, this function aborts the run with an error.	

Syntax Example

```
ListSlotSet("AllResOutflows")
```

returns:

```
{"Mead.Outflow", "Powell.Outflow", "Havasu.Outflow"}
```

```
ListSlotSet("My Special Slots")
```

returns:

```
{"BigRes.MySlot", "Small Res.Trigger", "RedRiver.Seepage"}
```

• ListSubbasin

This function evaluates to a list of the objects in a given subbasin.

Description	The objects in the given subbasin.	
Type	LIST {OBJECT}	
Arguments	**Type**	**Meaning**
1	STRING	the name of the subbasin
Evaluation	The list of subbasins in the model is searched for a match to the given string. Then, a list is generated from the member objects of that subbasin.	
Comments	Both predefined and user defined subbasins may be referenced. Member objects are listed in the order in which they appear in the Edit Subbasins dialog. If there is no subbasin with the given name in the model, this function aborts the run with an error.	

Syntax Example

```
ListSubbasin("LevelPowerReservoir")
ListSubbasin("Colorado above GJ")
```

Return Example

```
{%"Mead", %"Powell", %"Havasu"}
```

Ln

This function evaluates to the natural logarithm of the given number.

Description	The natural logarithm of a number.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the value
2	NUMERIC	the units in which to compute the logarithm
Evaluation	Converts the value into the desired units, and then computes the natural logarithm of this value. The solution is this number in the units of the converted value.	
Comments	The natural logarithm may only be evaluated for values greater than zero. This function aborts the run with an error, if it is evaluated with a value less than or equal to zero. The two arguments must have compatible units, otherwise the run is aborted with an error. This function does not use the scalar portion of the units argument.	

Syntax Example

```
Ln(1.0 "cfs", 0.0 "cms")
```

Return Example

```
-3.56429837 "cms"
```

Log

This function evaluates to the base10 logarithm of the given number.

Description	The base10 logarithm of a number.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the value
2	NUMERIC	the units in which to compute the logarithm
Evaluation	Converts the value into the desired units, and then computes the base 10 logarithm of this value. The solution is this number in the units of the converted value.	
Comments	The base10 logarithm may only be evaluated for values greater than zero. This function aborts the run with an error, if it is evaluated with a value less than or equal to zero. The two arguments must have compatible units, otherwise the run is aborted with an error. This function does not use the scalar portion of the units argument.	

Syntax Example

```
Log(100.0 "cfs", 0.0 "cms")
```

Return Example

```
0.45204489 "cms"
```

Max

This function evaluates to the greater of its two arguments.

Description	The greater value.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the first value
2	NUMERIC	the second value
Evaluation	The two numbers are converted to a common unit and compared. The greater of the two numbers is returned.	
Comments	If the values are of a different unit type, this function aborts the run with an error.	

Syntax Example

```
Max(100"cfs", 10"cms") = 10.000 "cms"
Max(Powell.Storage[], Mead.Storage[]) = 1233481837.55 "m3"
```

MaxByIndex

This function finds the maximum numeric value at a given location within a list of lists.

Description	Given a list of lists and an index, return the maximum numeric value at the given index from all the sub lists.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST{LIST}	an input list of lists.
2	NUMERIC	an index (zero-based)
Evaluation	All values located at the given index in each list contained within the input list are visited; the maximum value is returned.	

Mathematical Expression	FOR sublist IN inputlist WITH result = (inputlist<0>)<i>
	  IF (sublist<i> > result)
	    sublist<i>
	  ELSE
	    result
	END FOR	

Comments	The input list must be non-empty. The index must be non-negative and a legal index for each of the lists contained within the values list. For example, if the index value is 3, the sublists must each contain at least 4 items. All items at the given index must be numeric and have the same unit type. If any of these conditions is not met, this function aborts with an error.	

Syntax Example

```
MaxByIndex( {{true, 2.0 “cms”}, {false, 1.0 “cms”}}, 1.0 )
```

Return Example

```
2.0 "cms"
```

• MaxItem

This function evaluates to the greatest number in a given list.

Description	The greatest value.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST	the list of values
Evaluation	The numbers in the input list are converted to a common unit and compared. The greatest number is returned.	
Comments	If the list is empty, one of the items is not numeric, or they do not all have compatible types, this function aborts the run with an error.	

Syntax Example

```
MaxItem({100"cfs", 10"cms", 50 "cfs"})
```

Return Example

```
10.00 "cms"
```

• MaxObjectsAggregatedOverTime

This function returns a list. The second item is a numeric value, which is the largest of several objects' aggregated slot values. The first item is the object with the largest value, and the third item is the timestep of the largest value. The objects' slot values may be aggregated as a SUM, AVG, MIN, or MAX over a specified time range.

Description	The largest of several object's values, each of which is the result of aggregating a slot's values over time.	
Type	LIST {OBJECT, NUMERIC, DATETIME}	
Arguments	**Type**	**Meaning**
1	STRING	subbasin name
2	STRING	slot name
3	STRING	aggregation function (SUM, AVG, MIN, or MAX)
4	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
5	BOOLEAN	time conversion option (TRUE or FALSE)
6	DATETIME	start date
7	DATETIME	end date

Evaluation	A list of slots is generated by searching all of the objects in the subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, each slot's values are aggregated according to the aggregation function argument over the time range of the datetime arguments. During each of these slot aggregations, any values which do not satisfy the aggregation filter argument are ignored. Finally, the largest of the objects' aggregated slot values is determined. This value is returned as the first value in a list. If there is a date/time associated with this value, it is returned as the second value in the list. This will be the case if the MIN or MAX aggregation function is specified for the fifth argument.
Mathematical Expression	$Max(\forall_{(obj \text{ in Subbasin})}[\forall_{(t \text{ from } start \text{ to } end)}[AggFunction_{(obj)}(obj.slotname)]])$
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, this function aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.

Syntax Example

```
MaxObjectsAggregatedOverTime( "upper basin", "Inflow", "MAX", "ALL", TRUE
   @"October, Previous Year",
   @"September, Current Year" )
```

Return Example

```
{BlueLake, 644.32 "1000 * acre-ft", 24:00 April 30, 2017}
```

• MaxObjectsAtEachTimestep

This function evaluates to a list. Each item of the list is a sub-list comprised of the datetime at which the largest value was determined, the value itself, and the object with the largest value.

Description	The largest of several object's slot values for each timestep in a range.	
Type	LIST{LIST{DATETIME, NUMERIC, OBJECT}}	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
4	BOOLEAN	time conversion option (TRUE or FALSE)
5	DATETIME	start date
6	DATETIME	end date

Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values whose maximum to find are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, all of the object's slot values are compared, yielding one maximum value for each timestep in the time range of the datetime arguments. The function returns a list of two items, where the first and second items of the inner lists are the datetime and the largest value, respectively.
Mathematical Expression	$\forall_{(t \text{ from } start \text{ to } end)}[\{t, Max(\forall_{(obj \text{ in Subbasin})}[obj.slotname])\}]$
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, RiverWare aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.

Syntax Example

```
MaxObjectsAtEachTimestep( "upper basin", "Storage", "ALL", FALSE
   @"October, Previous Year",
   @"September, Current Year" )
```

Return Example

```
{ { 24:00 October 31, 1996, 1233232.2 "m3", BlueLake},
   { 24:00 November 30, 1996, 1067478.3 "m3", BlueLake}, ...
   { 24:00 September 30, 1997, 1563456.7 "m3", GreenLake} }
```

MaxTimestepsAggregatedOverObjects

This function evaluates to a list, which is the timestep of the largest value resulting from aggregating several objects' slot values at each timestep, the value itself, and the object with the largest value.

Description	Largest over a timeseries of values, each of which is the result of aggregating several objects' slot values.	
Type	LIST {DATETIME, NUMERIC, OBJECT}	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation function (SUM, AVG, MIN, or MAX)
4	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
5	BOOLEAN	time conversion option (TRUE or FALSE)
6	DATETIME	start datetime
7	DATETIME	end datetime

Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, all of the objects’ slot values are aggregated according to the aggregation function argument for each timestep in the time range of the datetime arguments. During each of these slot aggregations, any values which do not satisfy the aggregation filter argument are ignored. Finally, the largest value in the timeseries of object aggregated slot values is determined. This value is returned as the second value in a list. The first item is the date/time associated with this value. If there is an object associated with the value, it is returned as the third value in the list. This will be the case if the MIN or MAX aggregation function is specified for the third argument.
Mathematical Expression	$Max(\forall_{(t \text{ from } start \text{ to } end)}[\forall_{(obj \text{ in Subbasin})}[AggFunction_{(t)}(obj.slotname)]])$
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, RiverWare aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.

Syntax Example

```
MaxTimestepsAggregatedOverObjects( "upper basin", "Storage", "MAX", "ALL", FALSE,
  @"October, Previous Year",
  @"September, Current Year" )
```

Return Example

```
{@"March 31, 2004", 2342343232.32"m3", Res1}
```

• MaxTimestepsForEachObject

This function evaluates to a list. Each item of the list is a list comprised of the object name, the largest value of the slot on that object for the time range specified and the timestep of the largest value.

Description	Largest value in a slot over a time range, for each object in a subbasin.	
Type	LIST {LIST {OBJECT, NUMERIC, DATETIME}}	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
4	BOOLEAN	time conversion option (TRUE or FALSE)
5	DATETIME	start datetime
6	DATETIME	end datetime

Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. For each object, the largest slot value over every timestep in the range of the datetime arguments is determined. Any values which do not satisfy the aggregation filter argument are ignored during the calculation. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are first multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME.
Mathematical Expression	$\forall_{(obj \text{ in Subbasin})}[\{obj, Max(\forall_{(t \text{ from } start \text{ to } end)}[obj.slotname])\}]$
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, this function aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, this function also aborts RiverWare with an error.

Syntax Example

```
MaxTimestepsForEachObject("upper basin", "Inflow", "ALL", TRUE,
   @"October, Previous Year",
   @"September, Current Year")
```

Return Example

```
{ {Res1, 12.23"cms", 24:00 April 30, 2017},
   {Reach2, 4.92 "cms", 24:00 March 31, 2017},
   {Res2, 23.2 "cms", 24:00 April 30, 2017} }
```

• MeetLowFlowRequirement

This function computes the necessary Low Flow Releases from contributing reservoirs to meet a low flow requirement at a specified control point.

Description	Computes the Low Flow Release from each reservoir so that the low flow requirement at the specified control point is met.	
Type	LIST{ LIST{Slot, Value, Object}}	
Arguments	**Type**	**Meaning**
1	STRING	The subbasin used to perform the calculations

2	OBJECT	The control point whose low flow requirement needs to be met
Evaluation	Returns a LIST of LISTs with the inner list containing a triplet. Each triplet is a slot (at index zero), the value to set on that slot (at index one), and the object of the slot (at index two). The function returns each Low Flow Release slot and the value to set on that slot. Also, the Outflow slot for each reservoir is returned with the new outflow value (Outflow plus Low Flow Release). The function computes the release for each contributing reservoir. The contributing reservoirs are specified in a slot called Low Flow Reservoirs on the Control Point. The rule executes as follows: First, the specified low flow reservoirs are sorted in descending order according to Operating Level. Reservoirs that are below the bottom of the conservation pool are excluded. Next, each reservoir (beginning with the most full reservoir) makes a release until the requirement (in the Computed Low Flow Requirement slot on the Control Point) is met, the Maximum Low Flow Delivery Rate (on the reservoir) is met, or the reservoir reaches the bottom of the conservation pool (whichever value is lowest). In addition, as each reservoir is making releases, the function calls the getMaxOutflowGivenInflow function to calculate the maximum flow that can be released from the reservoir. If the calculated low flow release is greater than this max, the release is reduced to the max. The Low Flow Release and updated Outflow value (limited by max constraints) for each reservoir is returned by the RPL function to the calling rule. The rule sets these slots using the syntax given below. After the rule executes, the system solves and routes the values downstream. **Note:** Each time this rule function is evaluated, it adds to the existing value in the Low Flow Release slot on reservoir objects. This is because each reservoir may contribute to the low flow requirement of more than one control point. So if the user wants to recompute all the low flow releases, they must all be reset to zero. In other words, this function is designed to execute once for each control point, adding to the Low Flow Releases made for previous control points.	
Comments	A rule needs to be created for each Control Point that has a low flow requirement. Each rule will call the MeetLowFlowRequirement function for the specified Control Point. After simulating the new releases, the next rule will be executed for the next low flow Control Point. The specified subbasin needs to include all the relevant objects (reservoirs, control points). Within the function execution, no routing of low flow releases is considered between the reservoir and the control point. But, during the simulation after the rule finishes, routing is considered. As a result, the water released may not make it to the control point on a single timestep. Each reservoir must have the Conservation and Flood Pools method selected in the Operating Levels category. The reservoirs specified in the Low Flow Reservoirs slot on each control point MUST be upstream of the control point. RiverWare does not check this and the results will be incorrect if the user does not enforce this. Also, no tandem operations are considered by this RPL function, i.e. the function assumes that reservoir releases can travel directly to a control point without passing through another reservoir. See “Low-flow Releases” in *USACE-SWD Modeling Techniques* for details on using this function for USACE-SWD.	

Syntax Example

```
MeetLowFlowRequirement("Basin")
```

where "Basin" contains Res1, Res2, and CP1.

Return Example

```
{ {"Res1.Low Flow Release", 9.75 "cms", "Res1"},
  {"Res1.Outflow", 9.75 "cms", "Res1"},
```

```
  {"Res2.Low Flow Release", 2.35 "cms", "Res2"}
  {"Res2.Outflow", 3.25 "cms", "Res2"} }
```

Use Example

```
FOR EACH ( LIST result IN MeetLowFlowRequirement("Basin", %"CP1")) DO
  ( result<0> )[] = result<1>
END FOR EACH
```

MemoryUsage

Description	Returns the RiverWare process’s current virtual memory usage, in kilobytes (KB).	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
Evaluation		
Comments	This function can be useful for performance analysis, for example to identify which rules in a rulebased simulation use the most memory.	

Syntax Example

```
MemoryUsage() returns 72,212
```

Min

This function evaluates to the smaller of its two arguments.

Description	The lesser value.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the first value
2	NUMERIC	the second value
Evaluation	The two numbers are converted to a common unit and compared. The lesser of the two numbers is returned.	
Comments	If the values are of a different unit type, this function aborts the run with an error.	

Syntax Example

```
Min(100"cfs", 10"cms")
```

returns:

```
100 "cfs"
```

```
Min(Powell.Storage[], Mead.Storage[])
```

returns:

```
12236343.55 "m3"
```

MinByIndex

This function finds the minimum numeric value at a given location within a list of lists.

Description	Given a list of lists and an index, return the minimum value at the given index from all the sublists.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST{LIST}	an input list of lists.
2	NUMERIC	an index (zero-based)
Evaluation	All values located at the given index in each list contained within the input list are visited; the minimum value is returned.	

Mathematical Expression	`FOR sublist IN inputlist WITH result = (inputlist<0>)<i>`
	`IF (sublist<i> < result)`
	`sublist<i>`
	`ELSE`
	`result`
	`END FOR`	

Comments	The input list must be non-empty. The index must be non-negative and a legal index for each of the lists contained within the input list. For example, if the index value is 3, the sublists must each contain at least 4 items. All items at the given index must be numeric and have the same unit type. If any of these conditions is not met, this function aborts with an error.	

Syntax Example

```
MinByIndex( {{true, 2.0 “cms”}, {false, 1.0 “cms”}}, 1.0 )
```

Return Example

```
1.0 "cms"
```

MinItem

This function evaluates to the least number in a given list.

Description	The smallest value.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST	the list of values
Evaluation	The numbers in the input list are converted to a common unit and compared. The smallest number is returned.	
Comments	If the list is empty, one of the items is not numeric, or they do not all have compatible types, this function aborts the run with an error.	

Syntax Example

```
MinItem( {100"cfs", 10"cms", 50 "cfs"} )
```

Return Example

```
50.0 "cfs"
```

• MinObjectsAggregatedOverTime

This function returns a list. The second item is a numeric value, which is the smallest of several objects' aggregated slot values. The first item is the object with the smallest value, and the third item is the timestep of the smallest value. The objects' slot values may be aggregated as a SUM, AVG, MIN, or MAX over a specified time range.

Description	The smallest of several object's values, each of which is the result of aggregating a slot's values over time.	
Type	LIST{OBJECT, NUMERIC, DATETIME}	
Arguments	**Type**	**Meaning**
1	STRING	subbasin name
2	STRING	slot name
3	STRING	aggregation function (SUM, AVG, MIN, or MAX)
4	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
5	BOOLEAN	time conversion option (TRUE or FALSE)
6	DATETIME	start date
7	DATETIME	end date
Evaluation	A list of slots is generated by searching all of the objects in the subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, each slot's values are aggregated according to the aggregation function argument over the time range of the datetime arguments. During each of these slot aggregations, any values which do not satisfy the aggregation filter argument are ignored. Finally, the smallest of the objects' aggregated slot values is determined. This value is returned as the first value in a list. If there is a date/time associated with this value, it is returned as the second value in the list. This will be the case if the MIN or MAX aggregation function is specified for the third argument.	
Mathematical Expression	$Min(\forall_{(obj \text{ in Subbasin})}[\forall_{(t \text{ from } start \text{ to } end)}[AggFunction_{(obj)}(obj.slotname)]])$	
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, this function aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.	

Syntax Example

```
MinObjectsAggregatedOverTime("upper basin", "Inflow", "MAX", "ALL", TRUE
   @"October, Previous Year",
   @"September, Current Year")
```

Return Example

```
{BlueLake, 0.24 "cms", @"February 3, 2003"}
```

• MinObjectsAtEachTimestep

This function evaluates to a list. Each item of the list is a sub-list comprised of the datetime at which the smallest value was determined, the value itself, and the object with the smallest value.

Description	The smallest of several object's slot values, for each timestep in a range.	
Type	LIST{LIST{DATETIME, NUMERIC, OBJECT}}	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
4	BOOLEAN	time conversion option (TRUE or FALSE)
5	DATETIME	start date
6	DATETIME	end date
Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values whose maximum to find are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, all of the object's slot values are compared, yielding one minimum value for each timestep in the time range of the datetime arguments. The function returns a list of two items, where the first and second items of the inner lists are the datetime and the smallest value, respectively.	
Mathematical Expression	$\forall_{(t \text{ from } start \text{ to } end)}[\{t, Min(\forall_{(obj \text{ in Subbasin})}[obj.slotname])\}]$	
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, RiverWare aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.	

Syntax Example

```
MinObjectsAtEachTimestep("upper basin", "Storage", "ALL", FALSE
  @"October, Previous Year",
  @"September, Current Year")
```

Return Example

For a model with a 1 Month timestep, something like:

```
{ { 24:00 October 31, 1996, 1232.2 "m3", BlueLake},
  { 24:00 November 30, 1996, 1074.3 "m3", GreenLake}, ...
  { 24:00 September 30, 1997, 1564.0 "m3", BlueLake} }
```

• MinTimestepsAggregatedOverObjects

This function evaluates to a list, which is the timestep of the smallest value resulting from aggregating several objects' slot values at each timestep, the value itself, and the object with the smallest value.

Description	Smallest over a timeseries of values, each of which is the result of aggregating several objects' slot values.	
Type	LIST {DATETIME, NUMERIC, OBJECT}	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation function (SUM, AVG, MIN, or MAX)
4	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
5	BOOLEAN	time conversion option (TRUE or FALSE)
6	DATETIME	start datetime
7	DATETIME	end datetime
Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, all of the objects' slot values are aggregated according to the aggregation function argument for each timestep in the time range of the datetime arguments. During each of these slot aggregations, any values which do not satisfy the aggregation filter argument are ignored. Finally, the smallest value in the timeseries of object aggregated slot values is determined. This value is returned as the second value in a list. The first item is the date/time associated with this value. If there is an object associated with the value, it is returned as the third value in the list. This will be the case if the MIN or MAX aggregation function is specified for the third argument.	
Mathematical Expression	$Min(\forall_{(t \text{ from } start \text{ to } end)}[\forall_{(obj \text{ in Subbasin})}[AggFunction_{(t)}(obj.slotname)]])$	
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, RiverWare aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.	

Syntax Example

```
MinTimestepsAggregatedOverObjects( "upper basin", "Storage", "MAX", "ALL", FALSE,
   @"October, Previous Year",
   @"September, Current Year" )
```

Return Example

```
{ @"March 31, 2001", 0.23"cms", Res1 }
```

• MinTimestepsForEachObject

This function evaluates to a list. Each item of the list is a list comprised of the object name, the smallest value of the slot on that object for the time range specified, and the timestep of the smallest value.

Description	Smallest value in a slot over a time range, for each object in a subbasin.	
Type	LIST {LIST {OBJECT, NUMERIC, DATETIME}}	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
4	BOOLEAN	time conversion option (TRUE or FALSE)
5	DATETIME	start datetime
6	DATETIME	end datetime
Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. For each object, the smallest slot value over every timestep in the range of the datetime arguments is determined. Any values which do not satisfy the aggregation filter argument are ignored during the calculation. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are first multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME.	
Mathematical Expression	$\forall_{(obj \text{ in Subbasin})}[\{obj, Min(\forall_{(t \text{ from } start \text{ to } end)}[obj.slotname])\}]$	
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, this function aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, this function also aborts RiverWare with an error.	

Syntax Example

```
MinTimestepsForEachObject( "upper basin", "Inflow", "ALL", TRUE,
  @"October, Previous Year",
  @"September, Current Year" )
```

Return Example

```
{ {Res1, 0.0"cms", 24:00 August 31, 2017},
  {Reach2, 0.02 "cms", 24:00 September 30, 2017},
  {Res2, 3.2 "cms", 24:00 August 31, 2017} }
```

Mod

This function computes the integer modulus of two numbers.

Description	Integer modulus of two numbers.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the numerator
2	NUMERIC	the units to which to convert the numerator
3	NUMERIC	the denominator

4	NUMERIC	the units to which to convert the denominator
Evaluation	Converts numerator and denominator into the specified units, then returns the integral modulus of the converted values, where integral modulus of x and y returns the integral remainder after integral division of x and y, which can be defined as follows: $\mathrm{Mod}(x, y) \equiv y \cdot \left(\frac{\lfloor x \rfloor}{\lfloor y \rfloor} - \left\lfloor \frac{\lfloor x \rfloor}{\lfloor y \rfloor} \right\rfloor\right)$	
Comments	If the denominator is equal to zero, the run is aborted with an error. Each of the units arguments must have units which are compatible with the value they are associated with, otherwise the run is aborted with an error. This function does not use the scalar portion of either of the units arguments.	

Syntax Example

```
Mod(3.9 "m", 0.0 "ft", 5.0 "sec", 0.0 "sec")
```

Return Example

```
2.0
```

• NetNonShortDiversionRequirement

This function computes the diversion required to satisfy all of Water Users' requests in an Aggregate Diversion Site.

Description	Minimum diversion required to meet all Water Users' requests.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the aggregate diversion site or diversion object
2	DATETIME	the timestep

Evaluation	If the object is an Aggregate Diversion Site and is linked with the No Structure or Lumped Structure, this function evaluates to the current value of the Total Diversion Requested slot. If the object is an Aggregate Diversion Site and if the Aggregate Diversion Site is linked with the Sequential Structure, the function sets a total diversion requirement equal to the topmost Water User's Diversion Requested and then loops over the remaining Water Users. The water available at each element is calculated based on the upstream elements' diversions and their return flows. If this water is enough to satisfy the Water User's Diversion Requested, the total diversion requirement is not modified. If this water is not enough to satisfy the Water User's Diversion Requested, the total diversion requirement is increased to satisfy this Water User. If the object is a diversion object, this function evaluates to the current value of the Diversion Request slot. **Note:** The diversion object cannot use the Percent of Available method.
Mathematical Expression	For sequential agg diversion sites: $Max\left(\forall_{(WU \text{ in } Agg)} \left[DivReq_{(WU)} + \sum_{Upstream\ WU} DivReq - \sum_{Upstream\ WU} ReturnFlow\right]\right)$
Comments	This function exits with an early termination if any of the required data used to solve the diversion is unknown. The required data is the same as that needed for the objects to fully dispatch, except Total/ Incoming Available Water, which need not be known. For sequentially linked Agg Diversion Sites, the function takes into account whether Return Flow, or Surface Return Flow are linked to another object, or unlinked and available to the next Water User in the diversion.

Syntax Example

```
NetNonShortDiversionRequirement( %"CAP Diversion", @"t" )
```

Return Example

```
9.25 "cms"
```

• NetSubbasinDiversionRequirement

This function computes the inflow to a subbasin required to satisfy all diversions in the subbasin while meeting minimum flow requirements below all diversion points.

Description	Minimum Inflow required to meet all diversion requests and minimum flows.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST	the subbasin's Reach and Confluence objects in downstream order
2	DATETIME	the timestep at which to calculate

Evaluation	The subbasin diversion requirement is originally set to zero. Each of the objects in the subbasin list is processed in the downstream order in which they are provided. If the object is a Reach, the following calculations are performed: • If an Aggregate Diversion Site or Diversion Object is linked to the Reach's Diversion slot, the NetNonShortDiversionRequirement function is executed on the diversion. If a water user is linked, then the Water User's Diversion Request is used. This step determines the diversion request from the Reach. • The Reach's minimum flow just below the diversion point is determined from the Reach's Minimum Diversion Bypass slot. If this slot does not exist because of the selected User Method in the Min Diversion Bypass category, the minimum flow requirement is zero. • If nothing is linked to the Diversion slot, but a value exists in the slot, this value is assumed to be the diversion requirement for this reach. In this case, there is no minimum flow requirement below the diversion point. • The subbasin diversion requirement is recalculated as the greater of the previous subbasin diversion requirement or the Reach diversion requirement plus the minimum flow requirement plus any cumulative upstream diversions minus any cumulative upstream return flows minus any cumulative upstream tributary inflows. • Any Local Inflow to the Reach is added to the cumulative tributary inflows. • If the Return Flow slot has a valid value, it is added to the cumulative return flows. If the Return Flow slot does not have a valid value, but a Water User or an Aggregate Diversion Site object is linked to it, the return flow is estimated. Return flow is estimated by subtracting the object's (Total) Depletion Requested from its (Total) Diversion Requested. The estimated return flow is then added to the cumulative return flows. If the object is a Confluence, the Inflow1 and Inflow2 slots are checked to determine which is the main subbasin flow, and which is the tributary inflow. The objects linked to the inflow slots are checked against the last Reach object to be processed. When a match is found, the other Inflow, if valid, is added to the cumulative tributary inflows. The loop continues until all objects in the list have been processed. The largest subbasin diversion requirement calculated at any diversion point is the total subbasin diversion requirement.
Mathematical Expression	$Max(\forall Reach \text{ in Subbasin})$ $\Big[NetNonShortDiversionRequirement(AggDivSite)$ $+ minimum\ flow_{(Reach)} + \sum_{Upstream\ Reach} Diversion$ $- \sum_{Upstream\ Reach} Return\ Flow - \sum_{Upstream\ Reach,\ Confluence} Tributary\ Inflow$

Comments	This function exits its calling rule with an early termination if any of the required data used to solve the diversions are unknown. The required data is the same as that needed for the NetNonShortDiversionRequirement predefined function for each Aggregate Diversion Site along the subbasin. This function aborts the run with an error if an object other than a Reach or Confluence is in the subbasin list. One of the Confluence Inflows must be linked to the previous Reach object upstream, or an Aggregate Reach which contains the previous Reach object upstream as its last element. If this condition is not met, the Confluence cannot determine which slot is the tributary inflow and the function aborts the run with an error. All subbasin diversion requirement calculations are performed at the given timestep. Subbasin diversion requirement will not be correct if there are lags in Reaches. This predefined function is recommended for use in long timestep models or for subbasins where there is no lag between top and bottom.

Syntax Example

```
NetDiversionRequirement( {%"La Plata", %"Hesperus", %"Pine Ridge"}, @"t" )
```

Return Example

```
9.25 "cms"
```

• NextDate

Returns the next date which matches a partially specified date.

Description	Resolves a partially specified date/time into the next (with respect to a reference date) date/time which matches the specified fields.	
Type	DATETIME	
Arguments	**Type**	**Meaning**
1	DATETIME	a reference date/time.
2	DATETIME	a partially specified date/time.
Evaluation	The unspecified fields with a coarser resolution are resolved into the future with respect to the reference date. If there are finer resolution fields, they are filled in with default values (e.g., time with 24:00, day of the month with the last day of the month). **Note:** If the partial date can be resolved into the current date, it is. See the Syntax Examples below.	
Comments	If the reference date/time is not fully specified or if the partial date/time is, then the run is aborted with an error.	

Syntax Example

```
NextDate(@"t", @"March")
```

assuming the current timestep is any date between March, 1994 and March, 1995, returns:

```
24:00 March 31, 1995
```

```
NextDate(@"24:00 February 28, 1995", @"March")
```

returns:

```
24:00 March 31, 1995
```

```
NextDate(@"24:00 May 10, 1995", @"March 20")
```

returns:

```
24:00 March 20, 1996
```

```
NextDate(@"24:00 February 28, 1994", @"MAX DayOfMonth")
```

returns:

```
February 28, 1994
```

```
NextDate(@"24:00 February 28, 1994", @"Tuesday")
```

returns:

```
24:00 March 1, 1994
```

```
NextDate(@"24:00 February 28, 1994", @"6:00 MAX DayOfYear")
```

returns:

```
6:00 December 31, 1994
```

```
NextDate(@"24:00 February 28, 1994", @"6:00")
```

returns:

```
6:00 March 1, 1994
```

```
NextDate(@"", @"")
```

returns:

```
( )
```

• NumberToDate

Description	Given a numeric encoding of a date/time, returns the corresponding date/time value.	
Type	DATETIME	
Arguments	**Type**	**Meaning**
1	NUMERIC	The numeric encoding of a date/time value.
Comments	Slots representing date/time values have unit type DateTime. Internally these values are represented as numbers although the interface displays them as date/times. Looking up a value on such a slot will retrieve the numeric encoding, this function converts that number to a date/time value as required to treat it as a date within policy. If the unit for the slot corresponds to a partially specified date/time format, then the result will a partially specified date/time value.	

Syntax Example

```
NumberToDate(Data.PriorityDate[])
```

Return Example

If the Data.PriorityDate slot has units “MonthAndDate”, this call might return:

```
@"January 12"
```

Use Example

This function should be used in conjunction with Dates on Series slots and the DateToNumber function; see “DateTime Values in Slots” in *User Interface* and “DateToNumber”. For a use example, see “Access to DateTime Values Using RPL” in *User Interface*.

NumberToYear

Description	Given a numeric value, NumberToYear returns a DATETIME with only the year.	
Type	DATETIME	
Arguments	**Type**	**Meaning**
1	NUMERIC	A number.
Evaluation	NumberToYear truncates the specified numeric value and returns the value as a year DATETIME.	

Syntax Example

```
NumberToYear(2013.987 “s”)
```

Return Example

```
@“Year 2013”
```

NumColumns, NumRows

Description	Returns the number of columns/rows in a table slot or periodic slot.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	a table or periodic slot.
Comments	If the slot is not a table or periodic slot, the run is aborted with an error.	

Syntax Example

```
NumRows($"Data.MyTable") = 3
```

ObjAcctSupplyByWaterTypeRelTypeDestType

Description	This function finds a list of objects, accounts and supplies that match the given arguments. It returns a list of triplets{ OBJECT object, STRING account, STRING supply }, where the object^account is served by the supply, and the object is in the given subbasin (argument 1), the supply has the given release type and destination type (arguments 3 and 4), and the *supplying account* (upstream end of the supply in the returned triplet) has the given water type (argument 2).	
Type	LIST (LIST { OBJECT, STRING, STRING })	
Arguments	**Type**	**Meaning**

1	STRING	The name of the subbasin in which to search.
2	STRING	The water type of the upstream end of the supplies returned. The string ALL means that any water type will satisfy this filter. The string NONE means that only supplying accounts lacking a water type will satisfy this filter.
3	STRING	The release type of the supply returned. The string ALL means that any release type will satisfy this filter. The string NONE means that only supplies lacking a release type will satisfy this filter.
4	STRING	The destination type of the supplies returned. The string ALL means that any destination type will satisfy this filter. The string NONE means that only supplies lacking a destination type will satisfy this filter.
Comments	This function is meant to be used in conjunction with the water rights solvers (SolveWaterRights()). It looks for supplies that are *appropriation points* for legal water accounts as defined for the water rights solver. In the solver, these supplies are identified by the water type of the account at the point of appropriation. Usually these supplies directly supply the object^account in the returned triplets. The one exception to this is when the supply serves an offstream reservoir. In this case, the offstream reservoir is supplied through a diversion object, and so a passthrough account on the diversion object sits between the point of diversion and the receiving object^account. This is the only case in which any indirection is detected, and the function looks two hops upstream to check the supplying account's water type. In all other cases, the function looks only one hop upstream.	

Syntax Example

```
ObjAcctSupplyByWaterTypeRelTypeDestType( "WRA", "MyWT", "MyRel", "MyDest" )
```

Return Example

```
{ {"Res1", "Farmer1", "Res1 Farmer1 Diversion to Farmer1 Diversion"},
  {"Res1", "Farmer2", "Res1 Farmer2 Diversion to Farmer2 Diversion"} }
```

• ObjectAttributeValue

Description	For the specified Object and a string representing an attribute, return the value for that particular Object, as a string	
Type	STRING	
Arguments	**Type**	**Meaning**
1	OBJECT	The Object
2	STRING	The name of the Attribute
Evaluation		
Comments	If the attribute is not found on the object, an error is issued. If the object or attribute is not found in the model, an error will be issued.	

Syntax Example

```
ObjectAttributeValue(%"Dolores", "State")
```

Return Example

```
"Colorado"
```

ObjectHasAttributeValue

Description	Return whether the particular Object has the specified Attribute Value.	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
1	OBJECT	The Object
2	STRING	The name of the Attribute
3	STRING	The Value of the Attribute.
Evaluation	When evaluated, the function looks at the particular object and checks to see if it has the given Attribute and Value.	
Comments	If the attribute or value is not found on the object, FALSE is returned. If the object, attribute or value is not found in the model, an error will be issued.	

Syntax Example

```
ObjectHasAttributeValue(%"Dolores", "State", "Colorado")
```

Return Example

```
TRUE
```

ObjectiveValue

Description	Returns the objective value from the last successful solution to the Optimization problem.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
Comments	If there is no solution information available (e.g., if an Optimization run has not occurred), the run is aborted with an error diagnostic.	

Syntax Example

```
ObjectiveValue()
```

Return Example

```
12.23
```

ObjectsFromAccountName

Description	Returns a list of the objects that contain an account with the given name and account type.	
Type	LIST{OBJECT}	
Arguments	**Type**	**Meaning**
1	STRING	The name of the account.
2	STRING	The name of the account type, e.g, “Storage”. The string ALL for account type designates all account types.
Comments		

Syntax Example

```
ObjectsFromAccountName("Municipal", "Storage")
```

Return Example

```
{%"Reservoir1", %"Reservoir2"}
```

ObjectsFromAttributeValue

Description	Given a string representing an attribute and a string representing a value, return a list of all of the objects that have that value for the attribute	
Type	LIST {OBJECT, OJBECT, ...}	
Arguments	**Type**	**Meaning**
1	STRING	The name of the Attribute
2	STRING	The Value of the that Attribute.
Evaluation	When evaluated, the function looks throughout the model and finds all objects that have the Attribute Value pair.The set of matching objects is returned in a list.	
Comments	If the attribute or value is not found in the model, an error will be issued.	

```
ObjectsFromAttributeValue("State", "Colorado")
```

Return Example

```
{%"Arkansas", %"RioGrande", %"SanJuan", %"Dolores", %"Gunnison"}
```

ObjectsFromWaterType

Description	Returns a list of the objects that have an account with given water type and account type.	
Type	LIST{OBJECTS}	
Arguments	**Type**	**Meaning**
1	STRING	The water type. The string ALL for water type designates all water types, and the string NONE designates the default water type.
2	STRING	The name of the account type, e.g, "Storage". The string ALL for account type designates all account types.
Comments		

Syntax Example

```
ObjectsFromWaterType("ALL", "Storage")
```

Return Example

```
{%"Reservoir1", %"Reservoir2"}
```

OffsetDate

This function adds some number of timesteps to a given date/time and returns the result.

Description	Returns the date/time which is some number of timesteps added to or subtracted from an input date/time.	
Type	DATETIME	
Arguments	**Type**	**Meaning**
1	DATETIME	a date/time.
2	NUMERIC	the number of timesteps to add (a negative number will subtract). Should have units of NONE.
3	STRING	a timestep specification. This specification includes an integer which can be positive or negative. Examples include “1 Months”, “2 hours”. Case is not important in this string. The valid timestep units are “Minutes,” “Hours,” “Days,” “Weeks,” “Months,” “Years.”
Evaluation	Adds the given timestep, to the given date, to the given number of times, then returns the result.	
Comments	If the second argument has units other than NONE, or if the third argument is not recognized as a timestep, then the run is aborted with an error. **Tip:** Although you can put any integral amount within the timestep specification, if you make this integer 1, then the second argument allows you to vary the increment (second argument) at the time of rule execution. See “Datetime Math” in *RiverWare Policy Language (RPL)* for details on datetime math.	

Syntax Example

```
OffsetDate(@"t", 1, "1 Months")
OffsetDate(@"January 1, 2000", getIncr(), "1 Hours")
```

Return Example

```
@"July 31, 2007"
```

Note on Leap Years:

If the third argument represents a variable length timestep, months or years, leap years are included as follows (using “1 Years” as an example):

- When mapping from a leap year to a non leap year, Feb 28th is returned for both the 28th and 29th:

  `OffsetDate(@ “February 28, 2024, -1, “1 Years”)` returns `@”February 28, 2023”`

  `OffsetDate(@ “February 29, 2024, -1, “1 Years”)` returns `@”February 28, 2023”`

- When mapping from a non leap year to a leap year, the February 29th is used.

  `OffsetDate(@ “February 28, 2025, -1, “1 Years”)` returns `@”February 29, 2024”`

  This is done to access the end of month value.

A similar approach is used for strings of “1 Months”.

• OperatingHeadToMaxRelease

This function performs a lookup in a Power Reservoir object's Max Turbine Q table based on a given operating head, and then evaluates to the corresponding maximum turbine release.

Description	Find the maximum turbine release at a given reservoir operating head.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	power reservoir object
2	NUMERIC	operating head
3	DATETIME	datetime context for unit conversions
Evaluation	The operating head argument is looked up in the Operating Head column, of the Max Turbine Q table, of the power reservoir object argument, to determine the Turbine Capacity. If the exact operating head is not in the table, the lookup performs a linear interpolation between the two nearest bounding operating heads and their corresponding turbine capacities.	
Mathematical Expression	$turbine\ capacity = turbine\ capacity_{(lesser)} + \frac{turbine\ capacity_{(greater)} - turbine\ capacity_{(lesser)}}{operating\ head_{(greater)} - operating\ head_{(lesser)}} \times (operating\ head - operating\ head_{(lesser)})$	
Comments	If the object is not a power reservoir, the function aborts the run with an error. If the Power Reservoir does not have a Max Turbine Q table, Plant Power Coefficient must be selected as the Power selected method, or this function exits the rule with an early termination.	

Syntax Example

```
OperatingHeadToMaxRelease(%"Hoover Dam", 508.63 "ft", @"t")
```

Return Example

```
152.23 "cms"
```

• OptDualPrice

Description	Returns a physical constraint's dual price as calculated during the most recent Optimization solution.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	Desired object
2	STRING	Physical constraint name

3	DATETIME	Date at which to return the value
Evaluation	This function returns the dual price calculated during the last optimization run for a physical constraint, where the constraint is specified by three inputs: a simulation object, the name of a physical constraint, and a date.	
Comments	If there is no physical constraint corresponding to the inputs, the run is aborted. If the physical constraint exists, but has no dual price, then an invalid value is returned. The interpretation of the dual price is the change in the objective function value per one unit increase in teh constraint right hand side. The units are objective function units / constraint units Currently the only physical constraint supported is Mass Balance.	

Syntax Example

```
OptDualPrice(%"Norris", “Mass Balance”, @”18:00 Jan 23, 2009”)
```

Return Example

```
152.23 "$/m3"
```

OptReducedCost

Description	Returns a slot variable’s reduced cost as calculated during the most recent Optimization solution.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	Desired Slot
2	DATETIME	Date at which to return the value
Evaluation	This function returns the reduced cost for the given slot at the given timestep as part of the most recent optimization problem solution of the last run using the Optimization controller.	
Comments	If there is no optimum reduced cost for the slot at that timestep, an invalid value is returned. This function supports the return of reduced costs which are associated with decision variables, that is to variables which are contained in the problem and are not replaced with linear combinations or substitutions of other variables. If called for slots whose variables are not decision variables, an error message will be posted, and the run will be aborted.	

Syntax Example

```
OptReducedCost("Norris.Spill", @”18:00 Jan 23, 2009”)
```

Return Example

```
152.23 "$/cms"
```

OptReducedCostByCol

Description	Returns a slot variable’s reduced cost as calculated during the most recent Optimization solution for a variable associated with a particular column of an agg series slot.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	Specified Agg Series Slot

2	NUMERIC	Column index (0-based)
3	DATETIME	Date at which to return the value
Evaluation	This function returns the reduced cost for the given slot and column at the given timestep as part of the most recent optimization problem solution of the last run using the Optimization controller.	
Comments	If there is no reduced cost for the slot and column at that timestep, an invalid value is returned. This function supports the return of reduced values which are associated with decision variables, that is, to variables which are contained in the problem and are not replaced with linear combinations or substitutions of other variables. If called for slots whose variables are not decision variables, an error message will be posted, and the run will be aborted.	

Syntax Example

```
OptReducedCostByCol("Thermal.Hydro Block Use", 7, @”18:00 Jan 23, 2009”)
```

Return Example

```
152.23 "$/MWH"
```

• OptValue

Description	Returns a slot variable’s optimal value as calculated during the most recent Optimization solution.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	Desired Slot
2	DATETIME	Date at which to return the value
Evaluation	This function returns the optimal value for the given slot at the given timestep as part of the most recent optimization problem solution of the last run using the Optimization controller. See “Create a Post-optimization Ruleset” in *Optimization* for an example of the typical use of this function.	
Comments	If there is no optimum value for the slot at that timestep, but there is a slot cache value, then the cached value is returned. If neither value is valid, an invalid value is returned. In the future the return of the cached value is likely to be eliminated. This function supports the return of slot values which correspond to decision variables, that is to variables which are contained in the problem and are not replaced with linear combinations of other variables. If called for slots whose variables are not decision variables, an error message will be posted and the run will be aborted. The OptValuePiecewise function allows access to the piecewise approximation of non-decision variables for which the piecewise approximation technique is applicable.	

Syntax Example

```
OptValue("Norris.Outflow", @”18:00 Jan 23, 2009”)
```

Return Example

```
152.23 "cms"
```

OptValueByCol

Description	Returns a slot variable’s optimal value as calculated during the most recent Optimization solution for a variable associated with a particular column of an agg series slot.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	Specified Agg Series Slot
2	NUMERIC	Column index (0-based)
3	DATETIME	Date at which to return the value
Evaluation	This function returns the optimal value for the given slot and column at the given timestep as part of the most recent optimization problem solution of the last run using the Optimization controller. See “Create a Post-optimization Ruleset” in *Optimization* for an example of the typical use of this function.	
Comments	If there is no optimum value for the slot and column at that timestep, but there is a slot cache value, then the cached value is returned. If neither value is valid, an invalid value is returned. In the future the return of the cached value is likely to be eliminated. This function supports the return of slot values which correspond to decision variables, that is, to variables which are contained in the problem and are not replaced with linear combinations of other variables. If called for slots whose variables are not decision variables, an error message will be posted and the run will be aborted.	

Syntax Example

```
OptValueByCol("Thermal.Hydro Block Use", 7, @”18:00 Jan 23, 2009”)
```

Return Example

```
152.23 "MWH"
```

OptValuePiecewise

Description	Returns a slot variable’s piecewise approximation value as calculated during the most recent Optimization solution.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	Specified Slot
2	DATETIME	Date at which to return the value
Evaluation	Given a Series Slot and a date, this function returns the piecewise approximation of the slot at the given date, as computed during the most recent problem solution of the last optimization run.	
Comments	The function will fail if the variables associated with the slot are not approximated (i.e., are decision variables or are replaced by a linear combination of other variables). If there is no approximation value available, perhaps because the quantity was not referenced in the optimization policy or there has been not been a successful optimization run, the function returns an invalid value.	

Syntax Example

```
OptValuePiecewise(%"Kumquat Reservoir.Power", @”18:00 Jan 23, 2009”)
```

Return Example

```
6.1 "MW"
```

• Percentile

Returns the pth percentile from a list of values.

Description	This function returns the pth percentile from a list of values. In other words, it returns the value with a given percentile from a distribution.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST	A list of NUMERIC values representing the distribution.
2	NUMERIC	The probability p for which you wish the corresponding value. **Note:** p should be between 0 and 1 inclusive.
Evaluation	The list in argument one describes the distribution by providing independently sampled values from that distribution. The function returns an estimate of the value which has the given probability p of being greater than a value taken from the distribution. Consequently, for an input probability p at most (100*p)% of the values in the data set will be less than the return value (and at most 100*(1-p)% will be greater than this value). Several methods exist for computing the percentile; the following is the technical definition used by the Percentile function: for the pth percentile, Percentile(list, p): Compute p x (N + 1) where N is the number of items in the data set. Then set k and d, where k + d = p x (N + 1), k is an integer, and d is a fraction greater than or equal to 0 and less than 1. Essentially k is the integer part and d is the decimal part of p x (N + 1). Then, sort the numeric values in the list in increasing order. The function $Y_{[i]}$ denotes the isorted value of the numeric list, where i is between 1 and N, inclusive. Then: If k = 0, Percentile(list, p) = $Y_{[1]}$ If 0 < k < N, Percentile(list, p) = $Y_{[k]} + (d)(Y_{[k+1]} - Y_{[k]})$ If k = N, Percentile(list, p) = $Y_{[N]}$ See *NIST/SEMATECH e-Handbook of Statistical Methods* for details: http://www.itl.nist.gov/div898/handbook	
Comments	This function is sometimes called the *quantile* function. Excel's, Percentile function sets 1+p(N-1) equal to k + d, then proceeds as above. The two methods give similar results.	

Syntax Example

```
Percentile( {1"cfs", 7"cfs", 3"cfs", 4"cfs"}, 0.3 )
```

Return Example

```
2.0"cfs"
```

• PercentRank

Description	Find the rank of a given value within a list of values as a percentage of the number of values in the list.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST	a list of numeric values
2	NUMERIC	the value for which to determine the rank
Evaluation	This function provides a measure of the relative standing of a value within a data set. If the value whose percent rank is being determined, x, is one of the input values, then the return value is computed by: $\frac{\text{\# of values less than x}}{\text{\# of values} - \text{\# of values equal to x}}$ Otherwise, interpolation is used to combine the percent ranks for the closest data points on either size of x. If the input values are viewed as a sample from some distribution, then PercentRank can be viewed as a smooth estimate of the empirical cumulative distribution function.	
Comments	This function produces the same results as Excel’s PercentRank function.	

Syntax Example

```
PercentRank( {4”cfs”, 4”cfs”, 3”cfs”, 1”cfs”}, 2.3”cfs” )
```

Return Example

```
0.21666667
```

• PreviousDate

Returns the previous date which matches a partially specified date.

Description	Resolves a partially specified date/time into the previous (with respect to a reference date) date/time which matches the specified fields.	
Type	DATETIME	
Arguments	**Type**	**Meaning**
1	DATETIME	a reference date/time.
2	DATETIME	a partially specified date/time.
Evaluation	The unspecified fields with a coarser resolution are resolved into the past with respect to the reference date. If there are finer resolution fields, they are filled in with default values (e.g., time with 24:00, day of the month with the last day of the month). **Note:** If the partial date can be resolved into the current date, it is. See Syntax Examples, below.	
Comments	If the reference date/time is not fully specified or if the partial date/time is, then the run is aborted with an error.	

Syntax Example

If “t” is any date between March, 1994 and March, 1995:

```
PreviousDate(@"t", @"March")
```

returns:

```
24:00 March 31, 1994
```

```
PreviousDate(@"24:00 May 10, 1995", @"March 20")
```

returns:

```
24:00 March 20, 1995
```

```
PreviousDate(@"24:00 February 28, 1994", @"MAX DayOfMonth")
```

returns:

```
February 28, 1994
```

```
PreviousDate(@"24:00 February 28, 1994", @"Tuesday")
```

returns:

```
24:00 February 22, 1994
```

```
PreviousDate(@"24:00 February 28, 1994", @"6:00 MAX DayOfYear")
```

returns:

```
6:00 December 31, 1993
```

```
PreviousDate(@"24:00 February 28, 1994", @"6:00")
```

returns:

```
6:00 February 28, 1994
```

• RanDev

Description	Returns the next number in a pseudo-random sequence.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	a number which is ignored except that the units are taken as the units to be returned.
Evaluation	Returns the next number in the pseudo-random series, given a seed.	
Comments	This function should not be called within a user-defined function which has no arguments, if that user-defined function might be called multiple times within a single block (rule). This is because functions with no arguments are actually evaluated only once per rule and return this same result on each function call during the execution of that block. This is not a very good random number generator, but is implemented in this way for historical reasons. If ResetRanDev() has not been called before this function, then the results are unpredictable.	

Syntax Example

```
RanDev(1)
```

Return Example

```
0.34105
```

• Random, RandomNormal

Description	Returns a given number in a pseudo-random sequence.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	a numeric value which is rounded down to an integer and used to identify a unique sequence of numbers -- calls with the same integral seed refer to the same random sequence of numbers.
2	NUMERIC	a numeric value which is rounded down to an integer and denotes the one-based index into the random sequence of the value to be returned.
3	NUMERIC	a number which is ignored except that the units are taken as the units to be returned.
Evaluation	`Random` returns a number from a random sequence of numbers uniformly distributed in the range [0, 1.0]. `RandomNormal` returns a number from a random sequence of numbers whose distribution is normal with a mean of 0 and a standard deviation of 1. The unique sequence of numbers associated with each integral seed is the same on all platforms supported by RiverWare, allowing for repeatable results. The sequences are generated using the linear congruential method described in Park and Miller (1988) Communication of the ACM, vol 31, pages 1192-1201. **Note:** Random number generators such as this are often referred to as *pseudo-random* because they are not the result of an intrinsically random process; they are in fact predictably determined by the seed.	
Comments	The time to evaluate a call to either of these functions is proportional to the magnitude of the index argument (because the entire sequence must be generated at least once per RiverWare execution). Thus, if performance is an important issue, one should choose to get numbers from the earlier portion of a sequence.	

Syntax Example

```
RandomNormal(1.0, 1.0, 1.0)
RandomNormal(1.0, 3.0, 0.0)
```

Return Example

Refer to the sequence: 0.09151046 0.33494915 -1.421276 -1.24931121 ...

Thus, the first call returns the first number in the sequence (0.09151046), and the second call returns the third in the sequence (0.33494915).

• ReleaseTypes

This function evaluates to the list of user-defined ReleaseTypes

Description	This function returns a list of the names of all ReleaseTypes defined in the Water Accounting System Configuration.
Type	LIST {STRING}
Arguments	none
Evaluation	
Comments	ReleaseTypes are properties of Supplies. The returned list does not include the default (NONE) ReleaseType.

Syntax Example

```
ReleaseTypes()
```

Return Example

```
{"MinimumFlows", "ProjectWater", "Flood"}
```

• ReleaseTypesFromObject

This function evaluates to the list of ReleaseTypes which represent outflows from an Object

Description	This function returns a list of unique names of ReleaseTypes of Supplies which represent outflows from a specified Object.	
Type	LIST {STRING}	
Arguments	**Type**	**Meaning**
1	OBJECT	The Object.
Evaluation	The set of Accounts on the Object are examined. The outflow Supplies on those Accounts which link a different downstream Object are considered. The names of the ReleaseTypes of those Supplies are added to the returned list -- but any given ReleaseType name will appear on the list only once.	
Comments	ReleaseTypes are properties of Supplies. The returned list can include the default (NONE) ReleaseType. Supplies which represent *internal flows* between two Accounts on the Object are not considered.	

Syntax Example

```
ReleaseTypesFromObject(%"Reservoir1")
```

Return Example

```
{"MinimumFlows", "Flood"}
```

ResetRanDev

Description	Initialize internal data structures to permit RanDev() to return a pseudo-random sequence of numbers. This involves reading a file, each line of which has a date associated with it. Basically, this is a *seeding* function.	
Type	BOOLEAN	
Arguments	**Type**	**Meaning**
1	BOOLEAN	True if some lines in the initialization file should be skipped.
2	DATETIME	The date of the line to be skipped.
Evaluation	Returns true if initialization was successful.	
Comments	The recommendation is that this function be called within a block that contains only the following statement: obj.slot[] = IF (NOT ResetRanDev(...)) STOP_RUN "ResetRanDev failed" ENDIF This will never assign any values but will always evaluate the function call. An alternative is to embed the call within a Print statement, but if diagnostics are turned off then this statement will not get executed. This, and the RanDev() function are scheduled to be removed in the future and replaced with a more convenient and effective means of generating a sequence of pseudo-random numbers.	

Syntax Example

```
ResetRanDev(TRUE, @"24:00:00 October Max DayOfMonth, 1983")
```

Return Example

```
TRUE or FALSE
```

Reverse

Description	Reverses the order of items in a list.	
Type	LIST	
Arguments	**Type**	**Meaning**
1	LIST	a list of values
Evaluation	Returns a list with the same values as the input list, in reverse order.	
Comments		

Syntax Example

```
Reverse( {1.0, {res1, 10}, "hello", 0.0, "bob"} )
```

Return Example

```
{"bob", 0.0, "hello", {res1, 10}, 1.0}
```

RoundToFactor

Description	Round the specified numeric value to a multiple of the factor	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the value
2	NUMERIC	the factor
Evaluation	Converts the value into the units of the factor, then rounds the value to the integral multiple of the factor.	
Comments	When the value is halfway between two multiples of the factor, the value is rounded away from zero. The value and factor must have the same unit type. See also “Ceiling” and “Floor”.	

Syntax Example

```
RoundToFactor(Wet Reservoir.Pool Elevation[], 100.0 ft)
```

returns:

```
5300.0 ft
```

```
RoundToFactor(6.5 cfs, 1 cfs)
```

returns:

```
7 cfs
```

```
RoundToFactor(2.33 m3, 0.1 m3)
```

returns:

```
2.3 m3
```

RowLabel

Description	Returns the label associated with a given row of a table slot.	
Type	STRING	
Arguments	**Type**	**Meaning**
1	SLOT	A table slot
2	NUMERIC	The row index (0-based).
Evaluation	Returns the label of the row of the table slot which has the given index.	
Comments	It is an error to provide an illegal index (e.g., an index of 4 with a table which has only 4 rows). If the row index is legal but there is no label for that row, then the empty string is returned: " ".	

Syntax Example

```
RowLabel(DataObjA.CoeffTable, 2)
```

Return Example

```
"Coefficient 1"
```

• RowLabels

Description	Returns a list containing the row labels of a given table slot, in order.	
Type	LIST of STRING values	
Arguments	**Type**	**Meaning**
1	SLOT	A table slot
Evaluation	Returns a list of strings that represent the row labels of the table slot	
Comments	It is an error if the input slot has a type other than table slot. For each row, if no label exists the empty string is returned.	

Syntax Example

```
RowLabels(DataObjA.CoeffTable)
```

Return Example

```
{“Coefficient 1”, “Coefficient 2”, “Coefficient 3”}
```

• RunStartDate, RunEndDate

Description	RunStartDate and RunEndDate return the start or end date of the currently active controller, respectively.	
Type	DATETIME	
Arguments	**Type**	**Meaning**
Comments	When evaluated from a Rule, Goal, or Method set, these functions are equivalent to @"Start Timestep" or @"Finish Timestep". But, for Expression Series Slots, the symbolic datetime specifications @"Start Timestep" and @"Finish Timestep" refer to the expression slot’s evaluation range, not the controller’s start or end dates. Thus, RunStartDate() may not be equivalent to @"Start Timestep" and RunEndDate() may not be equivalent to @"Finish Timestep". But, regardless of the set from which they are called, RunStartDate and RunEndDate functions provide a fixed references to the controller’s start and end dates, respectively.	

Syntax Example

```
RunStartDate()
```

Return Example

```
@"January 1, 2003"
```

• RunTime

Description	Returns the number of seconds which have elapsed since the current run began, or if called from outside a run, the total number of seconds within the last run.
Type	NUMERIC

Syntax Example

```
RunTime()
```

Return Example

```
22.000 "s"
```

• SlotCacheValue

Description	Returns a series slot’s value from the slot cache.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	Series slot whose cache value is desired
2	DATETIME	Date for which the value is desired
Evaluation	Returns the slot cache value for the given series slot at the given date. If there is no such value, an invalid value is returned.	
Comments	The slot cache is a repository of series slot values that can be created from workspace series slots, allowing access within one run to values computed by a previous run. See “Slot Cache” in *User Interface* for details. **Note:** The slot cache is under development. For information on the current status of this feature, email: riverware-support@colorado.edu	

Syntax Example

```
SlotCacheValue(%"Berkley.Outflow", @”18:00 Jan 23, 2009”)
```

Return Example

```
152.23 "cms"
```

• SlotCacheValueByCol

Description	Returns an aggregate series slot’s value from the slot cache.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	Aggregate series slot for which a slot cache value is desired.
2	NUMERIC	Index of the column for which a value is desired.
3	DATETIME	Date for which a value is desired.
Evaluation	Returns the slot cache value for the given series slot at the given date. If there is no such value, an invalid value is returned.	
Comments	The slot cache is a repository of series slot values which can be created from workspace series slots, allowing access within one run to values computed by a previous run. See “Slot Cache” in *User Interface* for details. **Note:** The slot cache is under development. For information on the current status of this feature, email: riverware-support@colorado.edu	

Syntax Example

```
SlotCacheValueByCol(%"Klamath Data.Precip", 2, @”18:00 Jan 23, 2009”)
```

Return Example

```
1.23 "m"
```

SlotWeightedAverageOverTime

Description	Computes a Series Slotvalues in that same time range as the weights.	
Type	NUMERIC	
Arguments		
1	SLOT	Slot 1: the weighting slot
2	SLOT	Slot 2: the slot being averaged
3	DATETIME	Begin timestep for period of average - partially specified
4	DATETIME	End timestep for period of average - partially specified
	DATETIME	Reference timestep
Evaluation	The partially specified begin and end timesteps are converted to fully specified timesteps using the reference timestep to complete the missing information. The weighted average is then computed as the quotient of two summations over the averaging time period (begin timestep to end timestep): $\dfrac{\sum_{Begin}^{End} \text{Slot } 1[i] \times \text{Slot } 2[i]}{\sum_{Begin}^{End} \text{Slot } 1[i]}$	
Comments	For a calendar year weighting of 1 Month timesteps, the time arguments would be @“January”, @“December”, and @“t” For a water year weighting of 1 Month timesteps, the time arguments would be @“October 31, Previous Year”, @“September 30”, @“t”. For a monthly weighting, the time arguments would be @“DayOfMonth 1”, @“Max DayOfMonth”, @“t”.	

Syntax Example

```
SlotWeightedAverageOverTime(Mead.Outflow, Mead.Salt Concentration,
  @”January”, @”December”, @”t”)
```

SolveInflow

This performs a mass balance and evaluates to the inflow of a reservoir given its outflow, previous storage, and storage at the specified timestep.

Description	The inflow to a reservoir given outflow and storage.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the reservoir object for which to calculate
2	NUMERIC	the outflow over the timestep
3	NUMERIC	the storage value to use in the mass balance calculation
4	NUMERIC	the previous (beginning) storage
5	DATETIME	the timestep at which to calculate
Evaluation	This function computes the mass balance on the given reservoir object at the given timestep, and provides it with the outflow over the timestep, beginning storage, and desired storage. The function computes the end of timestep pool elevation, and then determines the inflow, taking into account the following sources and sinks. • The Evaporation and Precipitation category selected Method. • The Bank Storage category selected Method. • The Seepage category selected Method. • Side inflows like Hydrologic Inflow, Return Flow, and Diversion are NOT included. The total inflow is then calculated as the difference between the specified storage and beginning storage over the timestep, plus the outflow, evaporation, bank storage, and seepage, minus precipitation.	
Mathematical Expression	$Total\ Inflow = \frac{storage_{previous} - storage_{ending}}{\Delta t_{timestep}} + outflow$ $+ evaporation_{flow} + bank\ storage_{flow} + seepage$ $- precipitation_{flow}$	
Comments	The given outflow is a total outflow and should include any spills. The calculated inflow is a total inflow. **Caution:** Although previous storage is passed into this function in argument 4, the value on the pool elevation and/or surface area slot must be valid at the timestep before the specified argument 5 timestep. For example, the prior slot values for Pool Elevation or Surface Area must be known for the Evaporation method to succeed.	

Syntax Example

```
SolveInflow(%"Hoover Dam", 13651 "cfs",
  19853486 "acre-feet", 19787262 "acre-feet",
  @"June 30, 1984")
```

Return Example

```
12.5 "cms"
```

• SolveMonthlyOutflowAnnualTimestep

This function solves for the monthly outflow of reservoir in a model with a 1 year timestep. It performs a mass balance and evaluates to the outflow of a reservoir given the inflow, previous month's storage, and desired storage at the specified month. Because of this timestep difference, the function makes many assumptions about the physical processes involved, see the evaluation section below. This function was developed for use in the CRSS Lite model and typically used on Powell.

<table>
<tr><td>Description</td><td colspan="2">The monthly outflow of reservoir in a model with a 1 year timestep, given inflow and storage.</td></tr>
<tr><td>Type</td><td colspan="2">NUMERIC</td></tr>
<tr><td>Arguments</td><td>Type</td><td>Meaning</td></tr>
<tr><td>1</td><td>OBJECT</td><td>reservoir object for which to calculate</td></tr>
<tr><td>2</td><td>NUMERIC</td><td>the inflow over the timestep, often the reservoir's inflow slot value</td></tr>
<tr><td>3</td><td>NUMERIC</td><td>desired storage at the end of the specified month</td></tr>
<tr><td>4</td><td>NUMERIC</td><td>previous month's storage</td></tr>
<tr><td>5</td><td>DATETIME</td><td>the monthly timestep at which to calculate</td></tr>
<tr><td>Evaluation</td><td colspan="2">This function can only be called if the run timestep is 1 Year. Also, the last argument, the DATETIME, must be a valid 1 Month timestep.
This function solves the mass balance equation on the given reservoir object at the given timestep and provides it with the inflow, beginning storage, and ending storage. The function determines the outflow over the timestep, taking into account the following sources and sinks, and thus they should not be included in the inflow value for Argument 2.
• Evaporation is modeled using the Monthly Evaporation approach
• Precipitation is not included
• Bank Storage is modeled using CRSS Bank Storage
• No Seepage is included
• No other side inflows are included
The outflow is then calculated as the difference between the ending and beginning storage over the timestep, plus the inflow, minus evaporation, and bank storage.</td></tr>
<tr><td>Mathematical Expression</td><td colspan="2">$$Outflow = \frac{storage_{previous} - storage_{ending}}{\Delta t_{timestep}} + inflow$$ $$- evaporation_{flow} - bank\ storage_{flow}$$</td></tr>
<tr><td>Comments</td><td colspan="2">The calculated outflow is a total outflow and will include any spills.
Caution: Although previous storage is passed into this function in argument 4, the value on the pool elevation and/or surface area slot must be valid at the timestep before the specified argument 5 timestep. For example, the prior slot values for Pool Elevation or Surface Area must be known for the Evaporation method to succeed.</td></tr>
</table>

Syntax Example

```
SolveMonthlyOuflowAnnualTimestep( %"Powell",
   13651 "cfs",
```

```
19853486 "acre-feet",
19787262 "acre-feet",
@"June 30, 1984" )
```

Return Example

```
12022 "cfs"
```

• SolveMonthlyStorageAnnualTimestep

This function solves for the monthly storage of reservoir in a model with a 1 year timestep. It performs a mass balance and evaluates to the storage of a reservoir given the inflow, outflow, previous month's storage, and the specified month. Because of this timestep difference, the function makes many assumptions about the physical processes involved, see the evaluation section below. This function was developed for use in the CRSS Lite model and typically used on Powell.

Description	The monthly storage of a reservoir in a model with a 1 year timestep	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	reservoir object for which to calculate
2	NUMERIC	the inflow over the timestep, often the reservoir's inflow slot value
3	NUMERIC	the outflow over the timestep
4	NUMERIC	previous month's storage
5	DATETIME	the monthly timestep at which to calculate

Evaluation	This function can only be called if the run timestep is 1 Year. Also, the last argument, the DATETIME, must be a valid 1 Month timestep. This function solves the mass balance equation on the given reservoir object at the given timestep and provides it with the inflow and outflow over the timestep, and previous month storage. The function must iterate to convergence due to the storage and pool elevation dependence of the following sources and sinks, which are included automatically: • Evaporation is modeled using the Monthly Evaporation approach • Precipitation is not included • Bank Storage is modeled using CRSS Bank Storage • No Seepage is included • No other side inflows are included At each iteration, the ending storage is calculated as the previous storage plus the inflow over the timestep, minus the outflow, evaporation, and bank storage over the timestep.
Mathematical Expression	$Storage = storage_{previous} + ((inflow)\Delta t_{timestep}) - ((outflow)\Delta t_{timestep}) - evaporation_{volume} - bank\ storage_{volume}$
Comments	The specified outflow is a total outflow and should include any spills. **Caution:** Although previous storage is passed into this function in argument 4, the value on the pool elevation and/or surface area slot must be valid at the timestep before the specified argument 5 timestep. For example, the prior slot values for Pool Elevation or Surface Area must be known for the Evaporation method to succeed.

Syntax Example

```
SolveMonthlyStorageAnnualTimestep( %"Powell",
  13651 "cfs",
  14255 "cfs",
  19,787,262 "acre-feet",
  @"July 31, 1978" )
```

Return Example

```
19,786,132 "acre-feet"
```

• SolveOutflow

This performs a mass balance and evaluates to the outflow from a reservoir given its inflow, previous storage, and the desired storage at the specified timestep.

Description	Compute the outflow from a reservoir given inflow and storage.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the reservoir object for which to calculate
2	NUMERIC	the inflow over the timestep, often the reservoir's inflow slot value
3	NUMERIC	the desired storage value to use in the mass balance calculation

4	NUMERIC	the previous (beginning) storage
5	DATETIME	the timestep at which to calculate
Evaluation	This function solves the mass balance on the given reservoir object at the given timestep and provides it with the inflow, previous (beginning) storage, and desired storage. The function computes the end of timestep pool elevation, and then determines the outflow over the timestep, taking into account the following sources and sinks, and thus they should not be included in the inflow value for Argument 2. • The Evaporation and Precipitation category selected Method. • The Bank Storage category selected Method. • The Seepage category selected Method. • Side inflows including: Inflow 2 (Slope Power Reservoir only), Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. These slots are automatically added as dependencies to the calling rule. The outflow is then calculated as the difference between the ending and beginning storage over the timestep, plus the inflow, side inflows, and precipitation, minus evaporation, bank storage, and seepage.	
Mathematical Expression	$Outflow = \frac{storage_{previous} - storage_{desired}}{\Delta t_{timestep}} + inflow + side\ inflows$ $- evaporation_{flow} - bank\ storage_{flow} - seepage + precipitation_{flow}$	
Comments	The given inflow in argument 2 represents the main inflow only and should not include any side inflows. This is the same value which would be in the Inflow slot. The calculated outflow is a total outflow. It includes both Release/Turbine Release and Spill. The given timestep's Inflow 2 (Slope Power Reservoir only), Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. are automatic dependencies of this function. Since the function evaluation depends on these slots, any change to their values at the indicated timestep, may impact the function result. **Caution:** Although previous storage is passed into this function in argument 4, the value on the pool elevation and/or surface area slot must be valid at the timestep before the specified argument 5 timestep. For example, the prior slot values for Pool Elevation or Surface Area must be known for the Evaporation method to succeed.	

Syntax Example

```
SolveOutflow( %"Hoover Dam",
  11651 "cfs",
  19853486 "acre-feet",
  19787262 "acre-feet",
  @"June, 1984" )
```

Return Example

```
21.32 "cms"
```

• SolveOutflowGivenEnergyInflow

This function evaluates to Outflow from a LevelPowerReservoir with the given Energy and Inflow at the specific timestep.

Description	The outflow from a LevelPowerReservoir.	
Type	NUMERIC	
Arguments		
1	OBJECT	the reservoir object for which to calculate (must be a LevelPowerReservoir)
2	NUMERIC	the energy value
3	NUMERIC	the inflow value
4	DATETIME	the timestep at which to calculate
Evaluation	This function behaves identically to the solution of the LevelPowerReservoir in simulation.	
Comments	This function assumes that the LevelPowerReservoir has solved for all the timesteps prior to the date specified in argument 4. This is necessary because the solution requires previous storage, inflow, and energy. This information is retrieved from slots on the object at timesteps prior to the date specified in argument 4. If any of this information is missing, an error is posted and the rule fails. If this function is called on the first timestep, the initial input data are used. These data are already required for the LevelPowerReservoir to dispatch in simulation mode. This function takes into account the following sources and sinks automatically, and thus they should not be included in the inflow value for Argument 2. • The Evaporation and Precipitation category selected Method. • The Bank Storage category selected Method. • The Seepage category selected Method. • Side inflows including: Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. These slots are automatically added as dependencies to the calling rule. **Caution:** The Pool Elevation or Storage slot must have a valid value on the timestep before the timestep passed in on Argument 4. Otherwise, an error will be issued.	

Syntax Example

```
SolveOutflowGivenEnergyInflow( %"HooverDam", HooverDam.Energy[],
   HooverDam.Inflow[], @"t" )
SolveOutflowGivenEnergyInflow( %"HooverDam", 20.0 "MWH", 1000.0 "cfs", @"t" )
```

Return Example

```
16.342 "cms"
```

• SolveShortage

Given some total available water, this method solves for the Diversion Shortage and Depletion Shortage on a Water User, or the Total Diversion Shortage and Total Depletion Shortage on an AggDiversionSite. It evaluates to a list which contains the two values.

Description	List containing the (Total) Diversion Shortage and (Total) Depletion Shortage	
Type	LIST {NUMERIC, NUMERIC}	
Arguments	**Type**	**Meaning**
1	OBJECT	the object on which to perform the calculations (either an AggDiversionSite or a Water User)
2	NUMERIC	the total water available for diversion
3	DATETIME	the timestep at which to calculate
Evaluation	This function behaves identically to the solution of the object in simulation. It mimics the dispatch method of the given object. However, instead of setting slots, the method just returns the values for (Total) Diversion Shortage and (Total) Depletion Shortage.	
Comments	This function exits its calling rule with an early termination if any of the required data used to solve the diversions are unknown. Depletion Requested is not required, if not specified it will be set equal to Diversion Requested. This function aborts the run with an error if an object other than a Water User or an AggDiversionSite is given as the first argument.	

Syntax Example

```
SolveShortage(%"San Juan Diversion", 100 "cfs", @"t")
```

Return Example

```
{1.25 "cms", 1.02 "cms"}
```

• SolveSlopeStorageGivenInflowHW

This function is used to solve a Slope Power Reservoir object when inflow and pool elevation are known. A LIST is returned which contains the resulting outflow as the first argument and the resulting storage as the second argument.

Description	List containing the resulting outflow and the resulting storage value	
Type	LIST {NUMERIC, NUMERIC}	
Arguments	**Type**	**Meaning**
1	OBJECT	the object on which to perform the calculations (must be a Slope Power Reservoir)
2	NUMERIC	the inflow value
3	NUMERIC	the pool elevation value

4	DATETIME	the timestep at which to calculate
Evaluation	This function behaves identically to the solution of the object in simulation.	
Comments	This function assumes that the Slope Power Reservoir has solved (through simulation) for all timesteps prior to the date specified in argument 4. This is necessary because the solution requires previous inflow, outflow, storage and pool elevation data. This information is retrieved from slots on the object at timesteps prior to the date specified in argument 4. If any of this information is missing, an error is posted and the rule fails. If this function is called on the first timestep, the initial input data is used. This data is already required for the Slope Power Reservoir to dispatch in simulation mode. This function takes into account the following sources and sinks automatically, and thus they should not be included in the inflow value for Argument 2. • The Evaporation and Precipitation category selected Method. • The Bank Storage category selected Method. • The Seepage category selected Method. • Side inflows including: Inflow 2, Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. These slots are automatically added as dependencies to the calling rule.	

Syntax Example

```
SolveSlopeStorageGivenInflowHW( %"FtLoudoun",
   FtLoudoun.Inflow[],
   FtLoudoun.Pool Elevation[],
   @"t" )
SolveSlopeStorageGivenInflowHW( %"FtLoudoun",
   100.0 "cfs",
   240.45 "ft",
   @"t")
```

Return Example

```
{16.342 "cms", 123348183.75 "m3"}
```

• SolveSlopeStorageGivenInflowOutflow

This function is used to solve a Slope Power Reservoir object when inflow and outflow are known. A LIST is returned which contains the resulting pool elevation as the first argument and the resulting storage as the second argument.

Description	List containing the resulting pool elevation and the resulting storage value	
Type	LIST {NUMERIC, NUMERIC}	
Arguments	**Type**	**Meaning**
1	OBJECT	the object on which to perform the calculations (must be a Slope Power Reservoir)
2	NUMERIC	the inflow value
3	NUMERIC	the outflow value

4	DATETIME	the timestep at which to calculate
Evaluation	This function behaves identically to the solution of the object in simulation.	
Comments	This function assumes that the Slope Power Reservoir has solved (through simulation) for all timesteps prior to the date specified in argument 4. This is necessary because the solution requires previous inflow, outflow, storage and pool elevation data. This information is retrieved from slots on the object at timesteps prior to the date specified in argument 4. If any of this information is missing, an error is posted and the rule fails. If this function is called on the first timestep, the initial input data is used. This data is already required for the Slope Power Reservoir to dispatch in simulation mode. This function takes into account the following sources and sinks automatically, and thus they should not be included in the inflow value for Argument 2. • The Evaporation and Precipitation category selected Method. • The Bank Storage category selected Method. • The Seepage category selected Method. • Side inflows including: Inflow 2, Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. These slots are automatically added as dependencies to the calling rule.	

Syntax Example

```
SolveSlopeStorageGivenInflowOutflow( %"FtLoudoun",
   FtLoudoun.Inflow[],
   FtLoudoun.Outflow[],
   @"t" )
SolveSlopeStorageGivenInflowOutflow( %"FtLoudoun",
   100.0 "cfs",
   110.45 "cfs",
   @"t" )
```

Return Example

```
{1253.2 "m", 123348183.75 "m3"}
```

• SolveStorage

This performs a mass balance and evaluates to the end of timestep storage of a reservoir, given its previous storage, inflow, and outflow at the specified timestep.

Description	The storage of a reservoir.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the reservoir object for which to calculate
2	NUMERIC	the specified inflow over the timestep
3	NUMERIC	the specified outflow over the timestep
4	NUMERIC	the previous (beginning) storage
5	DATETIME	the timestep at which to calculate

Evaluation	This function computes the mass balance on the given reservoir object at the given timestep and provides it with the inflow and outflow over the timestep, and beginning storage. The function must iterate to convergence due to the storage and pool elevation dependence of the following sources and sinks, which are included automatically: • The Evaporation and Precipitation Category selected Method. • The Bank Storage category selected Method. • The Seepage category selected Method. • Side inflows including: Inflow 2 (Slope Power Reservoir only), Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. These slots are automatically added as dependencies to the calling rule. At each iteration, the ending storage is calculated as the previous storage plus the inflow, side inflows, and precipitation over the timestep, minus the evaporation, bank storage, and seepage over the timestep
Mathematical Expression	$Storage = storage_{previous} + ((inflow + side\ inflows)\Delta t_{timestep}) - ((outflow)\Delta t_{timestep}) - evaporation_{volume} - bank\ storage_{volume} - ((seepage)\Delta t_{timestep}) + precipitation_{volume}$
Comments	The given inflow in Argument 2 represents the main inflow only and should not include any side inflows. This is the same value which would be in the Inflow slot. The given outflow represents the total outflow. It should include both Release/Turbine Release and Spill. The given timestep's Inflow 2 (Slope Power Reservoir only), Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage are automatic dependencies of this function. Since the function evaluation depends on these slots, any change to their values at the indicated timestep, may impact the function result. **Caution:** Although previous storage is passed into this function in argument 4, the value on the pool elevation and/or surface area slot must be valid at the timestep before the specified argument 5 timestep. For example, the prior slot values for Pool Elevation or Surface Area must be known for the Evaporation method to succeed.

Syntax Example

```
SolveStorage( %"Hoover Dam",
  11651 "cfs",
  13672 "cfs",
  19787262 "acre-feet",
  @"June, 1984" )
```

Return Example

```
 123348183.75 "m3"
```

• SolveSubbasinDiversions

This function evaluates to a list of two values. The first value, is the minimum inflow to a subbasin required to satisfy all of its diversions. The second value, is the outflow from the subbasin when this minimum flow is available.

Description	Minimum Inflow required to meet all diversion requests and resulting Outflow.	
Type	LIST {NUMERIC, NUMERIC}	
Arguments	**Type**	**Meaning**
1	LIST	the subbasin’s Reach and Confluence objects in downstream order (can be included in a subbasin)
2	DATETIME	the timestep at which to calculate

Evaluation	The subbasin diversion requirement is originally set to zero. Each of the objects in the subbasin list is processed in the downstream order in which they are provided. If the object is a Reach, the following calculations are performed: • If an Aggregate Diversion Site is linked to the Reach's Diversion slot, the NetNonShortDiversionRequirement function is executed on the diversion. This determines the diversion requirement from the Reach. • If an Aggregate Diversion Site is linked to the Reach's Diversion slot, the minimum flow just below the diversion point is determined from the Reach's Minimum Diversion Bypass slot. If this slot does not exist because of the selected User Method in the Min Diversion Bypass category, the minimum flow requirement is zero. • If nothing is linked to the Diversion slot, but a value exists in the slot, this value is assumed to be the diversion requirement for this reach. In this case, there is no minimum flow requirement below the diversion point. • The subbasin diversion requirement is recalculated as the greater of the previous subbasin diversion requirement or the Reach diversion requirement plus the minimum flow requirement plus any cumulative upstream diversions minus any cumulative upstream return flows minus any cumulative upstream tributary inflows. • Any Local Inflow to the Reach is added to the cumulative tributary inflows. • If the Return Flow slot has a valid value, it is added to the cumulative return flows. If the Return Flow slot does not have a valid value, but a Water User or an Aggregate Diversion Site object is linked to it, the return flow is estimated. Return flow is estimated by subtracting the object's (Total) Depletion Requested from its (Total) Diversion Requested. The estimated return flow is then added to the cumulative return flows. If Depletion Requested is not specified, it will be set equal to Diversion Requested. If the object is a Confluence, the Inflow1 and Inflow2 slots are checked to determine which is the main subbasin flow and which is the tributary inflow. The objects linked to the inflow slots are checked against the last Reach object to be processed. When a match is found, the other Inflow, if valid, is added to the cumulative tributary inflows. The loop continues until all objects in the list have been processed. The largest subbasin diversion requirement to have been calculated at any diversion point is the total subbasin diversion requirement.
Mathematical Expression	$Max(\forall Reach \text{ in Subbasin})$ $\Big[NetNonShortDiversionRequirement(AggDivSite)$ $+ minimum\ flow_{(Reach)} + \sum_{Upstream\ Reach} Diversion$ $- \sum_{Upstream\ Reach} Return\ Flow - \sum_{Upstream\ Reach,\ Confluence} Tributary\ Inflow$

Comments	This function exits its calling rule with an early termination if any of the required data used to solve the diversions are unknown. The required data is the same as that needed for the NetNonShortDiversionRequirement predefined function for each Aggregate Diversion Site along the subbasin. This function aborts the run with an error if an object other than a Reach or Confluence is in the subbasin list. One of the Confluence Inflows must be linked to the previous Reach object upstream, or an Aggregate Reach which contains the previous Reach object upstream as its last element. If this condition is not met, the Confluence cannot determine which slot is the tributary inflow and the function aborts the run with an error. All subbasin diversion requirement calculations are performed at the given timestep. Subbasin diversion requirement will not be correct if there are lags in Reaches. This predefined function is recommended for use in long timestep models or for subbasins where there is no lag between top and bottom.

Syntax Example

```
SolveSubbasinDiversions( ListSubbasin("AnimasBasin"), @"t" )
```

Return Example

```
{ 0.954 "cms", 0.00 "cms" }
```

• SolveTurbineRelGivenEnergyInflow

Description	This function computes the Turbine Release necessary to meet the specified Energy. If that energy cannot be met, the maximum turbine release is returned.	
Type	LIST {NUMERIC, BOOLEAN}	
Arguments		
1	OBJECT	the reservoir object for which to calculate (must be a LevelPowerReservoir)
2	NUMERIC	the energy value
3	NUMERIC	the inflow value

4	DATETIME	the timestep at which to calculate
Evaluation	This function behaves like the solution of the LevelPowerReservoir in simulation. If the given energy can be met by the turbine release, TRUE is returned in the list boolean. If the given energy cannot be met, the turbine release is calculated to be the maximum release as computed by GetMaxReleaseGivenInflow and FALSE is returned in the list boolean.The maximum turbine release may be reduced by specified Plant Power Limits and Plant Power Cap Fractions, as appropriate for the selected power method.	
Comments	This function assumes that the LevelPowerReservoir has solved for all the timesteps prior to the date specified in argument 4. This is necessary because the solution requires previous Storage. This information is retrieved from slots on the object at timesteps prior to the date specified in argument 4. If any of this information is missing, an error is posted and the rule fails. If this function is called on the first timestep, the initial input data are used. These data are already required for the LevelPowerReservoir to dispatch in simulation mode. This function takes into account the following sources and sinks automatically, and thus they should not be included in the inflow value for Argument 3. • The Evaporation and Precipitation category selected Method. • The Bank Storage category selected Method. • The Seepage category selected Method. • Side inflows including: Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. These slots are automatically added as dependencies to the calling rule. Also, if there are Unregulated Spills, the unregulated spill is limited to be no greater than the max unregulated spill. This is described in the Unregulated Spill documentation on the Level Power Reservoir. For the Plant Power Coefficient and Plant Efficiency Curve power methods, if you have an input value on the Power Coefficient slot, the result of solveTurbineRelGivenEnergyInflow is non-unique. A value will be found, but there may be multiple solutions that meet the specified energy. **Caution:** The Pool Elevation or Storage slot must have a valid value on the timestep before the timestep passed in on Argument 4. Otherwise, an error will be issued.	

Syntax Example

```
SolveTurbineRelGivenEnergyInflow( %"HooverDam",
  HooverDam.Energy[],
  HooverDam.Inflow[],
  @"t" )
SolveTurbineRelGivenEnergyInflow( %"HooverDam",
  20.0 "MWH",
  1000.0 "cfs,
  @"t" )
```

Return Example

```
{16.342 "cms", TRUE}
```

• SolveWaterRights, SolveWaterRightsWithLags

This water accounting function invokes the Water Rights Allocation method on a computational subbasin; see "Modeling Approach" in *Accounting* for details. The subbasin identifies a set of accounts for which to solve; the Water Type identifies the supply chain that models the allocatable flow of water in the subbasin. The date controls the behavior of instream flow rights during the solution.

Description	Invokes computational subbasin's Water Rights Allocation method.	
Type	SolveWaterRights: LIST { LIST { SLOT, NUMERIC } } SolveWaterRightsWithLags: LIST{ LIST{SLOT, DATETIME, NUMERIC} }	
Arguments	**Type**	**Meaning**
1	STRING	the name of the computational subbasin
2	STRING	the name of the Water Type that identifies the allocatable flow supply chain
3	DATETIME	*Controlling date* for instream flow rights. Rights at or senior to (i.e., with priority date earlier or equal to) this date can make calls; instream flow rights junior to this date compute their Available Allocatable Flow.
Evaluation	Runs the selected Water Rights Allocation method on the subbasin. For each water right account (has priority) in the subbasin, the function returns {slot, value} pairs for the following slots: • Appropriation Request on all rights, • Available Allocatable Flow on Instream Flow Accounts whose priority date is later than the controlling date (3rd argument), • Supply slot values representing appropriations to the water right accounts. For storage rights on in-line reservoirs, this is a Transfer In supply; for off-stream storage rights, this is a Diversion supply to the passthrough account on a Diversion object that supplies the off-stream storage right account. For diversion rights, this is a Diversion supply. A changing set of temporary slots (whose names begin with Temp) on the rights is also returned, for use by RiverWare developers. If no appropriation is to be made, a value of zero is returned so that old appropriations that are no longer valid will be invalidated by this rule. The SolveWaterRightsWithLags() predefined rule function works much like SolveWaterRights(), but is used when the subbasin passthrough accounts contain lags. It returns a list of {slot name, date-time, value} triplets, which the rule uses to place the value in the appropriate slot at the appropriate timestep. The timestep given will reflect the Local Timestep Offset of the account on which the slot resides. It is some number of timesteps after the current rules-controller timestep, and reflects the relationships of the account to other accounts in the subbasin based on their respective cumulative lag times to the end of the subbasin. See "Solution Algorithm for SolveWaterRights" in *Accounting* for detailed descriptions of the solution methods.	
Comments	The calling rule is expected to make the assignments of the values to the slots.	

Use Example

To cause all instream flow rights to compute their Available Allocatable Flow values:

```
FOREACH
  ( LIST pair IN SolveWaterRights("Network", "Allocatable Flow",
  @"20:00:00 January 1, 1800") ) DO
    ( pair<0> )[] = pair<1>
```

```
ENDFOREACH
```

Use Example

To cause all instream flow rights to make calls, using their already-computed Available Allocatable Flow slot values:

```
FOREACH
   (LIST pair IN SolveWaterRights( "Network", "Allocatable Flow",
   @"20:00:00 December 31, 2030")) DO
      ( pair<0> )[] = pair<1>
ENDFOREACH
```

Use Example

Or if lags are to be considered:

```
FOREACH(LIST triplet IN SolveWaterRightsWithLags("Network",
   "Allocatable Flow", @"20:00:00 December 31, 2030")) DO
      ( triplet<0> )[triplet<1>] = triplet<2>
ENDFOREACH
```

• Sort

Description	Sort the items in a list.	
Type	LIST	
Arguments	**Type**	**Meaning**
1	LIST	a list of values to be sorted
Evaluation	Returns a list with the same values as the input list, in increasing order.	
Comments	Comparisons across type are defined by the following arbitrary ordering, on which users are advised against relying: BOOLEAN < NUMERIC < STRING < OBJECT < SLOT < DATETIME < LIST Within each type, ordering is as: • BOOLEAN: TRUE < FALSE • NUMERIC: values involving different dimensions are sorted by lexicographic ordering on the names of the units; within values of the same dimensionality, the sorting is based on standard numeric comparisons. • STRING: Lexicographic ordering • OBJECT: Lexicographic ordering on the object’s name • SLOT: Lexicographic ordering on the slot’s name • DATETIME: Same as RPL operator • LIST: Based on comparison of items within the list (left to right).	

Syntax Example

```
Sort( {1.0, {res1, 10}, "hello", 0.0, "bob"} )
```

Syntax Example

```
{0.0, 1.0, "bob", "hello", {res1, 10} }
```

• SortPairsAscending, SortPairsDescending

Description	Sort a list of two-item lists.	
Type	LIST	
Arguments	**Type**	**Meaning**
1	LIST {LIST}	the list of lists to be sorted.
Evaluation	The input list must be a list of lists, each member list must contain at least two items. The pairs are sorted into ascending/descending order by the second item's value, and a list containing the first items of this sorted list of pairs is returned. Duplicates are not removed.	
Comments	In the case that the items being compared are equal, the return order for SortPairsAscending will correspond to the order of the original list passed in as an argument, and the order for SortPairsDescending will be the reverse of the original list passed in as an argument. In other words the result from SortPairsDescending will always be the reverse of the result from SortPairsAscending. Comparisons across type are defined by the following arbitrary ordering, on which users are advised against relying: BOOLEAN < NUMERIC < STRING < OBJECT < SLOT < DATETIME < LIST Within each type, ordering is as: • BOOLEAN: TRUE < FALSE • NUMERIC: values involving different dimensions are sorted by lexicographic ordering on the names of the units; within values of the same dimensionality, the sorting is based on standard numeric comparisons. • STRING: Lexicographic ordering • OBJECT: Lexicographic ordering on the object's name • SLOT: Lexicographic ordering on the slot's name • DATETIME: Same as RPL operator • LIST: Based on comparison of items within the list (left to right).	

Syntax Example

```
SortPairsAscending(
  { {"a", 10.0}, {"b", 2.0}, {"c", 5.0}, {"d", 10.0} } )
```

Return Example

```
{"b", "c", "a", "d"}
```

Syntax Example

```
SortPairsDescending(
  { {"a", 10.0}, {"b", 2.0}, {"c", 5.0}, {"d", 10.0} } )
```

Return Example

```
 {"d", "a", "c", "b"}
```

SourceAccountAndObject

Description	Given a supply (specified by name), returns a list containing the source (upstream) account and object.	
Type	LIST {STRING, OBJECT}	
Arguments	**Type**	**Meaning**
1	STRING	The name of the supply.
Evaluation		
Comments		

Syntax Example

```
SourceAccountAndObject("ResA Fish to ReachB Fish")
```

Return Example

```
{"Fish", %"ResA"}
```

Split

Description	Split a string up into component pieces.	
Type	LIST {STRING}	
Arguments	**Type**	**Meaning**
1	STRING	the primary string
2	STRING	the separator string
Evaluation	This function returns a list of the strings which are contained within the primary string and separated by the separator string. Where ambiguity exists the left most occurrence of the separator string is used.	
Comments	It is an error for the separator string to be the empty string.	

Syntax Example

```
Split("ABabcdefABcdABcdef", "AB")
```

returns:

```
{ "", "abcdef", "cd", "cdef" }
```

```
Split("ResA^MyAccount.Inflow", "^")
```

returns:

```
{ "ResA", "MyAccount.Inflow" }
```

StorageToArea

This function performs a lookup in a Reservoir object's ***Elevation Volume Table*** based on a given storage to determine the corresponding pool elevation. The function then uses this pool elevation for a lookup in the Reservoir's Elevation Area Table and evaluates to the corresponding area.

Description	Find the area of a given reservoir with a given storage.	
Type	NUMERIC	
Arguments	Type	Meaning
1	OBJECT	reservoir object
2	NUMERIC	storage
Evaluation	The storage argument is looked up in the Storage column of the Elevation Volume Table of the reservoir object argument to determine the Pool Elevation. If the exact storage is not in the table, the lookup performs a linear interpolation between the two nearest bounding storages and their corresponding pool elevations. The pool elevation is then looked up in the Pool Elevation column of the Elevation Area Table to determine the Surface Area. If the exact elevation is not in the table, another linear interpolation is performed. The function evaluates to the computed surface area.	
Mathematical Expression	$pool\ elevation = elevation_{(lesser)} + \left(\frac{elevation_{(greater)} - elevation_{(lesser)}}{storage_{(greater)} - storage_{(lesser)}} \times (storage - storage_{(lesser)})\right)$ $area = area_{(lesser)} + \left(\frac{area_{(greater)} - area_{(lesser)}}{pool\ elevation_{(greater)} - pool\ elevation_{(lesser)}} \times (pool\ elevation - pool\ elevation_{(lesser)})\right)$	
Comments	If the object is not a reservoir or the reservoir does not have an Elevation Area Table, the function aborts the run with an error (CRSSEvaporationCalc, DailyEvaporationCalc, PanAndIceEvaporation, heatBudgetEvaporation, or InputEvaporation must be selected as the Evaporation and Precipitation Category selected Method). If the reservoir is a Slope Power Reservoir, the calculation is based only on level storage and does not include any wedge storage effects. This function will issue an error if the Time Varying Elevation Volume method or Time Varying Elevation Area method is selected (see “Time Varying Elevation Volume” in *Objects and Methods* and “Time Varying Elevation Area” in *Objects and Methods*). Instead, use the StorageToAreaAtDate function, described next.	

Syntax Example

```
StorageToArea(%"WattsBar", 442.39 "1000 cfsday")
```

Return Example

```
12203.231 "m2"
```

• StorageToAreaAtDate

This function performs a lookup in the Reservoir object’s Elevation Volume Table or Elevation Volume Table Time Varying based on a given storage and datetime and computes the corresponding pool elevation. The function then uses this pool elevation for a lookup in the Reservoir’s Elevation Area Table or Elevation Area Table Time Varying and evaluates to the corresponding surface area. See “Elevation Volume Table Time Varying” in *Objects and Methods* for details.

This function must be used when the Time Varying Elevation Volume method or Time Varying Elevation Area is selected. Otherwise, the StorageToArea function can be used and no DATETIME argument is required.

Description	Find the surface area corresponding to a reservoir's storage.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	reservoir object
2	NUMERIC	storage
3	DATETIME	the datetime at which to do the conversion
Evaluation	On the specified reservoir object argument, if the Time Varying Elevation Volume method is selected the function will reference the Elevation Volume Table Time Varying table. The function will select the appropriate column to use based on the datetime argument. On timesteps that exactly match a modification date, the previous column is used. The relationship changes at the end of that timestep and is taken into account when the reservoir dispatches. For this algorithm the previous timestep's relationship is used. See "Time Varying Elevation Volume" in *Objects and Methods* and "Elevation Volume Table Time Varying" in *Objects and Methods* for details. Otherwise, the Elevation Volume Table is used. Then, the storage argument is looked up in the appropriate storage column to determine the elevation from the Pool Elevation column. If the exact elevation is not in the table, the lookup performs a linear interpolation between the two nearest bounding elevations and their corresponding surface areas.On the specified reservoir object argument, if the Time Varying Elevation Area method is selected, the function will reference the Elevation Area Table Time Varying table. The function will select the appropriate column to use based on the datetime argument. On timesteps that exactly match a modification date, the previous column is used. The relationship changes at the end of that timestep and is taken into account when the reservoir dispatches. For this algorithm the previous timestep's relationship is used. Otherwise, the Elevation Area Table is used. Then, the computed pool elevation is looked up in the Pool Elevation column to determine the surface area from the appropriate column. If the exact elevation is not in the table, the lookup performs a linear interpolation between the two nearest bounding elevations and their corresponding surface areas.	
Mathematical Expression	$pool\ elevation = elevation_{(lesser)} + \left(\frac{elevation_{(greater)} - elevation_{(lesser)}}{storage_{(greater)} - storage_{(lesser)}} \times (storage - storage_{(lesser)})\right)$ $area = area_{(lesser)} + \left(\frac{area_{(greater)} - area_{(lesser)}}{pool\ elevation_{(greater)} - pool\ elevation_{(lesser)}} \times (pool\ elevation - pool\ elevation_{(lesser)})\right)$	
Comments	If the object is not a reservoir, or the reservoir does not have an Elevation Volume Table or Elevation Volume Table Time Varying AND Elevation Area Table or Elevation Area Table Time Varying, the function aborts the run with an error.	

Syntax Example

```
StorageToAreaAtDate(%"Lake Mead", 10520217087.2 "m3", @”t”)
```

Return Example

```
634547087.2 [m2]
```

• StorageToElevation

This function performs a lookup in a Reservoir object's Elevation Volume Table based on a given storage and evaluates to the corresponding pool elevation.

Description	Find the reservoir elevation at a given storage.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	reservoir object
2	NUMERIC	storage
Evaluation	The storage argument is looked up in the Storage column of the Elevation Volume Table of the reservoir object argument to determine the Pool Elevation. If the exact storage is not in the table, the lookup performs a linear interpolation between the two nearest bounding storages and their corresponding elevation values.	
Mathematical Expression	$pool\ elevation = elevation_{(lesser)} + \frac{elevation_{(greater)} - elevation_{(lesser)}}{storage_{(greater)} - storage_{(lesser)}} \times (storage - storage_{(lesser)})$	
Comments	If the object is not a reservoir, the function aborts the run with an error. If the reservoir is a Slope Power Reservoir, the calculation is based only on level storage and does not include any wedge storage effects. This function will issue an error if the Time Varying Elevation Volume method is selected (see “Time Varying Elevation Volume” in *Objects and Methods*). Instead, use the StorageToElevationAtDate function described next.	

Syntax Example

```
StorageToElevation(%"WattsBar", 442.39 "1000 cfsday")
```

Return Example

```
1792.25 "m"
```

• StorageToElevationAtDate

This function performs a lookup in the Reservoir object's Elevation Volume Table or Elevation Volume Table Time Varying based on a given elevation and datetime and evaluates to the corresponding volume. This function must be used when the Time Varying Elevation Volume method is selected. Otherwise, the StorageToElevation function can be used and no DATETIME argument is required. See “Elevation Volume Table Time Varying” in *Objects and Methods* and “Time Varying Elevation Volume” in *Objects and Methods* for details.

Description	Finds the elevation corresponding to a reservoir's storage.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	reservoir object
2	NUMERIC	storage
3	DATETIME	the datetime at which to do the conversion
Evaluation	On the specified reservoir object argument, if the Time Varying Elevation Volume method is selected, the function will reference the Elevation Volume Table Time Varying table. The function will select the appropriate column to use based on the datetime argument. On timesteps that exactly match a modification date, the previous column is used. The relationship changes at the end of that timestep and is taken into account when the reservoir dispatches. For this algorithm, the previous timestep's relationship is used. See "Time Varying Elevation Volume" in *Objects and Methods* and "Elevation Volume Table Time Varying" in *Objects and Methods* for details. Otherwise, the Elevation Volume Table is used and the datetime is not used. Then, the storage argument is looked up in the appropriate Storage column to determine the Pool Elevation. If the exact elevation is not in the table, the lookup performs a linear interpolation between the two nearest bounding elevations and their corresponding storages.	
Mathematical Expression	$pool\ elevation = elevation_{(lesser)} + \frac{elevation_{(greater)} - elevation_{(lesser)}}{storage_{(greater)} - storage_{(lesser)}} \times (storage - storage_{(lesser)})$	
Comments	If the object is not a reservoir, or the reservoir does not have an Elevation Volume Table or Elevation Volume Table Time Varying, the function aborts the run with an error.	

Syntax Example

```
StorageToElevationAtDate(%"Lake Mead", 634547087.2 "m3", @"t")
```

Return Example

```
1210.03 "ft"
```

• StringToTextSlotNumeric

Description	Converts a string into a numeric that can be set on a Text Series Slot	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	STRING	A string to encode.
Evaluation	Text Series Slots store text encoded as a number. See "Text Series Slots" in *User Interface*. This function takes a STRING and returns a numeric encoding that can be set on the Text Series Slot.	
Comments	See "TextSlotNumericToString" and "GetTextSlotValueAsString" for complimentary functions. Each numeric returned by this function is unique. Thus, StringToTextSlotNumeric("Apples") is not equivalent to StringToTextSlotNumeric("Apples") because each instance of "Apples" gets encoded to a different number. Instead, compare the strings directly.	

Syntax Example

```
Reservoir.OutflowReason[ ] = StringToTextSlotNumeric("Rafting Operations for Event")
```

In this rule assignment example, the slot Reservoir.OutflowReason would be set to the string "Rafting Operation for Event". Internally, the function returns an encoded index (numeric), which can be set on the slot.

Sum

This function sums a list of numbers.

Description	Sum a non-empty list of numbers.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST{NUMERIC}	a list of numeric values.
Evaluation	The numbers in the input list are added up, the total is returned.	
Mathematical Expression	$\text{total} = \sum_{x \in \text{input list}} x$	
Comments	If the input list is empty, one of the items in the list is not NUMERIC, or the unit types of items in the list are incompatible, this function aborts the run with an error.	

Syntax Example

```
Sum({1.0 "cfs", 2.0 "cms"})
```

Return Example

```
71.629333443 "cfs"
```

SumAccountSlotsByWaterType

This function sums the values of all accounting slots of a given name on accounts of a given water type.

Description	The sum of slots of a given name and water type.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the object on which to sum
2	STRING	the water type of accounts to sum
3	STRING	the name of the slots to sum
4	DATETIME	the date at which to sum

Evaluation	The function contains two nested loops. The outer loop iterates over all of the account types which may exist on the given object (Storage Account and/or Passthrough Account or Diversion Account). For each account type, a list is made of all of the accounts which are of the given water type. The inner loop iterates over all of these accounts and sums the values of the slots with the given name at the given time.
Mathematical Expression	$total = \sum_{(account\ type,(account = water\ type))} object.slotname_{timestep}$
Comments	If the object cannot accept accounts, has no accounts, or has no accounts of the given water type, this function aborts the run with an error. If none of the accounts of the given water type has a slot with the given name, or the slot is not a series slot, this function aborts the run with an error. Any slots which contain a NaN at the given datetime are ignored for the purpose of summation. If all of the slots contain NaNs at the given datetime, the function forces an early termination of the calling rule. If the given datetime argument does not land on an interval of the series slot, this function aborts the run with an error.

Syntax Example

```
SumAccountSlotsByWaterType( %"Heron Reservoir", "RioGrande", "storage", @"t" )
```

Return Example

```
71629333.443 "m3"
```

• SumByIndex

This function sums numbers at a given location within lists contained in a list.

Description	Given a list of lists and an index, sum the values at the given index in each list.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST{LIST}	a list of lists.
2	NUMERIC	an index.
Evaluation	All values located at the given index in each list contained within the input list are summed, the total is returned.	
Mathematical Expression	$\text{total} = \sum_{x \in \text{input list}} x_{\text{index}}$	
Comments	The input list must be non-empty. The index must be positive and a legal index for each of the lists contained within the values list. For example, if the index value is 3, the sublists must each contain at least 4 items. All items being summed must be numeric and have compatible dimensions. If any of these conditions is not met, this function aborts the run with an error.	

Syntax Example

```
SumByIndex( {{true, 2.0 “cms”}, {false, 1.0 “cms”}}, 1.0 )
```

Return Example

```
3.0 "cms"
```

• SumFlowsToVolume, SumFlowsToVolumeSkipNaN

This function sums a series slot’s FLOW values between a starting timestep and ending timestep and evaluates to the corresponding volume of water.

Description	The volume equivalent of flows over time.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the series or periodic slot to sum
2	DATETIME	the start date
3	DATETIME	the end date
Evaluation	The function loops through all of the slot values between the start and end datetime arguments. For each value, the flow is multiplied by the corresponding timestep’s length to convert it to a volume before adding it to the previous result. The function evaluates to the final result in units of m^3.	
Mathematical Expression	$volume = \sum_{start\ datetime - end\ datetime} (flow_{(slot)} \times \Delta t_{timestep})$	
Comments	If the unit type of the slot argument is not FLOW or the starting or ending datetime argument is not defined in the series slot, this function aborts the run with an error. For the SumFlowsToVolume function, if one of the slot values in the time range is a NaN, the function forces an early termination of the calling rule. The SkipNaN variation treats an invalid value (NaN) as 0.0. For periodic slots, the dates used are those within the range and falling on a run timestep and the column used is the first column.	

Syntax Example

```
SumFlowsToVolume(Crystal.Inflow, @"January, 1999", @"September, 1999")
```

Return Example

```
12.3023 "m3"
```

• SumFlowsToVolumeByCol, SumFlowsToVolumeByColSkipNaN

Description	This function sums a a column of a slot’s FLOW values between a starting timestep and ending timestep and evaluates to the corresponding volume of water	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the Agg Series Slot or periodic slot to sum

2	DATETIME	the start date
3	DATETIME	the end date
4	NUMERIC	the column (interpreted as a 0-based integral index)
Evaluation	The function loops through all of the slot values of the given column between the start and end datetime arguments. For each value, the flow is multiplied by the corresponding timestep's length to convert it to a volume before adding it to the previous result. The function evaluates to the final result in units of m^3.	
Mathematical Expression	$volume = \sum_{start\ datetime - end\ datetime} (flow_{(slot)} \times \Delta t_{timestep})$	
Comments	If the unit type of the given column of the slot is not FLOW or if the slot is an Agg Series Slot and the starting or ending datetime argument is not defined in the slot, this function aborts the run with an error. For the SumFlowsToVolumeByCol function, if one of the slot values in the time range is a NaN, the function forces an early termination of the calling rule. The SkipNaN variation treats an invalid value (NaN) as 0.0. For periodic slots, the dates used are those within the range and falling on a run timestep.	

Syntax Example

```
SumFlowsToVolumeByCol(Data.Coeff, @"January, 1999", @"September, 1999", 1)
```

Return Example

```
12.231 "m3"
```

• SumObjectsAggregatedOverTime

This function returns a single numeric value obtained by summing several objects' aggregated slot values. The objects' slot values may be aggregated as a SUM, AVG, MIN, or MAX over a specified time range.

Description	Sum several object's values, each of which is the result of aggregating a slot's values over time.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	STRING	subbasin name
2	STRING	slot name
3	STRING	aggregation function (SUM, AVG, MIN, or MAX)
4	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
5	BOOLEAN	time conversion option (TRUE or FALSE)
6	DATETIME	start date
7	DATETIME	end date

Evaluation	A list of slots is generated by searching all of the objects in the subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, each slot's values are aggregated according to the aggregation function argument over the time range of the datetime arguments. During each of these slot aggregations, any values which do not satisfy the aggregation filter argument are ignored. Finally, all of the object's aggregated slot values are summed
Mathematical Expression	$\sum_{obj \text{ in Subbasin}} (\forall_{(t \text{ from } start \text{ to } end)}[AggFunction_{(obj)}(obj.slotname)])$
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, RiverWare aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.

Syntax Example

```
SumObjectsAggregatedOverTime( "upper basin",
  "Inflow", "MAX", "ALL", TRUE
  @"October, Previous Year",
  @"September, Current Year" )
```

Return Example

```
234.3 "cms"
```

• SumObjectsAtEachTimestep

This function evaluates to a list. Each item of the list is a list comprised of the datetime at which the summation was performed and the value of the sum.

Description	Sum several object's slot values, for each timestep in a range.	
Type	LIST{LIST{DATETIME, NUMERIC}}	
Arguments	**Type**	**Meaning**
1	STRING	subbasin name
2	STRING	slot name
3	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
4	BOOLEAN	time conversion option (TRUE or FALSE)
5	DATETIME	start date
6	DATETIME	end date

Evaluation	A list of slots is generated by searching all of the objects in the subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values to be summed are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, all of the object's slot values are summed, yielding one value for each timestep in the time range of the datetime arguments. The function returns a list of two items, where the first and second items of the inner lists are the datetime and the summation value, respectively.
Mathematical Expression	$\forall_{(t \text{ from } start \text{ to } end)} \left[\{t, \sum_{obj \text{ in Subbasin}} obj.slotname\} \right]$
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, RiverWare aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.

Syntax Example

```
SumObjectsAtEachTimestep("upper basin",
   "Storage", "ALL", FALSE
   @"October, Previous Year",
   @"November, Previous Year")
```

Return Example

```
{ {@"October 31, 2003", 32303.2"m3"},
   {@"November 30, 2003", 43232.2"m3"} }
```

SumSlot, SumSlotSkipNaN

This function sums a series slot's values between a starting timestep and ending timestep.

Description	The sum of a slot's values over time.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the series or periodic slot to sum
2	DATETIME	the start date
3	DATETIME	the end date

Evaluation	The function loops through all of the slot values between the start and end datetime arguments. Each value is added to the previous result.
Mathematical Expression	$volume = \sum_{start\ datetime - end\ datetime} slot\ value_t$
Comments	If the starting or ending datetime argument is not defined in the series slot, this function aborts the run with an error. For the SumSlot function, if one of the slot values in the time range is a NaN, the function forces an early termination of the calling rule. The SkipNaN variation treats an invalid value (NaN) as 0.0. For periodic slots, the dates used are those in the first column within the range and falling on a run timestep.

Syntax Example

```
SumSlot(Crystal.Inflow, @"January 1, 1999", @"September 30, 1999")
```

Return Example

```
32.47 "cms"
```

• SumSlotByCol, SumSlotByColSkipNaN

Description	The sum the values in a column of a slot over a range of time.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the Agg Series Slot or periodic slot to sum
2	DATETIME	the start date
3	DATETIME	the end date
4	NUMERIC	the column (interpreted as a 0-based integral index)
Evaluation	The function loops through all of the slot values in the given column between the start and end datetime arguments. Each value is added to the previous result.	
Mathematical Expression	$volume = \sum_{start\ datetime - end\ datetime} slot\ value_t$	
Comments	If the slot is an Agg Series Slot and the starting or ending datetime argument is not defined in the slot, this function aborts the run with an error. For the SumSlotByCol function, if one of the slot values in the time range is a NaN, the function forces an early termination of the calling rule. The SkipNaN variation treats an invalid value (NaN) as 0.0. For periodic slots, the dates used are those within the range and falling on a run timestep.	

Syntax Example

```
SumSlotByCol(Data.Coeff, @"January 1, 1999", @"September 30, 1999", 2)
```

Return Example

```
25.323
```

• SumTableColumn

This function evaluates to the sum of a table slot's values, in the given column, from the given start row to the given end row.

Description	Sum of the values of a table slot column between two rows.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the table slot whose values to sum
2	NUMERIC	column
3	NUMERIC	start row
4	NUMERIC	end row
Evaluation	The function loops over each value in the given column of the given table slot, beginning with the start row and ending with the end row (inclusive). Each value is added to the previous sum. The function evaluates to this sum. Rows and columns are numbered beginning with zero.	
Mathematical Expression	$NUMERIC = \sum_{start\ row-end\ row} slot\ value_{(row,\ column)}$	
Comments	Units are not required for row and column indices and, if provided, will be ignored. If the column, start row, or end row do not exist in the slot or if the start row is greater than the end row, this function aborts the run with an error. If one of the slot values within the desired time range is a NaN, the function exits the rule with an early termination.	

Syntax Example

```
SumTableColumn(Chickamauga Data.Flow, 0, 0, 1)
```

Return Example

```
13.95
```

• SumTableRow

This function evaluates to the sum of a table slot's values, in the given row, from the given start column to the given end column.

Description	Sum of the values of a table slot row between two columns.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	the table slot whose values to sum
2	NUMERIC	row
3	NUMERIC	start column
4	NUMERIC	end column

Evaluation	The function loops over each value in the given row of the given table slot, beginning with the start column and ending with the end column (inclusive). Each value is added to the previous sum. The function evaluates to this sum. Rows and columns are numbered beginning with zero.
Mathematical Expression	$NUMERIC = \sum_{start\ column-end\ column} slot\ value_{(row,\ column)}$
Comments	Units are not required for row and column indices and, if provided, will be ignored. If the row, start column, or end column do not exist in the slot or if the start column is greater than the end column, this function aborts the run with an error. If one of the slot values within the desired time range is a NaN, the function exits the rule with an early termination.

Syntax Example

```
SumTableRow(Chickamauga Data.units, 0, 0, 1)
```

Return Example

```
13.95
```

• SumTimestepsAggregatedOverObjects

This function evaluates to a single numeric value, which is the sum over time of values resulting from aggregating several objects slot values at each timestep.

Description	Sum of a timeseries of values, each of which is the result of aggregating several objects' slot values.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation function (SUM, AVG, MIN, or MAX)
4	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
5	BOOLEAN	time conversion option (TRUE or FALSE)
6	DATETIME	start datetime
7	DATETIME	end datetime

Evaluation	A list of slots is generated by searching all of the objects in the subbasin argument for slots which match the slot name argument. If the time conversion option argument is TRUE, and the values to be aggregated are of the FLOW unit type, the values are multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME. Next, all of the objects' slot values are aggregated according to the aggregation function argument for each timestep in the time range of the datetime arguments. During each of these slot aggregations, any values which do not satisfy the aggregation filter argument are ignored. Finally, the timeseries of object aggregated slot values are summed.
Mathematical Expression	$\sum_{t \text{ from } start \text{ to } end} \forall_{(obj \text{ in Subbasin})}[AggFunction_{(t)}(obj.slotname)]$
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, RiverWare aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, the SUM aggregation function yields an aggregated value of 0.0 for that slot, while the AVG, MIN, and MAX aggregation functions abort RiverWare with an error.

Syntax Example

```
SumTimestepsAggregatedOverObjects("upper basin",
  "Inflow", "SUM", "ALL", FALSE,
  @"October, Previous Year",
  @"September, Current Year")
```

Return Example

```
133.43 "cms"
```

• SumTimestepsForEachObject

This function evaluates to a list. Each item of the list is a list comprised of the object name and the sum of the slot values on that object for the time range specified.

Description	Sum a slot's values over a time range, for each object in a subbasin.	
Type	LIST {LIST {OBJECT, NUMERIC}}	
Arguments	**Type**	**Meaning**
1	STRING	Subbasin name
2	STRING	slot name
3	STRING	aggregation filter (INPUT, OUTPUT, or ALL)
4	BOOLEAN	time conversion option (TRUE or FALSE)
5	DATETIME	start datetime
6	DATETIME	end datetime

Evaluation	A list of slots is generated by searching all of the objects in the Subbasin argument for slots which match the slot name argument. For each object, the slot’s values over every timestep in the range of the datetime arguments are summed. Any values which do not satisfy the aggregation filter argument are ignored during the calculation. If the time conversion option argument is TRUE, and the values to be summed are of the FLOW unit type, the values are first multiplied by their corresponding timestep length to convert them to values of the unit type VOLUME.
Mathematical Expression	$\forall_{(obj \text{ in Subbasin})} \left[\{obj, \sum_{(t \text{ from } start \text{ to } end)} obj.slotname\} \right]$
Comments	If the time conversion option argument is TRUE, but the unit of the slot values is not FLOW, this function aborts the run with an error. If none of the values for a slot satisfy the aggregation filter argument, this function also aborts RiverWare with an error.

Syntax Example

```
SumTimestepsForEachObject("upper basin",
  "Inflow", "ALL", TRUE,
  @"October, Previous Year",
  @"September, Current Year")
```

Return Example

```
{ {%"Res1", 12.23"cms"},
  {%"Reach2", 4.92 "cms"},
  {%"Res2", 23.2 "cms"} }
```

SupplyAttributes

Description	Given a supply (specified by name), returns a list containing the supply’s attributes, i.e., the supply’s release type and destination.	
Type	LIST {STRING, STRING}	
Arguments	**Type**	**Meaning**
1	STRING	The name of the supply.
Evaluation		
Comments		

Syntax Example

```
SupplyAttributes("ResA One to ResB Two")
```

Return Example

```
{"IrrigationWater", "FarmerB"}
```

• SupplyNamesFrom, SupplyNamesFrom1to1

Description	This function returns a list of names of Supplies which represent outflows from given Accounts and which have the indicated ReleaseType and Destination.	
Type	LIST {STRING}	
Arguments	Type	
1	LIST { LIST { OBJECT, STRING } }	Source List: A List of pairs (represented as Lists) containing an Object and an Account Name of an Account on that Object.
2	STRING	ReleaseType name or NONE or ALL
3	STRING	Destination name or NONE or ALL
Evaluation	A temporary list of Accounts is created from the Source List. For each of those Accounts, we examine the outflow Supplies that meet the following requirements: • Link an Account on a different downstream Object, and • Have the indicated ReleaseType, and • Have the indicated Destination In the case of SupplyNamesFrom, for each of these Accounts being considered, the names of all related Supplies matching the criteria are added to the returned List. In the case of SupplyNamesFrom1to1, there should be zero or one matching Supplies: • If there are no Supplies matching the criteria, then an empty string (" ") is added to the returned List, or • If there is exactly ONE Supply matching the criteria, then the name of that Supply is added to the returned List, or • If there is more than one Supply matching the criteria, then an error is generated. In this way, the list returned by SupplyNamesFrom is guaranteed to contain exactly one string for each Account in the Source List. If the ReleaseType argument or the Destination argument is NONE, then only Supplies having the default (unassigned) attribute of that type are considered. If the ReleaseType or the Destination argument is ALL, then that Supply attribute is ignored.	

Syntax Example

```
SupplyNamesFrom( {{%"ResA","One"}, {%"ResB","One"}},
   "Account Fill", "Abiquiu" )
```

Return Example

- SupplyNamesFrom:
  `{"ResA One to ResB One", "ResB One to ResB Three"}`
- SupplyNamesFrom1to1:
  `{"ResA One to ResB One"}`

• SupplyNamesFromIntra, SupplyNamesFromIntra1to1

Description	This function returns a list of names of Supplies which represent internal flows (i.e. Transfer supplies) from given Accounts and which have the indicated ReleaseType and Destination.	
Type	LIST {STRING}	
Arguments		
1	LIST { LIST { OBJECT, STRING } }	Source List: A List of pairs (represented as Lists) containing an Object and an Account Name of an Account on that Object.
2	STRING	ReleaseType name or NONE or ALL
3	STRING	Destination name or NONE or ALL
Evaluation	A temporary list of Accounts is created from the Source List. For each of those Accounts, we examine the outflow Supplies that meet the following requirements: • Link an Account on the SAME Object, and • Have the indicated ReleaseType, and • Have the indicated Destination In the case of SupplyNamesIntra, for each of these Accounts being considered, the names of all related Supplies matching the criteria are added to the returned List. In the case of SupplyNamesIntra1to1, there should be zero or one matching Supplies: • If there are no Supplies matching the criteria, then an empty string (“ “) is added to the returned List, or • If there is exactly ONE Supply matching the criteria, then the name of that Supply is added to the returned List, or • If there is more than one Supply matching the criteria, then an error is generated. In this way, the list returned by SupplyNamesIntra is guaranteed to contain exactly one string for each Account specified in the Source List. If the ReleaseType argument or the Destination argument is NONE, then only Supplies having the default (unassigned) attribute of that type are considered. If the ReleaseType or the Destination argument is ALL, then that Supply attribute is ignored.	
Comments	ReleaseTypes and Destinations are properties of Supplies.	

Syntax Example

```
SupplyNamesFromIntra( {{%"ResA","One"},
   {%"ResA","Two"}},
   "Account Fill", "Abiquiu" )
```

Return Example

- SupplyNamesFromIntra:
  `{"ResA One to ResA Two", "ResA Two to ResA Three"}`
- SupplyNamesFromIntra1to1:
  `{"ResA One to ResA Two"}`

• SupplyNamesTo, SupplyNamesTo1to1

Description	This function returns a list of names of Supplies which represent inflows to given Accounts and which have the indicated ReleaseType and Destination.	
Type	LIST {STRING}	
Arguments	**Type**	**Meaning**
1	LIST { LIST { OBJECT, STRING } }	Source List: A List of pairs (represented as Lists) containing an Object and an Account Name of an Account on that Object.
2	STRING	ReleaseType name or NONE or ALL
3	STRING	Destination name or NONE or ALL
Evaluation	A temporary list of Accounts is created from the Source List. For each of those Accounts, we examine the inflow Supplies that meet the following requirements: • Link an Account on a different upstream Object, and • Have the indicated ReleaseType, and • Have the indicated Destination In the case of SupplyNamesTo, for each of these Accounts being considered, the names of all related Supplies matching the criteria are added to the returned List. In the case of SupplyNamesTo1to1, there should be zero or one matching Supplies: • If there are no Supplies matching the criteria, then an empty string (“ “) is added to the returned List, or • If there is exactly ONE Supply matching the criteria, then the name of that Supply is added to the returned List, or • If there is more than one Supply matching the criteria, then an error is generated. In this way, the list returned by SupplyNamesTo is guaranteed to contain exactly one string for each Account in the Source List. If the ReleaseType argument or the Destination argument is NONE, then only Supplies having the default (unassigned) attribute of that type are considered. If the ReleaseType argument or the Destination argument is ALL, then that Supply attribute is ignored.	
Comments		

Syntax Example

```
SupplyNamesTo( {{%"ResA","One"}, {%"ResB","Two"}},
  "Account Fill", "Abiquiu" )
```

Return Example

- SupplyNamesTo:
  `{"ReachA One to ResA One", "Reach A Two to ResB Two"}`

SupplyNamesToIntra, SupplyNamesToIntra1to1

Description	This function returns a list of names of Supplies which represent internal flows (i.e. Transfer supplies) to given Accounts and which have the indicated ReleaseType and Destination.	
Type	LIST {STRING}	
Arguments	**Type**	**Meaning**
1	LIST { LIST { OBJECT, STRING } }	Source List: A List of pairs (represented as Lists) containing an Object and an Account Name of an Account on that Object.
2	STRING	ReleaseType name or NONE or ALL
3	STRING	Destination name or NONE or ALL
Evaluation	A temporary list of Accounts is created from the Source List. For each of those Accounts, we examine the inflow Supplies meet the following requirements: • Link an Account on the SAME Object, and • Have the indicated ReleaseType, and • Have the indicated Destination In the case of SupplyNamesToIntra, for each of these Accounts being considered, the names of all related Supplies matching the criteria are added to the returned List. In the case of SupplyNamesToIntra1to1, there should be zero or one matching Supplies: • If there are no Supplies matching the criteria, then an empty string (“ “) is added to the returned List, or • If there is exactly ONE Supply matching the criteria, then the name of that Supply is added to the returned List, or • If there is more than one Supply matching the criteria, then an error is generated. In this way, the list returned by SupplyNamesToIntra is guaranteed to contain exactly one string for each Account specified in the Source List. If the ReleaseType argument or the Destination argument is NONE, then only Supplies having the default (unassigned) attribute of that type are considered. If the ReleaseType or the Destination argument is ALL, then that Supply attribute is ignored.	
Comments	ReleaseTypes and Destinations are properties of Supplies.	

Syntax Example

```
SupplyNamesToIntra( {{%"ResA","One"}, {%"ResA","Two"}},
  "Account Fill", "Abiquiu" )
```

Return Example

- SupplyNamesToIntra:
  `{"ResA One to ResA Two", "ResA Three to ResA Two"}`

• SupplySlotsFrom, SupplySlotsFrom1to1

Description	This function returns a list of Supply slots of Supplies which represent outflows from given Accounts and which have the indicated ReleaseType and Destination.	
Type	LIST {SLOT}	
Arguments	**Type**	**Meaning**
1	LIST { LIST { OBJECT, STRING } }	Source List: A List of pairs (represented as Lists) containing an Object and an Account Name of an Account on that Object.
2	STRING	ReleaseType name or NONE or ALL
3	STRING	Destination name or NONE or ALL
Evaluation	A temporary list of Accounts is created from the Source List. For each of those Accounts, we examine the outflow Supplies that meet the following requirements: • Link an Account on a different downstream Object, and • Have the indicated ReleaseType, and • Have the indicated Destination In the case of SupplySlotsFrom, for each of these Accounts being considered, the Supply slots of all related Supplies matching the criteria are added to the returned List. In the case of SupplySlotsFrom1to1, there should be one matching Supply: • If there are no Supplies matching the criteria, or more than one, then an error is generated. • If there is exactly ONE Supply matching the criteria, then the Supply slot of that Supply is added to the returned List. In this way, the list returned by SupplySlotsFrom1to1 is guaranteed to contain exactly one slot for each Account in the Source List. If the ReleaseType argument or the Destination argument is NONE, then only Supplies having the default (unassigned) attribute of that type are considered. If the ReleaseType argument or the Destination argument is ALL, then that Supply attribute is ignored.	

Syntax Example

```
SupplySlotsFrom( {{%"ResA","One"}, {%"ResB","Two"}},
  "Account Fill", "Abiquiu" )
```

Return Example

- SupplySlotsFrom:
  `{$"ResA One to ResB One.Supply", $"ResA One to ResB Two.Supply"}`
- SupplySlotsFrom1to1:
  `{$"ResA One to ResB One.Supply"}`

• SupplySlotsFromIntra, SupplySlotsFromIntra1to1

Description	This function returns a list of Supply slots of Supplies which represent internal flows (i.e. Transfer supplies) from given Accounts and which have the indicated ReleaseType and Destination.	
Type	LIST {SLOT}	
Arguments	**Type**	**Meaning**
1	LIST { LIST { OBJECT, STRING } }	Source List: A List of pairs (represented as Lists) containing an Object and an Account Name of an Account on that Object.
2	STRING	ReleaseType name or NONE or ALL
3	STRING	Destination name or NONE or ALL
Evaluation	A temporary list of Accounts is created from the Source List. For each of those Accounts, we examine the outflow Supplies that meet the following requirements: • Link an Account on the SAME Object, and • Have the indicated ReleaseType, and • Have the indicated Destination In the case of SupplySlotsIntra, for each of these Accounts being considered, the Supply slots of all related Supplies matching the criteria are added to the returned List. In the case of SupplySlotsIntra1to1, there should be one matching Supply: • If there are no Supplies matching the criteria, or more than one, then an error is posted. • If there is exactly ONE Supply matching the criteria, then the Supply slot of that Supply is added to the returned List. In this way, the list returned by SupplySlotsIntra1to1 is guaranteed to contain exactly one slot for each Account in the input list. If the ReleaseType argument or the Destination argument is NONE, then only Supplies having the default (unassigned) attribute of that type are considered. If the ReleaseType argument or the Destination argument is ALL, then that Supply attribute is ignored.	
Comments		

Syntax Example

```
SupplySlotsFromIntra( {{%"ResA","One"},{%"ResA","Two"}},
  "Account Fill", "Abiquiu" )
```

Return Example

- SupplySlotsFromIntra:
  ```
  {$"ResA One to ResA Two.Supply", $"ResA Two to ResA Three.Supply"}
  ```
- SupplySlotsFromIntra1to1:
  ```
  {$"ResA One to ResA Two.Supply"}
  ```

SupplySlotsTo, SupplySlotsTo1to1

Description	This function returns a list of Supply slots of Supplies which represent inflows to given Accounts and which have the indicated ReleaseType and Destination.	
Type	LIST {SLOT}	
Arguments	**Type**	**Meaning**
1	LIST { LIST { OBJECT, STRING } }	Source List: A List of pairs (represented as Lists) containing an Object and an Account Name of an Account on that Object.
2	STRING	ReleaseType name or NONE or ALL
3	STRING	Destination name or NONE or ALL
Evaluation	A temporary list of Accounts is created from the Source List. For each of those Accounts, we examine the inflow Supplies meet the following requirements: • Link an Account on a different upstream Object, and • Have the indicated ReleaseType, and • Have the indicated Destination In the case of SupplySlotsTo, for each of these Accounts being considered, the Supply slots of all related Supplies matching the criteria are added to the returned List. In the case of SupplySlotsTo1to1, there should be one matching Supply: • If there are no Supplies matching the criteria, or more than one, then an error is generated. • If there is exactly ONE Supply matching the criteria, then the Supply slot of that Supply is added to the returned List. In this way, the list returned by SupplySlotsTo1to1 is guaranteed to contain exactly one slot for each Account in the Source List. If the ReleaseType argument or the Destination argument is NONE, then only Supplies having the default (unassigned) attribute of that type are considered. If the ReleaseType argument or the Destination argument is ALL, then that Supply attribute is ignored.	
Comments		

Syntax Example

```
SupplySlotsTo( {{%"ResA","One"}, {%"ResB","Two"}},
   "Account Fill", "Abiquiu" )
```

Return Example

```
{$"ReachA One to ResA One.Supply", $"Reach A Two to ResB Two.Supply"}
```

SupplySlotsToIntra, SupplySlotsToIntra1to1

Description	This function returns a list of Supply slots of Supplies which represent internal flows (i.e. Transfer supplies) to given Accounts and which have the indicated ReleaseType and Destination.	
Type	LIST {SLOT}	
Arguments	**Type**	**Meaning**

1	LIST { LIST { OBJECT, STRING } }	Source List: A List of pairs (represented as Lists) containing an Object and an Account Name of an Account on that Object.
2	STRING	ReleaseType name or NONE or ALL
3	STRING	Destination name or NONE or ALL
Evaluation	A temporary list of Accounts is created from the Source List. For each of those Accounts, we examine the inflow Supplies that meet the following requirements: • Link an Account on the SAME Object, and • Have the indicated ReleaseType, and • Have the indicated Destination In the case of SupplySlotsToIntra, for each of these Accounts being considered, the Supply slots of all related Supplies matching the criteria are added to the returned List. In the case of SupplySlotsToIntra1to1, there should be one matching Supply: • If there are no Supplies matching the criteria, or more than one, then an error is generated. • If there is exactly ONE Supply matching the criteria, then the Supply slot of that Supply is added to the returned List. In this way, the list returned by SupplySlotsToIntra1to1 is guaranteed to contain exactly one slot for each Account in the input list. If the ReleaseType argument or the Destination argument is NONE, then only Supplies having the default (unassigned) attribute of that type are considered. If the ReleaseType argument or the Destination argument is ALL, then that Supply attribute is ignored.	

Syntax Example

```
SupplySlotsToIntra( {{%"ResA","One"}, {%"ResA","Two"}},
   "Account Fill", "Abiquiu" )
```

Return Example

```
{$"ResA One to ResA Two.Supply", $"ResA Three to ResA Two.Supply"}
```

• TableInterpolation

This function performs a lookup in a given table slot, based on a given value in a given column, and evaluates to the corresponding value in the other given column.

Description	Table lookup with linear interpolation.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	table slot in which to do lookup
2	NUMERIC	"from" column
3	NUMERIC	"to" column
4	NUMERIC	value in "from" column
5	DATETIME	datetime context for unit conversions

Evaluation	The value argument is looked up in the “from” column of the given table slot to determine the corresponding value in the “to” column. If the exact value is not in the table, the lookup performs a linear interpolation between the two nearest bounding values in the “from” column and their corresponding values in the “to” column. **Note:** A key assumption of this function is that the “from” column has a value in every row and that those values are sorted in ascending order. NaNs at the bottom of the column are allowed.
Mathematical Expression	$to\ value = to\ value_{(lesser)} + \frac{to\ value_{(greater)} - to\ value_{(lesser)}}{from\ value_{(greater)} - from\ value_{(lesser)}}(from\ value - from\ value_{(lesser)})$
Comments	If the given slot is not a table slot or if the “from” column or “to” column does not exist in the table, the function aborts the run with an error. Column numbers are zero based with a unit type of NONE. If the “from” value is not the same unit type as the “from” column, or the “from” value is not between the first and last value of the “from” column, the function aborts the run with an error.

Syntax Example

```
TableInterpolation( Lake Mead.Elevation Volume Table,
   0, 1, 1210.03 "ft", @"t" )
TableInterpolation( Mead.Evaporation Table,
   GetColumnIndex(Mead.Evaporation Table, "Julian Day"),
   1, 1210.03 "ft", @"t" )
```

Return Example

```
234342422.32 "m"
```

• TableInterpolation3D

This function performs a double interpolation using two columns of data, and a value for each column, to compute the corresponding value in a third column of data. The data in the two columns used for the double interpolation must be in ascending order.

Description	Table lookup with double linear interpolation.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	Table slot on which to do the lookup
2	NUMERIC	This is the column number (zero based) corresponding to the first column of data in the table - the data to use for the outer/first interpolation
3	NUMERIC	The value to use for the first column
4	NUMERIC	This is the column number (zero based) corresponding to the second column of data in the table - the data to use for the inner/second interpolation
5	NUMERIC	The value to use for the second column

6	NUMERIC	This is the column number of the third column of data in the table - where the answer will be computed
7	DATETIME	datetime context for unit conversions
Evaluation		
Mathematical Expression		
Comments	If the given slot is not a table slot or if the columns do not exist in the table, the function aborts the run with an error. Column numbers are zero based with a unit type of NONE. If a value is not the same unit type as the values in the corresponding column, the function aborts the run with an error. If the values in the table do not encompass the values passed into the function, the function aborts the run with an error. Also, the values in the columns used for both the inner and outer interpolations MUST BE IN ASCENDING ORDER. NaNs at the bottom of the column are allowed.	

Syntax Example

```
TableInterpolation3D( LowerReach.Interpolated GainLoss Table,
   0, GetDayOfYear(@"t"), 1,
   LowerReach.Inflow[], 2, @"t" )
```

Return Example

```
1.344
```

• TableLookup

Description	Table lookup to the nearest value, using a user defined tolerance.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	table slot in which to do the lookup
2	NUMERIC	"from" column
3	NUMERIC	"to" column
4	NUMERIC	value in "from" column
5	DATETIME	datetime context for unit conversions
6	BOOLEAN	whether to round up (or down)

7	NUMERIC	The tolerance (including units) to use when comparing the values in the “from” column. See also “TableLookupDefaultTol” that does not have this argument, but uses a default tolerance value 10^{-6} in standard units (m, cms, m3,etc) of the “from” column.
Evaluation	The value argument is looked up in the “from” column of the given table slot to determine the corresponding value in the “to” column. If the round up argument is true and exact value is not in the table, the lookup finds the row whose value is the smallest value larger than the lookup value and returns the value in that row’s “to” column. If the round up argument is false and the exact value is not in the table, the lookup finds the row whose value is the largest value smaller than the lookup value and returns the value in that row’s “to” column. **Note:** A key assumption of this function is that the “from” column has a value in every row and that those values are sorted in ascending order. NaNs at the bottom of the column are allowed.	
Comments	Column numbers are zero based with a unit type of NONE. Error conditions include: • the given slot is not a table slot • one of the two column indices are not valid for the table • the lookup value has units which are inconsistent with the from column of the table • the search value is not contained within the table • the tolerance value has units which are inconsistent with the from column of the table **Note:** Prior to version 7.5, the TableLookup function only had 6 arguments and used a default tolerance of 10^{-6} using the standard units of the “from” column. In 7.5, calls to TableLookup were replaced automatically with calls to the function TableLookupDefaultTol, described in “TableLookupDefaultTol”.	

Syntax Example

```
TableLookup( Lake Mead.Elevation Volume Table,
   0, 1, 1210.0 "ft", @"t", TRUE, 0.0001 “ft” )
```

Return Example

```
1323342 "acre-feet"
```

• TableLookupDefaultTol

Description	Table lookup to nearest value using a default tolerance.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	SLOT	table slot in which to do lookup
2	NUMERIC	"from" column
3	NUMERIC	"to" column
4	NUMERIC	value in "from" column
5	DATETIME	datetime context for unit conversions

6	BOOLEAN	whether to round up (or down)
Evaluation	The value argument is looked up in the “from” column of the given table slot to determine the corresponding value in the “to” column. If the round up argument is true and exact value is not in the table, the lookup finds the row whose value is the smallest value larger than the lookup value and returns the value in that row’s “to” column. If the round up argument is false and the exact value is not in the table, the lookup finds the row whose value is the largest value smaller than the lookup value and returns the value in that row’s “to” column. **Note:** A key assumption of this function is that the “from” column has a value in every row and that those values are sorted in ascending order. NaNs at the bottom of the column are allowed.	
Comments	Column numbers are zero based with a unit type of NONE. Error conditions include: • the given slot is not a table slot • one of the two column indices are not valid for the table • the lookup value has units which are inconsistent with the from column of the table • the search value is not contained within the table • the tolerance value has units which are inconsistent with the from column of the table TableLookupDefaultTol uses a tolerance of 10^{-6} in standard units in its comparison. See “TableLookup” for a version of this function with a user specified tolerance. **Note:** Prior to version 7.5, the TableLookup function, described in “TableLookup”, only had 6 arguments and used a default tolerance of 10^{-6} in the standard units of the “from” column. In 7.5, calls to TableLookup were replaced automatically with calls to the function TableLookupDefaultTol.	

Syntax Example

```
TableLookupDefaultTol( Lake Mead.Elevation Volume Table,
   0, 1, 1210.0 "ft", @"t", TRUE )
```

Return Example

```
1323343 "acre-feet"
```

• TargetHWGivenInflow

This function computes the outflow required to meet a specified Pool Elevation at a specified future timestep. It performs a lumped mass balance across several timesteps specified as a target range. This function only works with Storage Reservoirs and Level Power Reservoirs.

Description	Computes the outflow required to meet a target pool elevation	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the reservoir on which to perform the calculation
2	DATETIME	the target begin date
3	DATETIME	the target date (target end date)

4	NUMERIC	the target pool elevation value
5	NUMERIC	the total inflow volume over the target range
6	NUMERIC	the previous storage value (before the target begin date)
Evaluation	This function takes the target pool elevation value and converts it to a storage. This is the storage value desired at the target date. The difference between the target storage and the previous storage is the change in storage over the target range. Since the total inflow volume over the target range is given as an argument, the total outflow volume can be computed. Side flows are automatically included in the mass balance computation and thus should not be included in the inflow value provided in Argument 5. These include Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. If any of the side flow slots contain NaN on one of the target period timesteps, the value will be assumed to be zero on that timestep. Other sources and sinks, such as Evaporation, Precipitation, Bank Storage and Seepage are NOT included in the mass balance computation. The total outflow volume is then converted to a flow rate and divided by the number of timesteps in the target range. The result is a single timestep outflow value. This value needs to be set on the outflow slot for every timestep in the target range in order to meet the target pool elevation.	
Mathematical Expression	$\text{Outflow Volume} = \text{Previous Storage} - \text{Target Storage} + \text{Inflow Volume} + \text{Side Flows}$	
Comments	This function is intended to be used with Storage Reservoirs and Level Power Reservoirs. If using a Slope Power Reservoir, the TargetSlopeHWGivenInflow function should be used. Target operations cannot be done on a Pump Storage Reservoir. This function will issue an error if the Time Varying Elevation Volume method is selected and the target begin and end dates bound a table modification date. See “Time Varying Elevation Volume” in *Objects and Methods* for details.	

Syntax Example

```
TargetHWGivenInflow( Lake Mead,
  @"24:00 January 1, 2002",
  @"24:00 January 5, 2002",
  1200 "ft", 70,000 "acre-ft",
  Lake Mead.Storage[@"24:00 December 31, 2002" )
```

Return Example

```
23.43 "cms"
```

• TargetSlopeHWGivenInflow

This function computes the outflow required to meet a specified Pool Elevation at a specified future timestep. It performs a lumped mass balance across several timesteps specified as a target range. This function should be used with Slope Power Reservoirs.

Description	Computes the outflow required to meet a target pool elevation	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	OBJECT	the reservoir on which to perform the calculation

2	DATETIME	the target begin date
3	DATETIME	the target date (target end date)
4	LIST	the inflow values over the target range (there should be one value for each date in the target range)
5	NUMERIC	the target pool elevation value
6	NUMERIC	the previous pool elevation value (before the target begin date)
7	NUMERIC	the previous storage value (before the target begin date)
Evaluation	This function takes the target pool elevation value and converts it to a storage. This is an iterative procedure because there is not a one to one relationship between pool elevation and storage on a slope power reservoir. Once the target storage value has been computed, the change in storage over the target range is determined. Since the total inflow volume over the target range is given as an argument, the total outflow volume can be computed. Side flows are automatically included in the mass balance computation and thus should not be included in the inflow value provided in Argument 4. These include Inflow 2, Hydrologic Inflow Net, Diversion, Return Flow, Canal Flow, Flow FROM Pumped Storage, and Flow TO Pumped Storage. If any of the side flow slots contain NaN on one of the target period timesteps, the value will be assumed to be zero on that timestep. Other sources and sinks, such as Evaporation, Precipitation, Bank Storage and Seepage are NOT included in the mass balance computation. The total outflow volume is then converted to a flow rate and divided by the number of timesteps in the target range. The result is a single timestep outflow value. This value needs to be set on the outflow slot for every timestep in the target range in order to meet the target pool elevation.	
Mathematical Expression	$\text{Outflow Volume} = \text{Previous Storage} - \text{Target Storage} + \text{Inflow Volume} + \text{Side Flows}$	
Comments		

Syntax Example

```
TargetSlopeHWGivenInflow( Lake Mead,
  @"24:00 January 1, 2002",
  @"24:00 January 5, 2002",
  {7500 "cfs", 7750 "cfs", 8125 "cfs", 8200 "cfs", 7900 "cfs"}, 1200 "ft",
  Lake Mead.Pool Elevation[@"24:00 December 31, 2002",
  Lake Mead.Storage[@"24:00 December 31, 2002" )
```

Return Example

```
23.43 "cms"
```

• TextSlotNumericToString

Description	Converts a Text Series Slot encoded text (a numeric) into a STRING	
Type	STRING	
Arguments	**Type**	**Meaning**

1	NUMERIC	The value of a text series slot
Evaluation	Text Series Slots store text as an encoded number. See "Text Series Slots" in *User Interface*. This function looks up the numeric value and returns the text as a STRING. If the numeric value is not found in the dictionary of text slot values, i.e. the numeric has never been used on any slot, an error is issued.	
Comments	See "StringToTextSlotNumeric" and "GetTextSlotValueAsString" for similar functions.	

Syntax Example

```
TextSlotNumericToString( Reservoir.OutflowReason[@"t"] )
```

Return Example

```
"Rafting Operations for Event"
```

ToCelsius, ToFahrenheit, ToKelvin

Description	Convert a temperature to a given temperature scale.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	the temperature to convert
Evaluation	These functions take as input a numeric value representing a temperature in some scale and return the equivalent value in another scale.	
Comments	It is an error to try to convert a value that is not a temperature (i.e., is not in units of degrees Celsius, degrees Fahrenheit, or Kelvin.	

Syntax Example

- ToCelsius
  `(63.23 "F") = 17.35 "C"`
- ToFahrenheit
  `(290.5 "K") = 63.23 "F"`

VolumeToFlow

This function evaluates to the average flow of water over a timestep, which corresponds to a given volume of water.

Description	The steady flow over a timestep corresponding to a volume.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	NUMERIC	volume to be converted
2	DATETIME	timestep over which to convert

Evaluation	The number of seconds in the timestep of the datetime argument is determined. Then, the volume argument is divided by this number of seconds.
Mathematical Expression	$flow = \frac{volume}{\Delta t_{(current\ timestep)}}$
Comments	When called from a series expression slot, the volume to flow conversion uses the expression slot timestep size, which may differ from the run timestep. (This is different from the FlowToVolume function, which always uses the run timestep size for the conversion.)

Syntax Example

```
VolumeToFlow($"Jemez Reservoir.Storage"[], @"t")
```

Return Example

```
23203.231 "cms"
```

WaterOwners

This function evaluates to a list of all user-defined WaterOwners.

Description	This function returns a list of the names of all WaterOwners defined in the Water Accounting System Configuration.
Type	LIST {STRING}
Arguments	none
Evaluation	
Comments	WaterOwners are properties of Accounts. The returned list does not include the default (NONE) WaterOwner.

Syntax Example

```
WaterOwners()
```

Return Example

```
{"IrrigationDistA","IrrigationDistB"}
```

WaterTypes

This function evaluates to the list of user-defined WaterTypes

Description	This function returns a list of the names of all WaterTypes defined in the Water Accounting System Configuration.
Type	LIST {STRING}
Arguments	none
Evaluation	
Comments	WaterTypes are properties of Accounts. The returned list does not include the default (NONE) WaterType.

Syntax Example

```
WaterTypes()
```

Return Example

```
{"CityWater", "Farmer1", "Farmer2"}
```

• WeightedSum

This function computes the normalized weighted sum of a list of numbers.

Description	The normalized weighted sum of a list of numbers.	
Type	NUMERIC	
Arguments	**Type**	**Meaning**
1	LIST {NUMERIC}	the values to be summed
2	LIST {NUMERIC}	the weights of the values
Evaluation	The following mathematical expression is computed and returned.	
Mathematical Expression	$\dfrac{\sum_i \mathrm{weight}_i \cdot \mathrm{value}_i}{\sum_i \mathrm{weight}_i}$	
Comments	All values must have the same dimensionality but may have different units (e.g., all values could be flows, but some in units of cms and others in cfs). Similarly, all weights must have the same dimensionality. Currently, if the dimensionality of the values or weights involves temperature, then all items in that list must have identical units (e.g., it would not be permitted for some values to have units of Celsius/meter and others to have units of Fahrenheit/meter).	

Syntax Example

```
WeightedSum( {2.0 “m”, 13.12 “ft”}, {0.5, 0.5} )
```

Return Example

```
= 2.99 “m”
```

Hypothetical Simulations

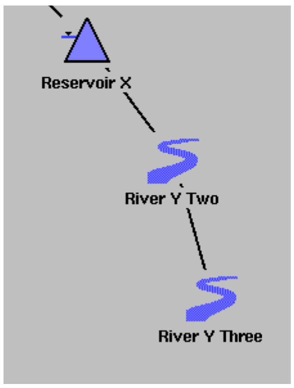
This section presents an overview of a set of functions that allow the user to run a *hypothetical simulation*, where:

- A limited number of objects on the workspace are involved.
- The simulation has no side-effects, i.e., after simulation the workspace is exactly as before.
- The objects involved initially have their current values, except for those values that the user provides as “fixed values” to the hypothetical simulation.
- At least one and possibly more values resulting from the hypothetical simulation are available for use within rulebased simulation.

  Note: See “Hypothetical Simulation Functions” for a list of the functions used for hypothetical simulations.

Hypothetical Simulation Overview

Why would you want to do this? Lets consider the following example. Suppose that you would like to maintain a minimum flow of 100 cfs at some point in River Y. Many miles upstream from this point you can control the outflow from Reservoir X. One question you might ask is: what is the release from X which will lead to the 100 cfs flow at the point of concern? A related but simpler question is: If I release 200 cfs from reservoir X, what will be the flow at the point of concern?



Even the answer to the second question can't be easily predicted; you might have to take into consideration many hydrologic inflows and flow-dependent physical processes like lags, losses, and diversions through different sections of River Y. you might even require that you know the release over some extended period of time in order to be able to determine the flow in the Y at a single time. At any rate, this is exactly the sort of computation performed by the objects in a RiverWare simulation.

On the other hand, answering the first question requires not only knowing the physical consequences of outflow from X, but a search for the release which has the target consequence. The target consequence cannot just be set and allowed to solve upstream because there are routing algorithms can only solve in the downstream direction.

Hypothetical Simulation Functions

The following functions can be used to do hypothetical simulations.

HypSim

Perform a hypothetical simulation with user specified values and returns user-requested result values. See "HypSim" on page 224 for more information on this function.

HypLimitSim

Perform hypothetical simulations iteratively to find a value which, when set on a given slot, will lead to another slot achieving but not exceeding a limiting value. See "HypLimitSim" on page 221 for more information on this function.

HypLimitSimWithStatus

Same as HypLimitSim but with information on whether a satisfying value was found or not. See "HypLimitSimWithStatus" on page 222 for more information on this function.

HypTargetSim

Perform hypothetical simulations iteratively to find a value which, when set on a given slot, will lead to a desired value on another slot. See "HypTargetSim" on page 225 for more information on this function.

HypTargetSimWithStatus

Same as HypTargetSim but with information on whether a satisfying value was found or not. See "HypLimitSimWithStatus" on page 222 for more information on this function.

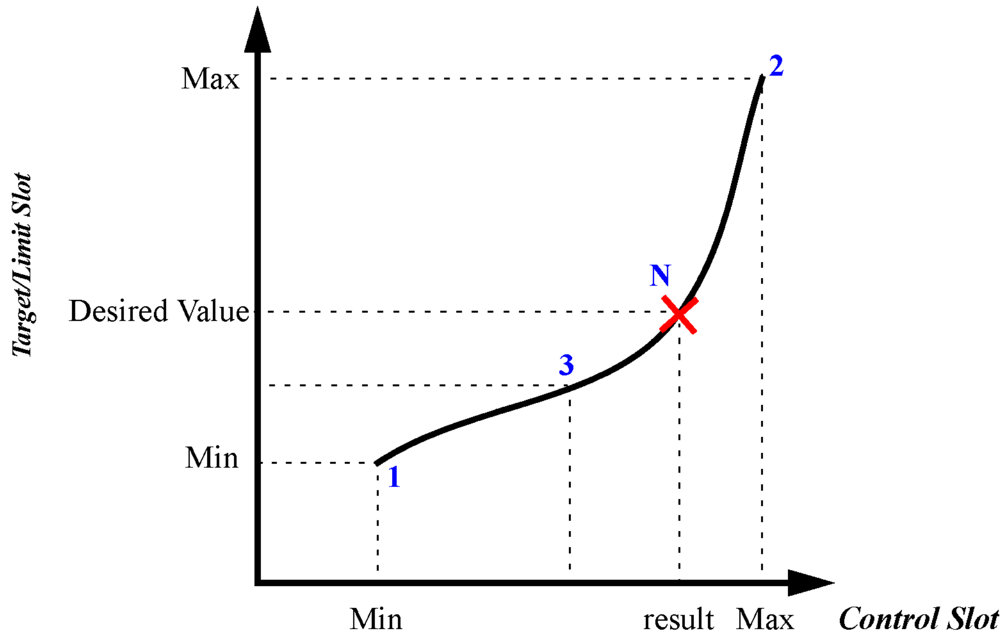
Hypothetical Simulation Process

Each hypothetical simulation function proceeds in the following manner. The first time the function is called, all of the objects and links in the specified subbasin are cloned. This copies each object including all of the slots and values. This cloning is necessary as all subsequent computations are performed on the cloned objects, thereby not affecting the real objects on the workspace. Once the objects are cloned, any values specified in the arguments as "fixed values" are set on the objects. Remember, the function also copied all its data at the time it was cloned, so the fixed values are values that are to be set on the object that are not already there.

Then, the hypothetical simulation performs the computations described for that function. For HypSim, the cloned objects will dispatch to simulate the effects of the fixed values. A list containing the values of the specified slots at the specified datetimes will be returned. The calling rule/function can then use these results in its computation.

HypLimitSim, HypLimitSimWithStatus, HypTargetSim, and HypTargetSimWithStatus perform an iterative solution. For each of these functions, you provide min and max values for a control slot and a target or limit of a downstream slot. The computations then boil down to univariate zero-finding, where each evaluation is a hypothetical simulation with different inputs. The solution is found using the bisection method as shown in the following figure; it simulates using the min control slot value (1), then the max control slot value (2), then bisects between the two(3). It continues bisecting until the tolerance or desired accuracy of the limit/target slot is met and a solution is found (N.) If the result of any try is outside of the range of previous tries, then a warning message is issued saying that the function is non-monotonic. There are either multiple solutions or the function is not well behaved. This typically leads to convergence issues.

Figure 5.2



Following are some notes to consider about hypothetical simulation and particularly the iterative hypothetical simulations:

- These are hypothetical simulations, not rulebased simulations. You can set fixed values in the arguments but they cannot change once the function is executed.
- Hypothetical simulation does not support accounting or optimization functionality.
- Each iteration within hypothetical simulation is setting the control value to a new value and re-simulating the system. Some object methods may have a dependence on previous solutions that will lead to different results. Also, slot convergence can be an issue. If setting a new value on a slot does not propagate down the system because convergence is too loose, then the target/limit slot value may never be achieved.
- Use the WithStatus version of the function if you suspect that the function may not work in all cases but you still need a result. If these functions do not find a result, the closest value is returned along with the status. But, make sure to check the status in the calling rule or function, don't just use the result blindly. Use the "without status" version if you reasonably expect there to be a solution. Then, if there is a problem with the computation and no solution is found, the run is stopped and a message is posted.
- For the iterative functions, if there are multiple timesteps involved, the control slot is always set to the same value for all timesteps in the hypothetical simulation range.
- These hypothetical simulation functions are expensive in terms of run time. Following are some approaches to limit slow down from these functions:
  - Limit their use as much as possible.
  - If you need to call a function multiple times per timestep with the same arguments, create a helper function with no arguments; functions with no arguments are executed once per timestep and the results are cached for later use.

– Only include the relevant objects in the subbasin; cloning objects and copying values is time and memory expensive.

– Only include as many *fixed values* as necessary. If you are only solving for the target slot value at t+2, don’t include fixed values from t through t+7. This will lead to unnecessary dispatching of the cloned objects.